

Livro HYDRA

Livro HYDRA reúne programas visuais de *live coding* feitos em Hydra por **Alexandre Rangel**. Cada entrada corresponde a um patch (uma data / variante), apresentado como um *spread* de duas páginas em A4 paisagem: à esquerda, a imagem em movimento condensada em stills; à direita, o código e um texto breve sobre as técnicas em jogo.

O livro não pretende ser um manual de Hydra, nem um catálogo fechado. É um recorte editorial de uma prática contínua — patches escritos, reescritos e reexecutados ao longo dos anos — fixado em papel para leitura lenta, com um QR que devolve o patch ao browser em tela cheia (GitHub Pages: `alexandrangel.github.io/hydras/{id}/`).

Método de captura

Para cada patch incluído, o fluxo de captura produz um **banco de dez stills nativos de GPU**, em **2560×1440**, a partir do instante em que o patch começa a rodar:

Frame	Tempo desde o start
f01	1 s
f02	3 s
f03	10 s
f04	30 s
f05	45 s
f06	60 s
f07	90 s
f08	120 s
f09	150 s
f10	180 s

Os spreads usam, por padrão, **f01/f03/f04/f05 (1/10/30/45 s)**. Esse conjunto sugere a animação no tempo da página; o banco completo (f01–f10) fica guardado em `captures/{id}/` para trocas posteriores sem reabrir o Hydra.

Notas de diagramação

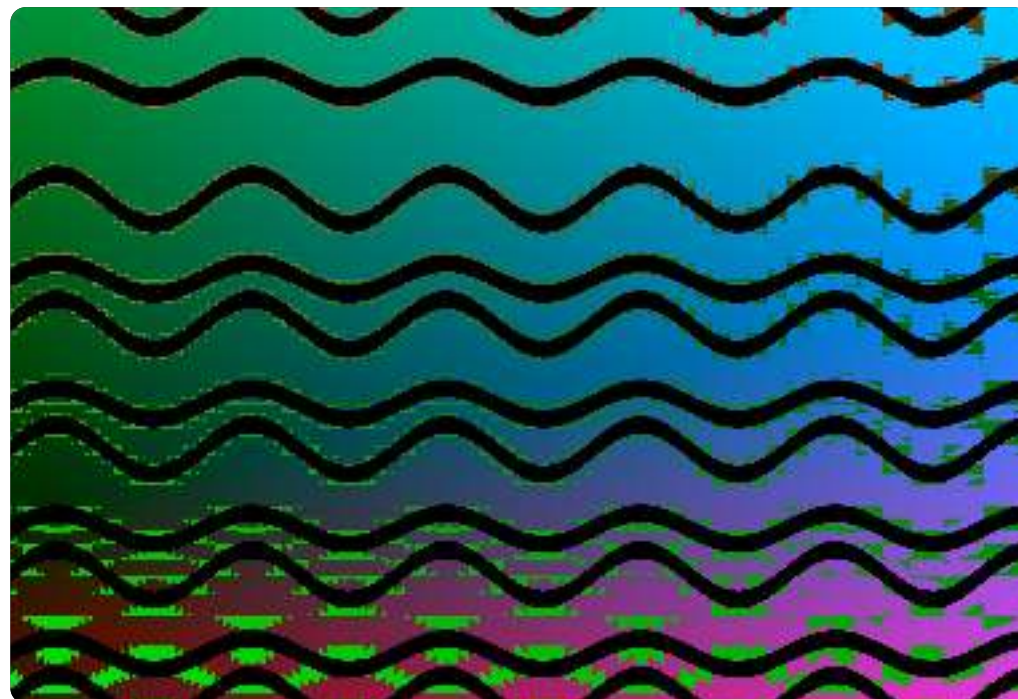
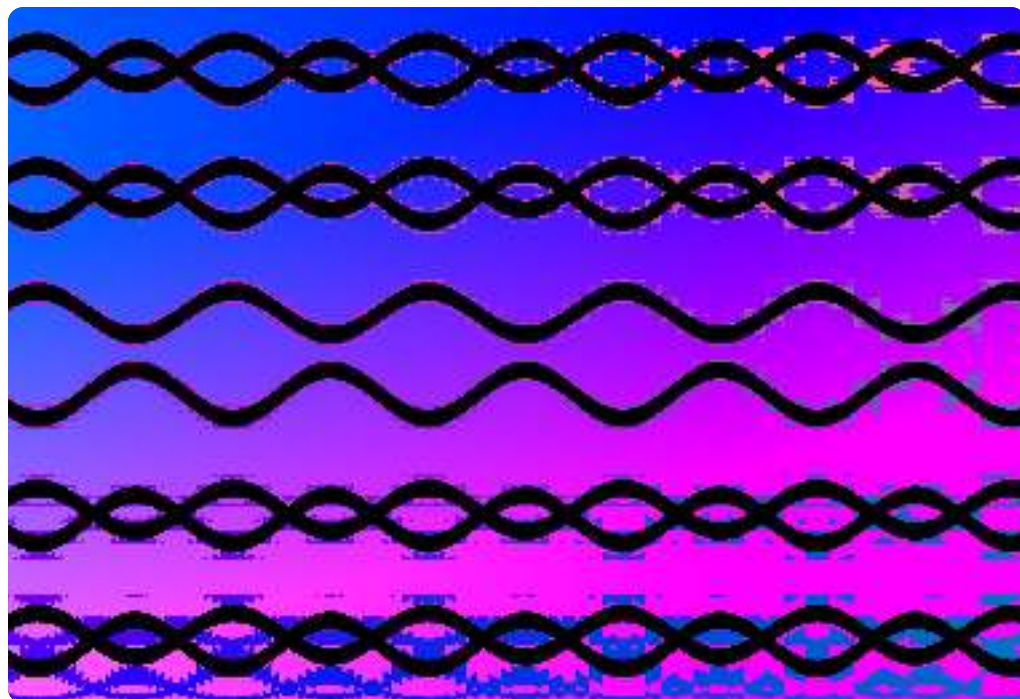
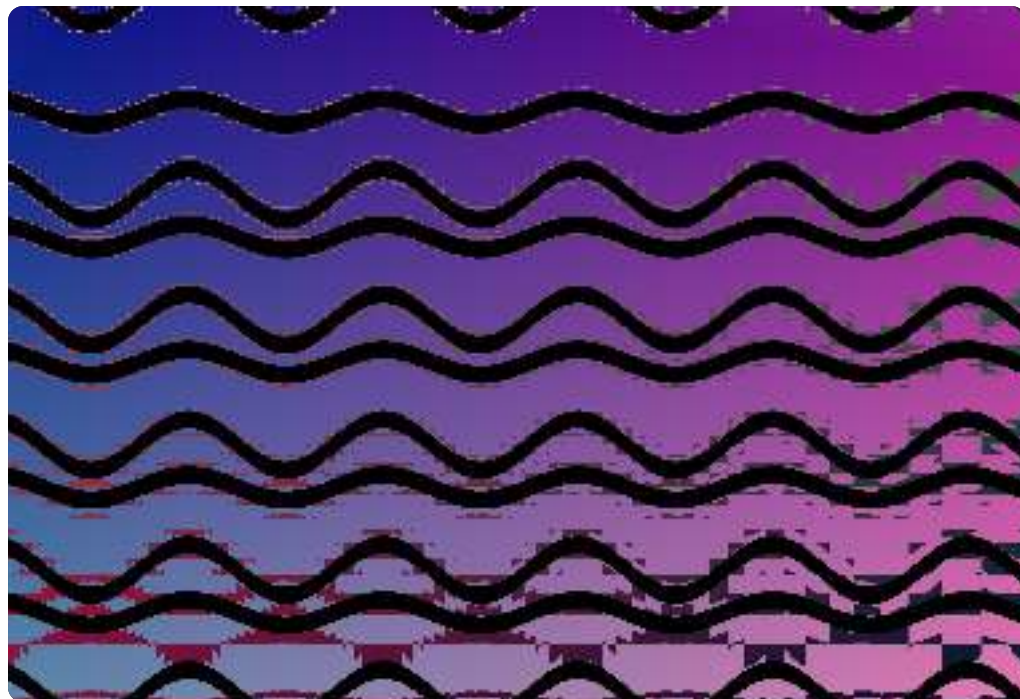
- Formato: **A4 paisagem**, um patch por *spread* (2 páginas).
- Página esquerda: grade **2×2** com os quatro stills escolhidos (f01/f03/f04/f05).
- Página direita: **código a 9,5 pt em Ubuntu Mono**, em caixa **#ededed (~7%)**, e **prosa técnica**.
- Código longo ocupa a coluna esquerda inteira; o excedente continua no topo da coluna direita, com a prosa abaixo.
- Patches com shader externo e `loadScript` foram incluídos após os patches/loader terem sido estabilizados.
- **QR** no canto inferior aponta para a página fullscreen no GitHub Pages.
- Margens verticais fixas de **8 mm** (laterais ~14 mm).

Entradas adiadas nesta montagem

A captura e a montagem dos patches de shader/`loadScript` foram concluídas. Permanecem adiadas **8 entradas**, registradas em `book/capture-skip.json` e fora deste PDF:

- **Falha de captura / WebGL:** 2022-06-27, 2022-08-19.
- **Não são patches Hydra JS:** 2023-04-12a, 2024-07-17, 2024-09-01, 2024-09-09.
- **Patches Tone.js / DOM com frames pretos:** 2025-04-22a, 2025-04-22b.

Podem voltar em revisões futuras quando a captura estiver resolvida.



```

speed=0.75

x()=>
osc(35,0,0).rotate(Math.PI/2).thresh(0.05,0)
.modulateScrollY( osc(40,-0.02,0) , ()=>Math.sin(time/2)*0.075 )
.mult(
osc(35,0,0).rotate(Math.PI/2).thresh(0.05,0).scrollY(0.1)
.modulateScrollY( osc(40,-0.02,0) , ()=>Math.cos(time/4)*0.075 )
)
.scrollY(.01,-.025)

x()
.mult(
  x()
  .modulatePixelate( gradient(1) , 100)
  .diff(gradient(0.25).rotate(()=>time/3)).color(1,0.5,1)
  .saturate( ()=>Math.sin(time/3)*2 )
)
.rotate(Math.PI)

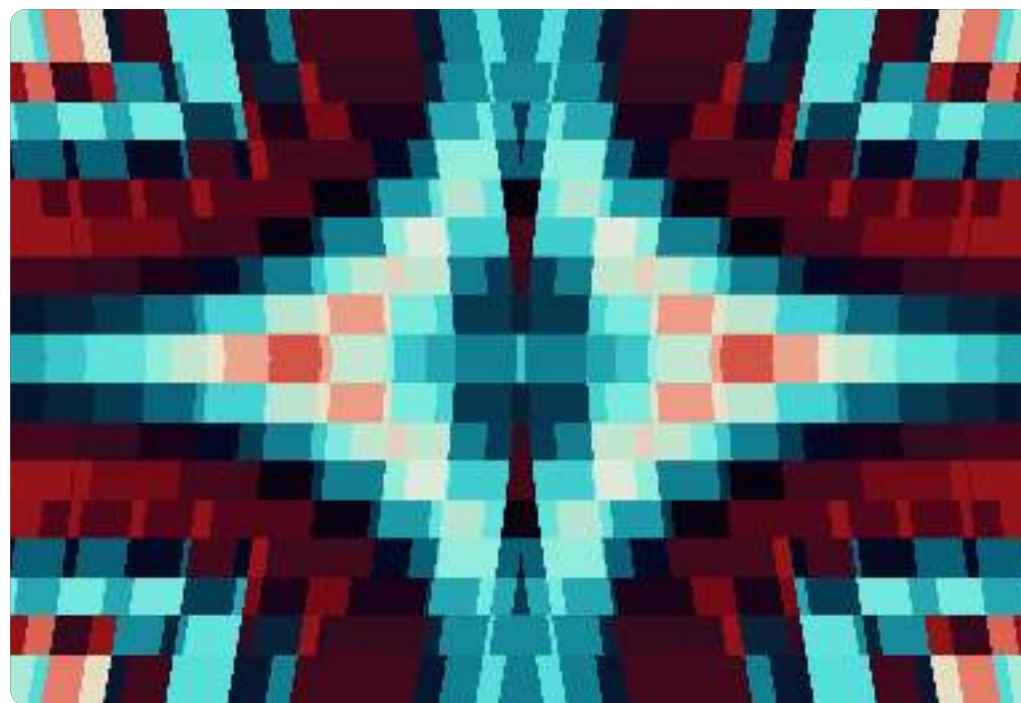
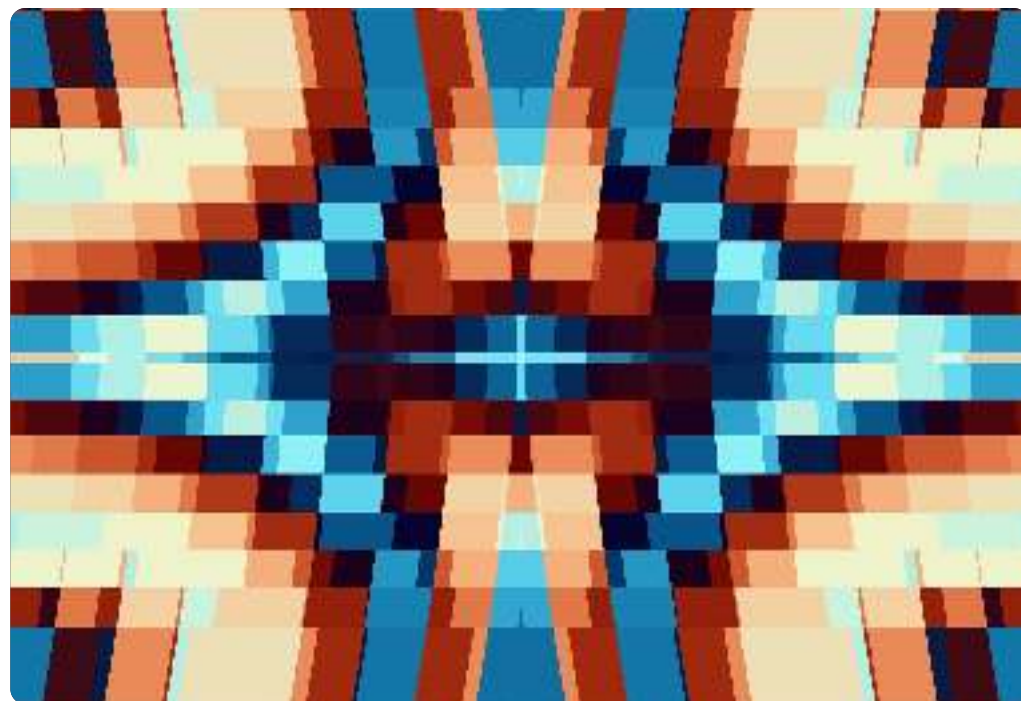
.out()

```

O patch se organiza em torno de rotação (**rotate**), osciladores (**osc**), modulação de scroll Y (**modulateScrollY**), gradientes (**gradient**) e deslocamento Y (**scrollY**). Osciladores viram padrões binários via **thresh**, e a geometria resultante alimenta o restante da cadeia. A composição usa **mult**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScrollY**, **modulatePixelate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.75** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.



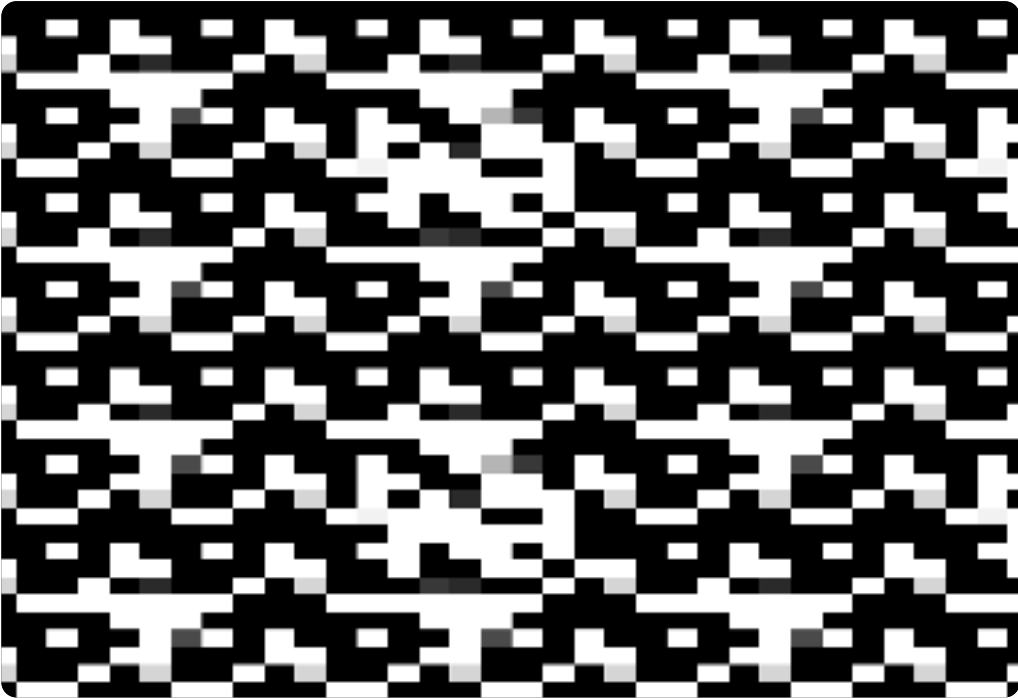
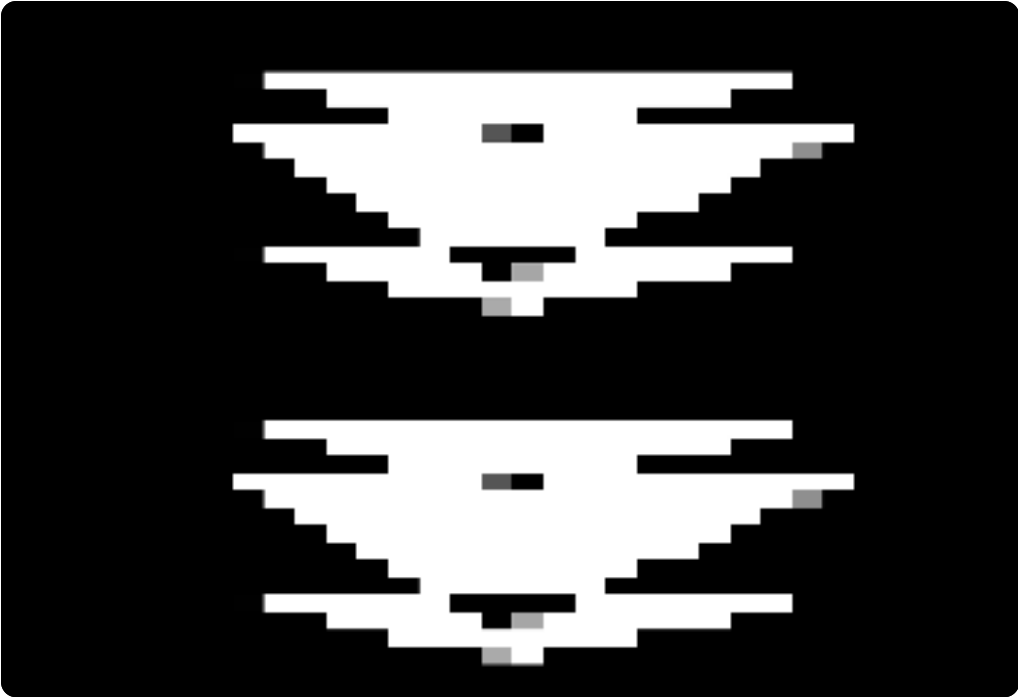


```
osc(16,-0.01,0.75).kaleid(30).pixelate()  
.modulateScrollX(osc(8,-0.003,0).kaleid(30).pixelate(),-1)  
.rotate(Math.PI/-2)  
.modulateScrollX( osc(2,0.1,0), .1)  
.modulateScrollY( osc(2,0.09,0), .1)  
.kaleid(2).colorama( ()=>Math.sin(time/5)*0.07 -0.1)  
.out()
```

O patch se organiza em torno de osciladores (**osc**), kaleidoscópio (**kaleid**), modulação de scroll X (**modulateScrollX**), pixelização (**pixelate**) e modulação de scroll Y (**modulateScrollY**). **osc** entra em **kaleid**, multiplicando eixos de simetria até o quadro parecer um ornamento vivo.

A modulação (**modulateScrollX**, **modulateScrollY**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.



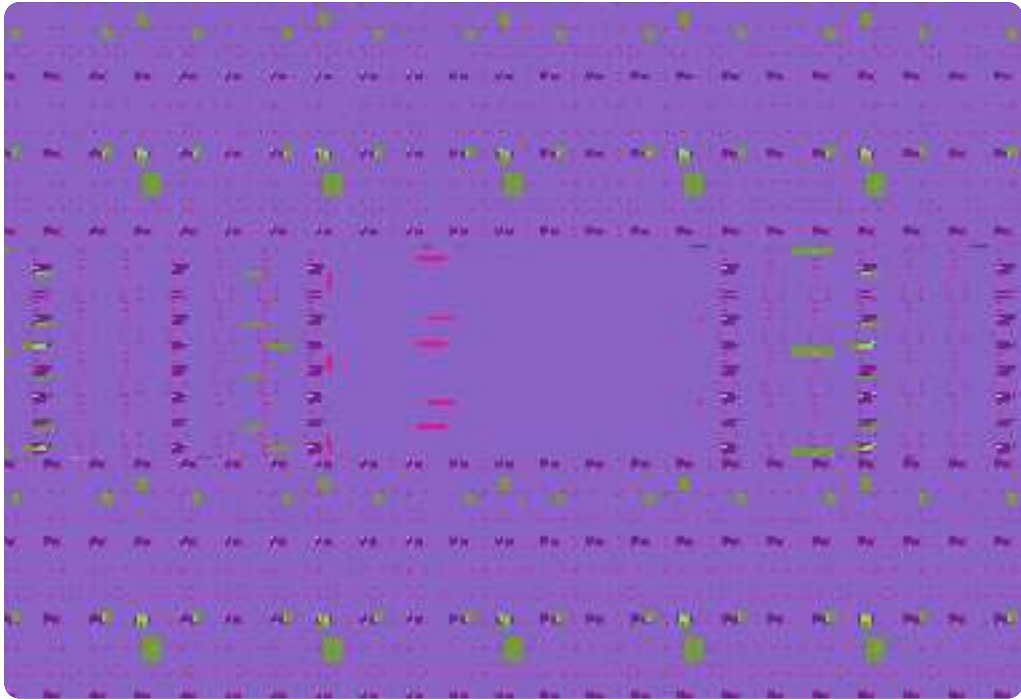
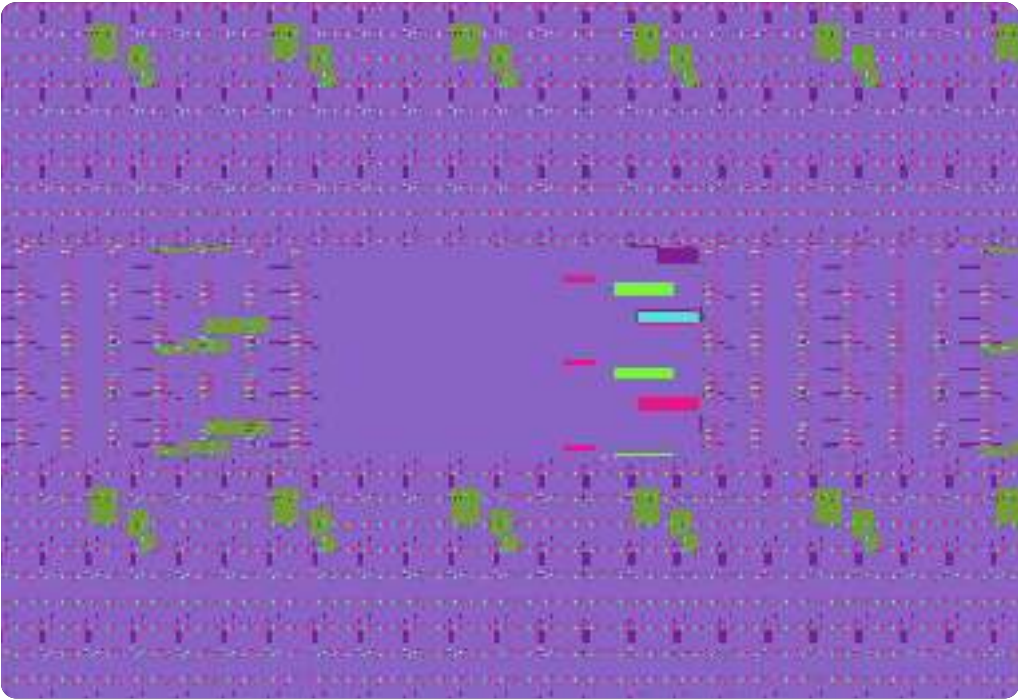
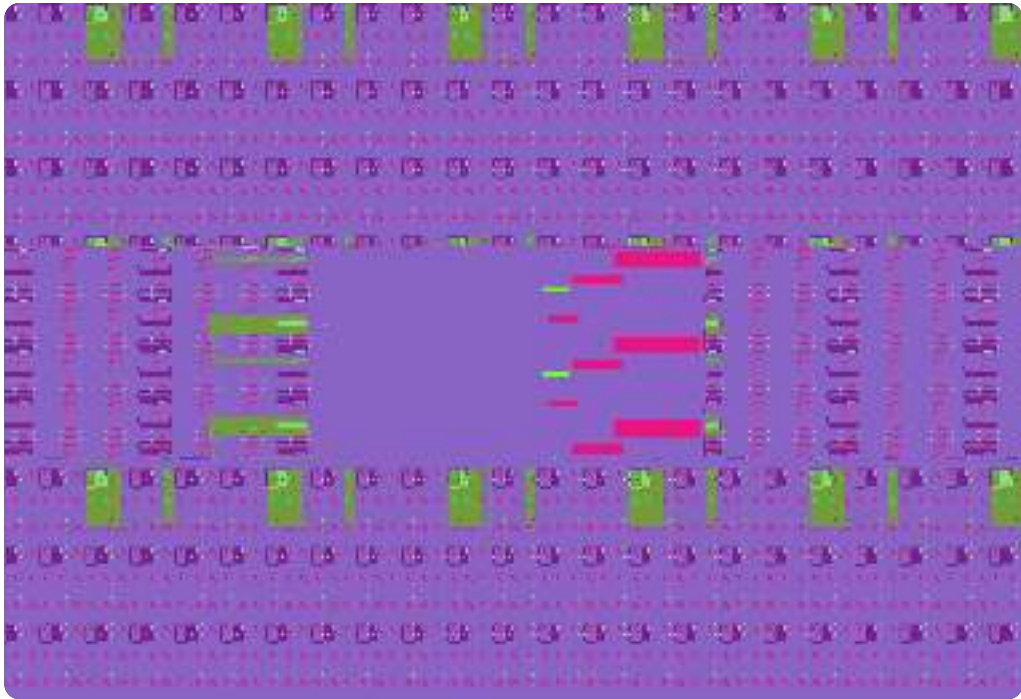
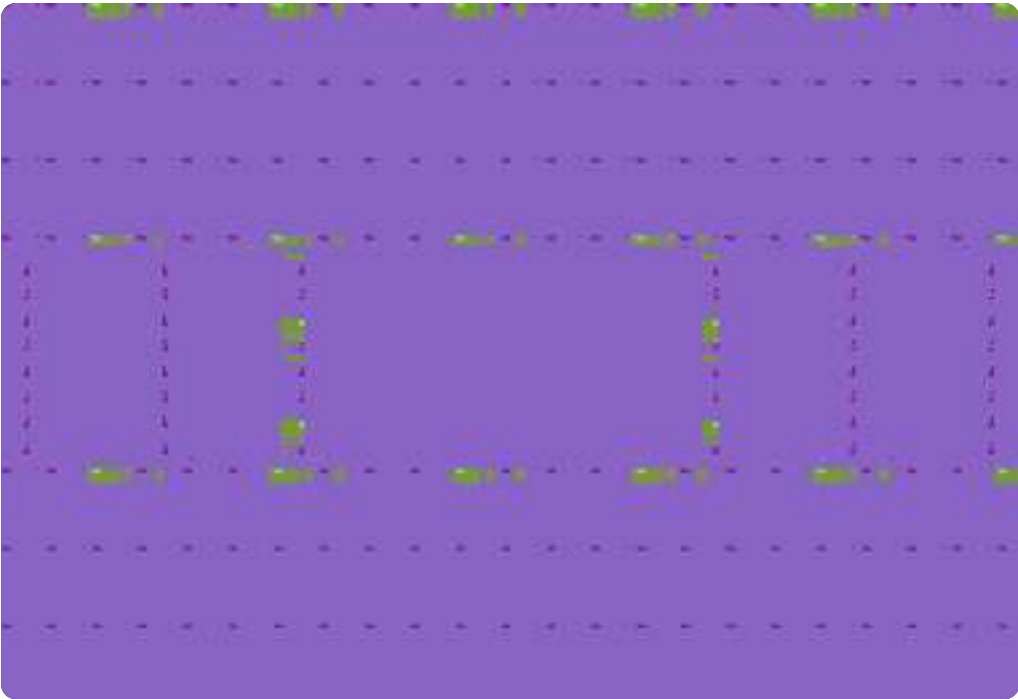


```
speed=0.5
shape([3,4,5,4].fast(0.3).smooth(0.5))
.diff(o0, ()=>Math.sin(time/3)*0.5+0.3 )
.repeat([1,2,1,2,1].fast(0.3),[1,2].fast(0.3))
.repeat(1,[2,1].fast(0.3))
.scrollX(0.01,-0.01)
.pixelate(40,40)
.out()
```

O patch se organiza em torno de arrays acelerados (**.fast**), repetição (**repeat**), pixelização (**pixelate**), deslocamento X (**scrollX**) e interpolação suave (**.smooth**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

speed = 0.5 define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra.





```

shape(4,0.1,0).scrollY(0.1,0.1).scale(
  ()=>Math.sin(time/1)+1.5).scrollX(Math.random())
.color( Math.random(), Math.random(), Math.random() )
.diff( shape(4,0.1,0).scrollY(0.1,0.1).scale(
  ()=>Math.sin(time/2)+1.5).scrollX(Math.random()) )
.diff( shape(4,0.1,0).scrollY(0.1,0.1).scale(
  ()=>Math.sin(time/3)+1.5).scrollX(Math.random()) )
.diff( shape(4,0.1,0).scrollY(0.1,0.1).scale(
  ()=>Math.sin(time/4)+1.5).scrollX(Math.random()) )
.repeat(3)

.colorama( Math.random() * -100 )

.diff( shape(4,0.1,0).scrollY(0.1,0.1).scale(
  ()=>Math.sin(time/5)+1.5).scrollX(Math.random()) )
.diff( shape(4,0.1,0).scrollY(0.1,0.1).scale(
  ()=>Math.sin(time/6)+1.5).scrollX(Math.random()) )
.diff( shape(4,0.1,0).scrollY(0.1,0.1).scale(
  ()=>Math.sin(time/7)+1.5).scrollX(Math.random()) )
.repeat(4)

.colorama( Math.random() * -100 )

.diff( shape(4,0.1,0).scrollY(0.1,0.1).scale(
  ()=>Math.sin(time/8)+1.5).scrollX(Math.random()) )
.diff( shape(4,0.1,0).scrollY(0.1,0.1).scale(
  ()=>Math.sin(time/9)+1.5).scrollX(Math.random()) )
.diff( shape(4,0.1,0).scrollY(0.1,0.1).scale(
  ()=>Math.sin(time/10)+1.5).scrollX(Math.random()) )
.repeat( Math.random()*10 )

.modulateScale( shape([2,4] ) , Math.random()* 10)

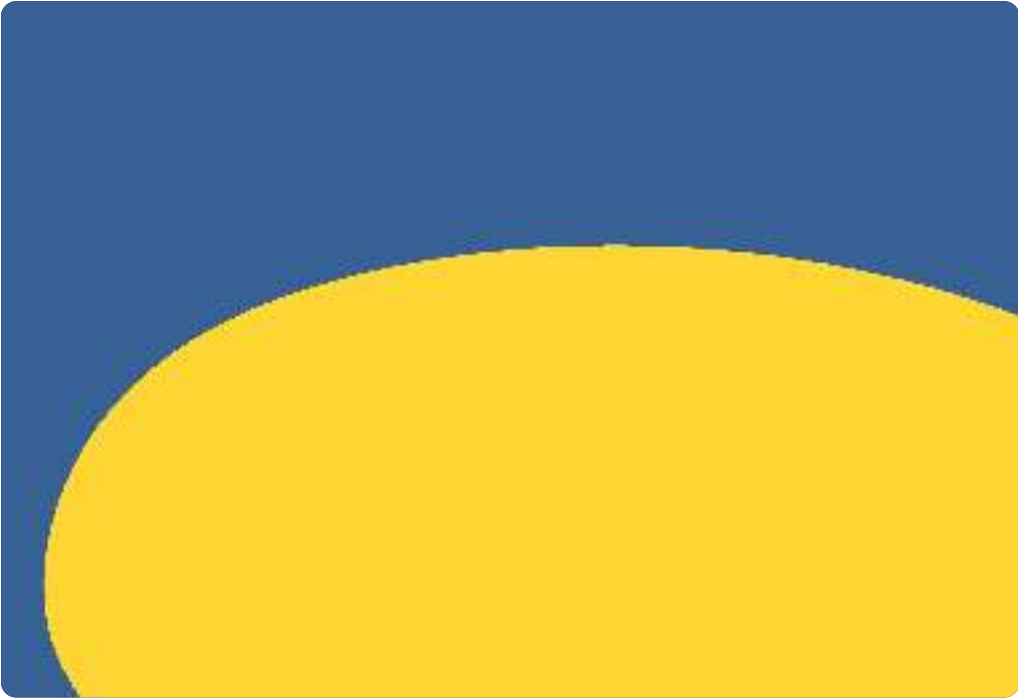
```

```
.out()
```

O patch se organiza em torno de formas geométricas (**shape**), deslocamento Y (**scrollY**), deslocamento X (**scrollX**), escala (**scale**) e diferença (**diff**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateRotate**, **modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. **Math.random** congela valores na carga do patch: cada abertura pode nascer com frequências e kaleidoscópios distintos. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.



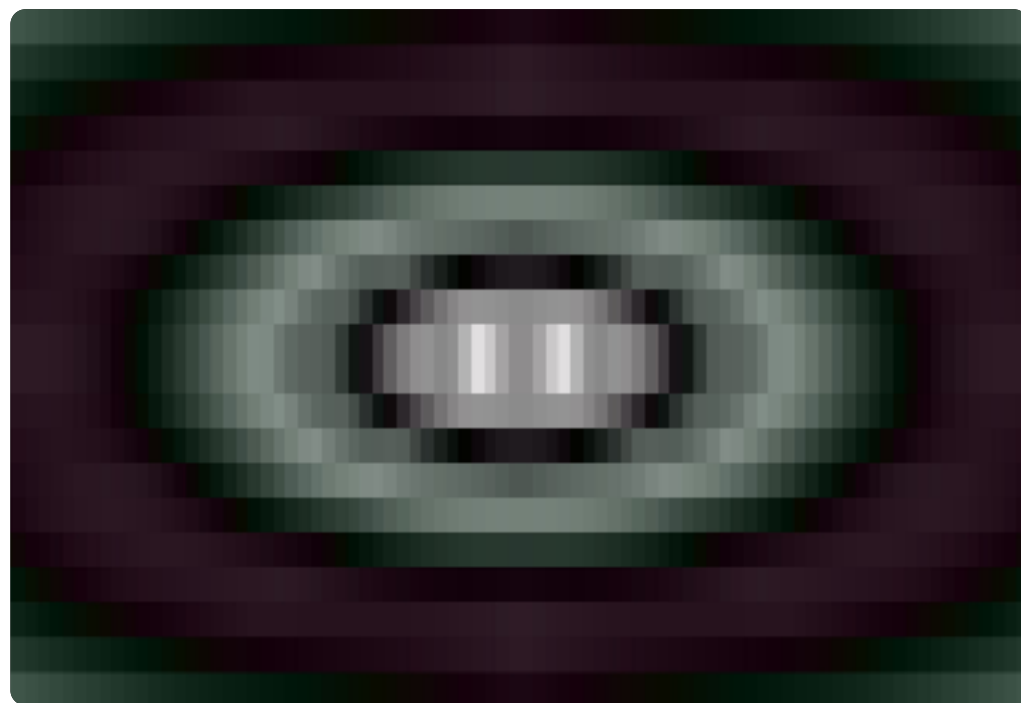
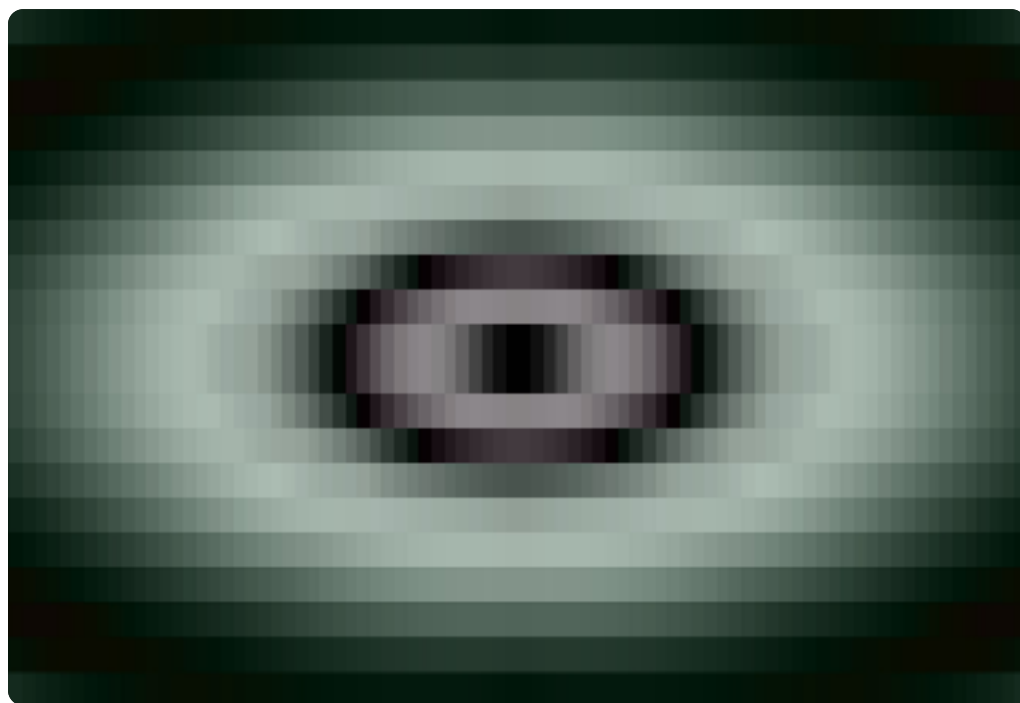
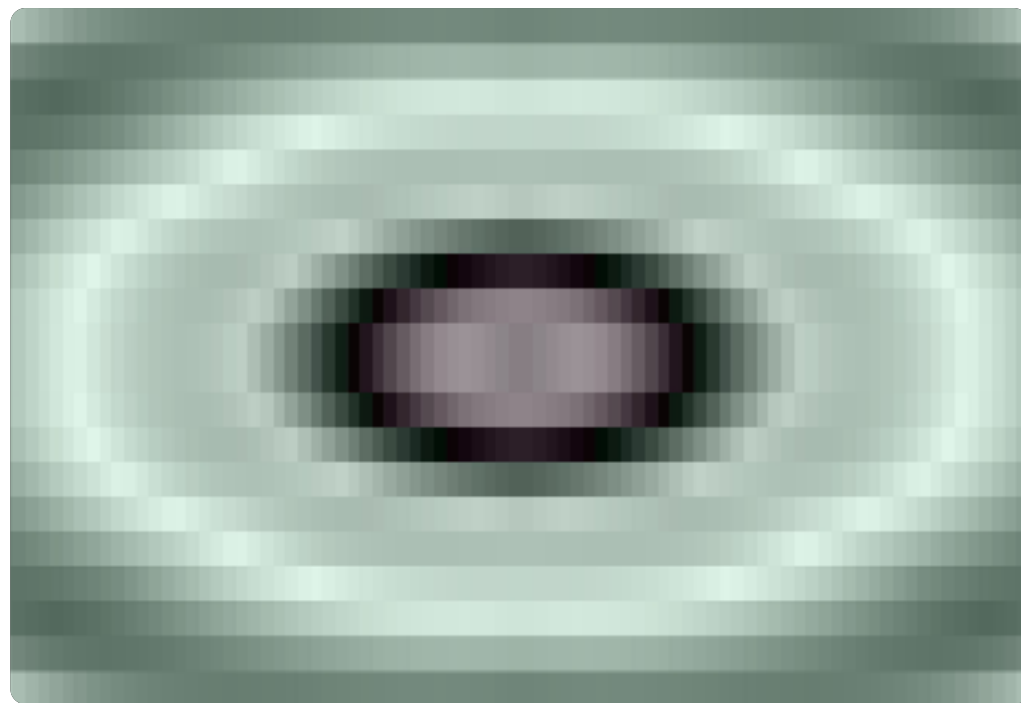
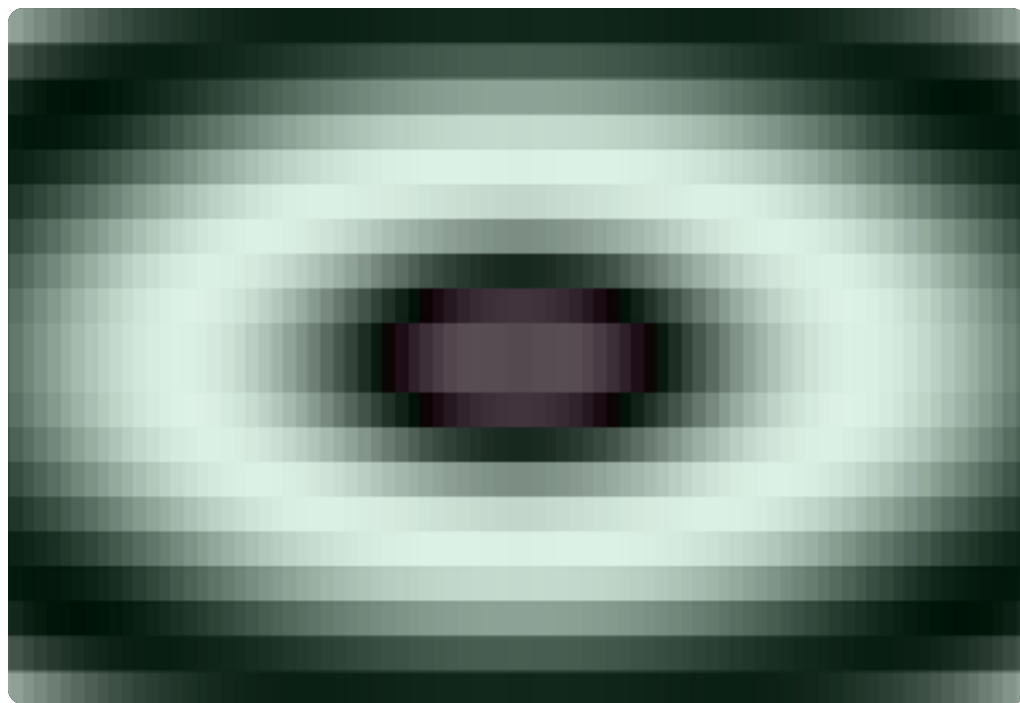


```
solid(0.22,0.38,0.59)
.layer(
noise(0.5,0.05).thresh(0.5,0).color(1,0.84,0.2).mask(noise(0.5,0.05).thresh(0.5,0))
)
.layer(
noise(0.4,0.04).thresh(0.5,0).color(0.43,0.58,0.24).mask(noise(0.4,0.04).thresh(0.5,0))
)
.layer(
noise(0.4,0.04).thresh(0.5,0).color(0.43,0.58,0.24).mask(noise(0.4,0.04).thresh(0.5,0))
)
.layer(
noise(0.35,0.035).thresh(0.5,0).color(0.19,0.34,0.51).mask(noise(0.35,0.035).thresh(0.5,0))
)
.layer(
noise(0.44,0.0222).thresh(0.5,0).color(0.97,0.67,0.35).mask(noise(0.44,0.0222).thresh(0.5,0))
)
.layer(
noise(0.44,0.032).thresh(0.5,0).color(0.87,0.91,0.94).mask(noise(0.44,0.032).thresh(0.5,0))
)
.out()
```

O patch se organiza em torno de limiar (**thresh**), ruído (**noise**), empilhamento (**layer**), colorização (**color**) e máscara (**mask**). Fundos **solid** recebem camadas com **layer/mask**, construindo o quadro por empilhamento seletivo. A composição usa **layer**, **mask** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.



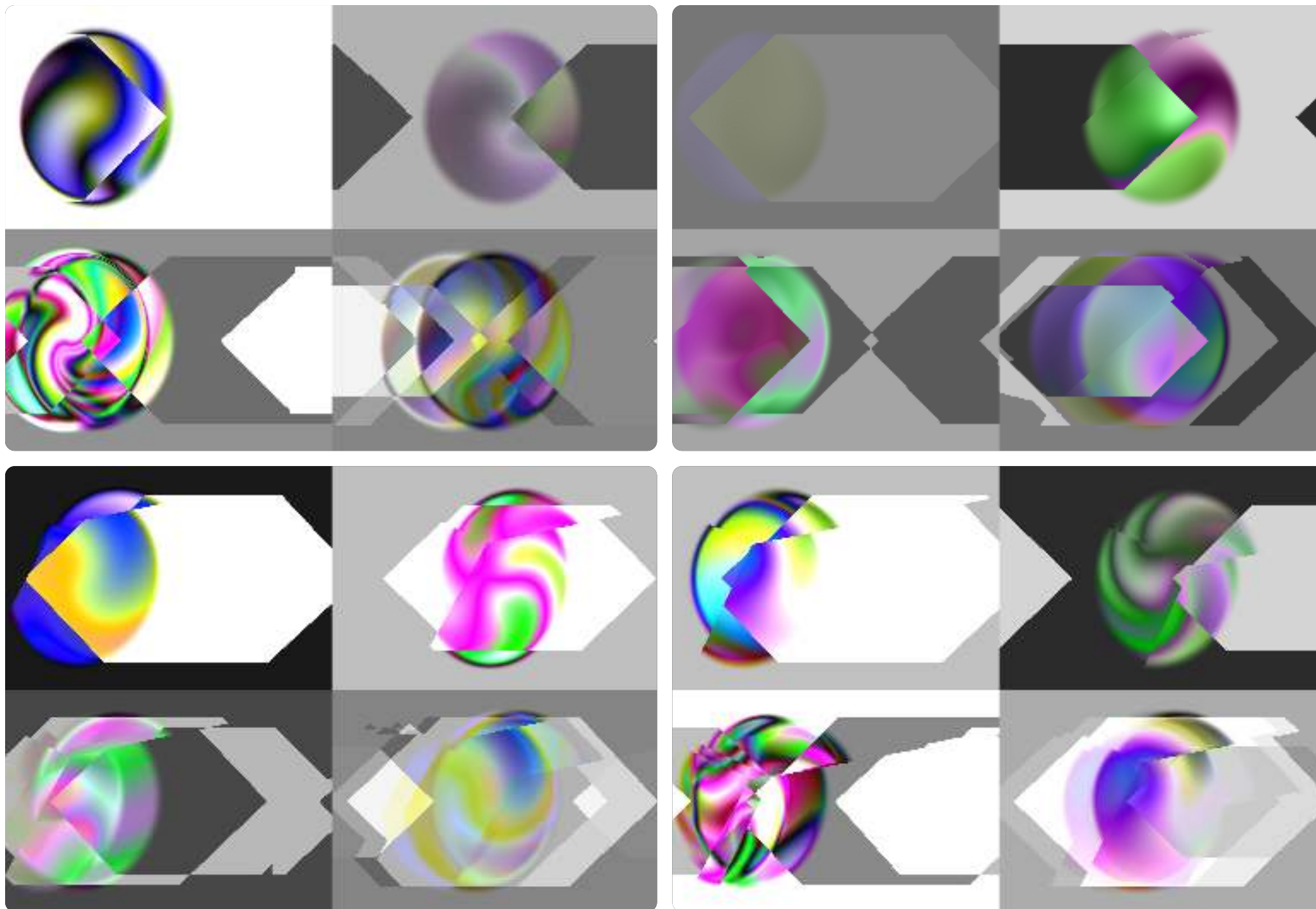


```
osc( Math.random()*50, Math.random()*-0.05 )
.kaleid( 100 )
.diff(
osc( Math.random()*40, Math.random()*0.04 )
.kaleid( 8 )
)
.diff(
osc( Math.random()*30, Math.random()*-0.03 )
.kaleid( 12 )
.colorama( Math.random()*-0.25 )
)
.modulateScale(
osc( Math.random()*20, Math.random()*0.02 )
.kaleid( 4 )
, (Math.random()*2)-1
)
.modulateScale(
osc(3,-0.01).kaleid(100)
,2)
.pixelate(100,20)
.out()
```

O patch se organiza em torno de kaleidoscópio (**kaleid**), osciladores (**osc**), modulação de escala (**modulateScale**), diferença (**diff**) e pixelização (**pixelate**). **osc** entra em **kaleid**, multiplicando eixos de simetria até o quadro parecer um ornamento vivo. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **Math.random** congela valores na carga do patch: cada abertura pode nascer com frequências e kaleidoscópios distintos. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

speed=0.7
const date = new Date();
let ms = ()=>date.getMilliseconds();
let ms2 = ()=>date.getMilliseconds()*2;
let ms3 = ()=>date.getMilliseconds()*3;
let ms4 = ()=>date.getMilliseconds()+9999;
let a = Math.random()*0.4+0.6
let b = Math.random()*0.4+0.6
let c = Math.random()*0.4-1.1
let d = Math.random()*0.4-1.1

solid(Math.random(),Math.random(),Math.random())
  .mult(noise(1.505,0.05))
  .diff(noise(2.71,0.022))
  .rotate(()=>Math.sin(time/600+ms())*Math.PI*2)
  .modulateScale(osc(()=>Math.sin(time/160+ms())+2,0.01).contrast(1
  .5).kaleid(50)
    ,2)
  .modulateRotate(osc(()=>Math.cos(time/298-
ms())+2.5,-0.005).contrast(1.8).kaleid(50)
    ,Math.PI*-1).saturate(2)
  .add(osc(2,0.05,
  ()=>Math.cos(time/170+ms3())*0.5+1.5).contrast(2),0.1)
    .mask(shape(100,0.7,0.13))
    .scale(1,innerHeight/innerWidth,1)
  .invert([0,-1,0,-1,a,-.75,0,c,.5,1,0.75,0,-0.5,-1].smooth(0.57).f
ast(0.154+(Math.random()*0.1)))
  .diff(shape(6,0.75,0.0).scrollX(0.075,-0.075))
  .modulateScrollX(voronoi(0.9,0.02),.2)
  .out(o0)

solid(Math.random(),Math.random(),Math.random())

```

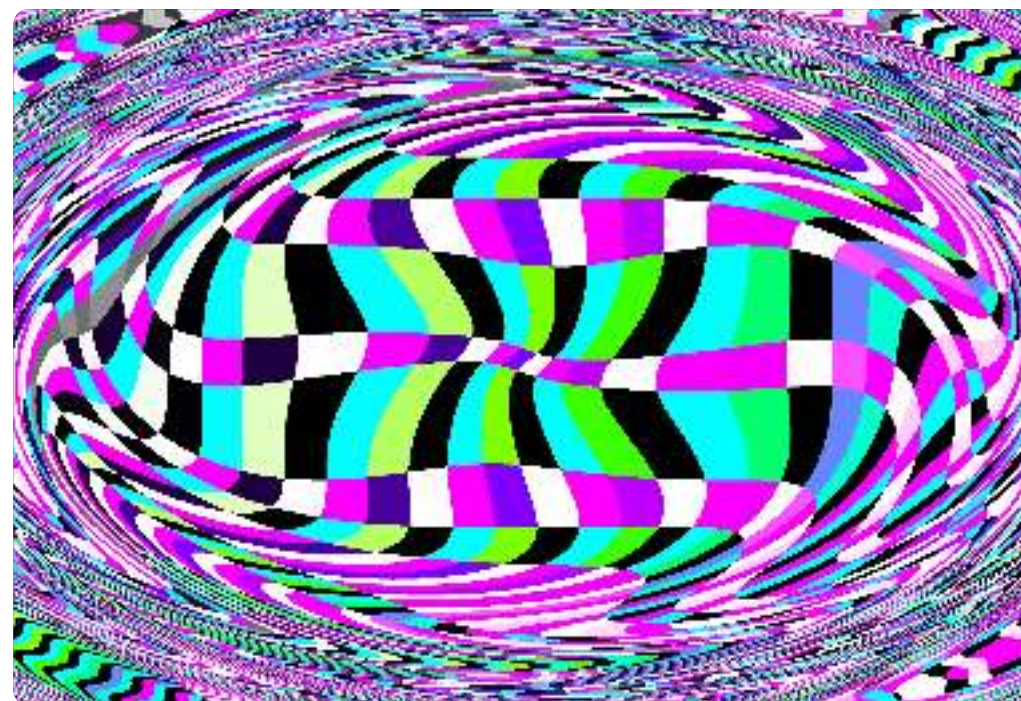
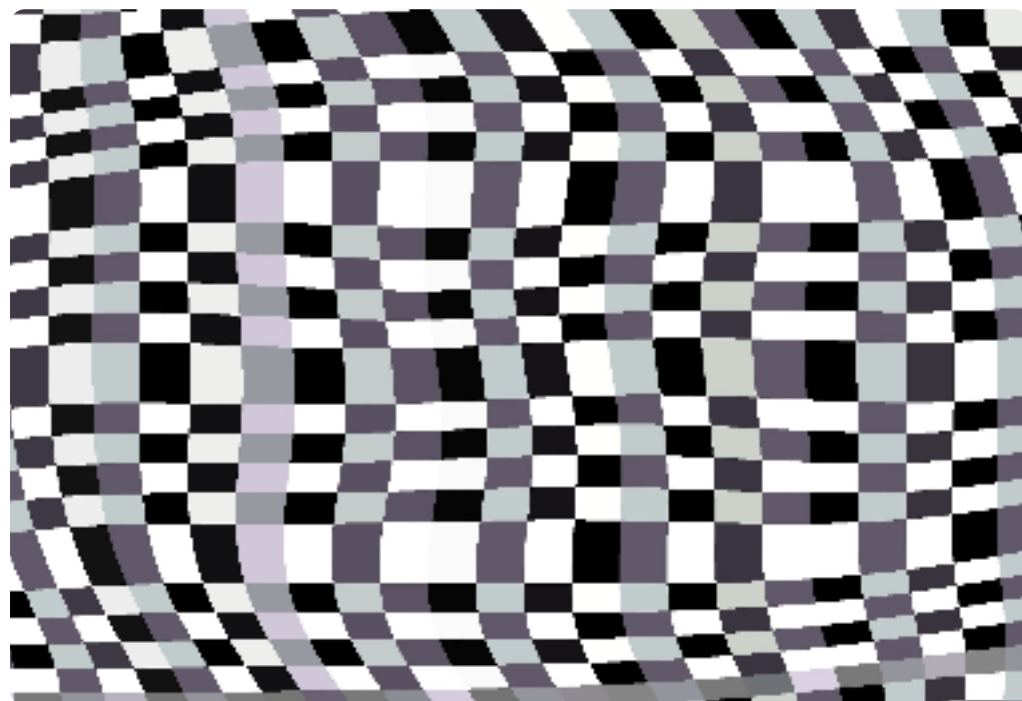
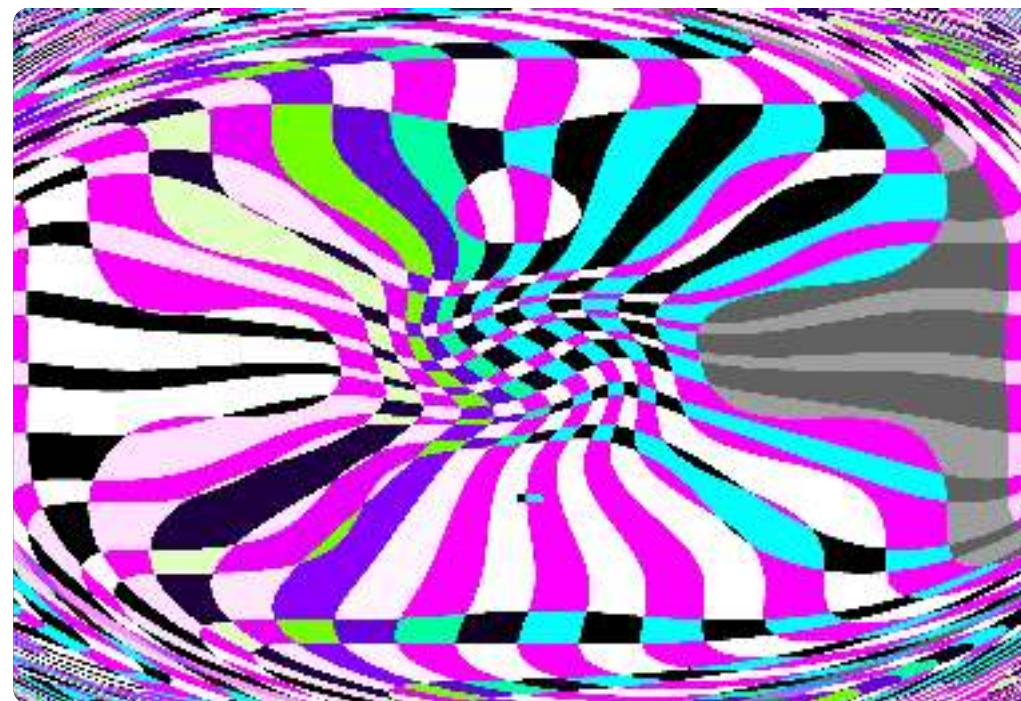
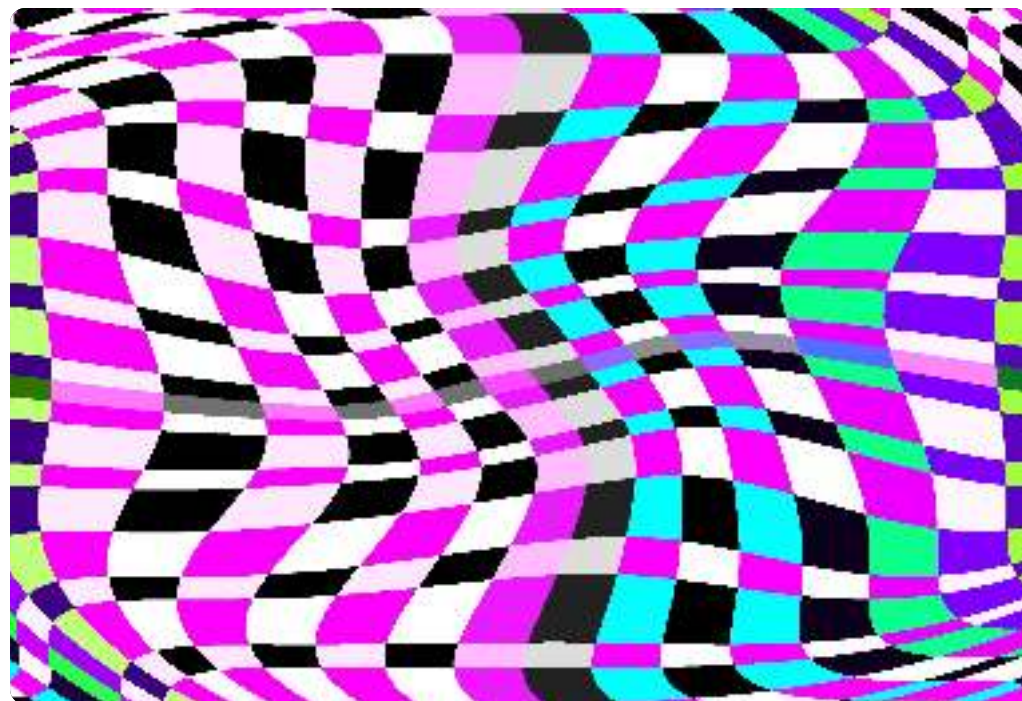
```

  .rotate(()=>Math.sin(time/600+ms2())*Math.PI*2)
  .modulateScale(osc(()=>Math.sin(time/155+ms2())+2,0.009).co
ntrast(1.5).kaleid(50)
    ,2)
  .modulateRotate(osc(()=>Math.cos(time/333+ms3())+2.5,0.005)
  .contrast(1.8).kaleid(50)
    ,Math.PI*-1).saturate(2)
  .add(osc(2,0.05,
  ()=>Math.cos(time/222+ms())*0.5+1.5).contrast(2),0.1)
    .mask(shape(100,0.7,0.13))
    .scale(1,innerHeight/innerWidth,1)
  .diff(src(o2).scale(1.015).invert(),
    ()=>Math.sin(time*0.6+ms())*0.26+0.58)
  .invert([b,c,.75,a,1,0.75,0,-0.75,d,-1,-.75].smooth(0.57).f
ast(0.15+(Math.random()*0.1)))
  .diff(shape(6,0.75,0.0).scrollX(0.05,0.05))
  .modulateScrollX(voronoi(0.8,0.02),.2)
  .out(o1)

solid(Math.random(),Math.random(),Math.random())
  .mult(noise(1.51,0.05))
  .diff(noise(2.42,0.021))
  .rotate(()=>Math.sin(time/600+ms2())*Math.PI*2)
  .modulateScale(osc(()=>Math.sin(time/157+ms3())+2,-0.008).c
ontrast(1.5).kaleid(50)
    ,2)
  .modulateRotate(osc(()=>Math.cos(time/310-
ms2())+2.5,0.005).contrast(1.8).kaleid(50)
    ,Math.PI*-1).saturate(2)
  .add(osc(2,0.05,(()=>Math.cos(time/220-
ms2())*0.5+1.5).contrast(2),0.1)
    .mask(shape(100,0.7,0.13))

```





```

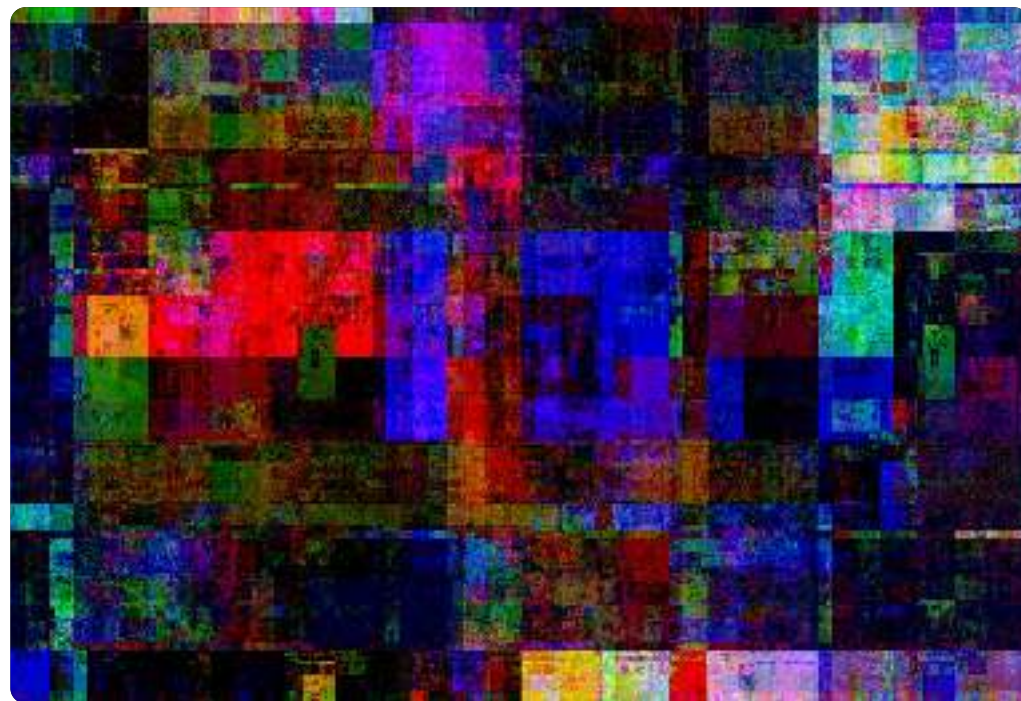
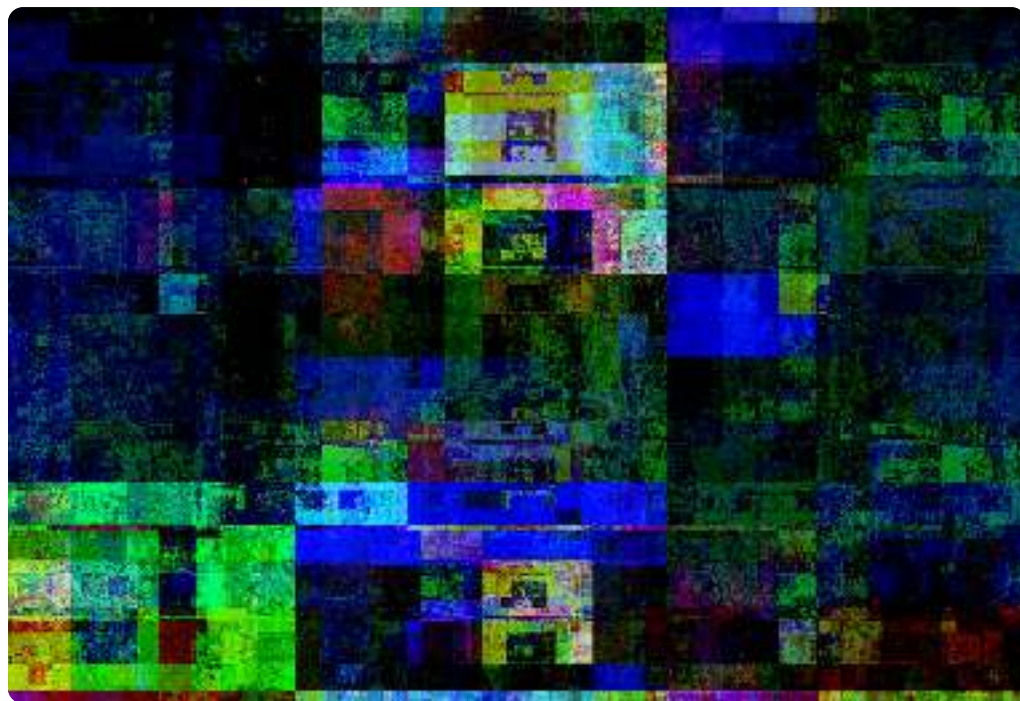
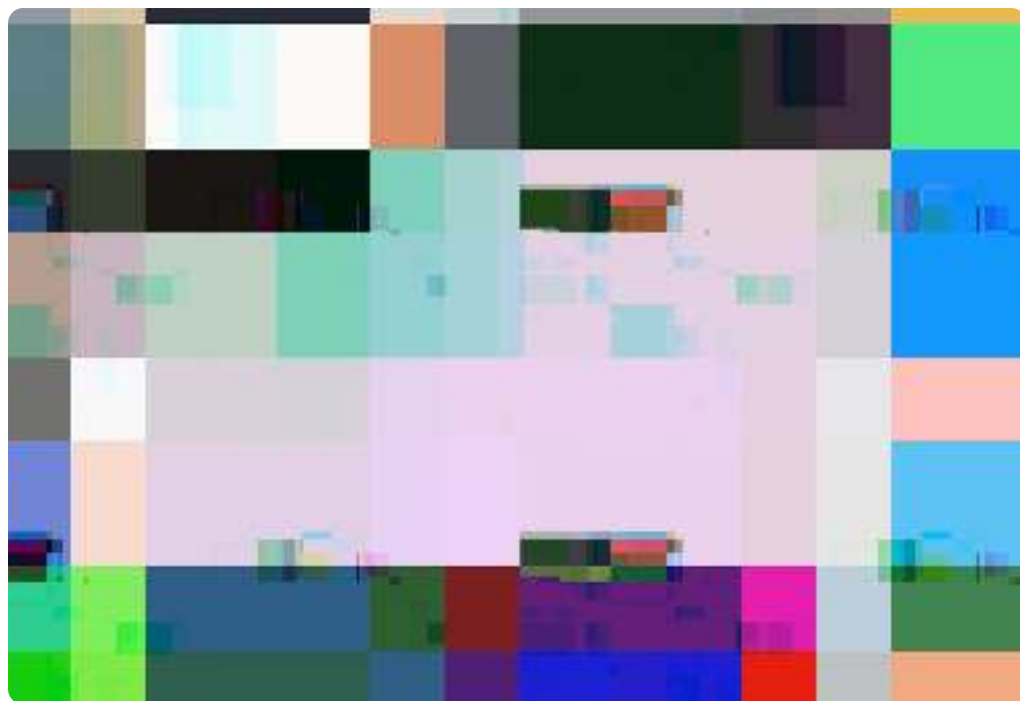
speed=0.5
osc(60,-0.04,0).thresh(0.5,0.1)
.add(
osc(60,-0.02/2,0).thresh(0.5,0.3).color(1,0,2)
)
.diff(
osc(60,-0.01/2,0).rotate(Math.PI/2).thresh(0.5,0.02)
)
.pixelate(
()=>Math.sin(time/21)*15+16,
()=>Math.cos(time/14)*15+16,
)
.saturate( ()=>Math.sin(time/3)*0.7+0.7 )
// them distorts
.modulateRotate(
osc(10,0.01).kaleid(99),()=>Math.sin(time*0.2)*-Math.PI/4
)
.modulateScale(
osc(10,0.01).kaleid(63),()=>Math.cos(time*0.4)*.3-0.4
)
.out()

```

O patch se organiza em torno de osciladores (**osc**), limiar (**thresh**), kaleidoscópio (**kaleid**), modulação de rotação (**modulateRotate**) e modulação de escala (**modulateScale**). Osciladores viram padrões binários via **thresh**, e a geometria resultante alimenta o restante da cadeia. A composição usa **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateRotate**, **modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.5** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```
speed=0.333

noise(2,0.01)
.pixelate(5,5)
.scrollX(-0.01,-0.01)
.contrast(2)

.diff(
  noise(2,0.01)
  .pixelate(10,10)
  .scrollX(0.01,0.01)
  .color(1.25,0,0)
  .contrast(2)
)

.add(
  noise(2,0.01)
  .pixelate(20,20)
  .scrollX(-0.007,-0.007)
  .color(0,0,1)
  .contrast(2)
)

.modulate(noise(0.15,-0.007))
.pixelate(50,50)
.colorama(()=>Math.sin(time/333)-1)
.scale(3)
.diff(src(o0).scale(.5))
.sub(src(o0).scale(2).scrollX(()=>random()*0.01,-0.01))
.out()
```

O patch se organiza em torno de pixelização (**pixelate**), deslocamento X (**scrollX**), ruído (**noise**), contraste (**contrast**) e escala (**scale**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo. A composição usa **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.333** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**src**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

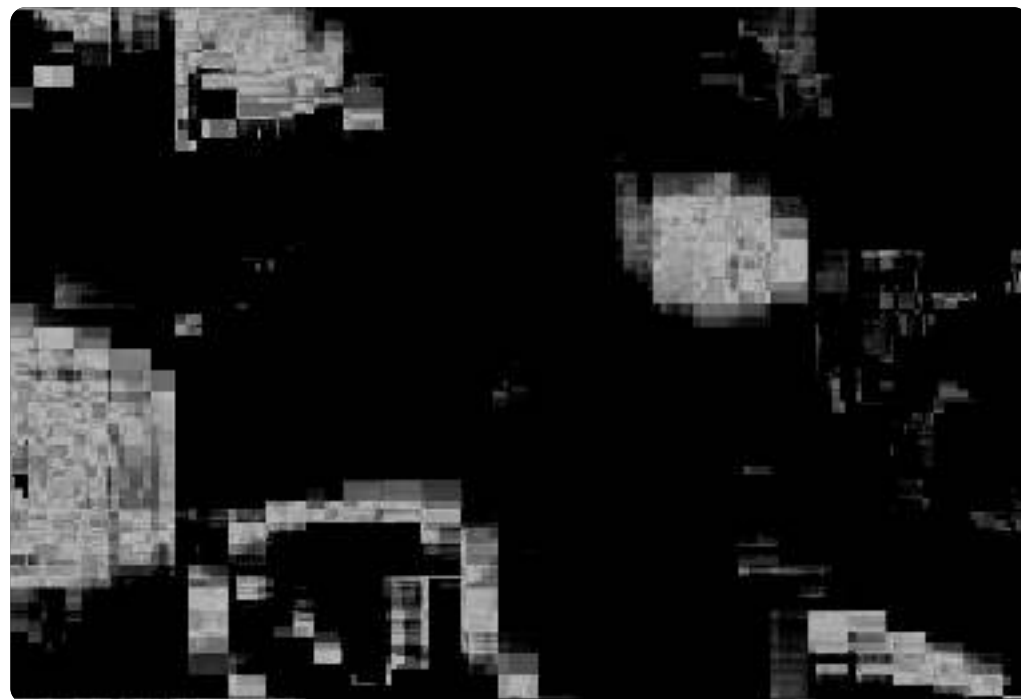
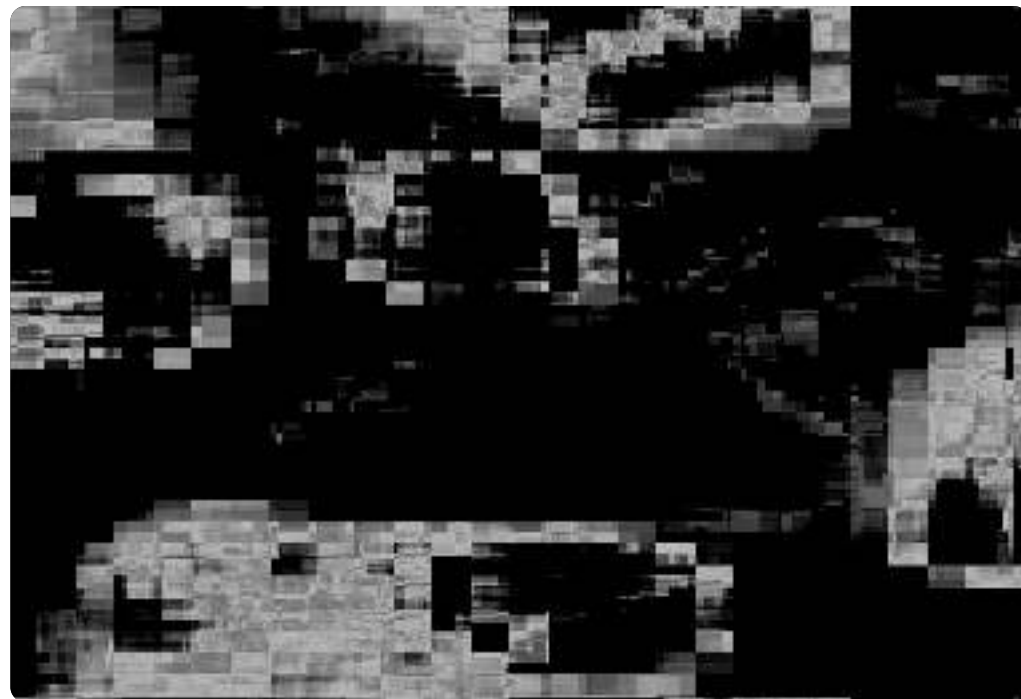
speed=0.7
noise(2,()=>Math.cos(time/44)*0.001 + 0.006)
.colorama(()=>Math.sin(time/99)*0.1)
.pixelate(50,50)
.modulateScale(
  voronoi(1.5,()=>Math.sin(time/55)*0.005+0.015,0)
  ,10.1)
.diff(
  noise(2,()=>Math.cos(time/44)*0.001 + 0.006)
.colorama(()=>Math.cos(time/34)*0.1)
  .mask( voronoi(1,()=>Math.sin(time/55)*0.005+0.015,0) )
)
.modulate( voronoi(1,()=>Math.sin(time/55)*0.005+0.015/2,
()=>Math.cos(time/77)*3+4 ) )
.out()

```

O patch se organiza em torno de Voronoi (**voronoi**), deslocamento de matiz (**colorama**), ruído (**noise**), modulação de escala (**modulateScale**) e modulação (**modulate**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo. A composição usa **diff**, **mask** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**, **modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.7** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

x1 = Math.random(); x2 = Math.random()
x3 = Math.random(); x4 = Math.random()
x5 = Math.random(); x6 = Math.random()
y1 = Math.random(); y2 = Math.random()
y3 = Math.random(); y4 = Math.random()
y5 = Math.random(); y6 = Math.random()

speed=0.6
noise(1,0.010).contrast(5).pixelate(16+(x1*5),16+(x1*5))
.scrollX(x1,0).scrollY(y1,0)
.modulate(noise(1.5,0.011).contrast(5).pixelate(16+(x1*5),16+(x1*5))
.scrollX(x2,0).scrollY(y2,0))
.diff(
  noise(1.5,0.011).contrast(5).pixelate(20,20)
  .modulate(noise(2,0.012).contrast(5).pixelate(80,80)
    .scrollX(x3,0).scrollY(y3,0))
)
.diff(
  noise(2,0.012).contrast(5).pixelate(80,80)
  .scrollX(x4,0).scrollY(y4,0)
  .modulate(noise(2.5,0.013).contrast(5).pixelate(40,40)
    .scrollX(x5,0).scrollY(y5,0))
)
.mult(noise(2.5,0.013).contrast(5).pixelate(40,40)
  .scrollX(x6,0).scrollY(y6,0))
//.color(0.7,0.5,0.5)
.invert()
.diff(src(o0))
.scale(1.25)
.scrollX(0.007,-0.007)

```

```

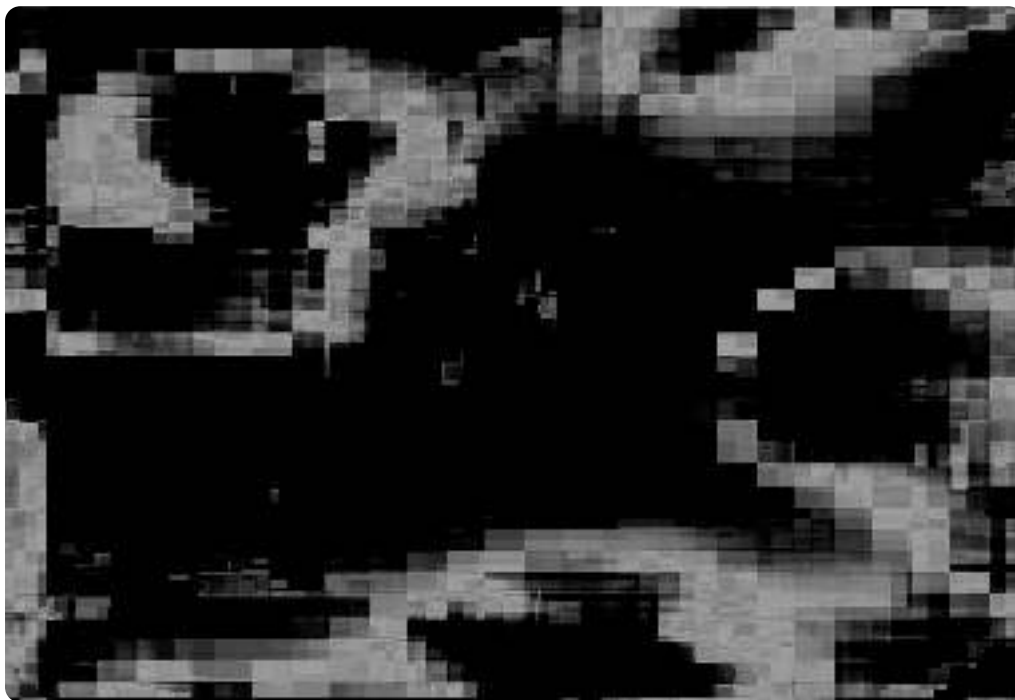
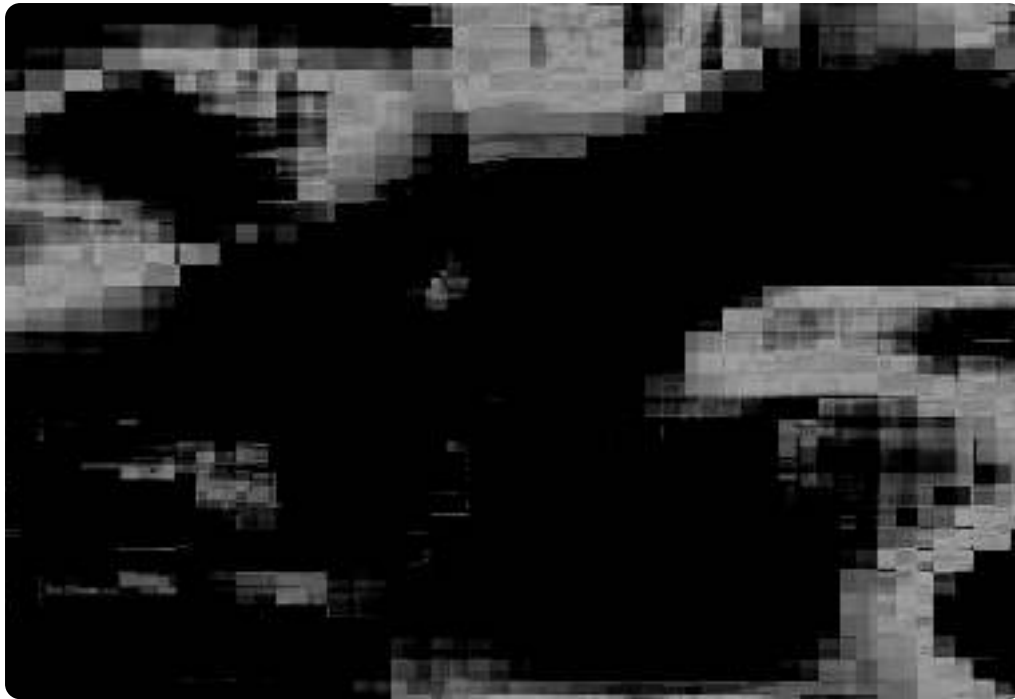
.diff(src(o0).scale(1.1),x1)
.scrollY( 0.01, ()=>Math.sin(time/999)*0.001 )
//.mult( noise(4,0.011).contrast(1).pixelate(20,20),0.5 )
.invert(x2)
.out()

```

O patch se organiza em torno de contraste (**contrast**), pixelização (**pixelate**), deslocamento Y (**scrollY**), deslocamento X (**scrollX**) e ruído (**noise**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo. A composição usa **mult**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.6** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. **Math.random** congela valores na carga do patch: cada abertura pode nascer com frequências e kaleidoscópios distintos. Há entrada externa (**src**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares.





```

x1 = Math.random(); x2 = Math.random()
x3 = Math.random(); x4 = Math.random()
x5 = Math.random(); x6 = Math.random()
y1 = Math.random(); y2 = Math.random()
y3 = Math.random(); y4 = Math.random()
y5 = Math.random(); y6 = Math.random()

speed=0.6
noise(1,0.010).contrast(5).pixelate(16+(x1*5),16+(x1*5))
.scrollX(x1,0).scrollY(y1,0)
.modulate(noise(1.5,0.011).contrast(5).pixelate(16+(x1*5),16+(x1*5))
.scrollX(x2,0).scrollY(y2,0))
.diff(
  noise(1.5,0.011).contrast(5).pixelate(20,20)
  .modulate(noise(2,0.012).contrast(5).pixelate(80,80)
    .scrollX(x3,0).scrollY(y3,0))
)
.diff(
  noise(2,0.012).contrast(5).pixelate(80,80)
  .scrollX(x4,0).scrollY(y4,0)
  .modulate(noise(2.5,0.013).contrast(5).pixelate(40,40)
    .scrollX(x5,0).scrollY(y5,0))
)
.mult(noise(2.5,0.013).contrast(5).pixelate(40,40)
  .scrollX(x6,0).scrollY(y6,0))
//.color(0.7,0.5,0.5)
.invert()
.diff(src(o0))
.scale(1.25)
.scrollX(0.007,-0.007)

```

```

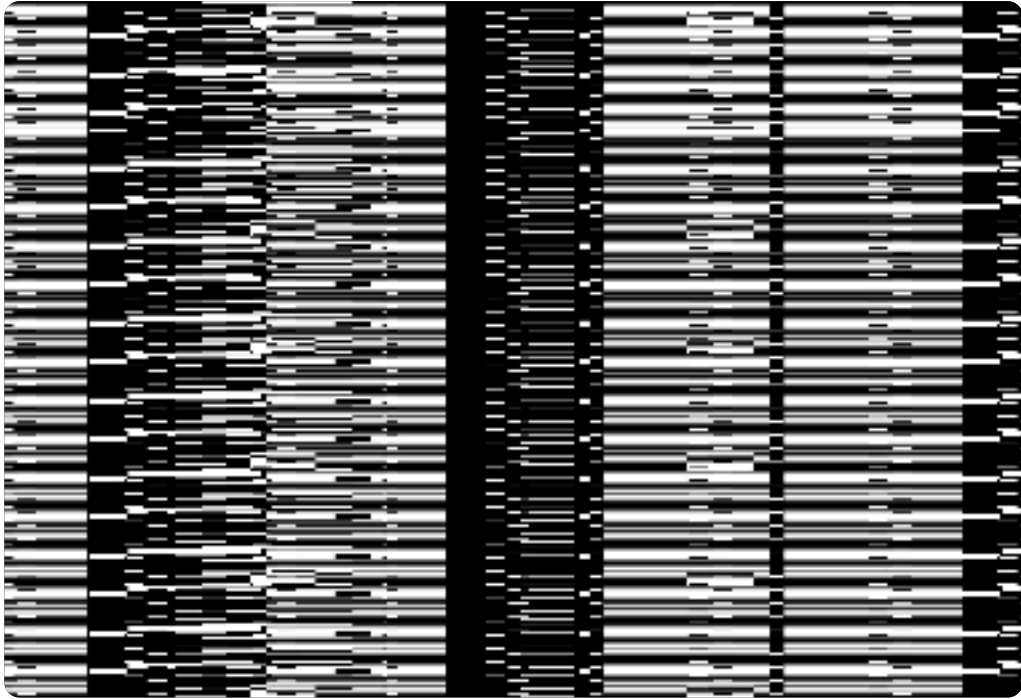
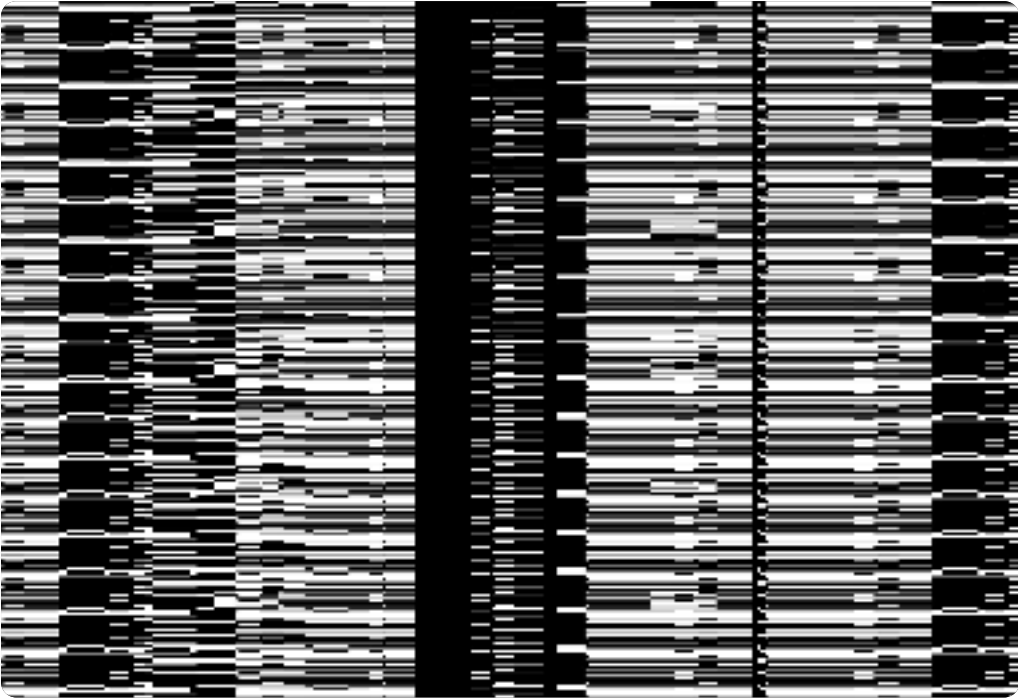
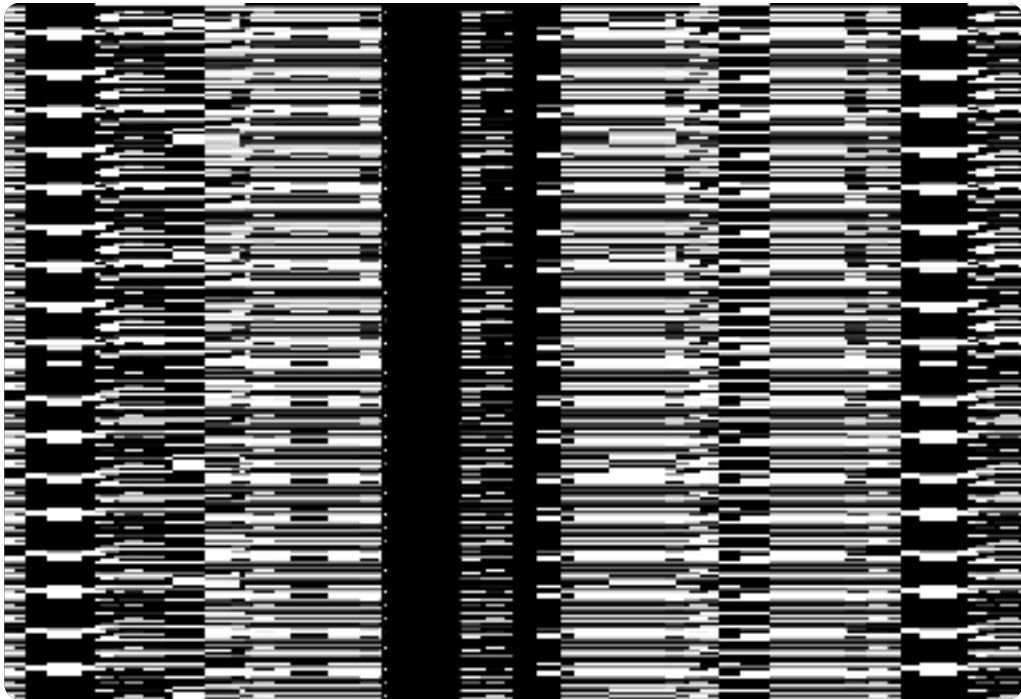
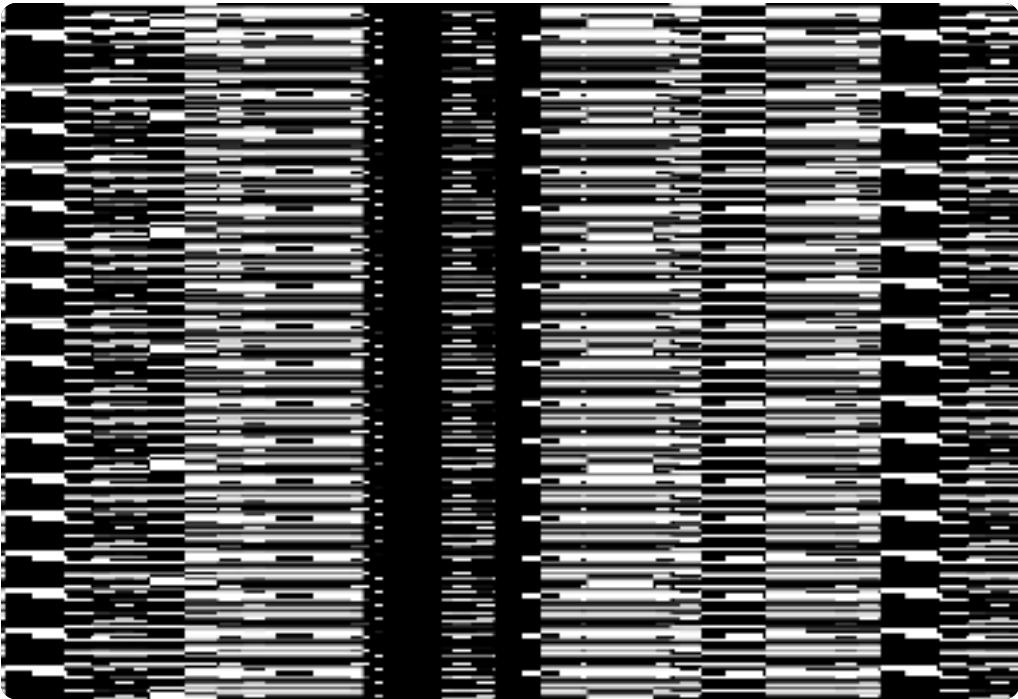
.diff(src(o0).scale(1.1),x1)
.scrollY( 0.01, ()=>Math.sin(time/999)*0.001 )
//.mult( noise(4,0.011).contrast(1).pixelate(20,20),0.5 )
.invert(x2)
.out()

```

O patch se organiza em torno de contraste (**contrast**), pixelização (**pixelate**), deslocamento Y (**scrollY**), deslocamento X (**scrollX**) e ruído (**noise**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo. A composição usa **mult**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.6** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. **Math.random** congela valores na carga do patch: cada abertura pode nascer com frequências e kaleidoscópios distintos. Há entrada externa (**src**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares.





```

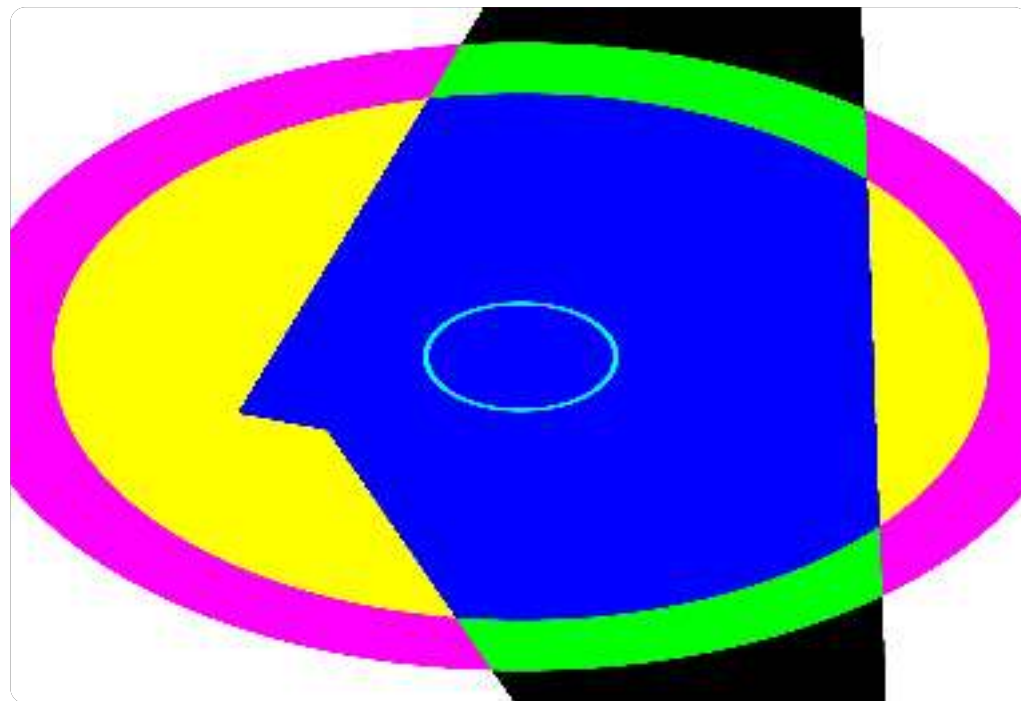
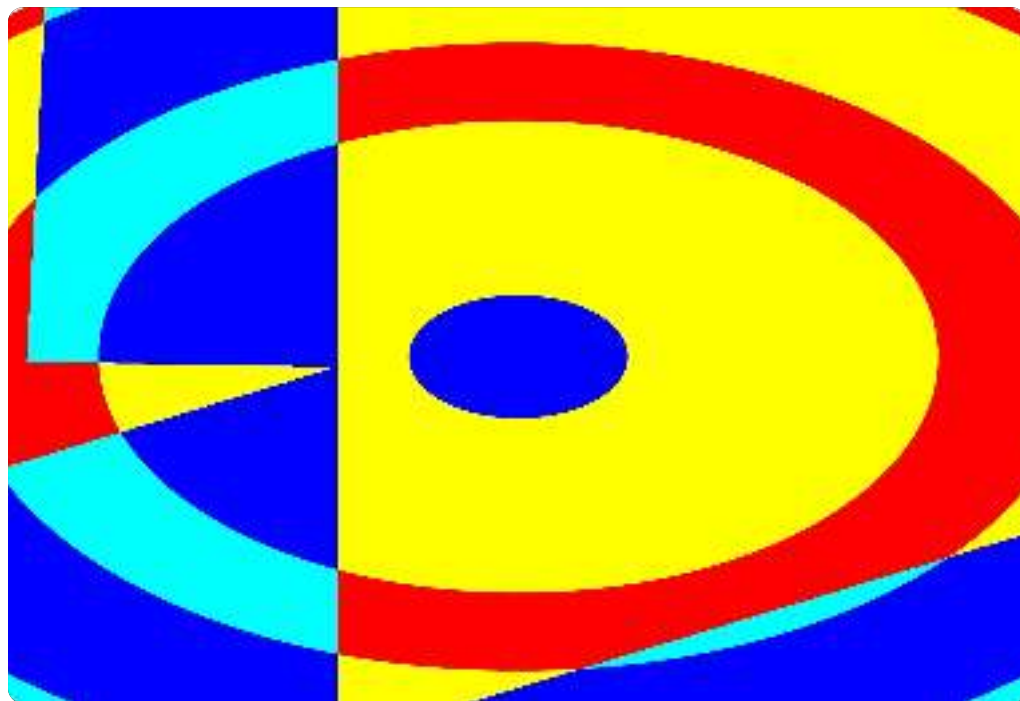
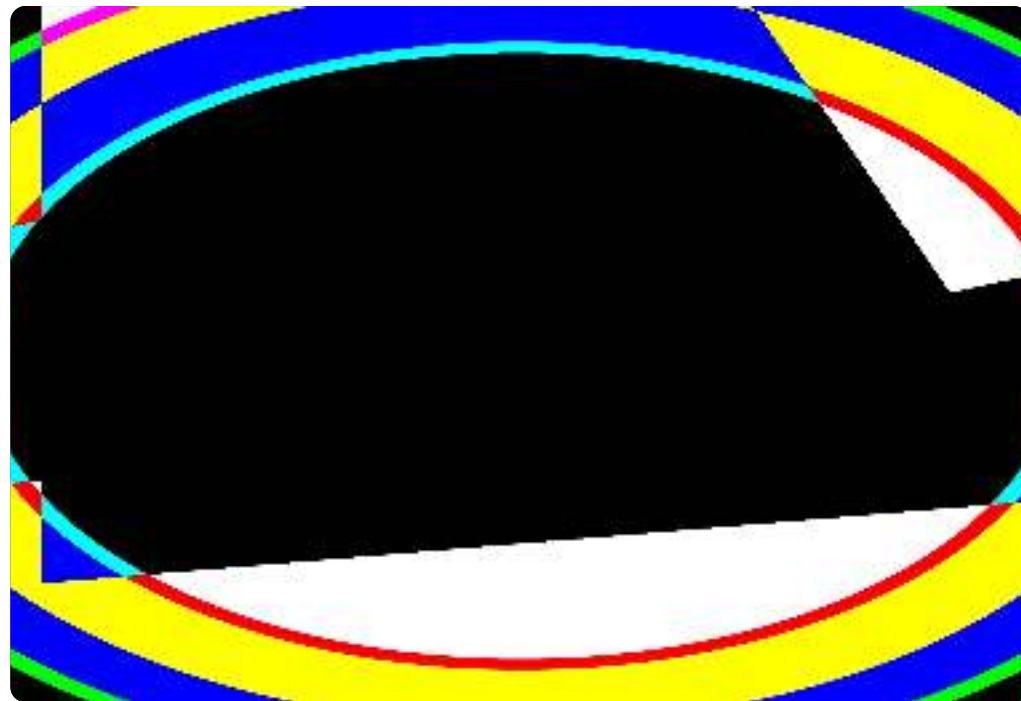
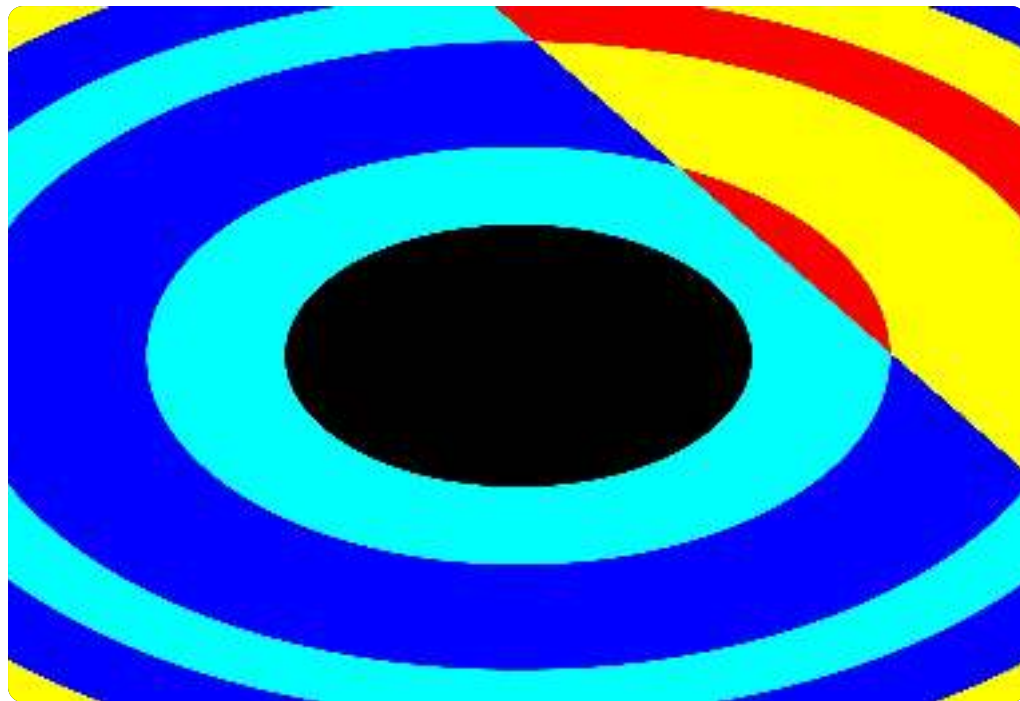
speed=0.5
shape(4,0.15,0).scrollX(0.01,-0.01).scrollX(Math.random()).scrollY(Math.random()).repeat(2,2)
.diff(
shape(4,0.15,0).scrollX(0.01,-0.01).scrollX(Math.random()).scrollY(Math.random()).repeat(2,2) )
.repeat(Math.random()*2)
.diff(
shape(4,0.15,0).scrollX(0.01,-0.01).scrollX(Math.random()).scrollY(Math.random()).repeat(2,2) )
.repeat(Math.random()*2)
.diff(
shape(4,0.5,0).scrollX(0.01,-0.01).scrollX(Math.random()).scrollY(Math.random()).repeat(2,2) )
.repeat(Math.random()*2)
.modulateScrollY(osc(2,0.15).thresh(0.5,0).rotate(Math.PI/2),0.1)
.diff(
shape(4,0.15,0).scrollX(0.01,-0.01).scrollX(Math.random()).scrollY(Math.random()).repeat(2,2) )
.repeat(Math.random()*2)
.diff(
shape(4,0.25,0).scrollX(0.01,-0.01).scrollX(Math.random()).scrollY(Math.random()).repeat(2,2) )
.repeat(Math.random()*2)
.diff(
shape(4,0.15,0).scrollX(0.01,-0.01).scrollX(Math.random()).scrollY(Math.random()).repeat(2,2) )
.repeat(Math.random()*2)
.diff(
shape(4,0.15,0).scrollX(0.01,-0.01).scrollX(Math.random()).scrollY(Math.random()).repeat(2,2) )
.repeat(Math.random()*2)
.modulateScrollY(osc(3,-0.1).thresh(0.5,0).rotate(Math.PI/2),0.2)
.out()

```

O patch se organiza em torno de deslocamento X (**scrollX**), repetição (**repeat**), deslocamento Y (**scrollY**), formas geométricas (**shape**) e diferença (**diff**). Osciladores viram padrões binários via **thresh**, e a geometria resultante alimenta o restante da cadeia. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScrollY**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.5** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. **Math.random** congela valores na carga do patch: cada abertura pode nascer com frequências e kaleidoscópios distintos.





```
// quad

shape(100,.9,.002)

.diff( shape(100,()=>Math.sin(time/9)+0.9,.002) ).color(1,1,0)
.diff( shape(100,()=>Math.cos(time/11)+0.7,.002) )

.diff( shape(100,()=>Math.sin(time/13)+0.5,.002) ).color(0,1,0)
.diff( shape(100,()=>Math.cos(time/15)+0.4,.002) )

.diff( shape(100,()=>Math.sin(time/17)+0.3,.002) ).color(0,1,1)
.diff( shape(100,()=>Math.cos(time/19)+0.2,.002))

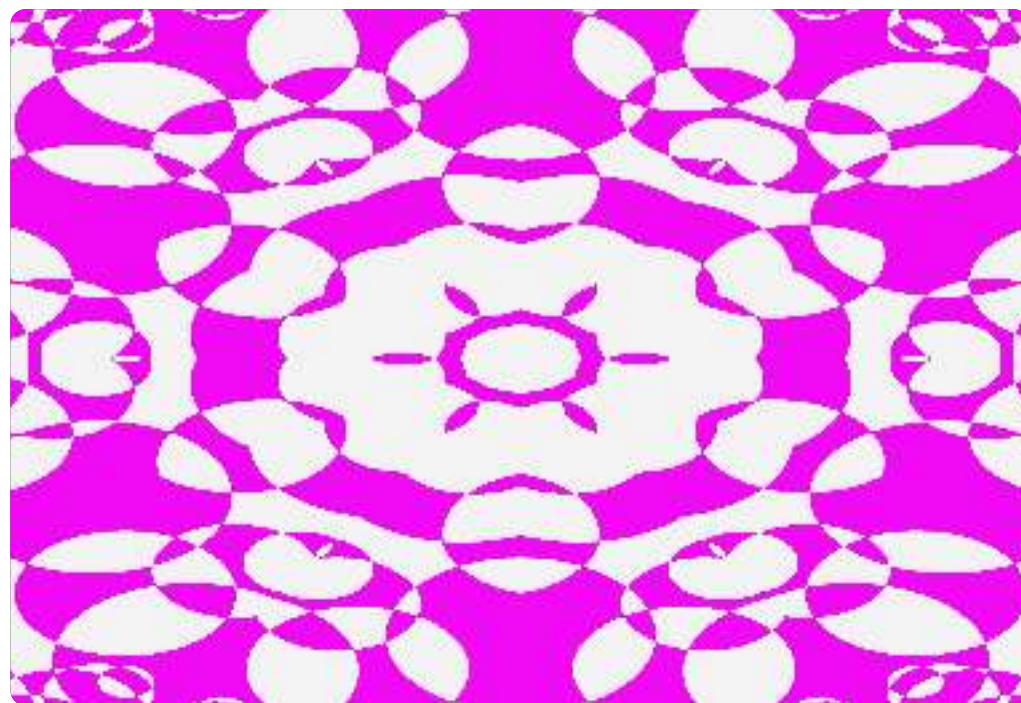
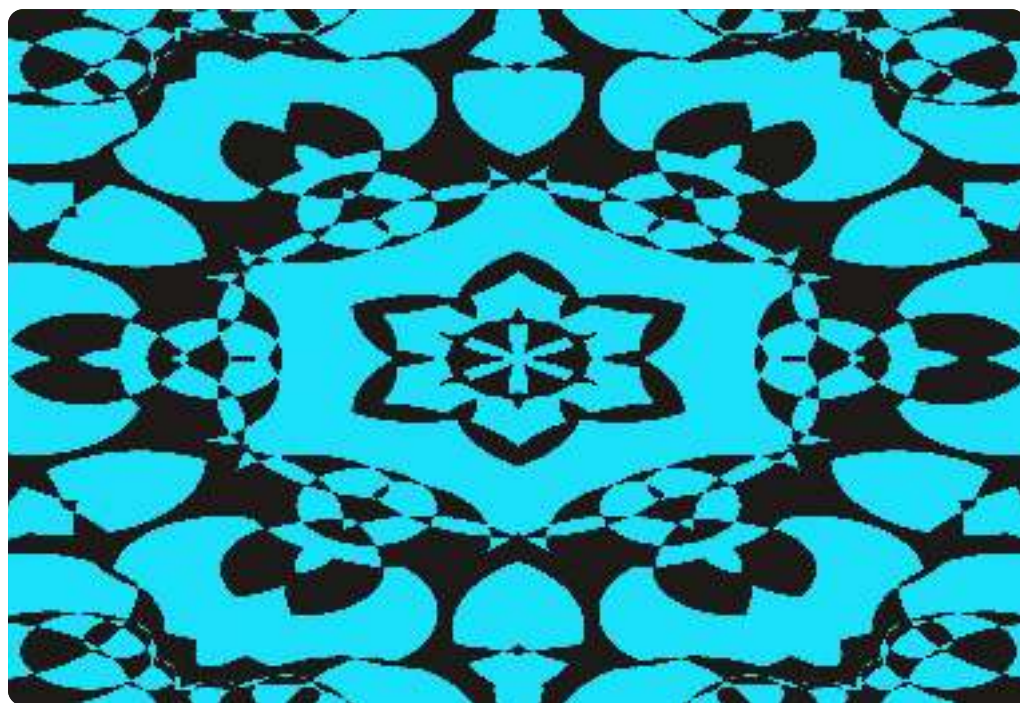
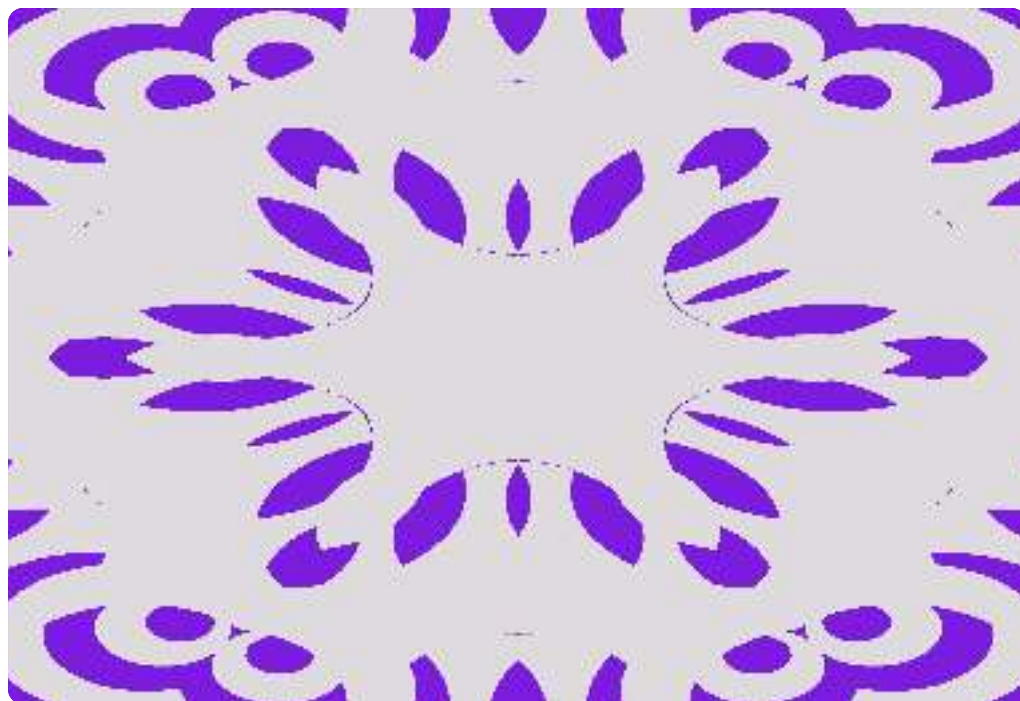
.diff(
shape([1,1.1,1.2,1.3,1.4,1.5,1.6,1.7,1.8,1.9].fast(0.8),0.5,0)

.rotate([0,0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,0.9,1].fast(0.4))
  .rotate(1)
  .scrollX(0.010,-0.012) )
.out()
```

O patch se organiza em torno de formas geométricas (**shape**), diferença (**diff**), colorização (**color**), rotação (**rotate**) e arrays acelerados (**.fast**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```
// as a mandala

// licensed with CC BY-NC-SA 4.0
https://creativecommons.org/licenses/by-nc-sa/4.0/
// "egg of the phoenix"
// Alexandre Rangel
// www.alexandrangel.art.br/hydra.html

speed=0.3

shape(99,.15*1.79,.5).color(0,1,2)

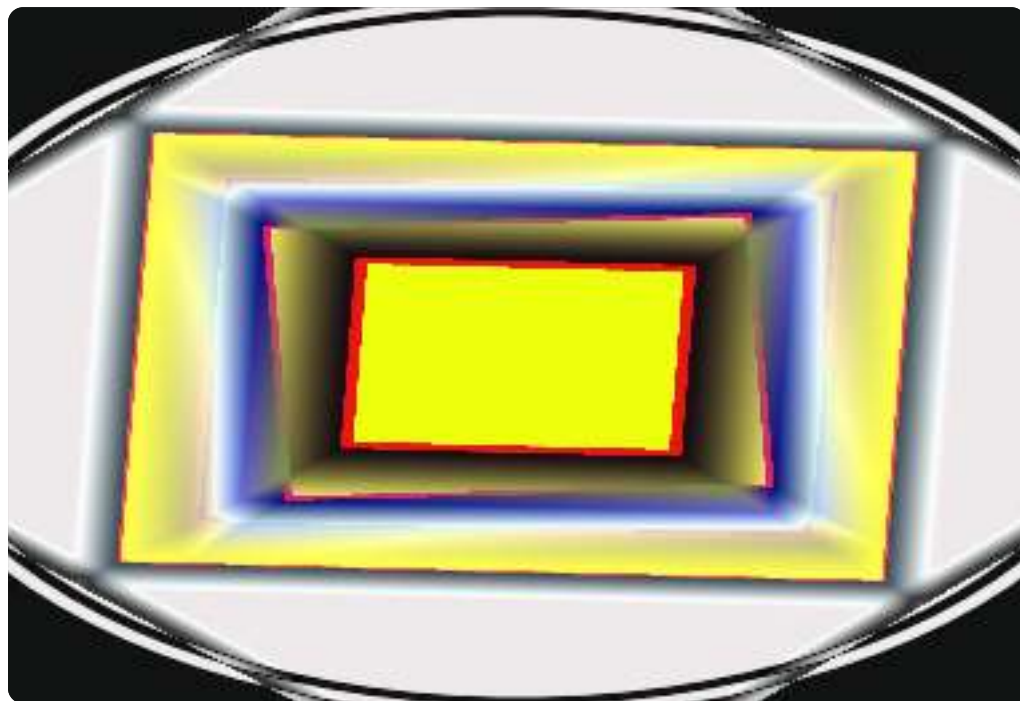
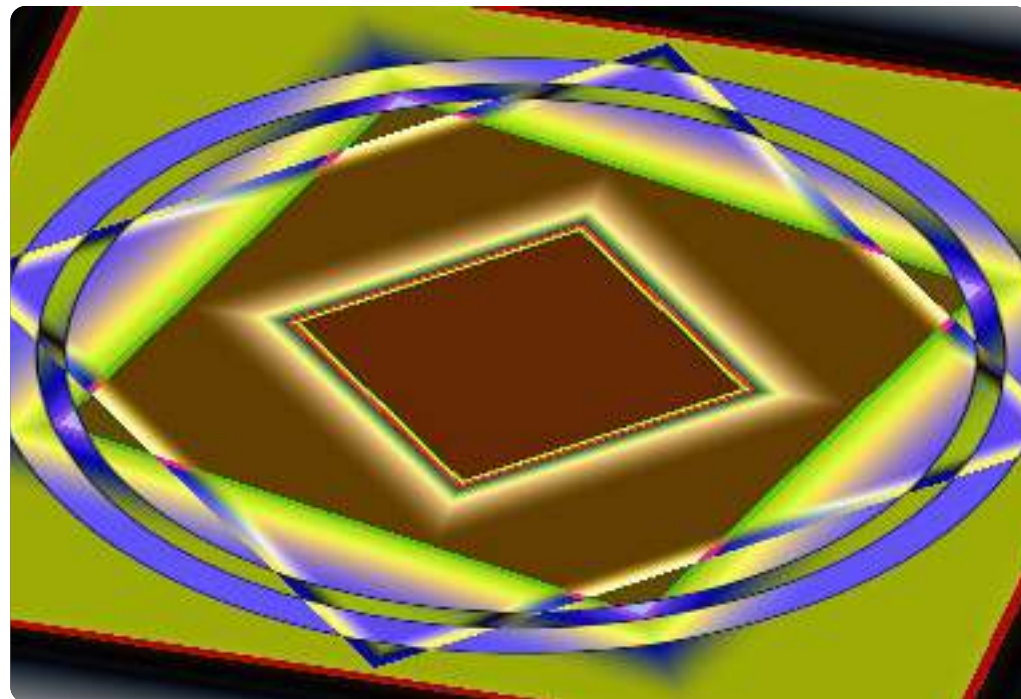
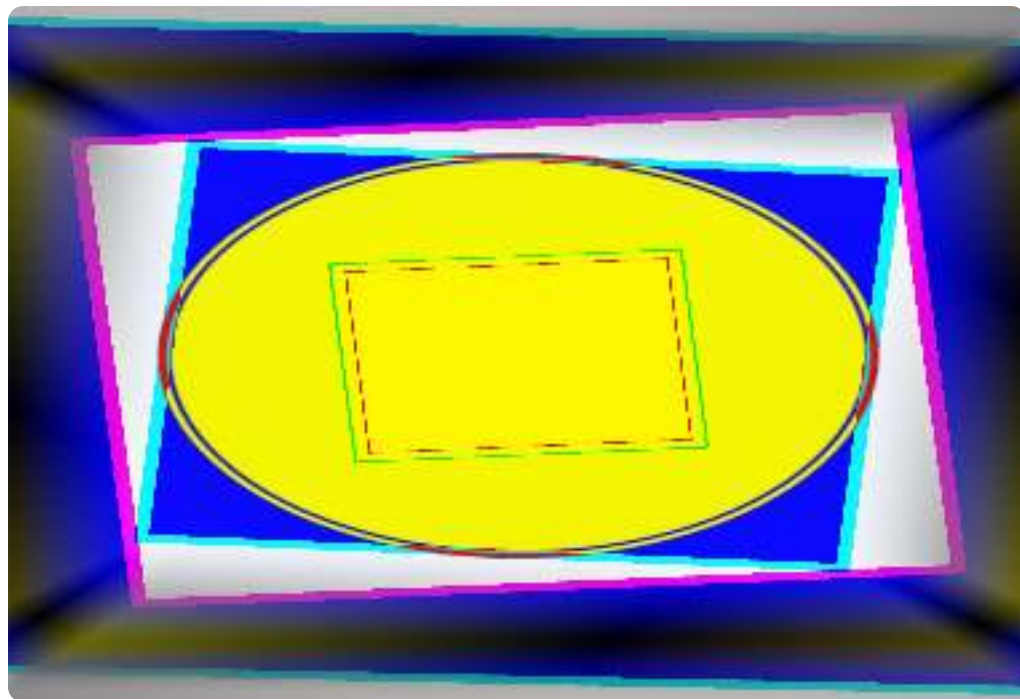
.diff( shape(240,.5*1.79,0).scrollX(.05).rotate( ()=>time/10
).color(1,0,.75) )
.diff( shape(99,.4*1.79,0).scrollX(.10).rotate( ()=>time/20
).color(1,0,.75) )
.diff( shape(99,.3*1.79,0).scrollX(.15).rotate( ()=>time/30
).color(1,0,.75) )
.diff( shape(99,.2*1.79,0).scrollX(.20).rotate( ()=>time/40
).color(1,0,.75) )
.diff( shape(99,.1*1.79,0).scrollX(.25).rotate( ()=>time/50
).color(1,0,.75) )

.modulateScale(
  shape(240,.5*1.79,0).scrollX(.05).rotate( ()=>time/10 )
  , ()=>(Math.sin(time/3)*.2)+.2 )
//.scale(1.1)
//.scale(1.6,.6,1)
.scale(0.25)
.colorama( ()=>(Math.sin(time*0.2)*.27)-0.15 )
.kaleid(6)
.out()
```

O patch se organiza em torno de formas geométricas (**shape**), deslocamento X (**scrollX**), rotação (**rotate**), colorização (**color**) e diferença (**diff**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.3** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

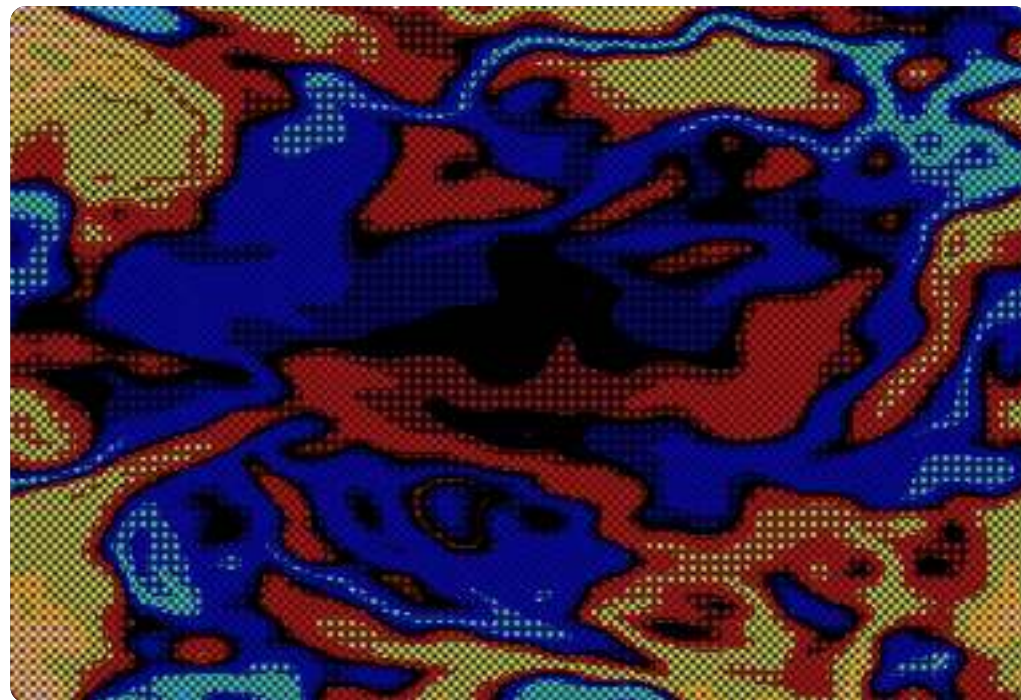
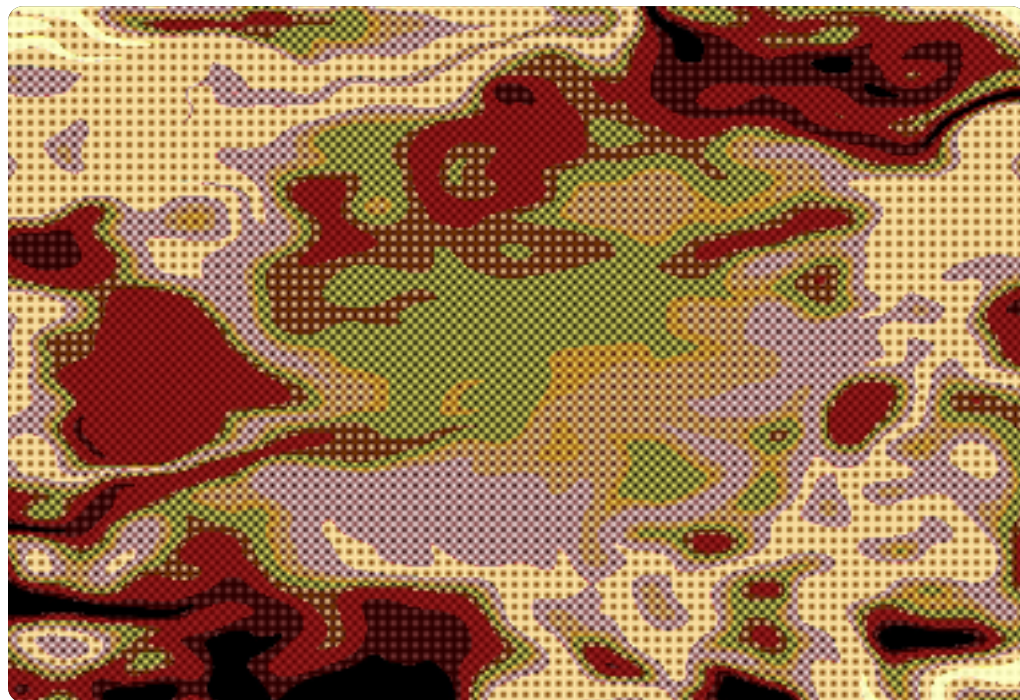
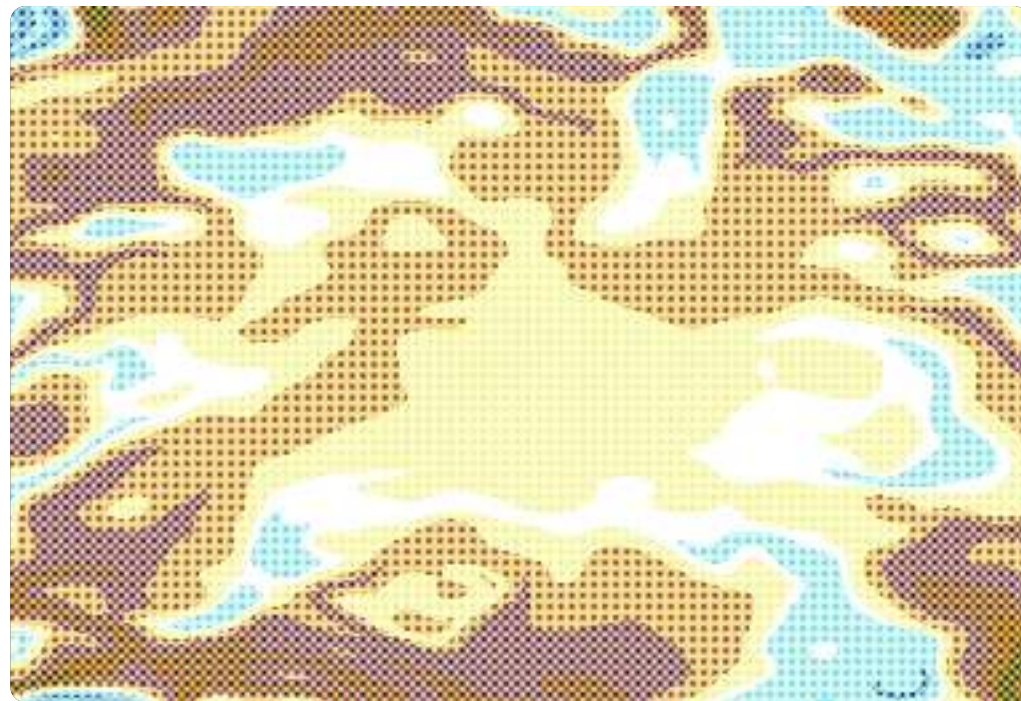
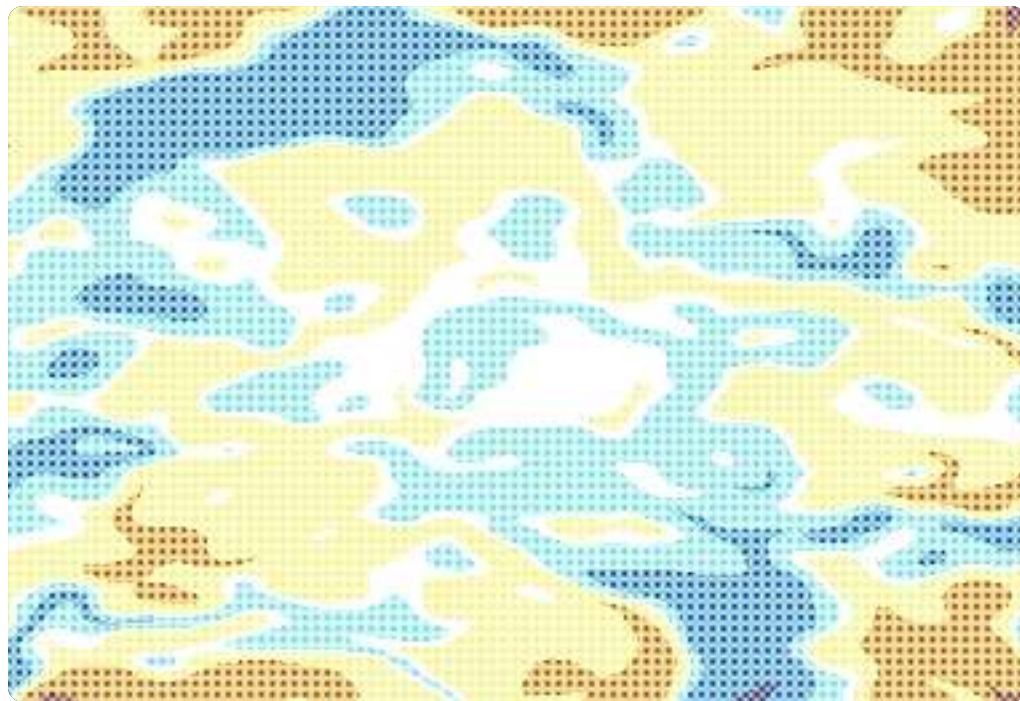
shape(4,0.9,()=>Math.cos(time/2)*0.2+0.25).rotate( ()=> time/40 )
.colorama( ()=>Math.sin(time/5)*0.1 )
.diff( shape(4,0.6,()=>Math.cos(time/3)*0.2+0.25)
    .rotate( ()=> time/-20 )
    .colorama( ()=>Math.cos(time/2.5)*0.1 ))
.diff( shape(4,0.25,()=>Math.cos(time/4)*0.2+0.25)
    .rotate( ()=> time/-20 )
    .colorama( ()=>Math.sin(time/7.5)*0.1 ))
.diff( shape(4,0.25,()=>Math.cos(time/5)*0.2+0.25)
    .rotate( ()=> time/-20 )
    .colorama( ()=>Math.sin(time/10)*0.1 ))
.diff( shape(4,0.5,()=>Math.cos(time/6)*0.2+0.25)
    .scale( ()=>Math.cos(time/22.5)*0.5+0.6 )
    .rotate( ()=> time/20 )
    .colorama( ()=>Math.sin(time/15)*0.1 ))
.diff( shape(90,()=>Math.sin(time/15)*0.5+0.525),
    ()=>Math.cos(time/7)*0.3+0.4)
.diff( shape(90,()=>Math.sin(time/20)*0.5+0.525),
    ()=>Math.cos(time/8)*0.3+0.4)
.diff( shape(90,()=>Math.sin(time/25)*0.5+0.525),
    ()=>Math.cos(time/9)*0.3+0.4)
.out()

```

O patch se organiza em torno de formas geométricas (**shape**), diferença (**diff**), deslocamento de matiz (**colorama**), rotação (**rotate**) e escala (**scale**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





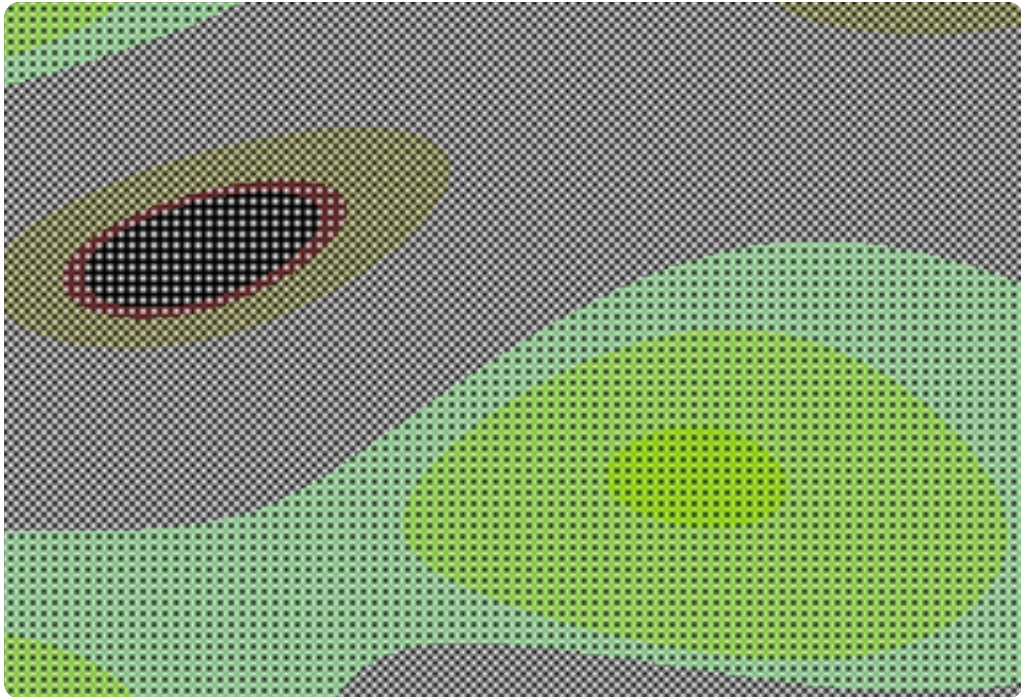
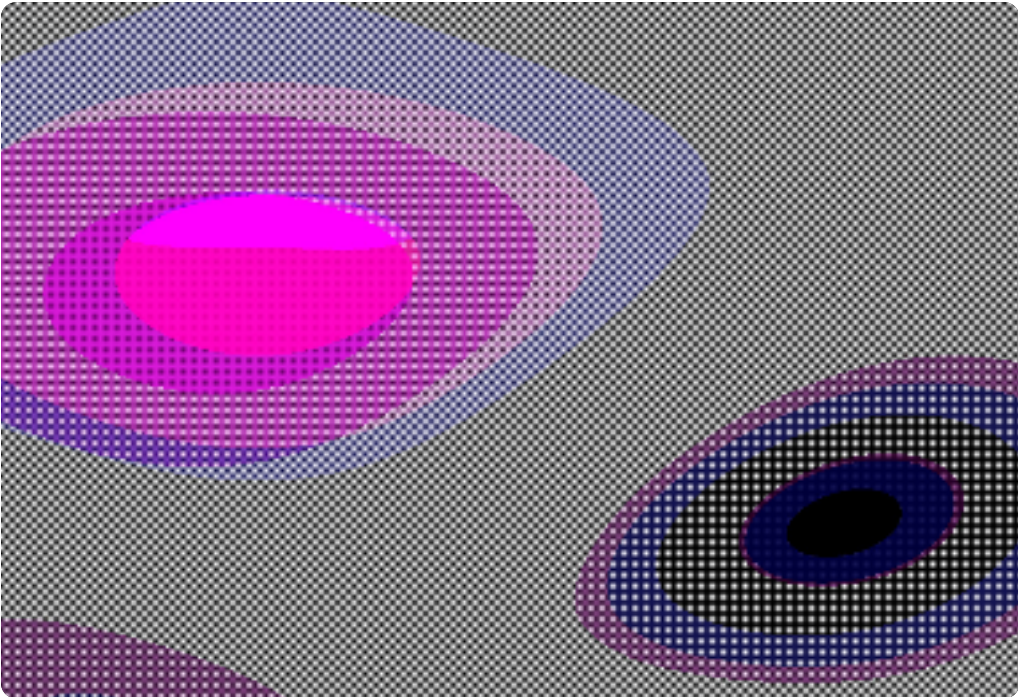
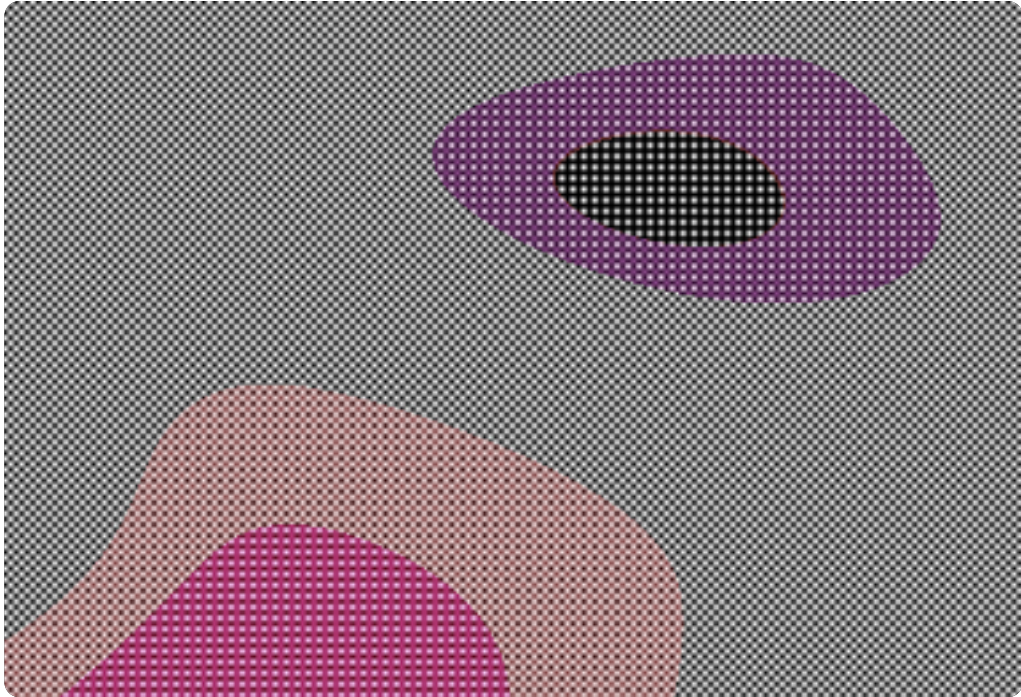
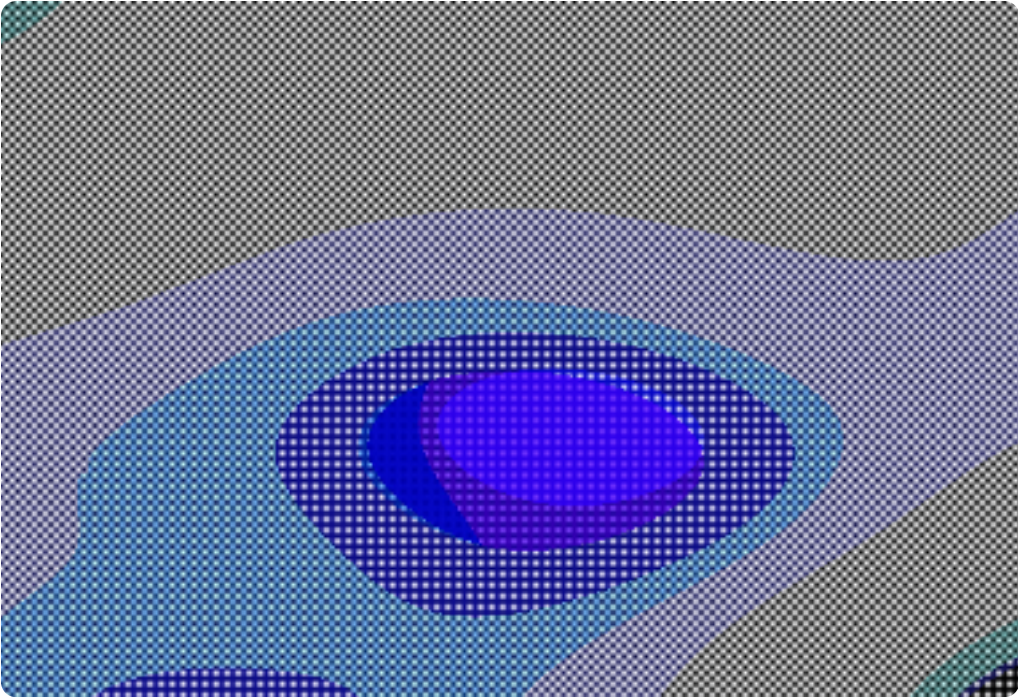
```
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-shaders-for-hydra/main/lib/lib-noise.js")
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-shaders-for-hydra/main/lib/lib-screen.js")

speed = 0.2
osc(0.75 * Math.PI, 0.15, 0.6)
  .modulateRotate(warp(0.7, 0.1, 1, 4)
    .diff(gradient(), -1), Math.PI*1.5)
  .dither2()
  .out()
```

O patch se organiza em torno de modulação de rotação (**modulateRotate**), gradientes (**gradient**), diferença (**diff**) e osciladores (**osc**). A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateRotate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.2** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares.





```
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-shaders-for-hydra/main/lib/lib-screen.js")

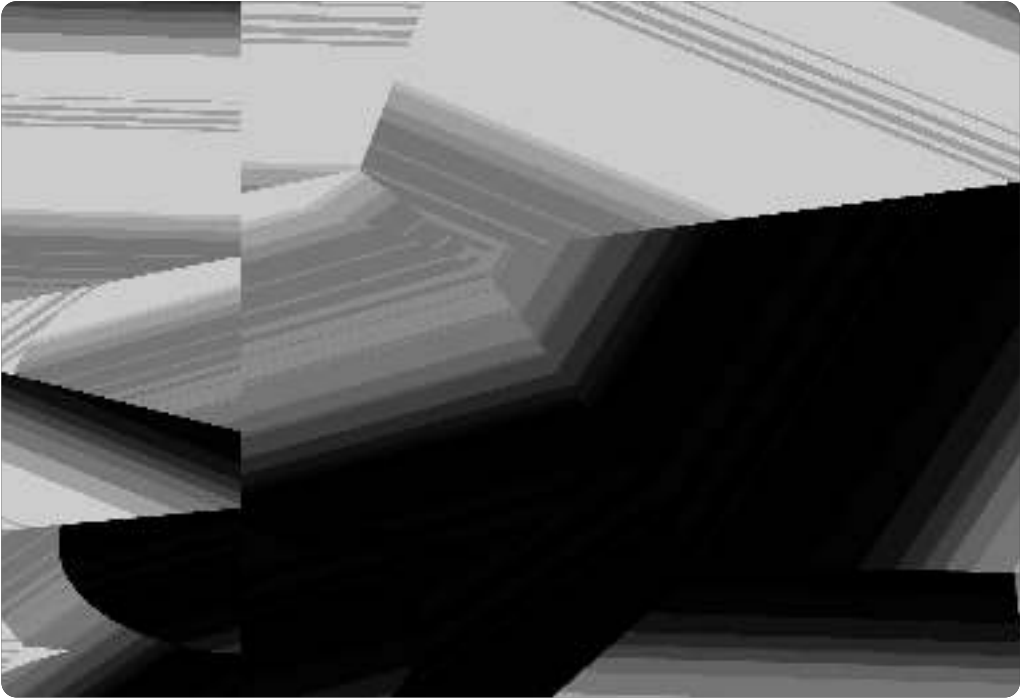
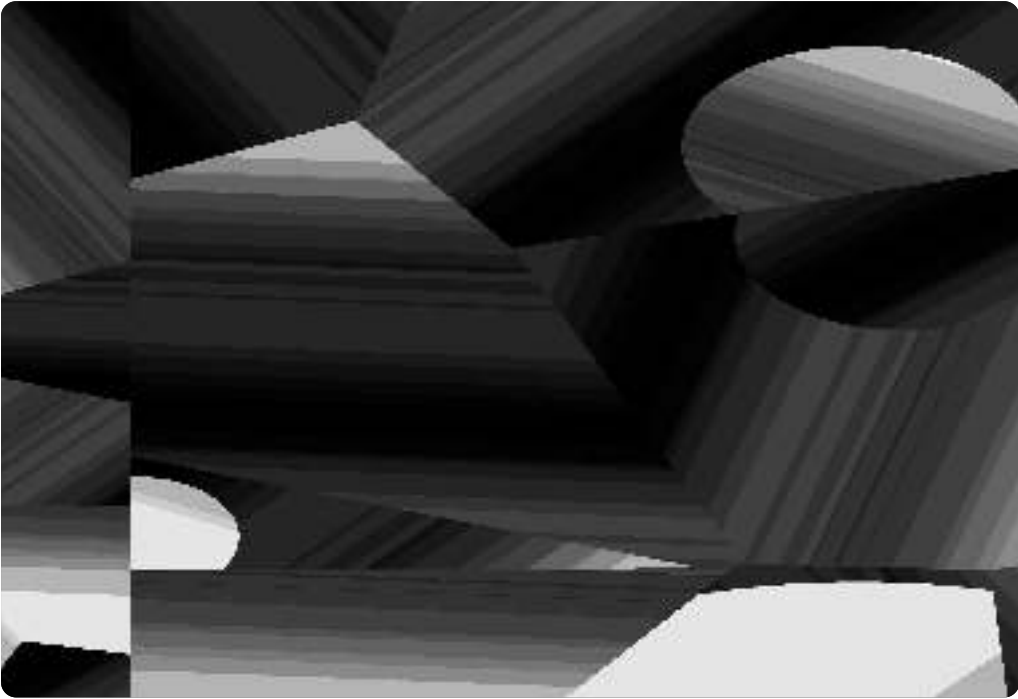
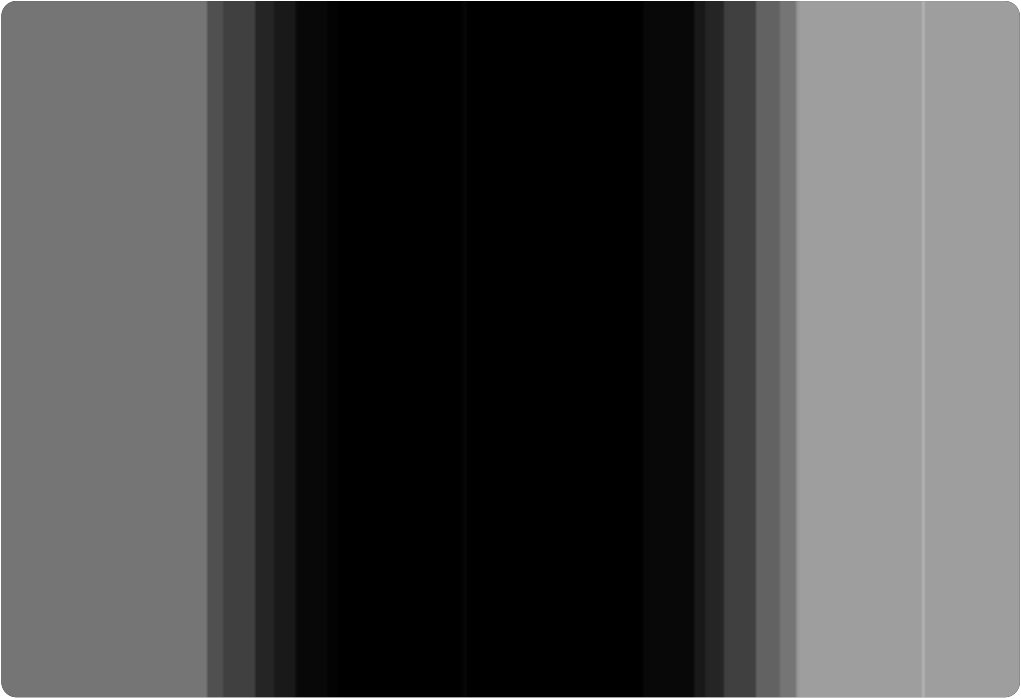
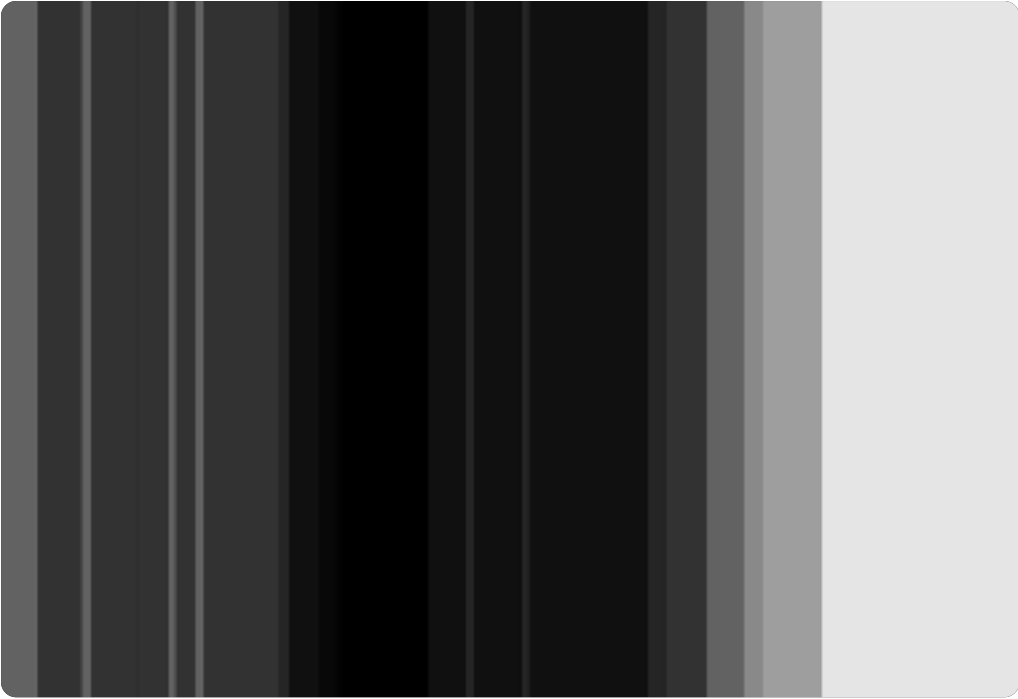
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-shaders-for-hydra/main/lib/lib-softpattern.js")

phasenoise( ()=>Math.sin(time/12)*0.5+0.5, 0.1, 1, 0.05, 0.025)
.dither2()
.out()
```

O patch se organiza em torno de técnicas Hydra em cadeia.

expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares.





```

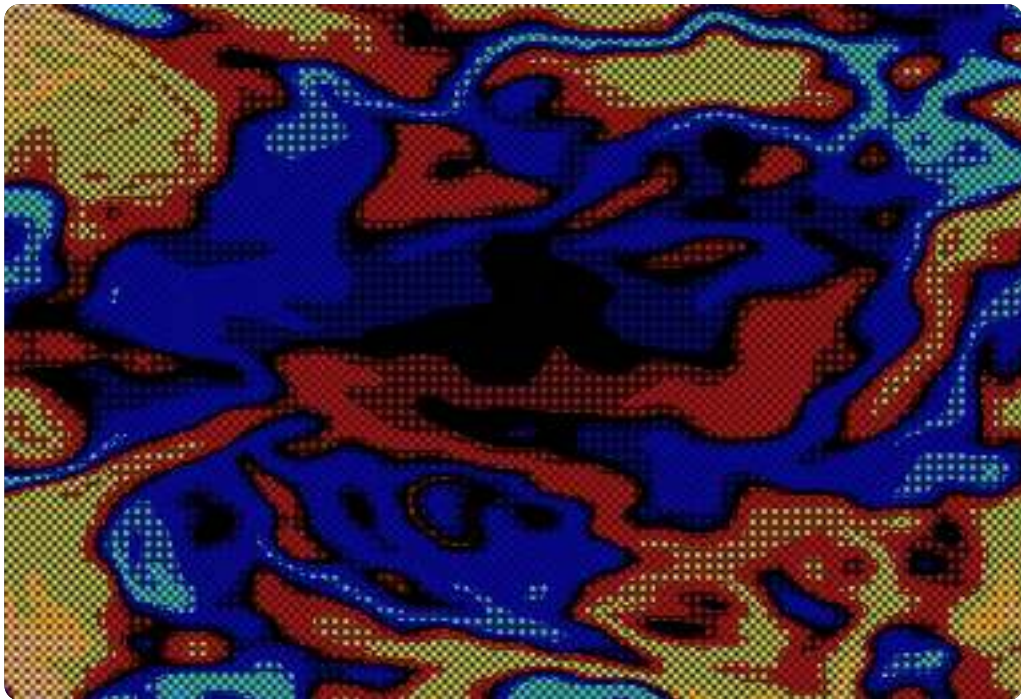
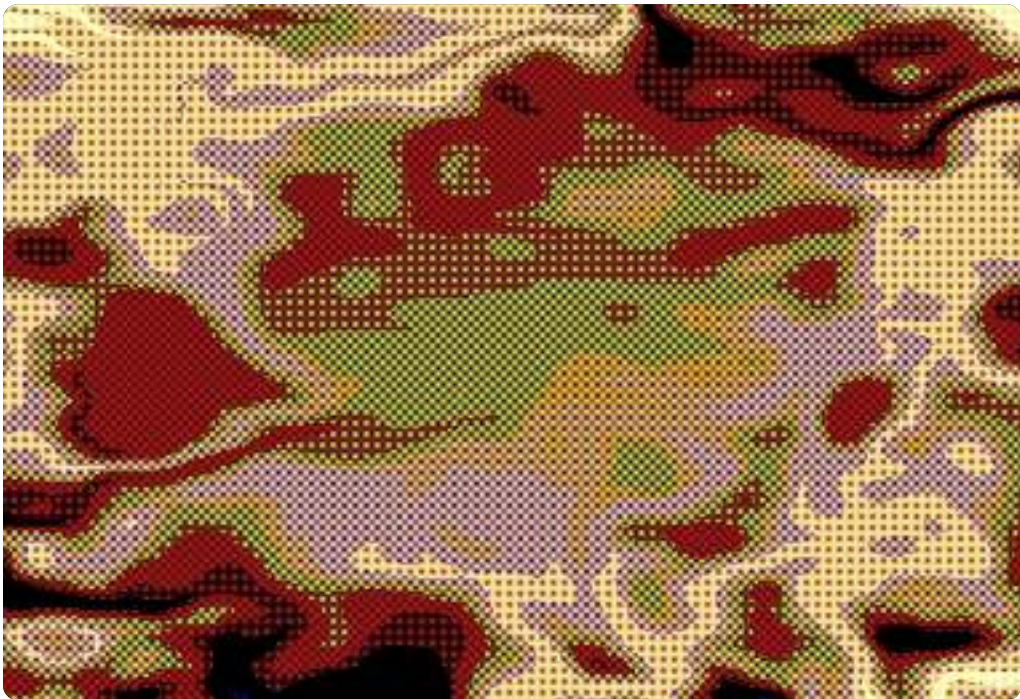
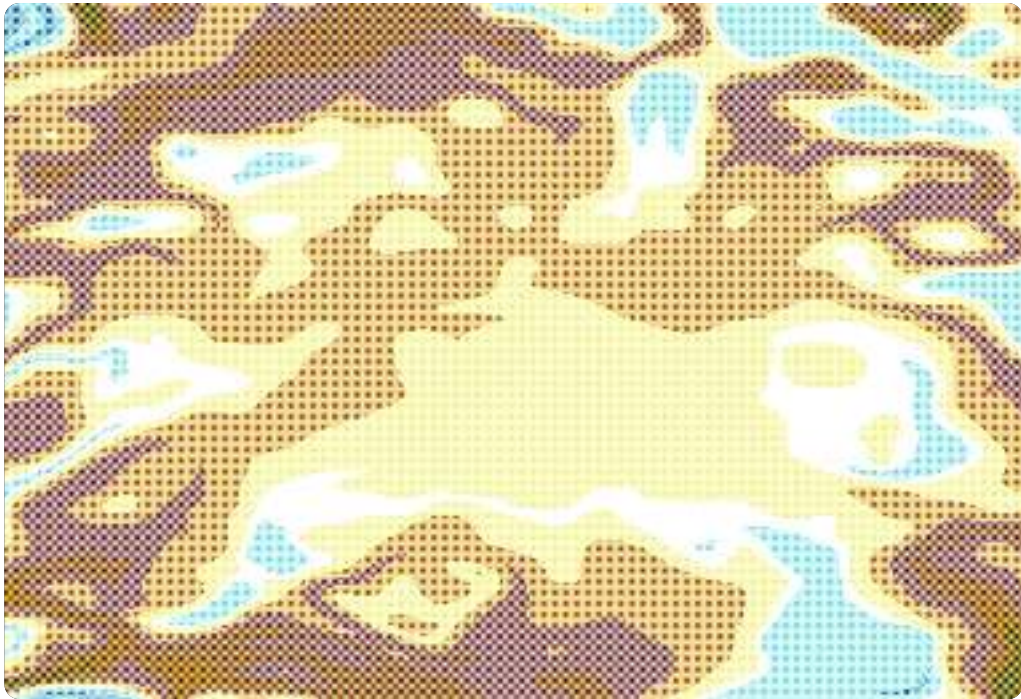
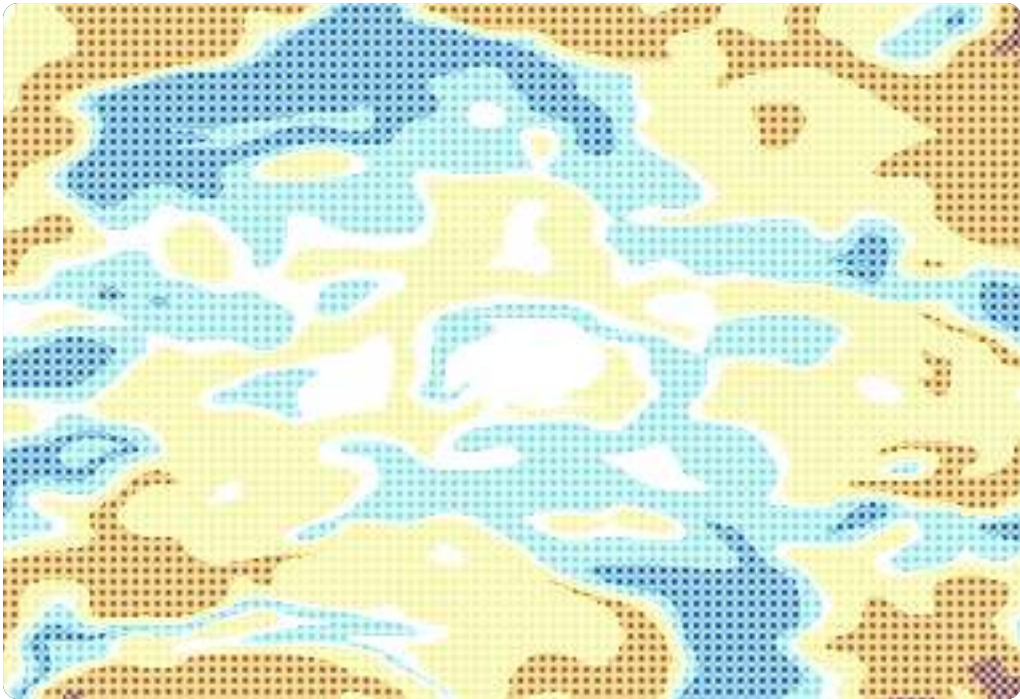
speed=0.2
osc(10,-0.02,0.1) .posterize( ()=>Math.sin(time/7)*4+5 )
.modulate(
  osc(30,0.01,0)
  .posterize( ()=>Math.sin(time/14)*4+5 )
)
.modulateRotate(
  voronoi(3,0.05)
  .posterize( ()=>Math.sin(time/14)*4+5 )
  , [0,Math.PI*2,Math.PI,Math.PI*-2] .fast(0.5)
)
.modulateScale(
  osc(3,-0.03,0)
  .posterize( 3 )
  , [1,1.5] .fast(1.2)
)
.saturate( 0 ) .posterize(16)
//.colorama( [-1,1,-0.2,0.03,-0.5,-0.7,-10,0.2,0.5,1] .fast(0.5) )
.out()

```

O patch se organiza em torno de posterização (**posterize**), osciladores (**osc**), arrays acelerados (**.fast**), modulação de rotação (**modulateRotate**) e modulação de escala (**modulateScale**).

A modulação (**modulateRotate**, **modulateScale**, **modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.2** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





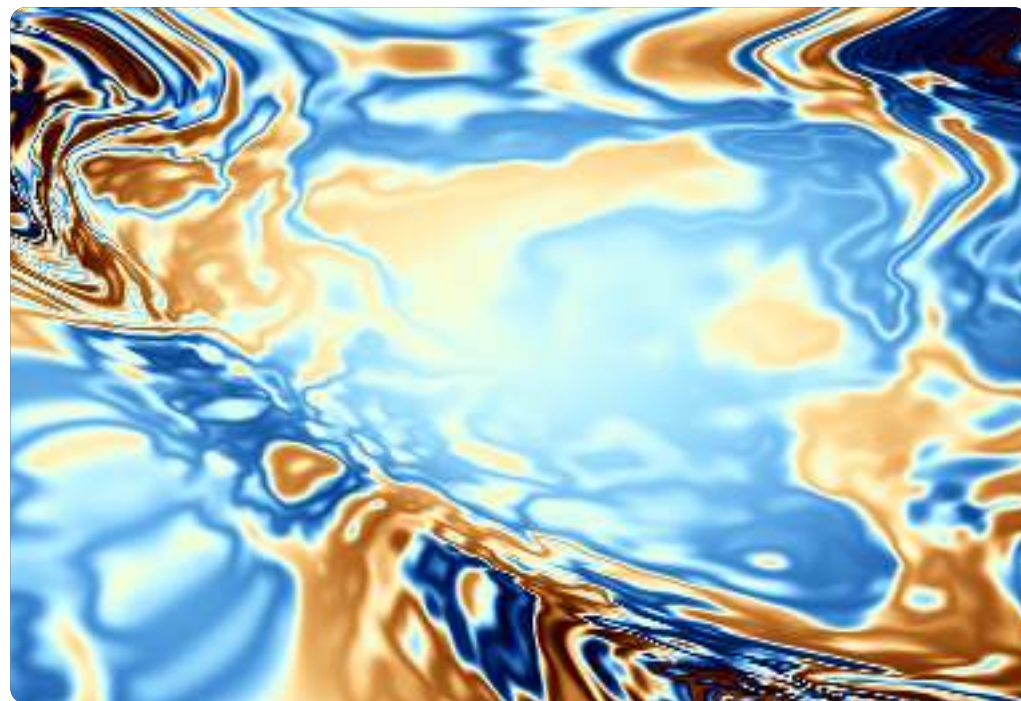
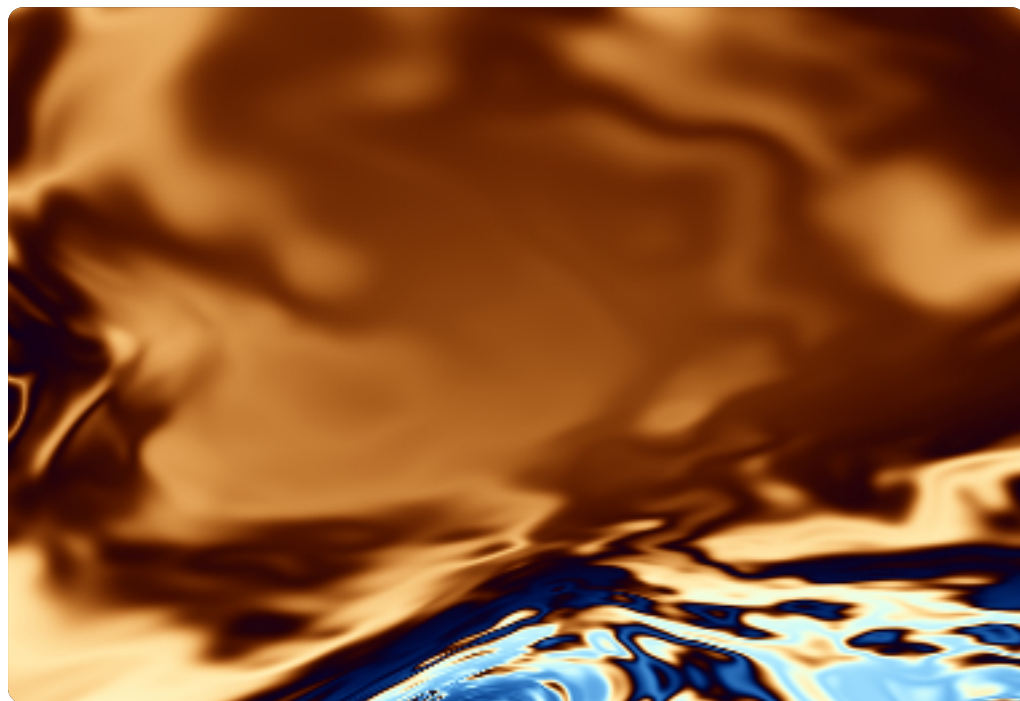
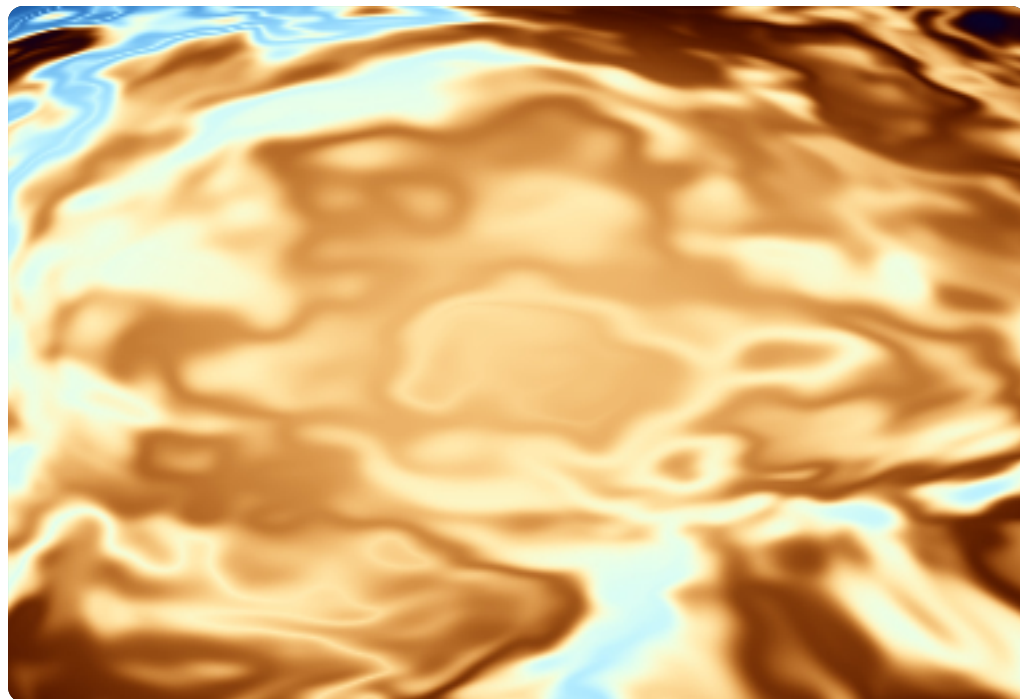
```
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-shaders-for-hydra@main/lib/lib-noise.js")
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-shaders-for-hydra@main/lib/lib-screen.js")

speed = 0.2
osc(0.75 * Math.PI, 0.15, 0.6)
  .modulateRotate(warp(0.7, 0.1, 1, 4)
    .diff(gradient(), -1), Math.PI*1.5)
  .dither2()
  .out()
```

O patch se organiza em torno de modulação de rotação (**modulateRotate**), gradientes (**gradient**), diferença (**diff**) e osciladores (**osc**). A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateRotate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.2** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares.





```
// made for https://hydra.ojack.xyz/dev

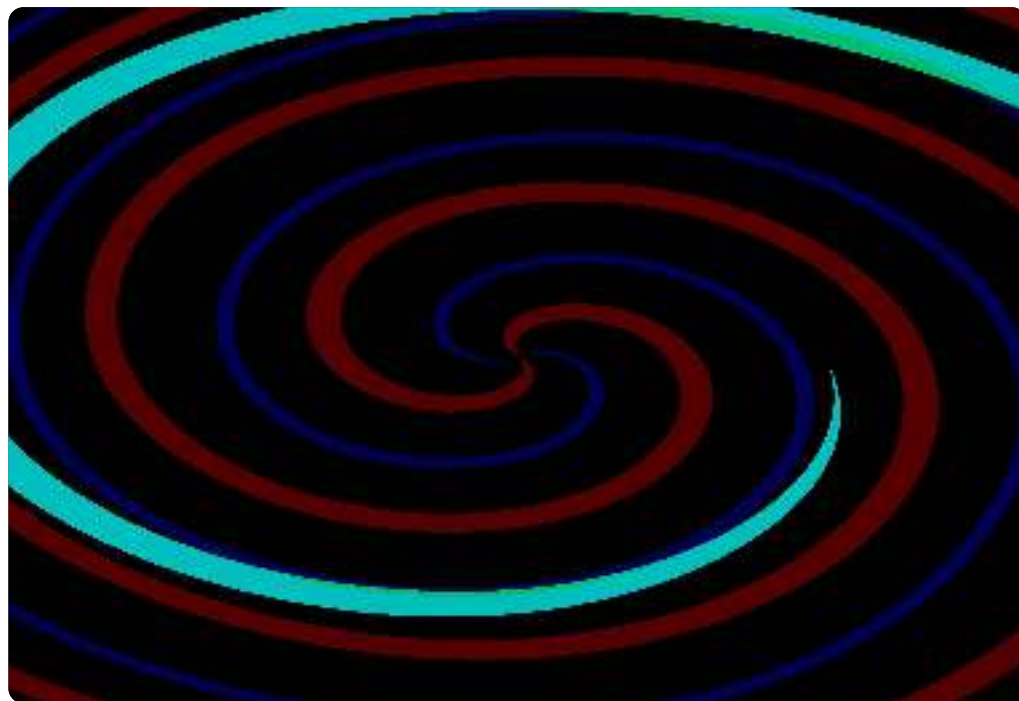
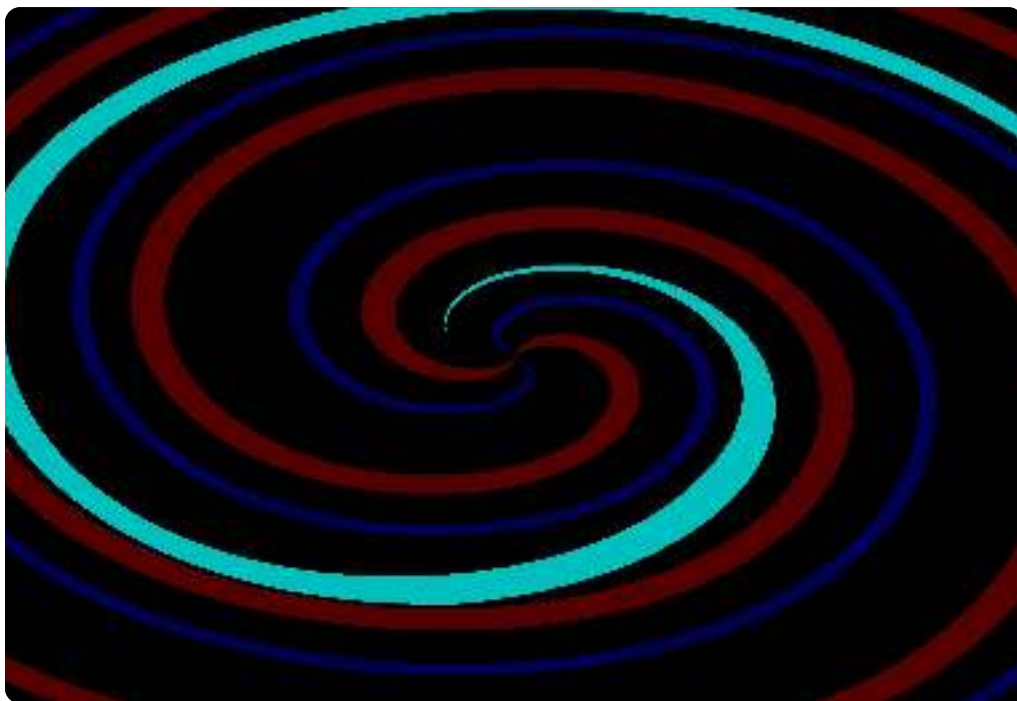
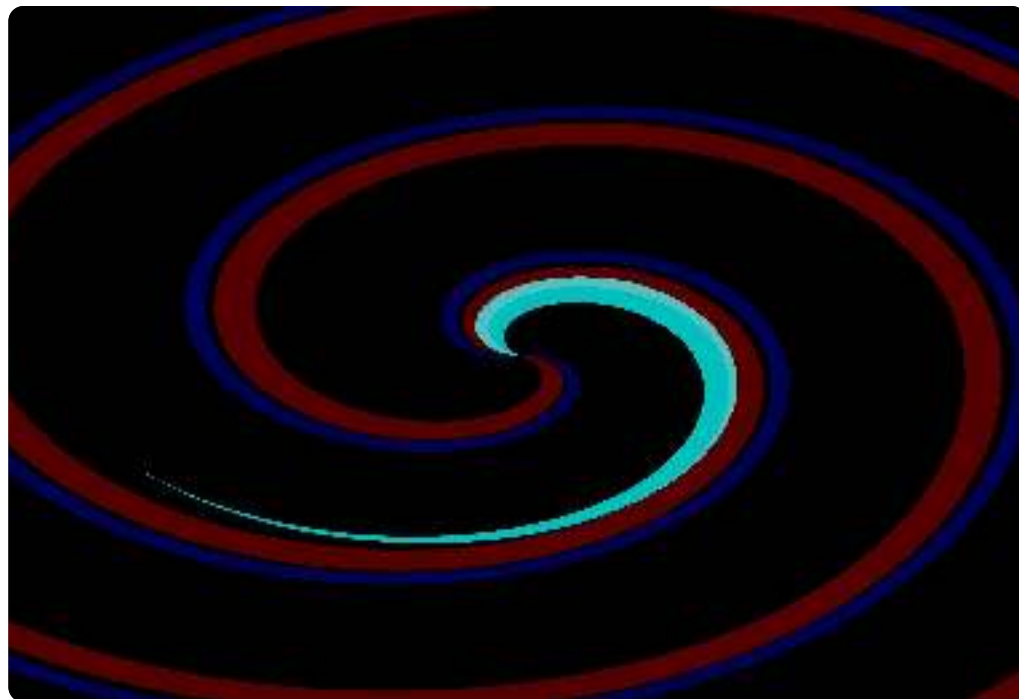
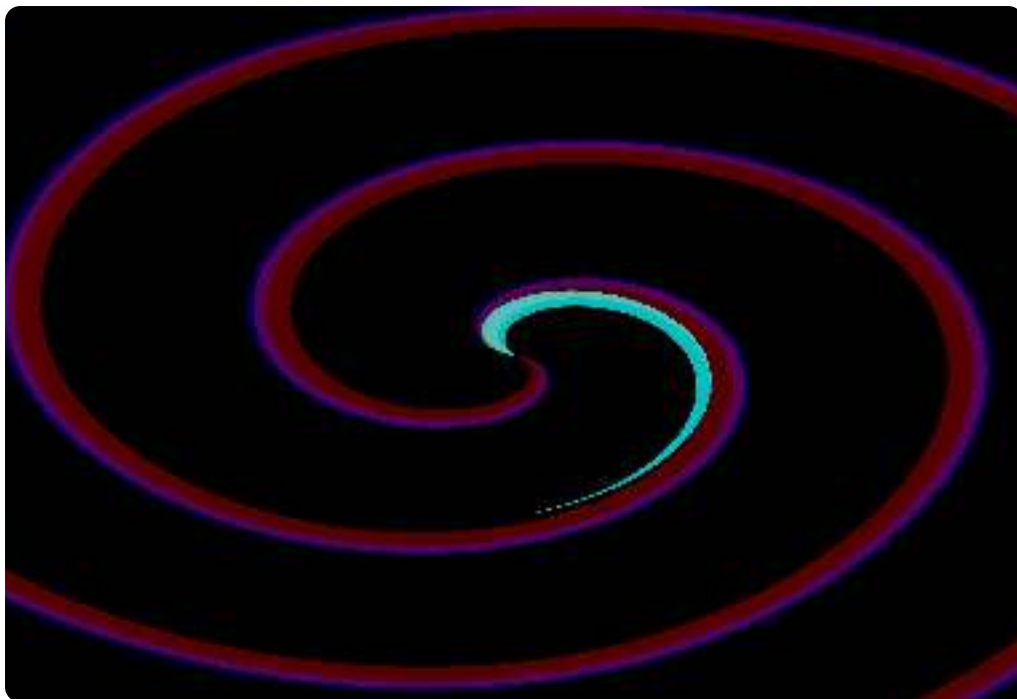
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-
shaders-for-hydra/main/lib/lib-noise.js")

osc(0.75 * Math.PI, 0.11, 0.6)
.modulateRotate(warp(0.7, 0.012, 1, 4)
.diff(gradient(), -1), Math.PI*1.5)
.rotate(()=>time*0.03)
.modulateScale(warp(0.2, 0.012, 1.5, 2),0.9)
.out()
```

O patch se organiza em torno de modulação de rotação (**modulateRotate**), modulação de escala (**modulateScale**), gradientes (**gradient**), rotação (**rotate**) e diferença (**diff**). A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateRotate**, **modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares.





```
// made for https://hydra.ojack.xyz/dev

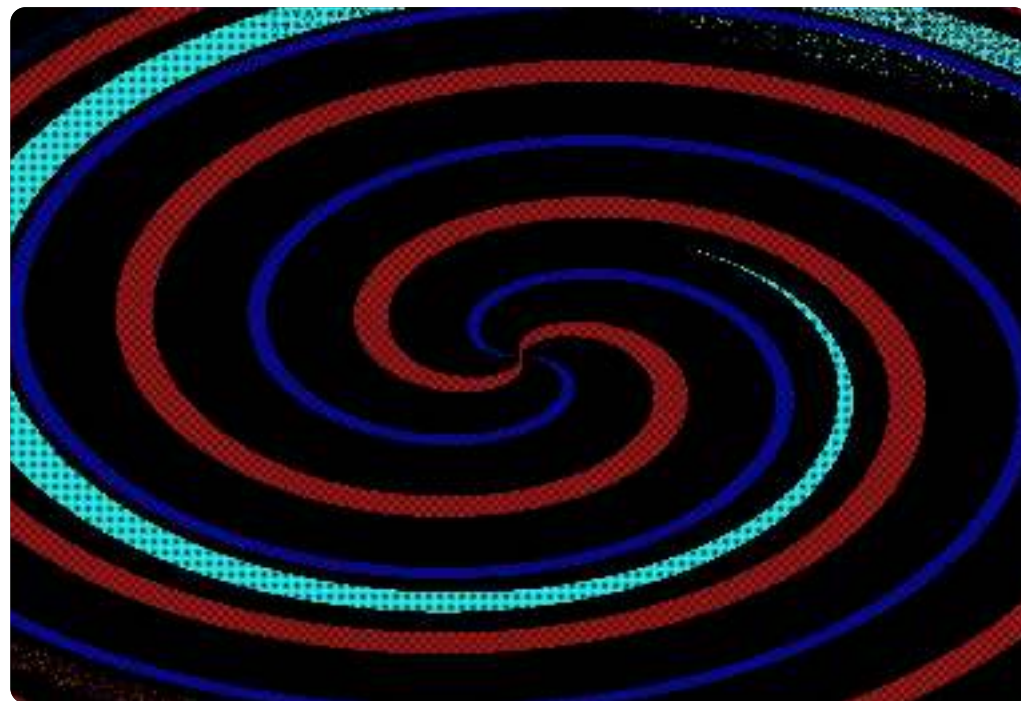
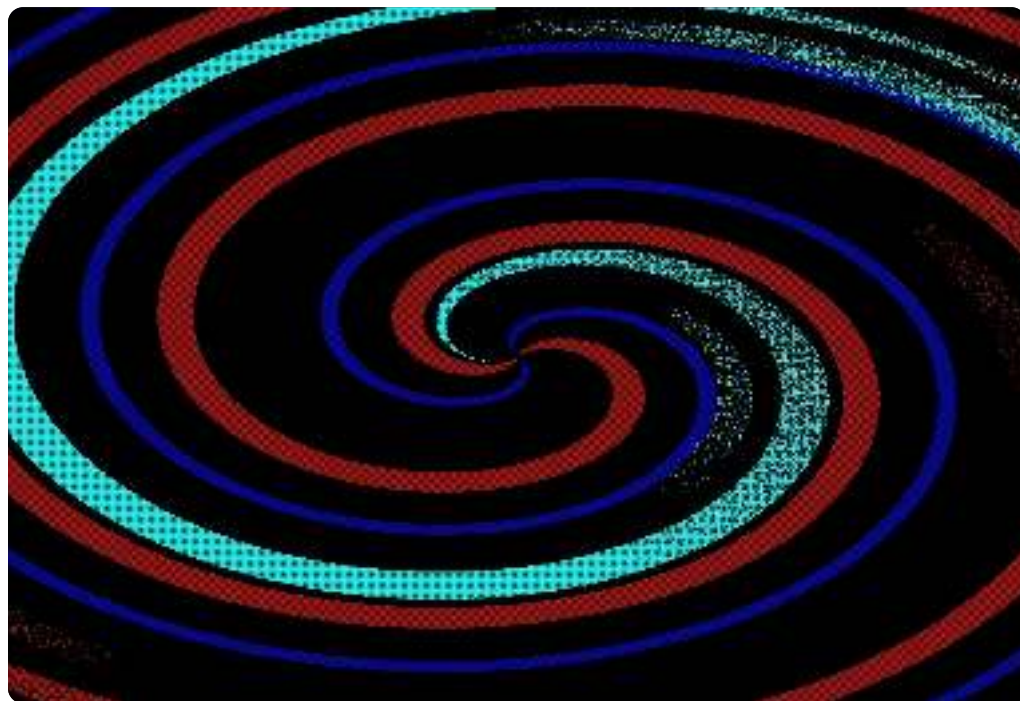
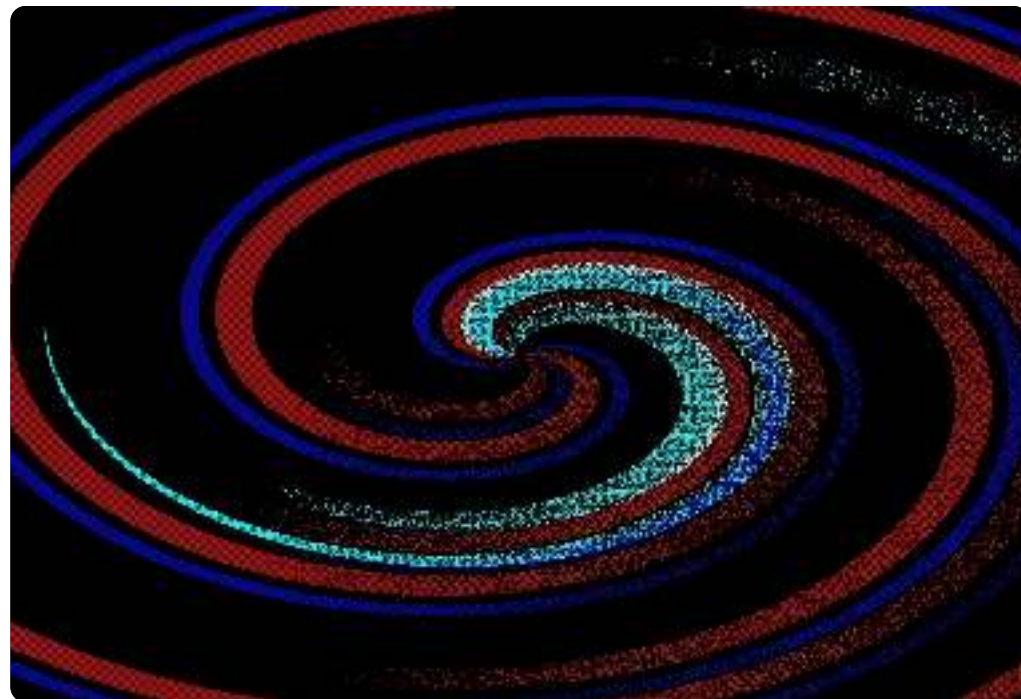
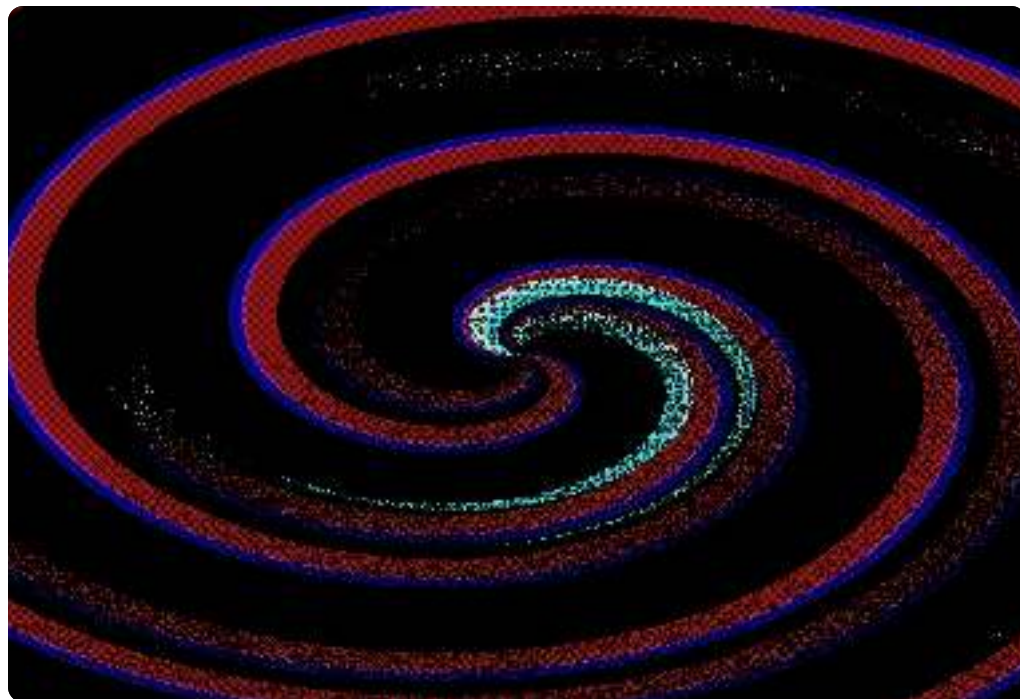
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-
shaders-for-hydra/main/lib/lib-pattern.js")

speed=0.75
r1 = () => 0.1 * time /2.0;
r2 = () => 0.05 * time /2.0;
r3 = () => -0.025 * time /2.0;
r4 = () => -0.075 * time;
spiral(2.0, 5.0, 0.2).rotate(r1).modulateRotate(
noise(0.75,0.015) )
  .mult( spiral(1.0, 3.0, 0.2).color(0,0.75,0.75).modulateRotate(
noise(0.75,0.020) ) )
  .diff( spiral(2.0, 5.0,
0.15).rotate(r2).color(0.333,0,0).modulateRotate(
noise(0.75,0.025) ) )
  .diff( spiral(2.0, 5.0,
0.15/2.0).rotate(r3).color(0,0,0.333).modulateRotate(
noise(0.75,0.030) ) )
  .rotate(r4)
.out()
```

O patch se organiza em torno de modulação de rotação (**modulateRotate**), rotação (**rotate**), ruído (**noise**), colorização (**color**) e diferença (**diff**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo. A composição usa **mult**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateRotate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.75** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```
// made for https://hydra.ojack.xyz/dev

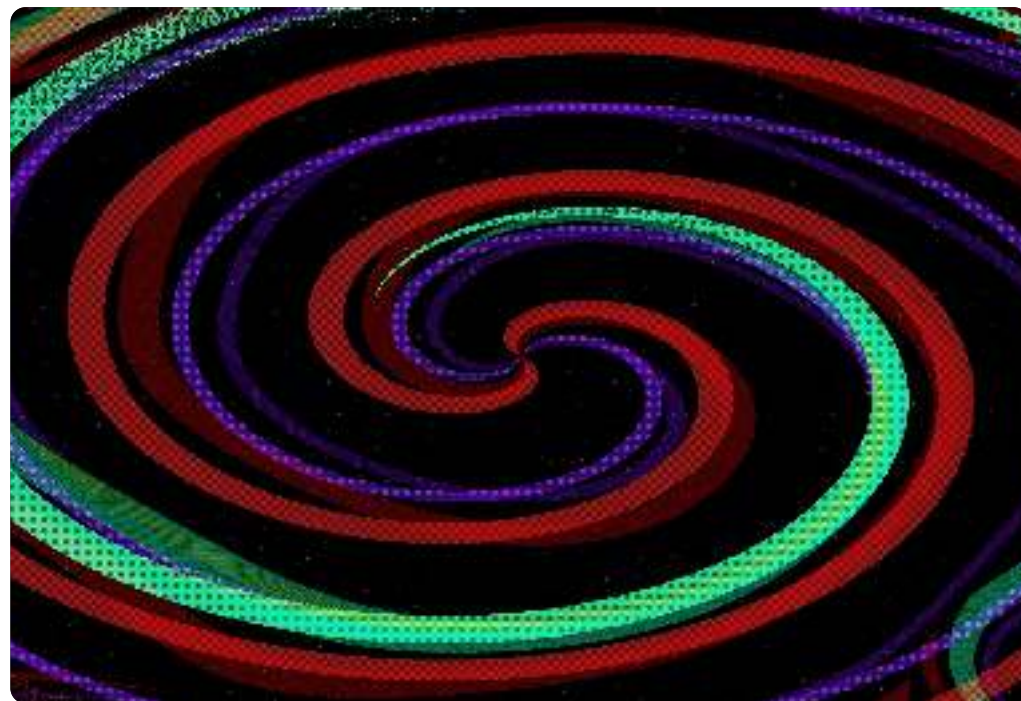
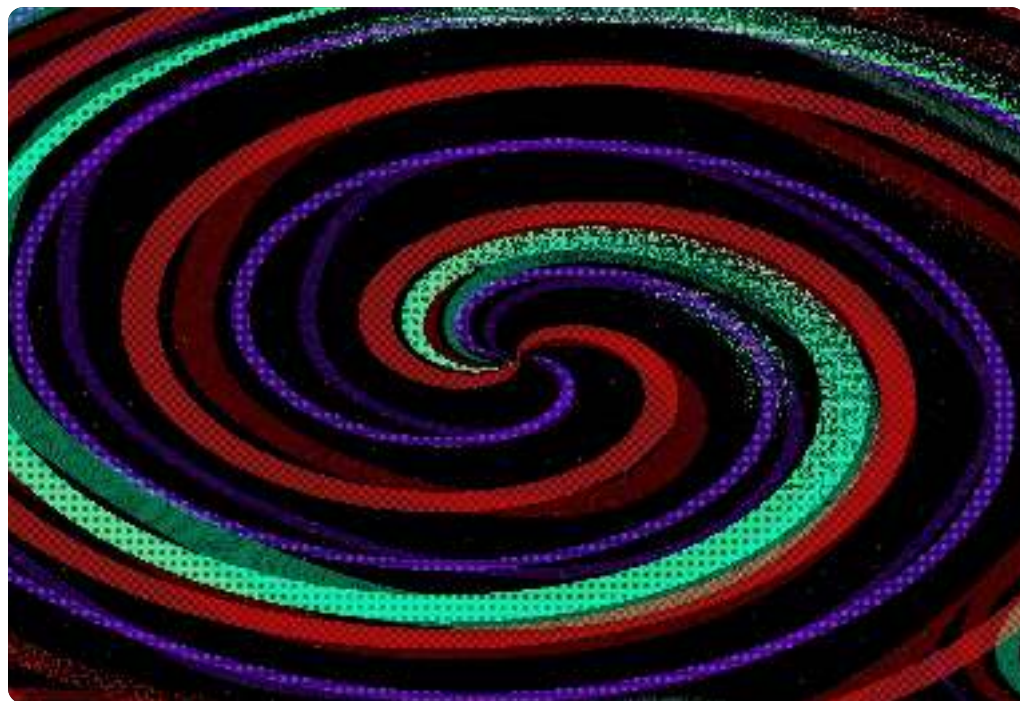
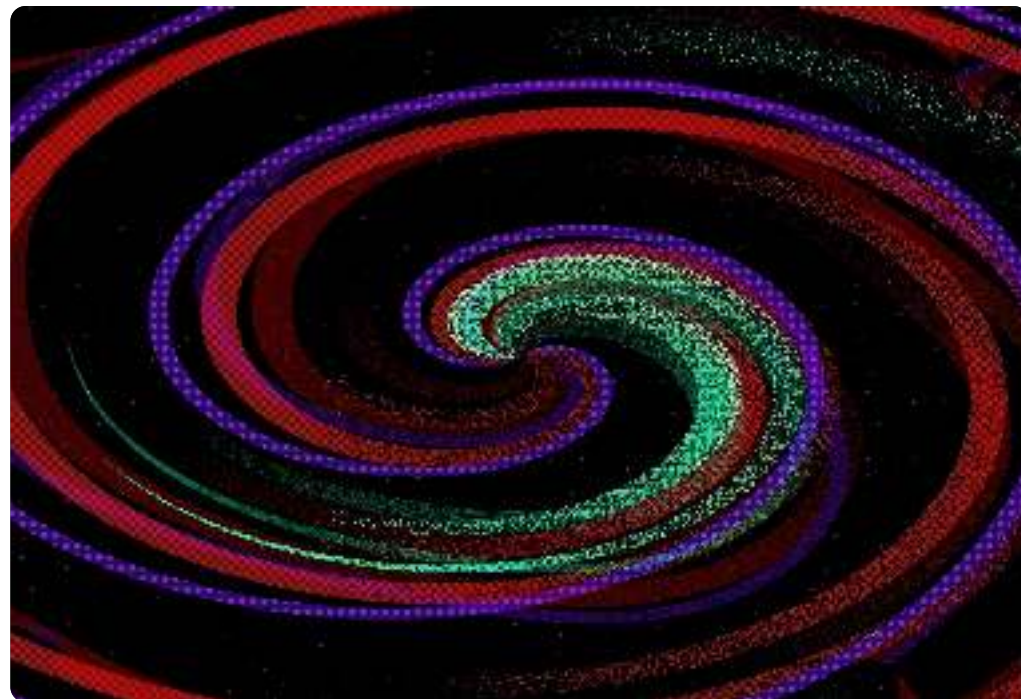
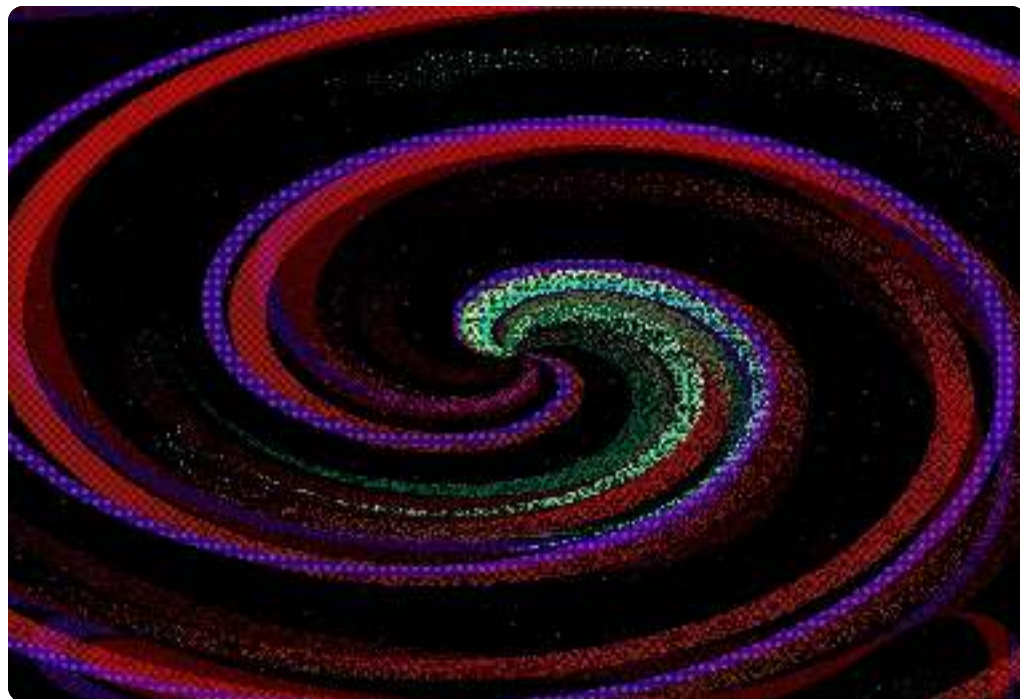
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-shaders-for-hydra/main/lib/lib-screen.js")
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-shaders-for-hydra/main/lib/lib-pattern.js")

speed=0.75
r1 = () => 0.1 * time /2.0;
r2 = () => 0.05 * time /2.0;
r3 = () => -0.025 * time /2.0;
r4 = () => -0.075 * time;
spiral(2.0, 5.0, 0.2).rotate(r1).modulateRotate(
noise(0.75,0.015).ditherrndcolor() )
  .mult( spiral(1.0, 3.0, 0.2).color(0,0.75,0.75).modulateRotate(
noise(0.75,0.020).ditherrndcolor() ) )
  .diff( spiral(2.0, 5.0,
0.15).rotate(r2).color(0.333,0,0).modulateRotate(
noise(0.75,0.025).ditherrndcolor() ) )
  .diff( spiral(2.0, 5.0,
0.15/2.0).rotate(r3).color(0,0,0.333).modulateRotate(
noise(0.75,0.030).ditherrndcolor() ) )
  .rotate(r4)
  .dither2()
.out()
```

O patch se organiza em torno de modulação de rotação (**modulateRotate**), rotação (**rotate**), ruído (**noise**), colorização (**color**) e diferença (**diff**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo. A composição usa **mult**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateRotate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.75** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```
// made for https://hydra.ojack.xyz/dev

await loadScript("https://hyper-hydra.glitch.me/hydra-
outputs.js")
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-
shaders-for-hydra/main/lib/lib-screen.js")
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-
shaders-for-hydra/main/lib/lib-pattern.js")

speed=0.72
oS.setLinear()
r1 = () => 0.1 * time / 2.2;
r2 = () => 0.05 * time / 2.2;
r3 = () => -0.03 * time / 2.2;
r4 = () => -0.09 * time;

n1 = () => Math.sin( time * 0.05 ) * 0.3 + 1.2;

spiral(2.0, 5.0, 0.2).rotate(r1).modulateRotate(
noise(0.75,0.015).ditherndcolor() )
  .mult( spiral(1.0, 3.0, 0.2).color(0,0.75,0.5).modulateRotate(
noise(0.75,0.020).ditherndcolor() ) )
  .diff( spiral(2.0, 5.0,
0.15).rotate(r2).color(0.333,0,0.1).modulateRotate(
noise(0.75,0.025).ditherndcolor() ) ) )
  .diff( spiral(2.0, 5.0,
0.15/2.0).ditherndcolor().rotate(r3).color(0.15,0,0.333).modulat
eRotate( noise(0.75,0.030) ) )
  .rotate(r4)
  .dither2()
  .out(o0)
```

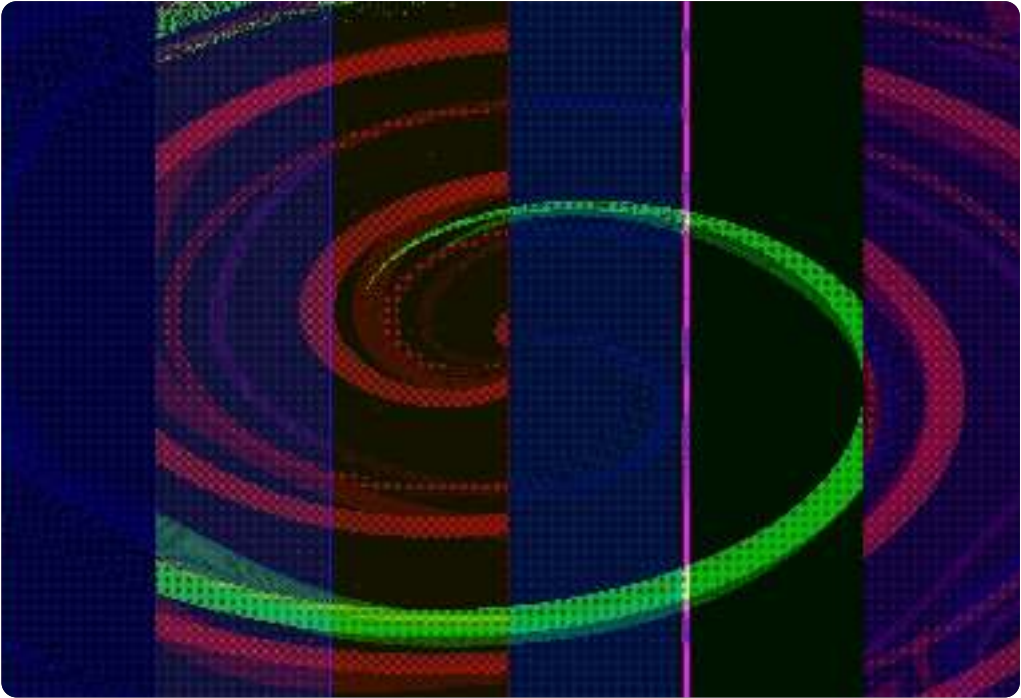
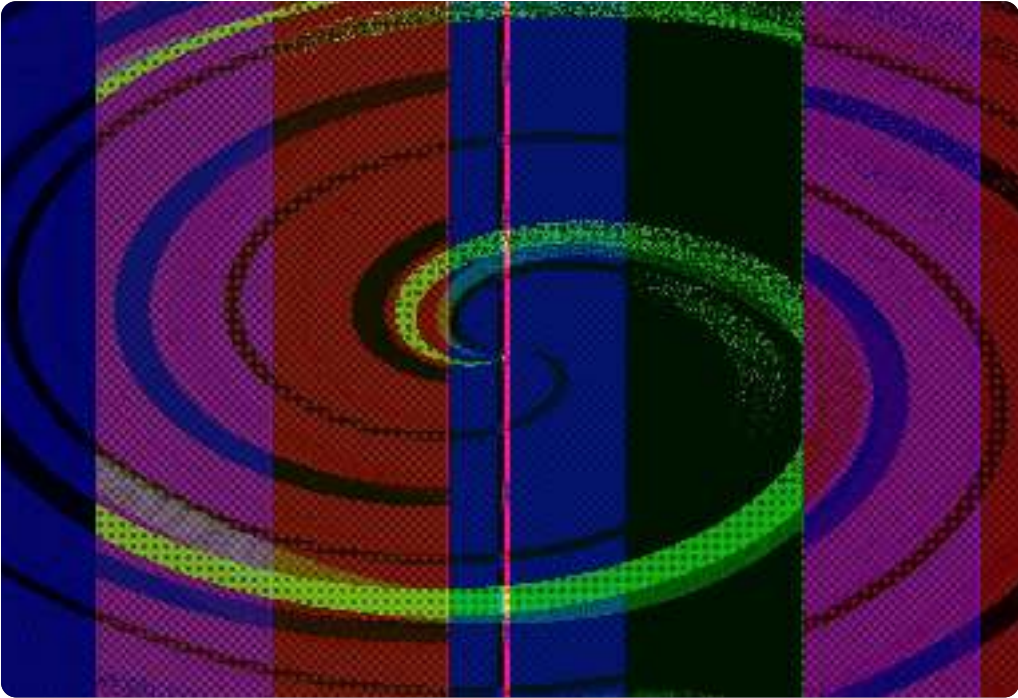
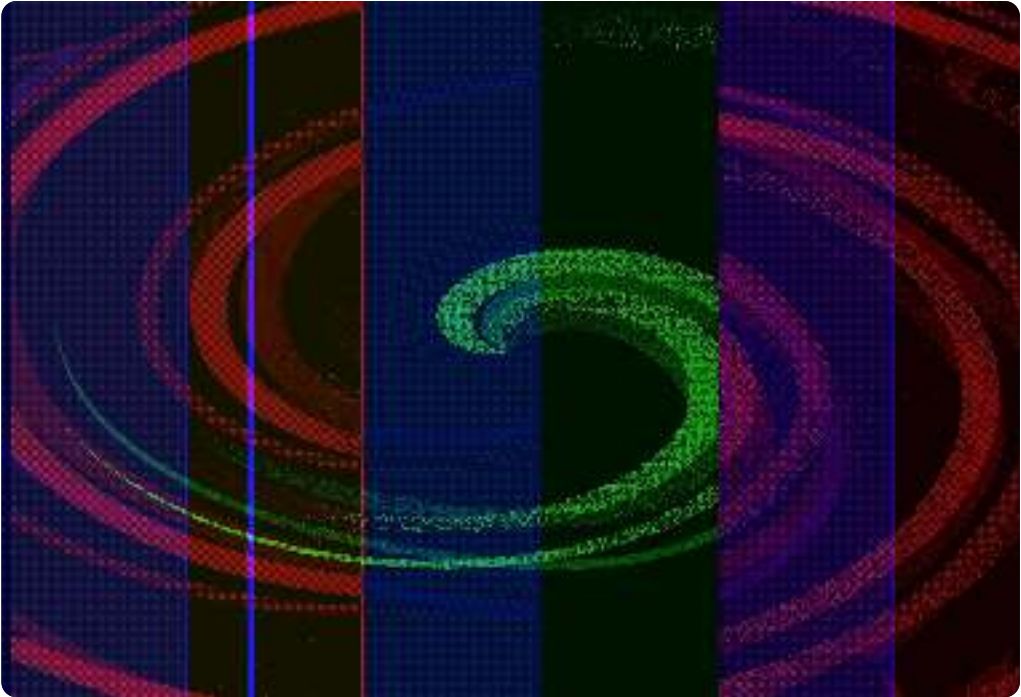
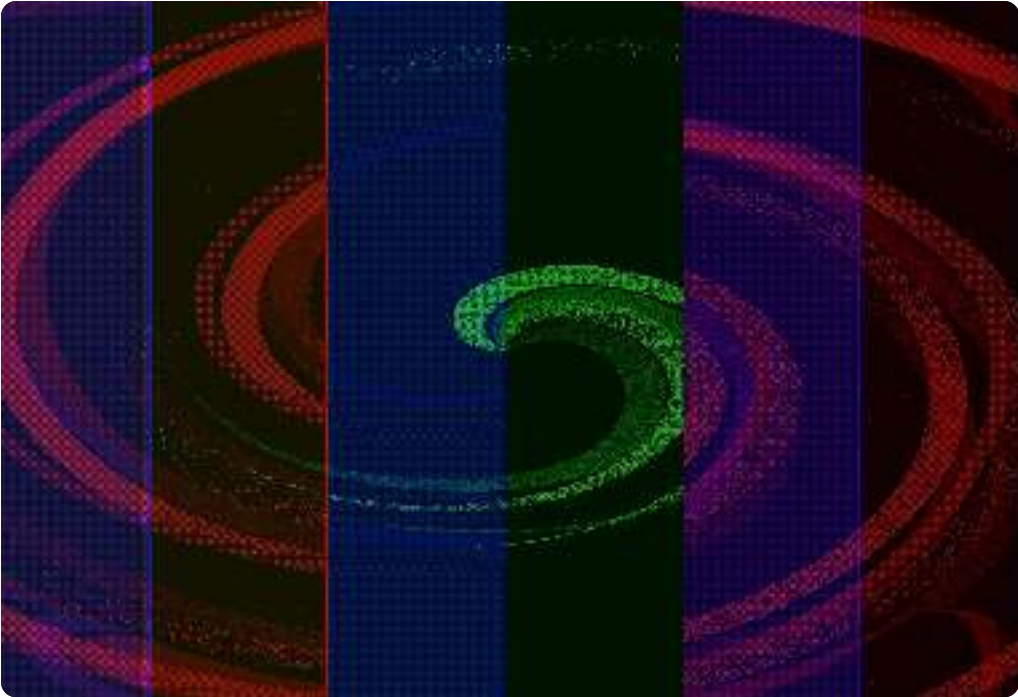
```
src(o0)
.add(
  src(o0).scale(0.996)
  .modulateRotate( noise( n1 ,0.007) )
  .modulateHue( noise( n1 ,0.007), 1 )
  ,0.5)
.out(o1)

render(o1)
```

O patch se organiza em torno de ruído (**noise**), modulação de rotação (**modulateRotate**), rotação (**rotate**), colorização (**color**) e diferença (**diff**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo. A composição usa **mult**, **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateRotate**, **modulateHue**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed** = **0.72** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**src**, **loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```
// made for https://hydra.ojack.xyz/dev

await loadScript("https://hyper-hydra.glitch.me/hydra-
outputs.js")
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-
shaders-for-hydra/main/lib/lib-screen.js")
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-
shaders-for-hydra/main/lib/lib-pattern.js")
s0.initCam()
//s1.initScreen() // run just once
speed=0.72
oS.setLinear()
r1 = () => 0.1 * time / 2.2;
r2 = () => 0.05 * time / 2.2;
r3 = () => -0.03 * time / 2.2;
r4 = () => -0.09 * time;

n1 = () => Math.sin( time * 0.05 ) * 0.3 + 1.2;
c1 = () => Math.sin( time * 0.2 ) * 0.5 + 1.4;

spiral(2.0, 5.0, 0.2).rotate(r1).modulateRotate(
noise(0.75,0.015).ditherndcolor() )
  .mult( spiral(1.0, 3.0, 0.2).color(0,0.75,0.5).modulateRotate(
noise(0.75,0.020).ditherndcolor() ) )
  .diff( spiral(2.0, 5.0,
0.15).rotate(r2).color(0.333,0,0.1).modulateRotate(
noise(0.75,0.025).ditherndcolor() ) )
  .diff( spiral(2.0, 5.0,
0.15/2.0).ditherndcolor().rotate(r3).color(0.15,0,0.333).modulat
eRotate( noise(0.75,0.030) ) )
  .rotate(r4)
  .dither2()
```

```
.out(o0)

src(o0)
.add(
  src(o0).scale(0.996)
  .modulateRotate( noise( n1 ,0.007) )
  .modulateHue( noise( n1 ,0.007), 1 )
  ,0.5)
.out(o1)

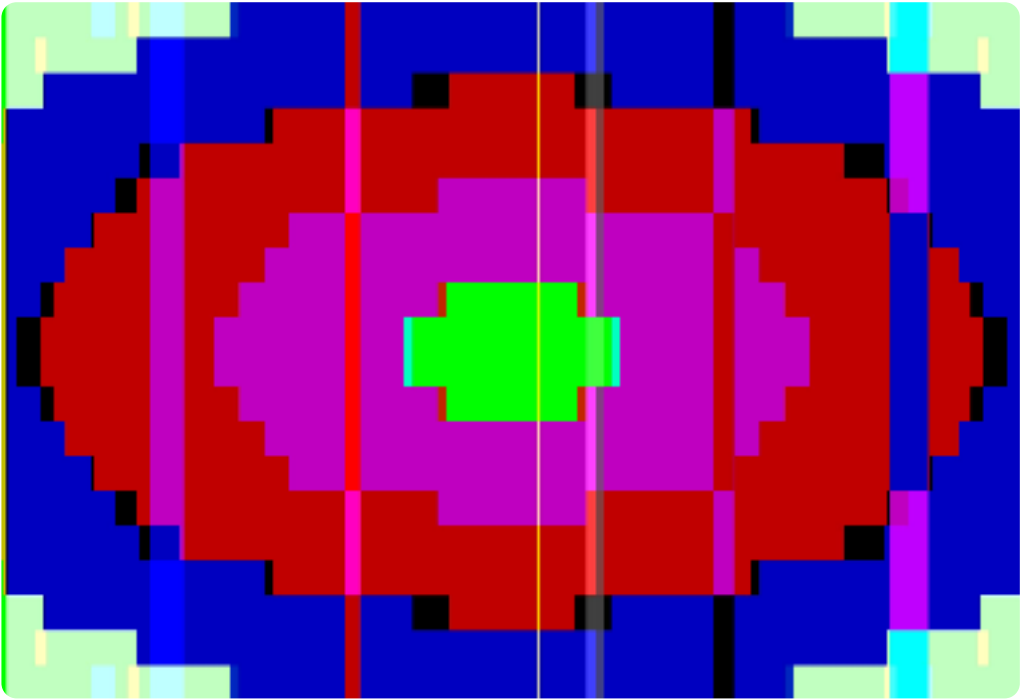
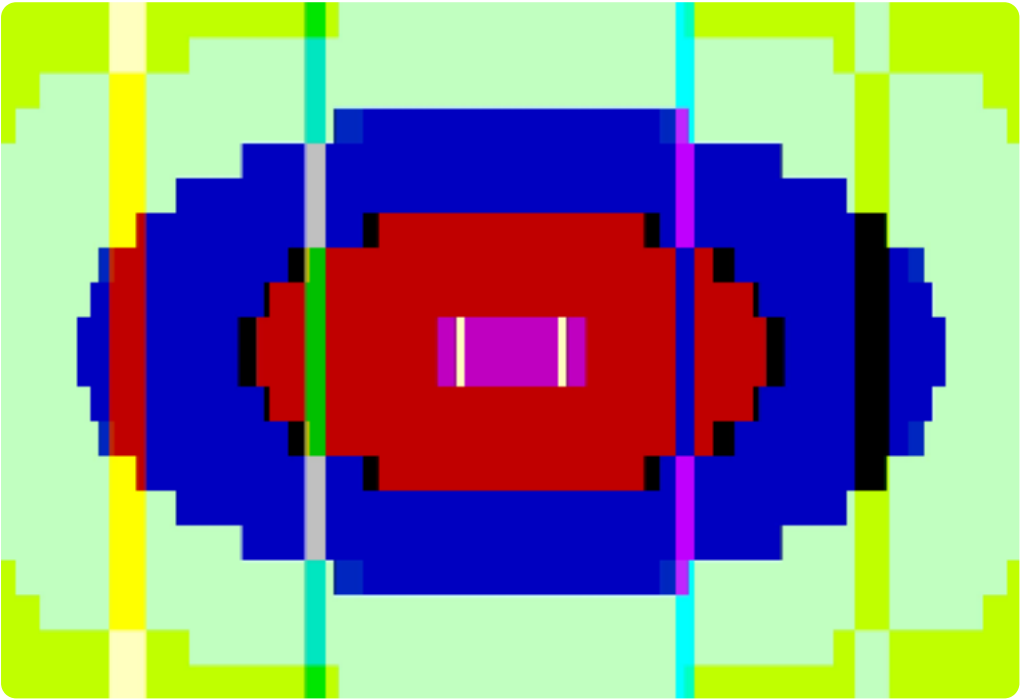
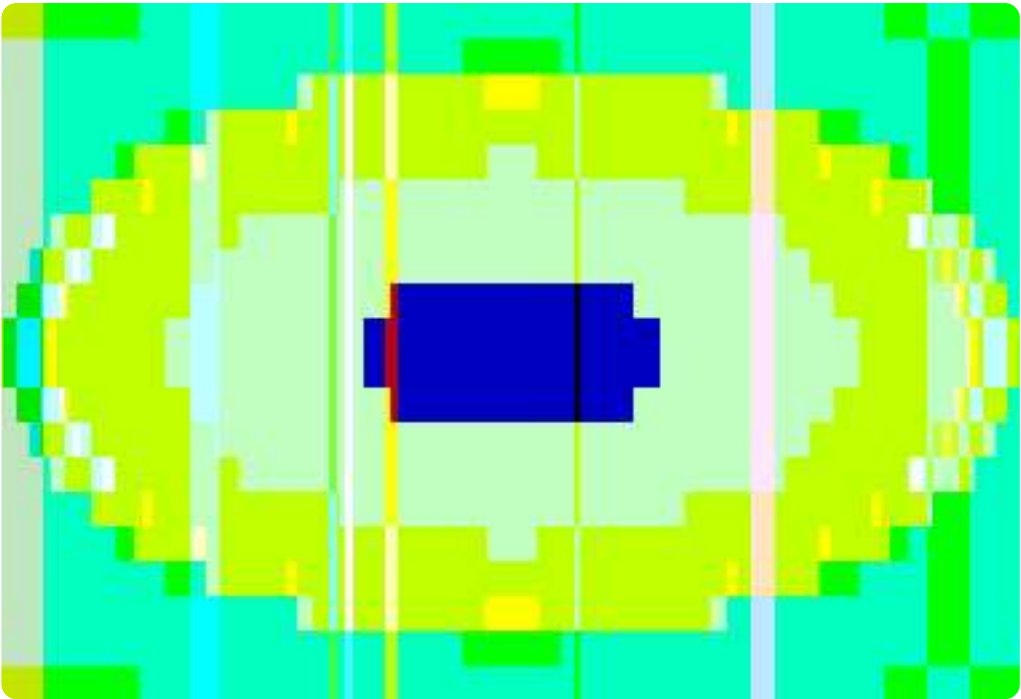
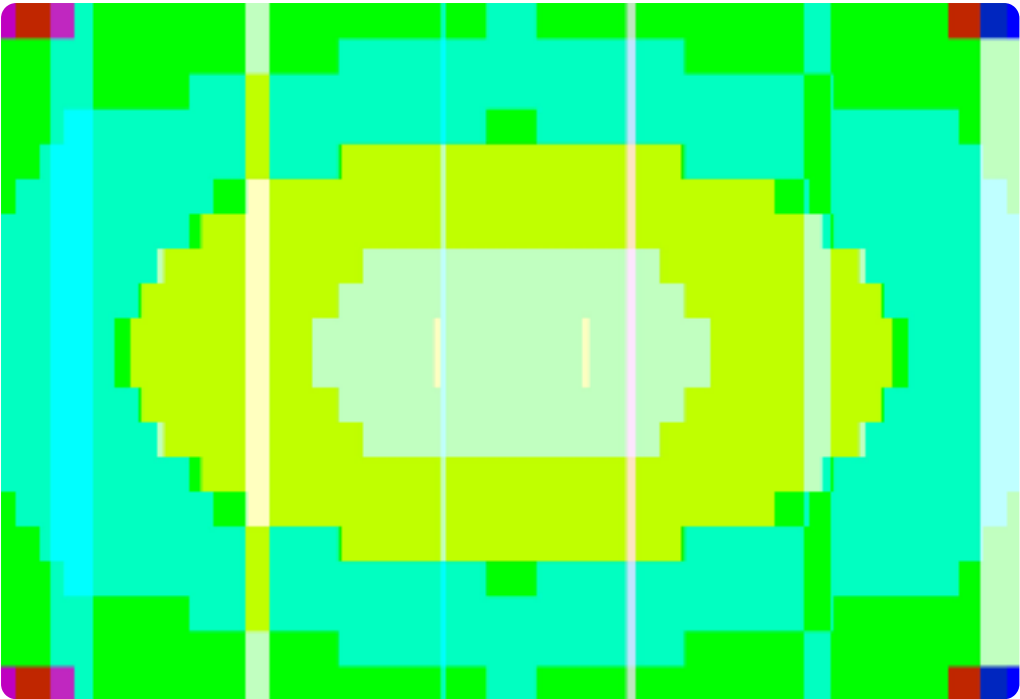
src(o1)
.diff(
  src(s0)
  .invert()
  .color(1,0.15,1.25)
  .contrast(c1)
  //.blur()
  .dither2()
).out(o2)

src(s0)
.mult(src(o2),0.9)
  .diff(src(s1).color(1,0,1))
.out(o3)

render(o3)
```

O patch se organiza em torno de fontes externas (**src**), ruído (**noise**), modulação de rotação (**modulateRotate**), colorização (**color**) e rotação (**rotate**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo. A composição usa **mult**, **add**, **diff** para misturar ou recortar sinais em vez





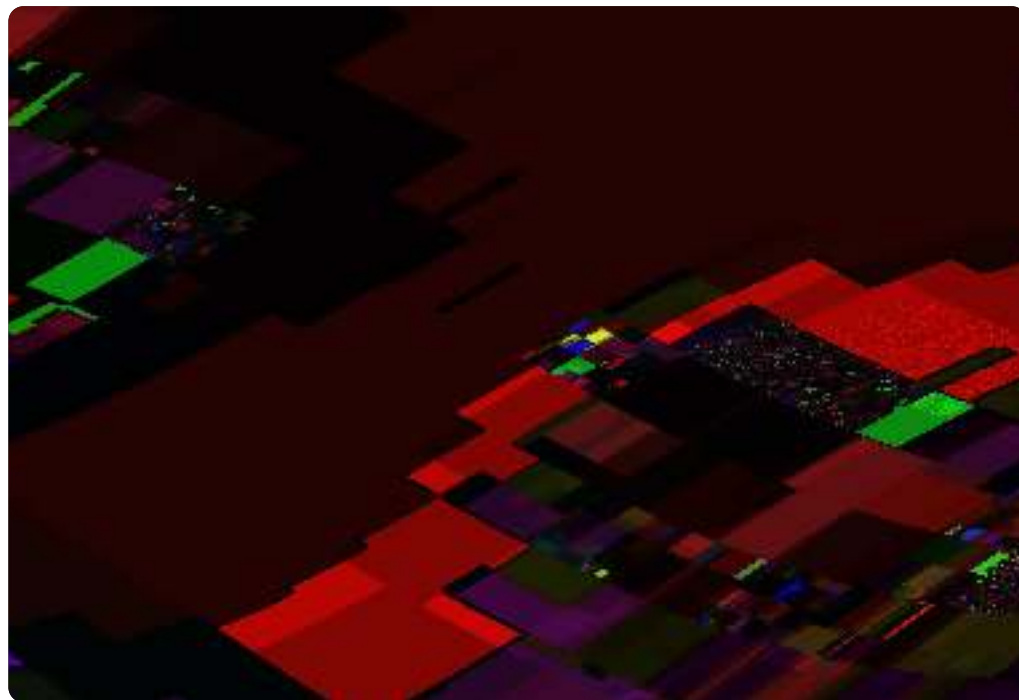
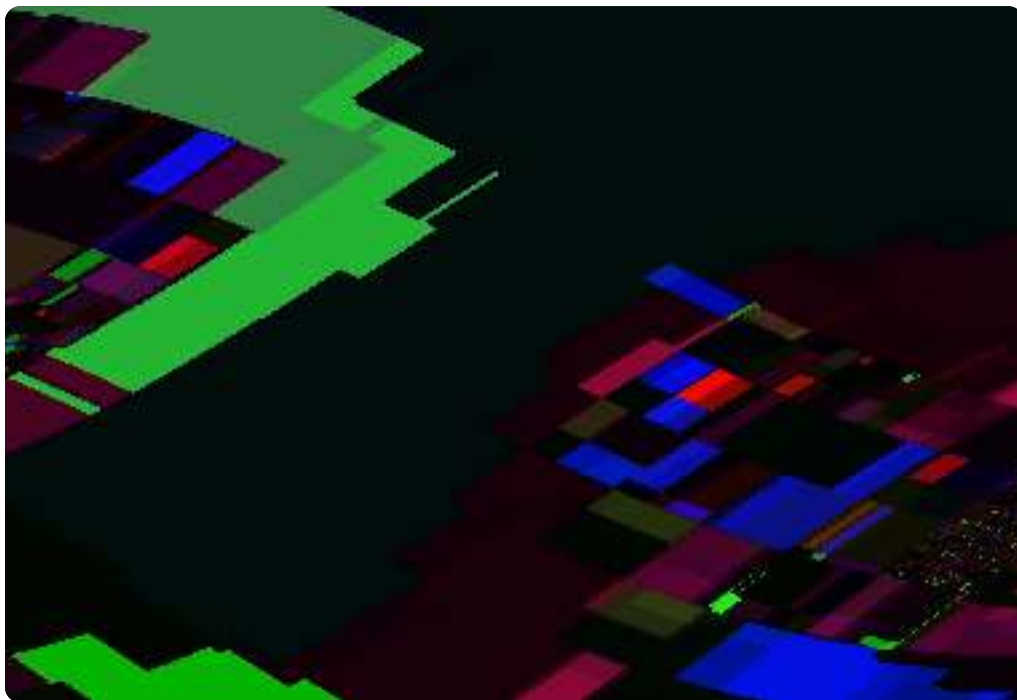
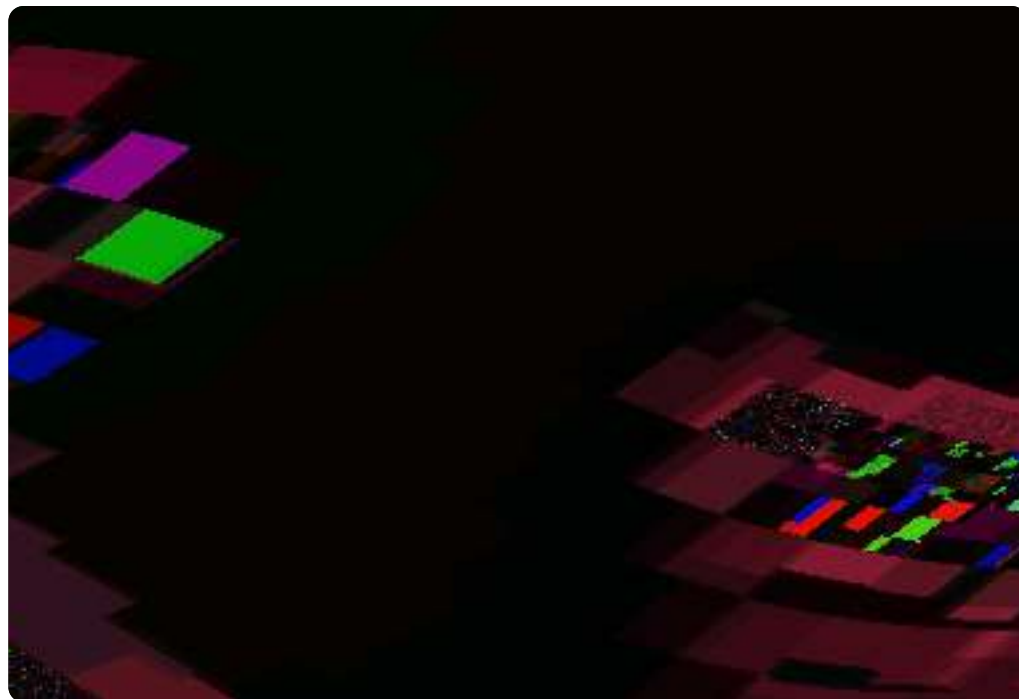
```
s0.initScreen() // grab Sonic Pi Lisajous graph

src(s0).scrollY(0.01,0.01).kaleid(8).color(0,0.2,1).pixelate(50,2
0)
.add(
src(s0).scrollY(0.005,0.005).kaleid(16).color(1,2,0).pixelate(400
) )
.add( src(s0).pixelate(400,10) )
.diff( src(s0).contrast(1).scale(1.1,1,1) )
.out()
```

O patch se organiza em torno de fontes externas (**src**), pixelização (**pixelate**), deslocamento Y (**scrollY**), kaleidoscópio (**kaleid**) e colorização (**color**). A composição usa **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

Há entrada externa (**src**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

await loadScript("https://cdn.statically.io/gl/metagrowing/extra-
shaders-for-hydra/main/lib/lib-noise.js")
//s1.initScreen() // run just once

speed=0.000001
whitenoise( 800,1.01 ).color(0,0,1)
.diff( whitenoise( 400,1.02 ).color(1,0,0).scrollX(0.1,-0.01) )
.diff( whitenoise( 200,1.03 ).color(1,1,0).scrollX(0.05,-0.01) )
.modulateScale( whitenoise( 25,1 ),0.5 )
.diff( whitenoise( 100,1.04 ).scrollX(0.025,-0.005) )
.diff( whitenoise( 50,1.05 ) )
.modulateScale( noise(3,3000).contrast(20).pixelate(30,30),1.01
)

.out(o1)

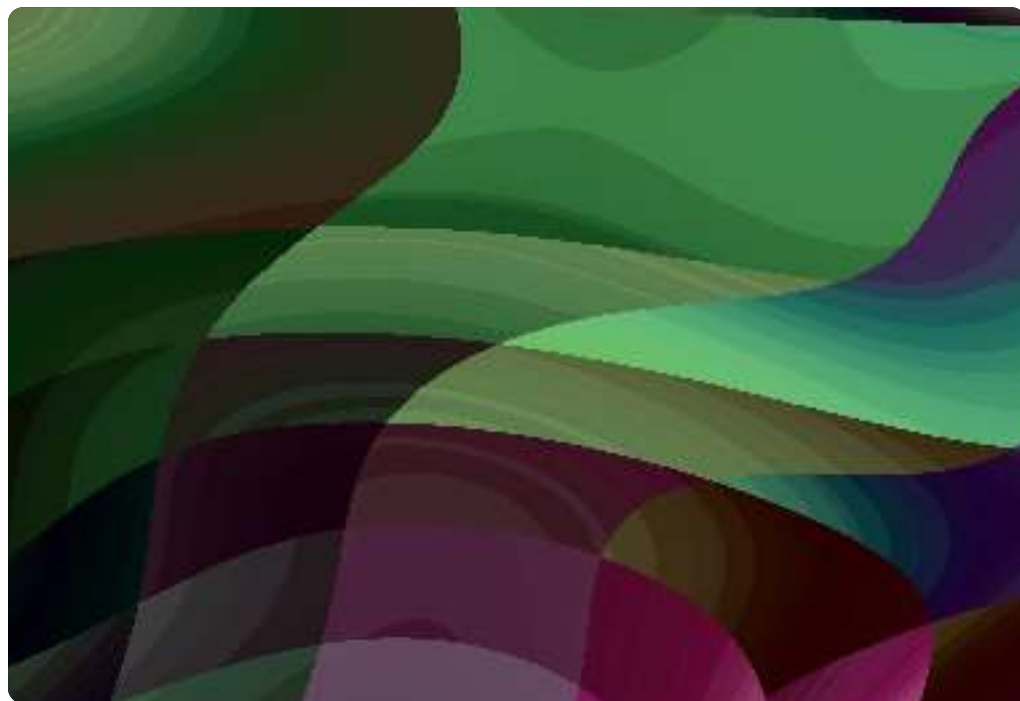
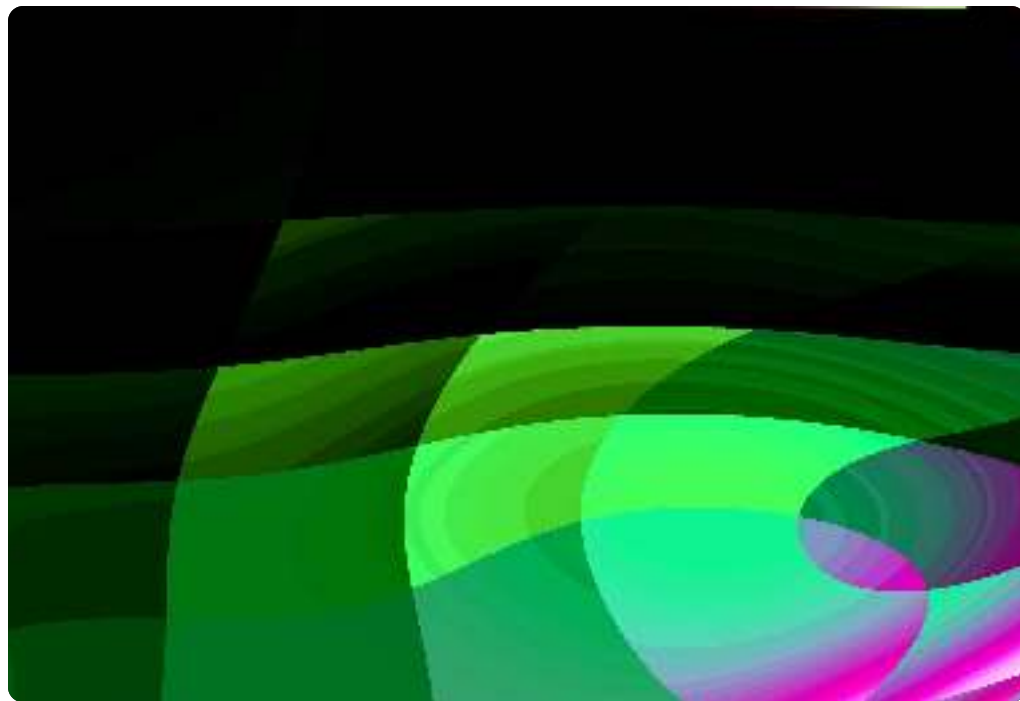
src(o1)
.color(1,0.333,1)
.colorama(-0.01)
.scale(1.2)
.mult(src(o1).scale( ()=>Math.sin(time)*0.333+1.2 ) )
.color(1,1,0.5)
.colorama(0.01)
.diff( src(s1) )
.mult(src(o1).scale(2), 0.5 )
.rotate( ()=>Math.PI*time*1000 )
.modulateScale( osc(3.5,0,29999900), 1.34 )
.add( src(s1) ,0.4)
.out()

```

O patch se organiza em torno de colorização (**color**), diferença (**diff**), fontes externas (**src**), modulação de escala (**modulateScale**) e deslocamento X (**scrollX**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo. A composição usa **mult**, **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.000001** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**src**, **loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





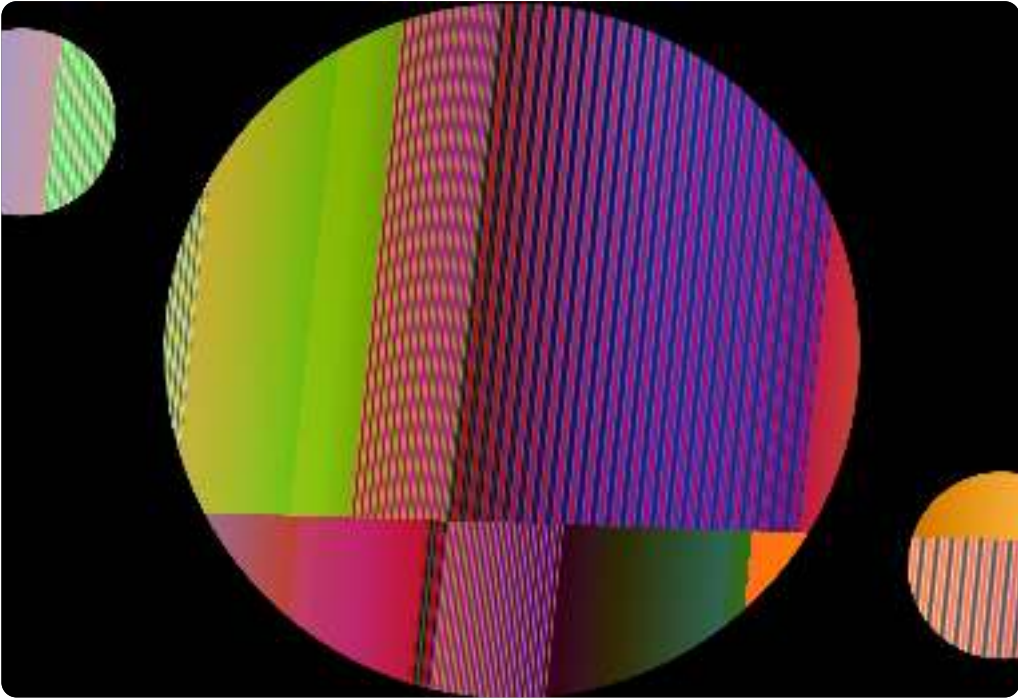
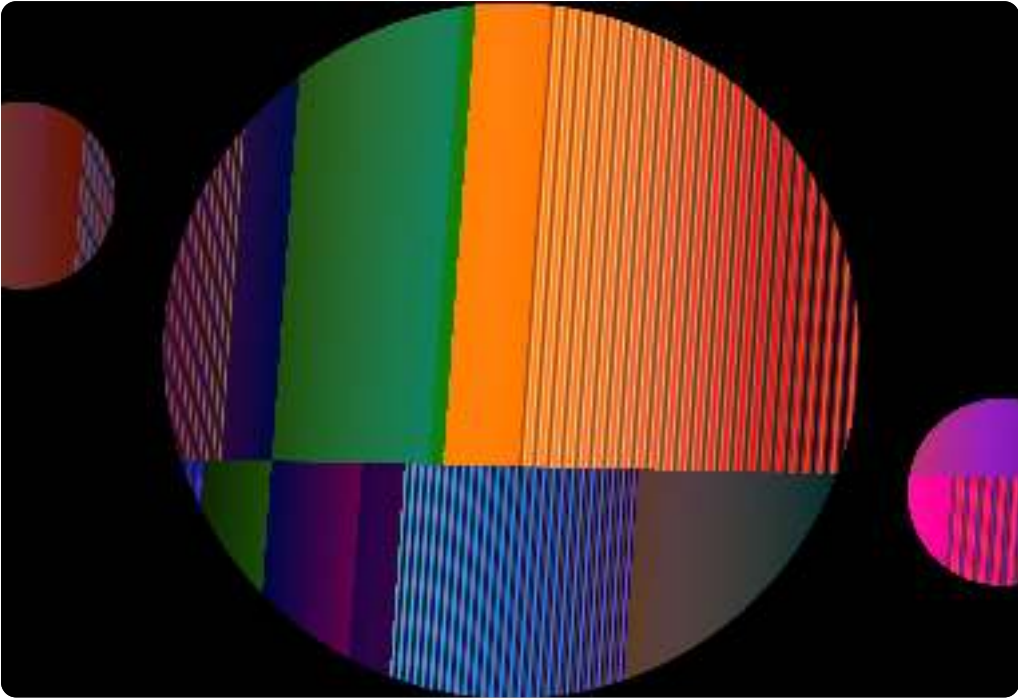
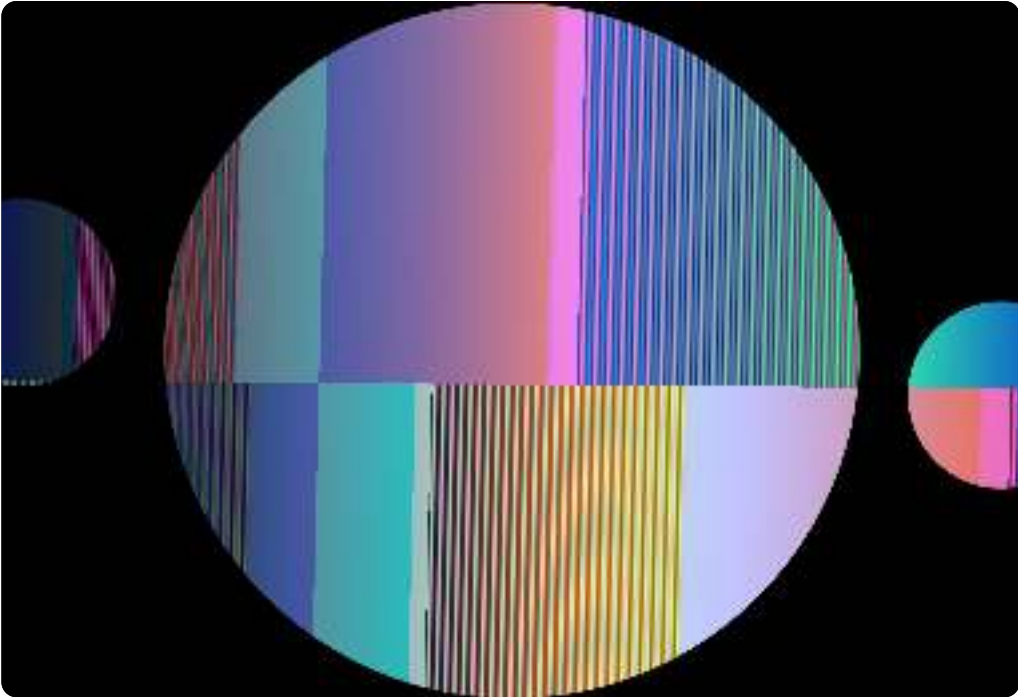
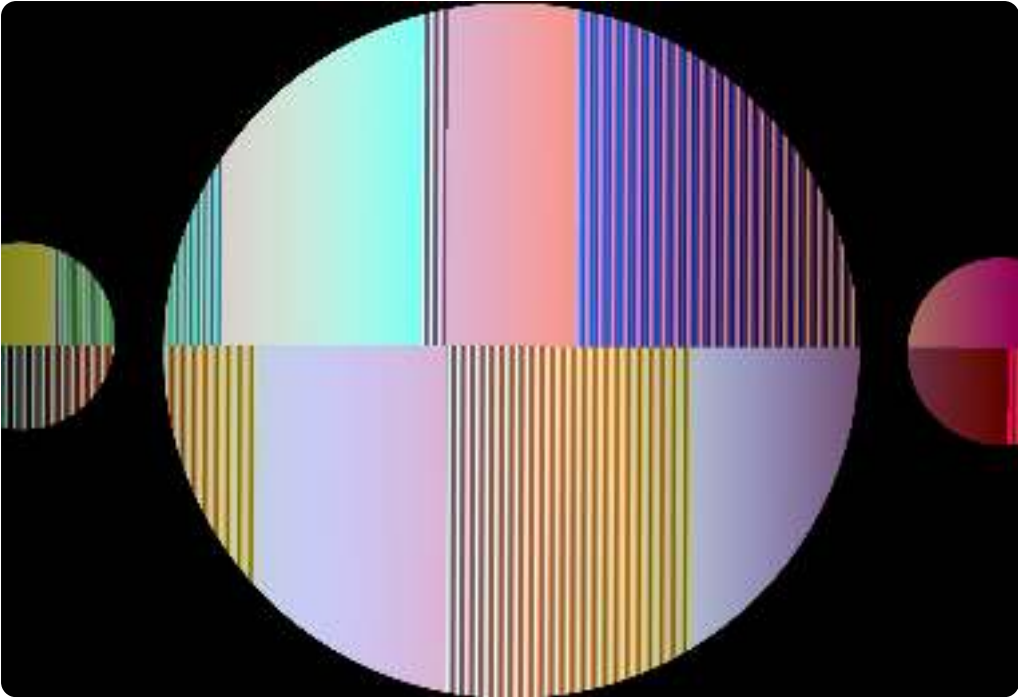
```
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-shaders-for-hydra/main/lib/lib-color.js")
```

```
noise(0.6,0.0125).color(1,0,0).pixelate(80,45)
.add( noise(0.7,0.015).color(0.5,0,0.5).pixelate(80,45) )
.add( noise(0.8,0.0175).color(0,0,1).pixelate(80,45) )
.diff( noise(0.9,0.02).pixelate(80,45) )
.modulatePixelate( noise(0.9,0.01), 3 )
.contrast( ()=>Math.sin(time/3)*0.5+1.5 )
.rotate( ()=> time * 0.002 )
.monotone(16, ()=>Math.sin(time*0.002)*0.5+0.5, 0.5)
.out()
```

O patch se organiza em torno de ruído (**noise**), pixelização (**pixelate**), colorização (**color**), soma de camadas (**add**) e modulação por pixelate (**modulatePixelate**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo. A composição usa **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulatePixelate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

speed=0.7
mySpeed=0.16
osc(2040,-0.002*mySpeed,0)
.mult( osc(40,-0.003*mySpeed,0).thresh(0.5,0) )
.diff( osc(20,0.002*mySpeed,0).thresh(0.5,0).color(2,0,0) )
.diff( osc(10,0.003*mySpeed,0).thresh(0.5,0).color(0,0,0.5) )
.diff( osc(10,-0.01*mySpeed,2) )
.modulateScale( osc(4.5,-0.04*mySpeed,2),0.81 )
.diff( osc(10,0.01*mySpeed,2) )
.color(1,0.81,1)
.modulate(
  osc( )=>Math.sin(time/9)*1.5+6.5 ,0,0).thresh(0.5,0)
  .diff( osc(6.27,0,0).rotate(Math.PI/2).thresh(0.5,0) )
  ,0.2
)
.rotate( )=>time/-90 )
.scrollY(0.01,-0.005)
.scale(1.75)
.rotate( )=>time/77 )
// .modulate(
//   osc( )=>Math.sin(time/18)*1.5+6.5 ,0,0).thresh(0.5,0)
//   .diff( osc(6.27,0.001,0).rotate(Math.PI/2).thresh(0.5,0)H )
//   ,0.1
// )
.diff( osc(4,-0.04, )=>time/54.0 ) )
.mult(
  shape(100,0.56,0.002).scale(1,1,16/9)

```

```

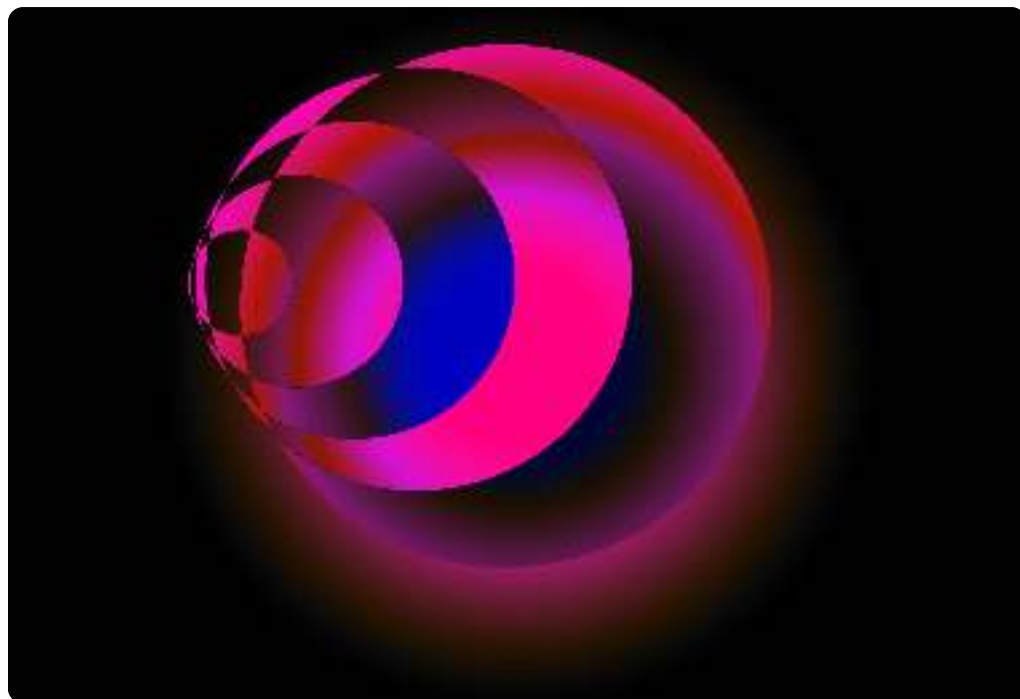
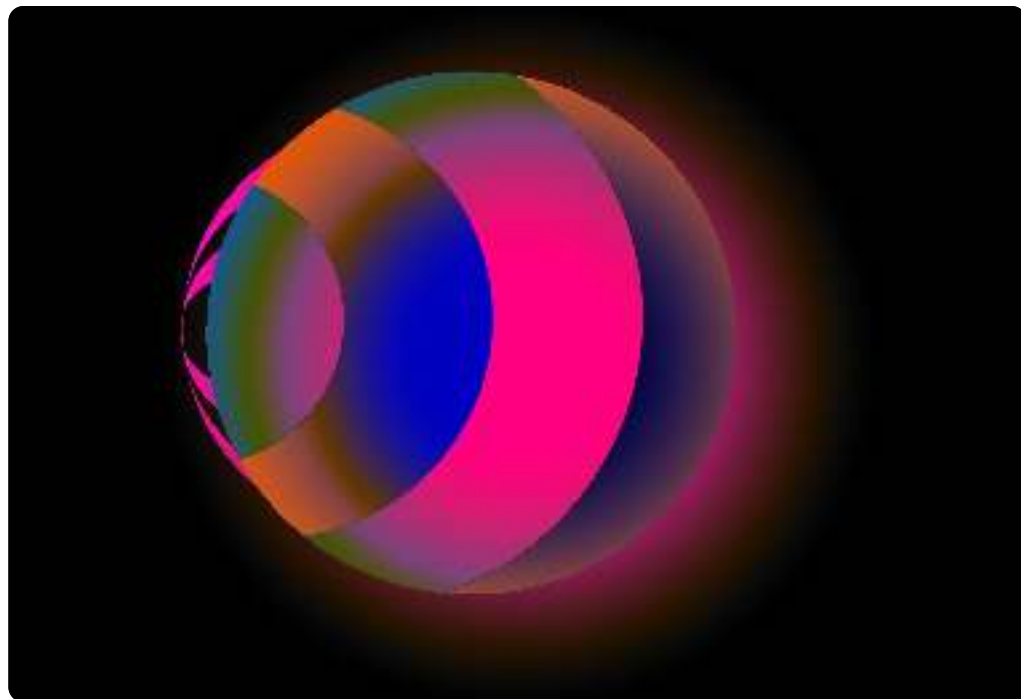
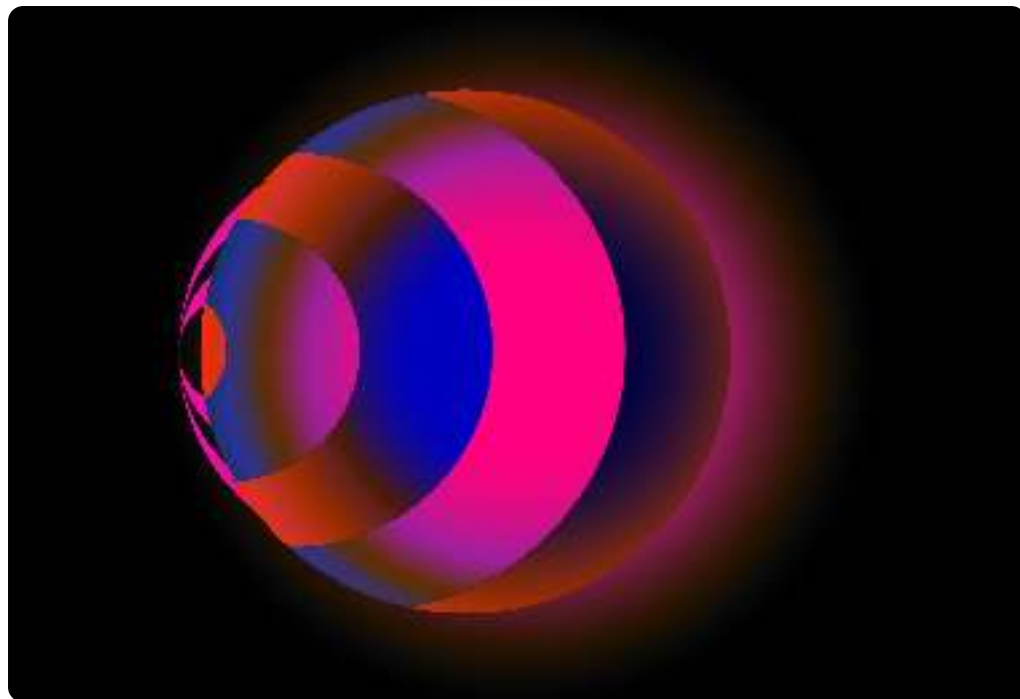
.add(
  shape(100,0.15,0.002).scale(1,1,16/9).scrollX(0.395,0).scrollY(0.01, 0.01) )
.add(
  shape(100,0.15,0.002).scale(1,1,16/9).scrollX(-0.395,0).scrollY(0.01, -0.01) )
)
.out()

```

O patch se organiza em torno de osciladores (**osc**), diferença (**diff**), limiar (**thresh**), escala (**scale**) e deslocamento Y (**scrollY**). Osciladores viram padrões binários via **thresh**, e a geometria resultante alimenta o restante da cadeia. A composição usa **mult**, **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**, **modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.7** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```
// licensed with CC BY-NC-SA 4.0
https://creativecommons.org/licenses/by-nc-sa/4.0/
// "egg of the phoenix"
// Alexandre Rangel
// www.alexandrangel.art.br/hydra.html

speed=0.43
shape(99,.15,.5).color(0,1,2)

.diff( shape(240,.5,0).scrollX(.05).rotate( ()=>time/10
).color(1,0,.75) )
.diff( shape(99,.4,.002).scrollX(.12).rotate( ()=>time/20
).color(1,0,.75) )
.diff( shape(99,.3,.002).scrollX(.17).rotate( ()=>time/30
).color(1,0,.75) )
.diff( shape(99,.2,.002).scrollX(.22).rotate( ()=>time/40
).color(1,0,.75) )
.diff( shape(99,.1,.002).scrollX(.27).rotate( ()=>time/50
).color(1,0,.75) )

.modulateScale(
  shape(240,.5,0).scrollX(.05).rotate( ()=>time/10 )
  , ()=>(Math.sin(time/3)*.2)+.2 )

.diff( shape(72,.18,.51) )

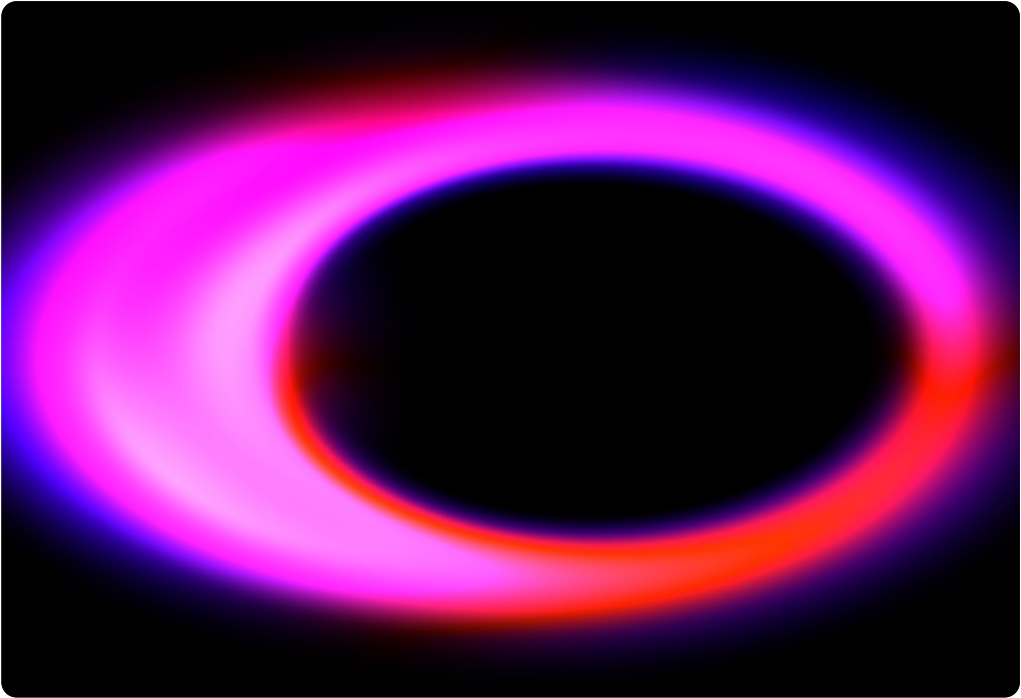
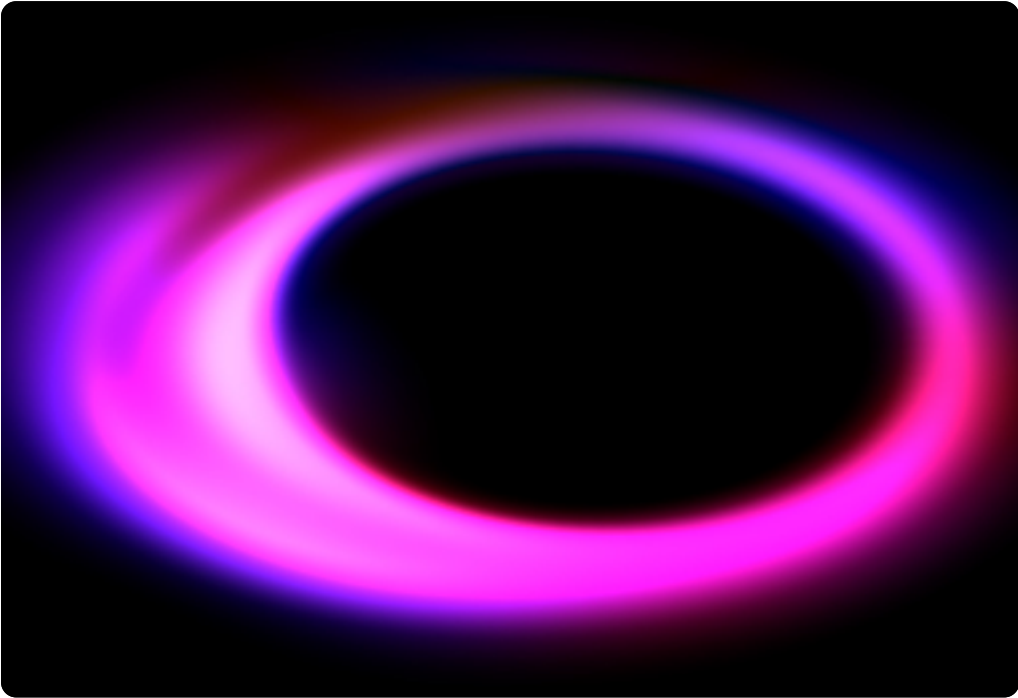
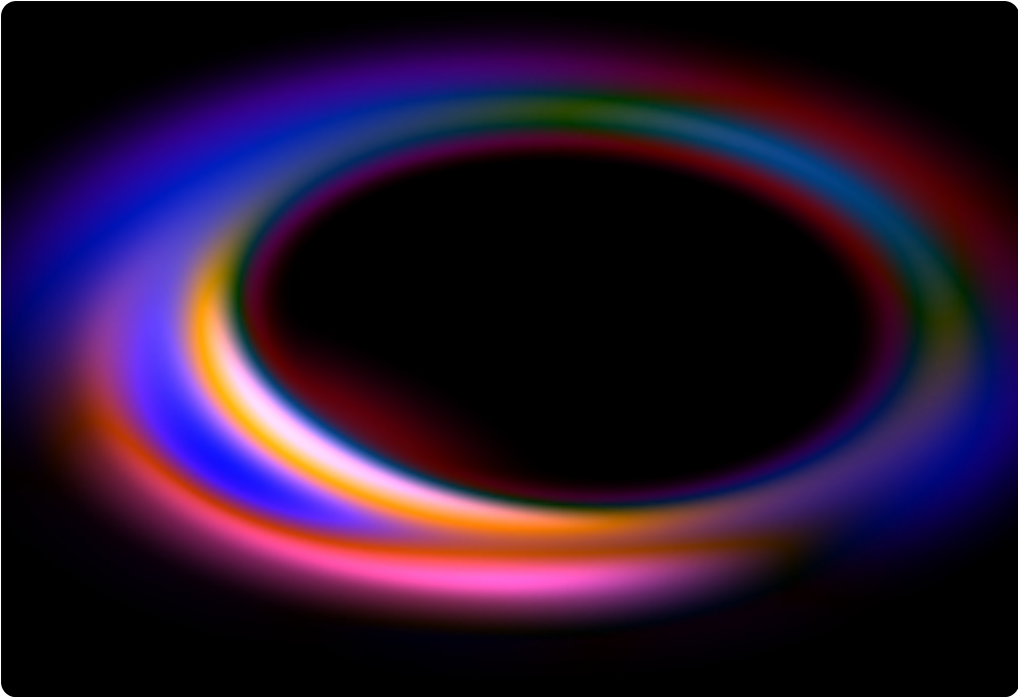
.scale(1.5,.57,1)

.out()
```

O patch se organiza em torno de formas geométricas (**shape**), deslocamento X (**scrollX**), rotação (**rotate**), colorização (**color**) e diferença (**diff**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.43** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

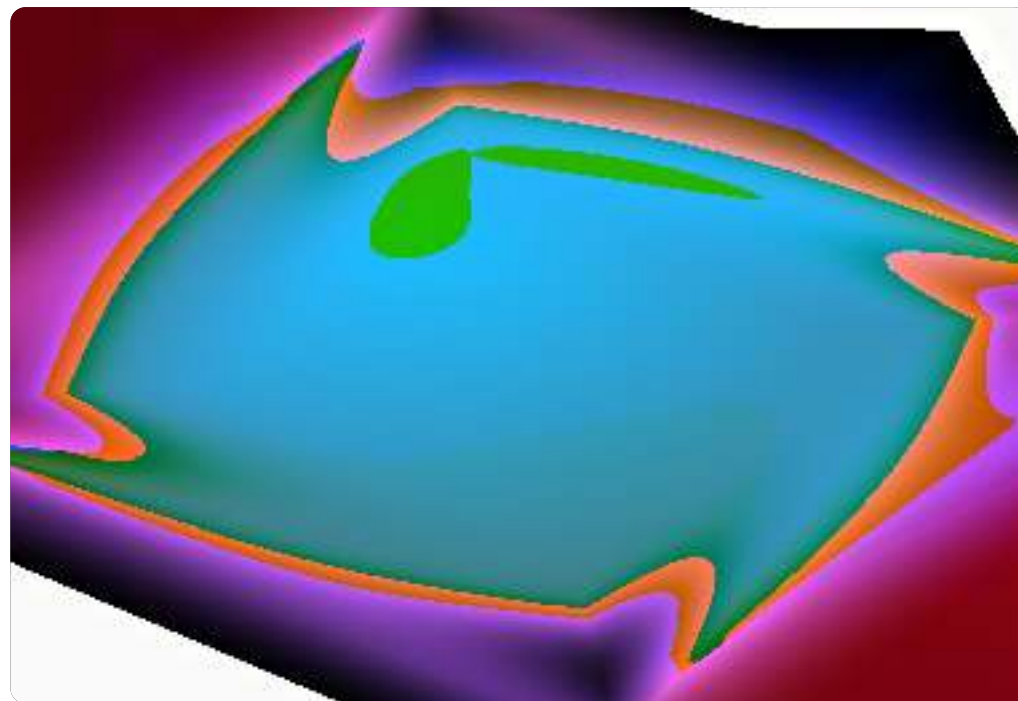
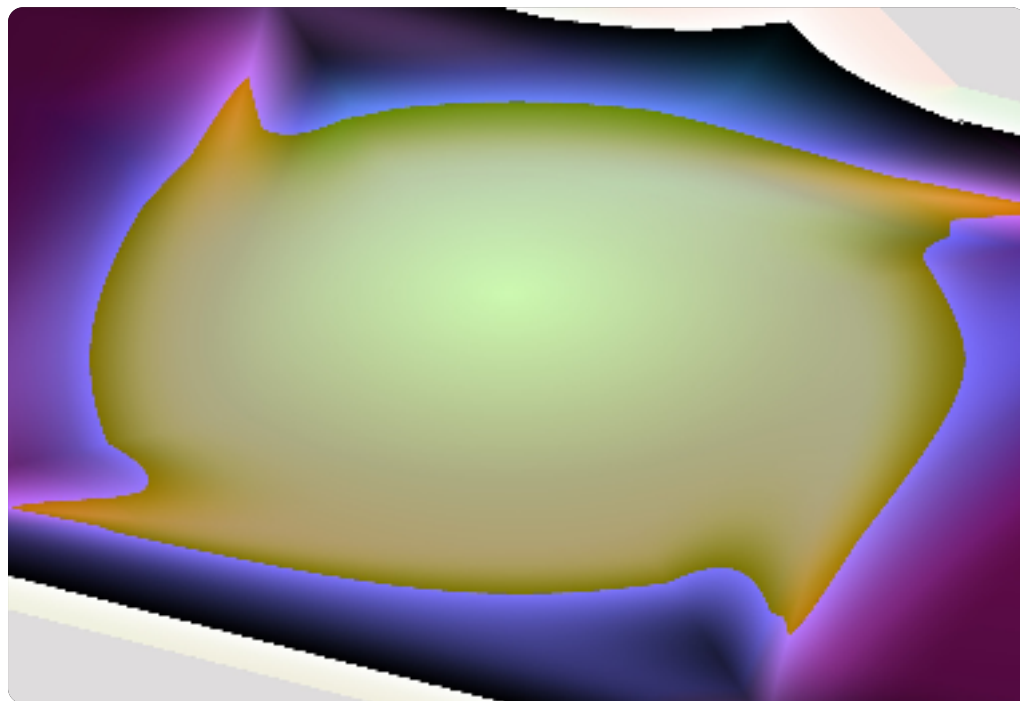
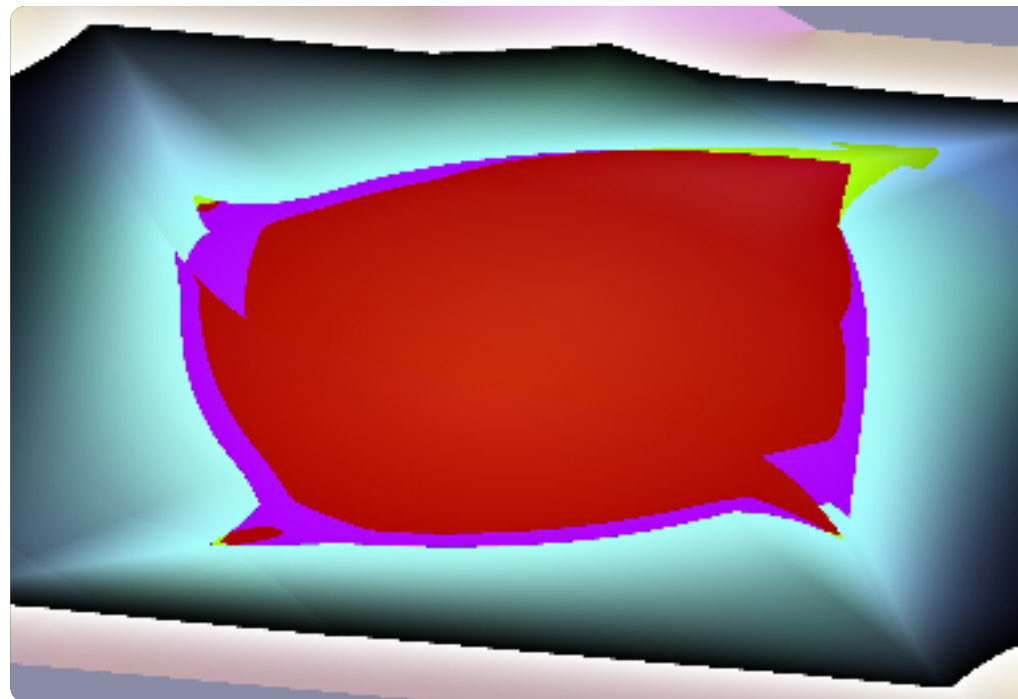
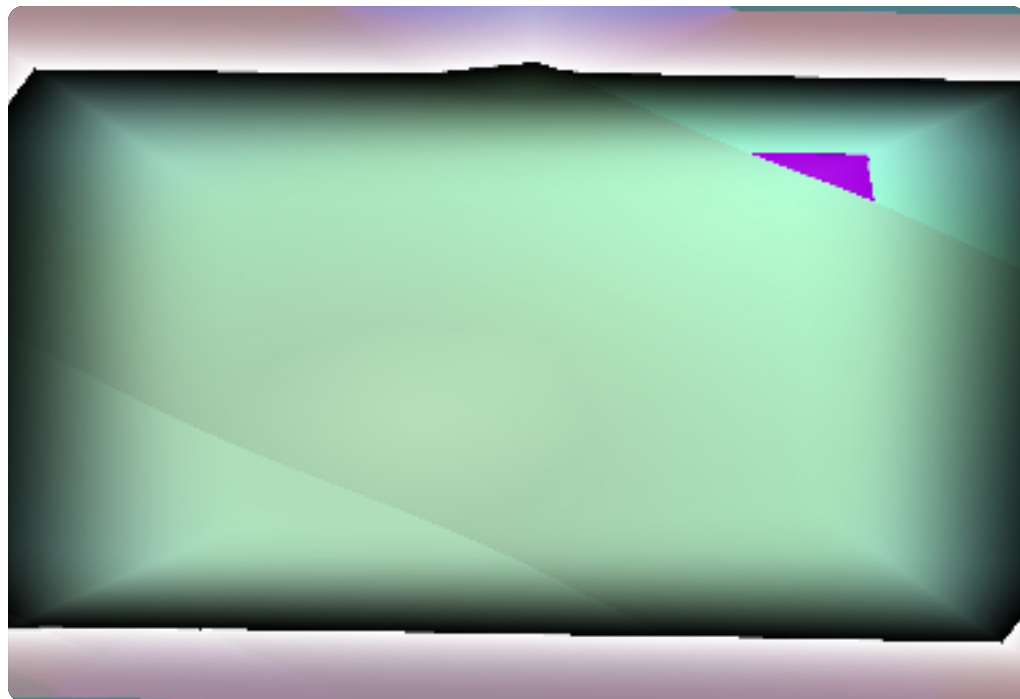
speed = 0.45
noise(0.25, 0.045)
  .color(1, 0, 0)
  .mult(shape(72, 0.45, 0.45, ))
  .add(noise(0.50, 0.050))
  .color(0, 0, 1)
  .mult(shape(72, 0.45, 0.55)))
  .add(noise(0.75, 0.045))
  .color(1, 0, 0)
  .mult(shape(72, 0.55, 0.45)))
  .add(noise(1.00, 0.050))
  .color(0, 0, 1)
  .mult(shape(72, 0.55, 0.45)))
  .add(noise(1.25, 0.055))
  .color(0, 0, 1)
  .mult(shape(72, 0.55, 0.45)))
  .diff(noise(1.50, 0.020))
  .invert()
  // .color( ()=>Math.sin(time/22)*0.5+0.333 ,
  ()=>Math.sin(time/31)*2-2, ()=>Math.sin(time/21)*0.5+0.333 )
  .mult(shape(108, 0.55, 0.3)))
  .color(() => Math.sin(time / 37) * 0.5 + 1.4, 0.5, () =>
Math.sin(time / 44) * 0.5 + 1.4)
  .modulate(shape(100, 0.33, 0.41), 0.4)
  .rotate(() => time / -27)
  .scale(-1, -1, 0.99)
  .out()

```

O patch se organiza em torno de formas geométricas (**shape**), colorização (**color**), ruído (**noise**), multiplicação de camadas (**mult**) e soma de camadas (**add**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **mult**, **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.45** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

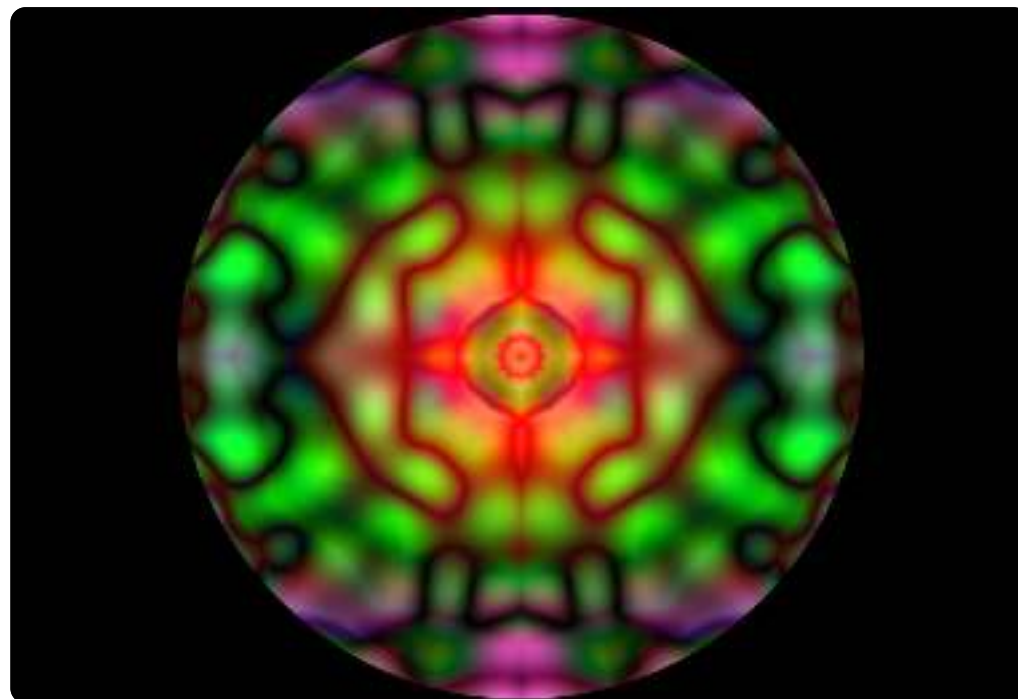
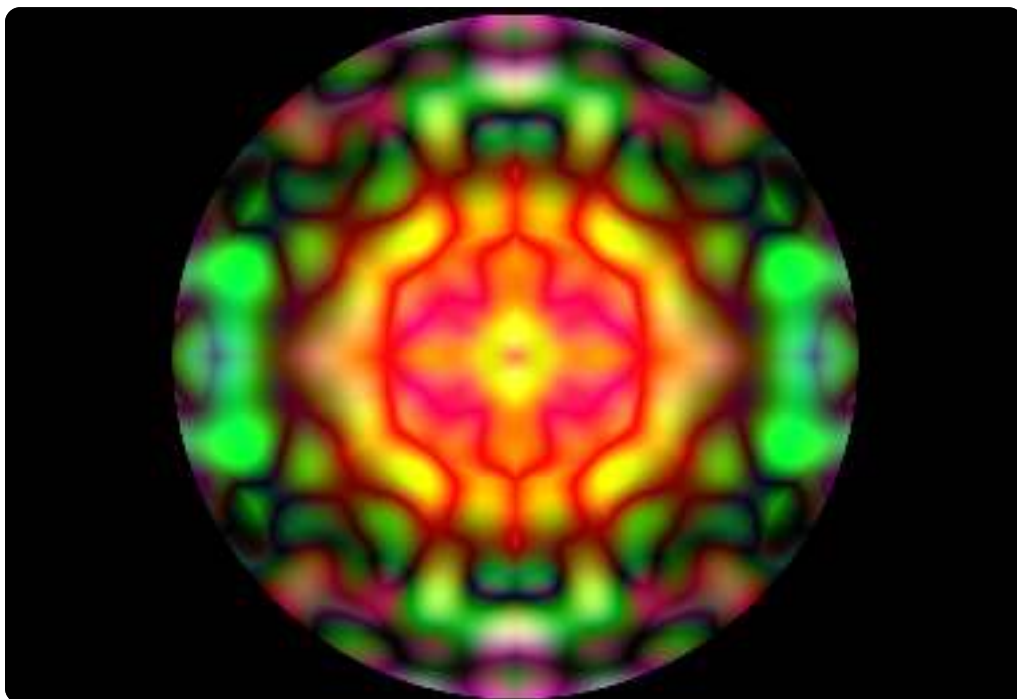
speed = 0.25
voronoi(0.25, 0.045,0.5)
  .color(1, 0, 0)
  .mult(shape(4, 0.45, 0.45, 0.5 ))
  .add(voronoi(0.50, 0.050,0)
    .color(0, 0, 1)
    .mult(shape(4, 0.45, 0.55)))
  .add(voronoi(0.75, 0.045,0.5)
    .color(1, 0, 0)
    .mult(shape(2, 0.55, 0.45)))
  .add(voronoi(1.00, 0.050,0.5)
    .color(0, 0, 1)
    .mult(shape(5, 0.55, 0.45)))
  .add(voronoi(1.25, 0.055,0.5)
    .color(-1, 1, 1)
    .mult(shape(4, 0.55, 0.45)))
  // .diff(voronoi(1.50, 0.020,0.5)
  // .invert()
  // .color( ()=>Math.sin(time/22)*0.5+0.333 ,
  ()=>Math.sin(time/31)*2-2, ()=>Math.sin(time/21)*0.5+0.333 )
  // .mult(shape(108, 0.55, 0.3)))
  //
  // .color(() => Math.sin(time / 37) * 0.5 + 1.4, 0.5, () =>
Math.sin(time / 44) * 0.5 + 1.4)
  .modulateRotate(shape(4, 0.33, 0.5),
  ()=>Math.sin(time/18)*Math.PI )
  .rotate(() => time / -18)
  .scale(-1, -1, 1)
  .colorama( ()=>Math.sin(time/9) * 0.5 - 0.5)
  .out()

```

O patch se organiza em torno de formas geométricas (**shape**), Voronoi (**voronoi**), colorização (**color**), multiplicação de camadas (**mult**) e soma de camadas (**add**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **mult**, **add** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateRotate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.25** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

speed=0.16

noise(6,.04)
.mult( osc(9,0, ()=>Math.sin(time/1.5)+2 ) )
.mult(
  noise(9,.025).brightness(1.2).contrast(2)
  .mult( osc(9,0, ()=>Math.sin(time/3)+13 ) )
  .kaleid(4).scale(0.75,1,16/9)
)
.diff(
  noise(15,.02).brightness(.2).contrast(1.3)
  .mult( osc(9,0, ()=>Math.sin(time/5)+2 ) )
  .rotate( ()=>time/45 )
  .kaleid(8).scale(0.75,1,16/9)
)
.scale( ()=>Math.sin(time/27)*.50+.60 )
.modulateScale(
  osc( ()=>Math.sin(time/36)*4+5 ,0.0052,0).mult(
osc(3,0,0).rotate(3.14/2) )
  .rotate( ()=>time/25 ).scale(.39).scale(1,.6,1).invert()
  , ()=>Math.sin(time/27)*1.5+3 )
  .kaleid(12).scale(0.72,1,16/9)

.diff(
  noise(6,.027).brightness(.2).contrast(1.3)
  .mult( osc(9,0, ()=>Math.sin(time/5)+2 ) )
  .rotate( ()=>time/81 )
  .kaleid(2).scale(0.52,1,16/9)
)

.mult( shape(100,.9,.005).scale(0.612,1,16/9) )

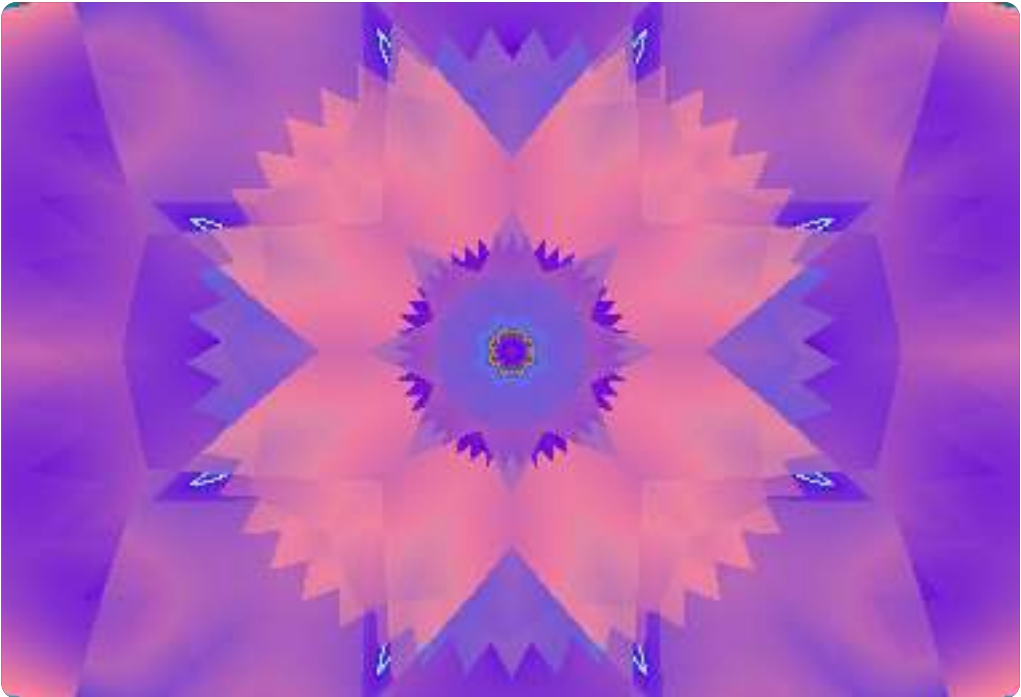
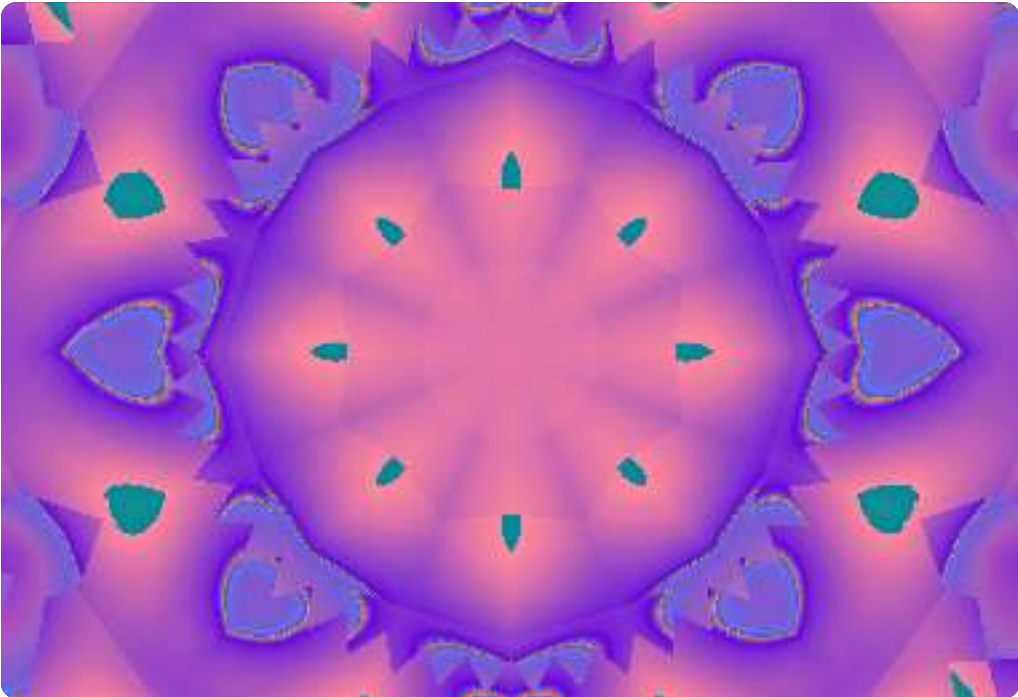
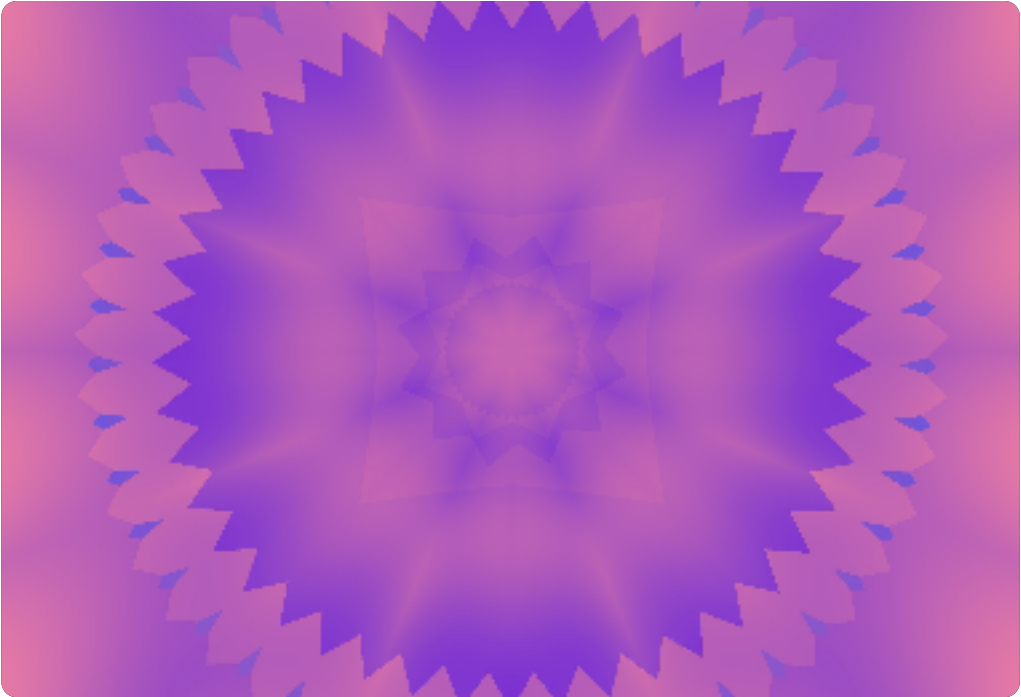
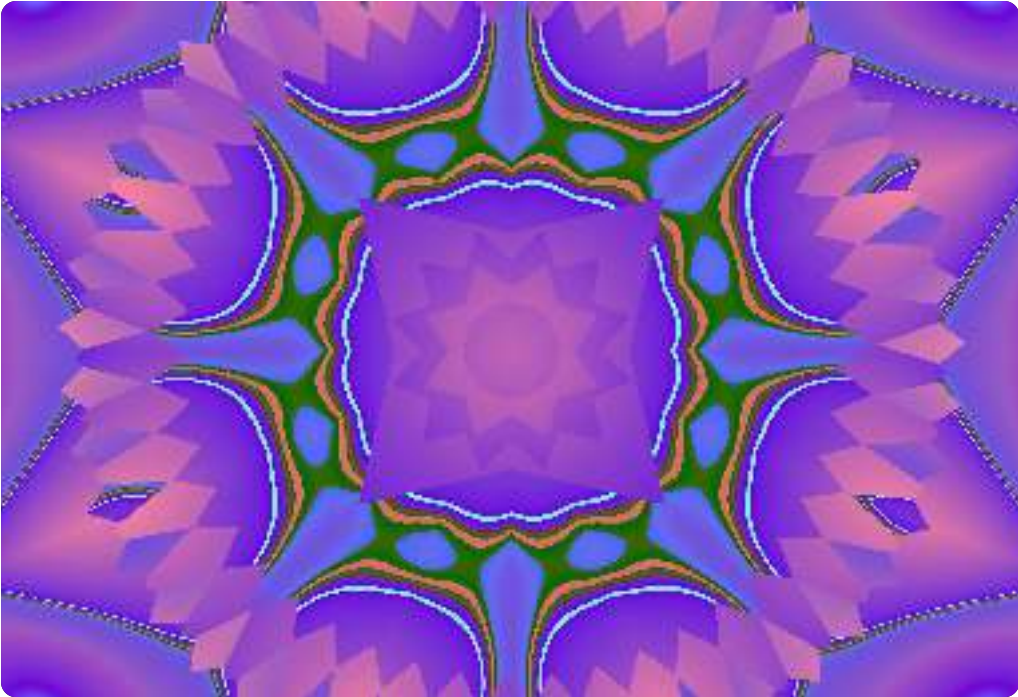
.out()

```

O patch se organiza em torno de escala (**scale**), multiplicação de camadas (**mult**), osciladores (**osc**), rotação (**rotate**) e kaleidoscópio (**kaleid**). **osc** entra em **kaleid**, multiplicando eixos de simetria até o quadro parecer um ornamento vivo. A composição usa **mult**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.16** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra.





```

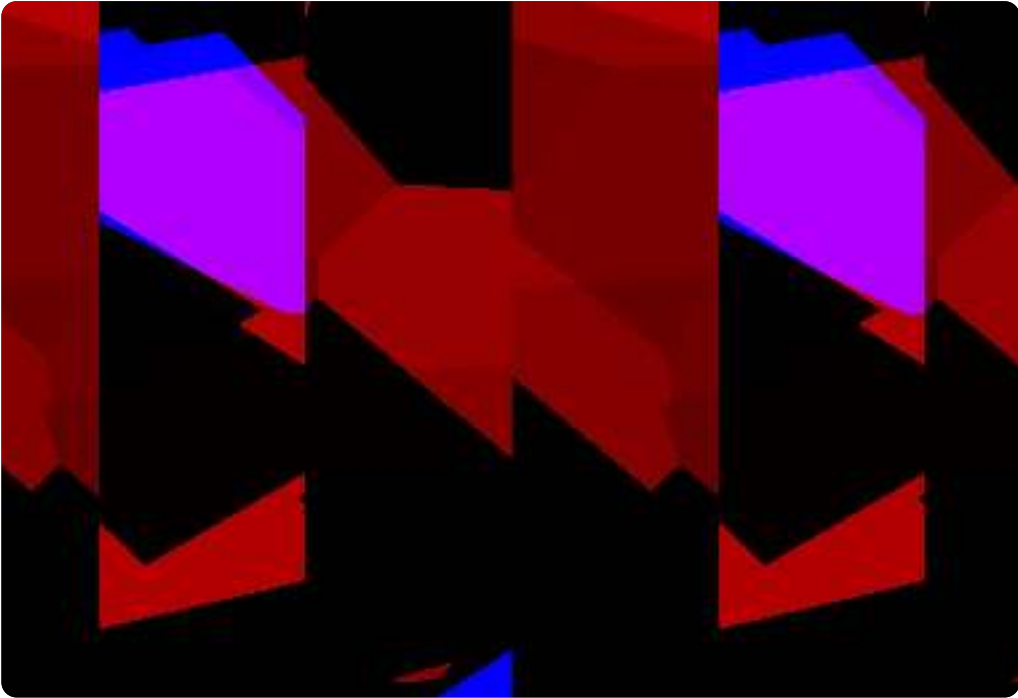
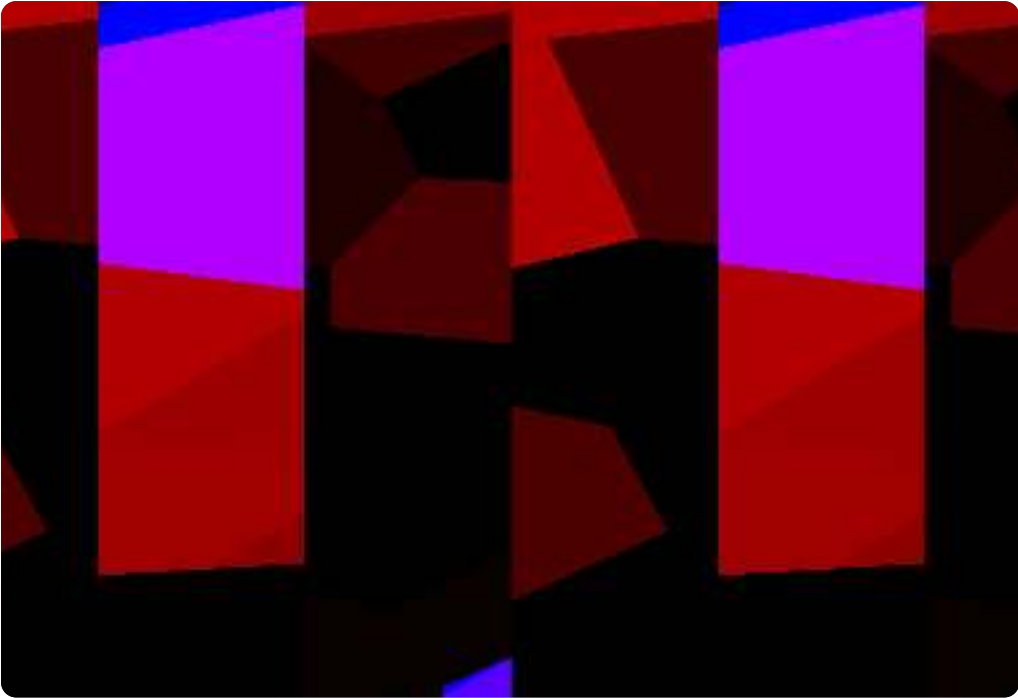
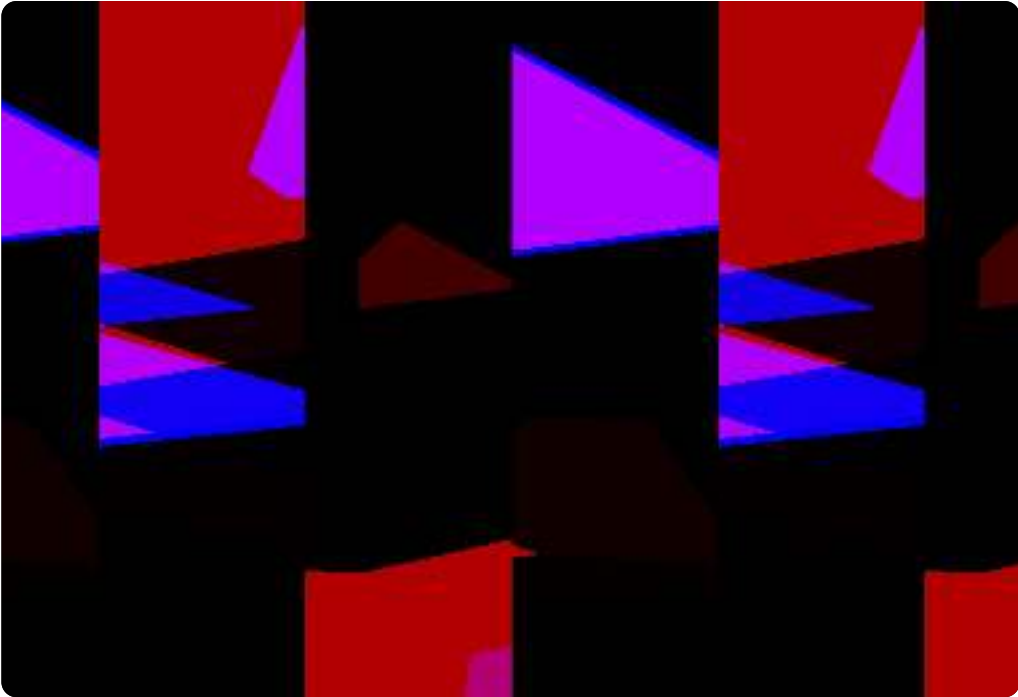
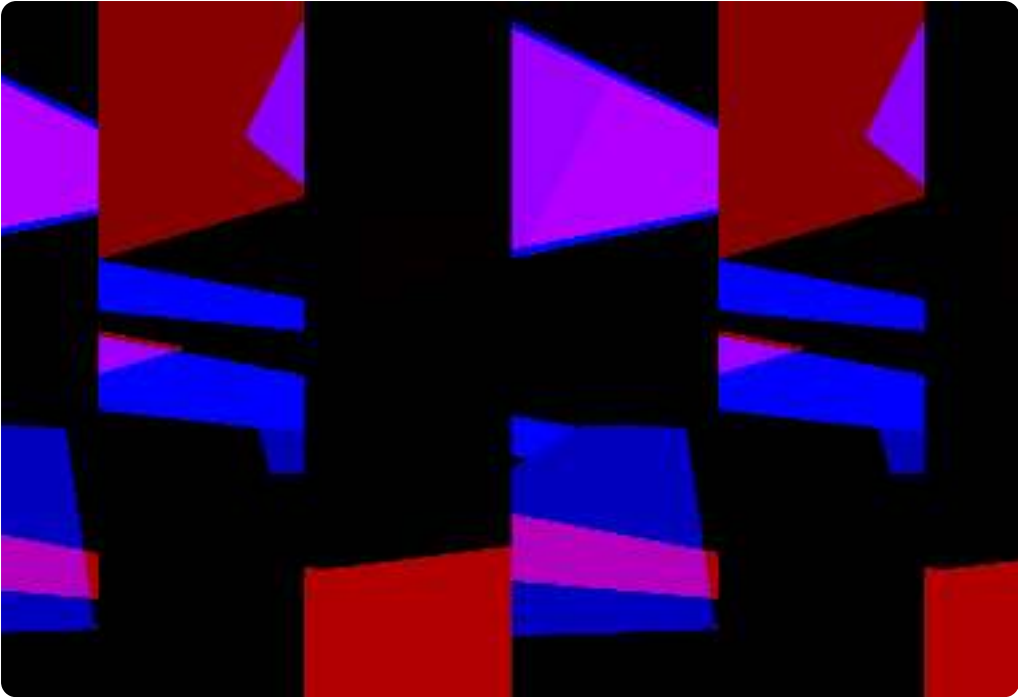
speed=0.52
voronoi(2,0.04,0.02).color(0.2,0,0.77).kaleid(4)
.add( voronoi(4,0.04,0.02).color(0.2,0,0.7).kaleid(8) )
.diff( voronoi(3,0.03,0.02).color(0.2,0,0.7).kaleid(12).rotate(
()=>time*-0.01 ) )
.diff( noise(5,0.07,0).color(0.2,0,0.7).kaleid(8) )
.modulateScale( osc(5,-0.05,0.02).kaleid(4), 0.9 )
.diff(
voronoi(5,0.0333,0.02).color(0.2,0,0.7).kaleid(45).contrast(1.007
5).rotate( ()=>time*0.1 ) )
.scale(1,1,16/9)
.colorama(0.4)
.out()

```

O patch se organiza em torno de kaleidoscópio (**kaleid**), colorização (**color**), Voronoi (**voronoi**), diferença (**diff**) e rotação (**rotate**). **osc** entra em **kaleid**, multiplicando eixos de simetria até o quadro parecer um ornamento vivo. A composição usa **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.52** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

await loadScript("https://hyper-hydra.glitch.me/hydra-
outputs.js")
oS.setLinear()
//oS.setNearest()

await loadScript("https://hyper-hydra.glitch.me/hydra-
outputs.js")
oS.setLinear()
//oS.setNearest()

speed=0.9
voronoi(2.5,0.0333,0).thresh(0.5,0.1)
//.color(1,0,0)
//.diff( voronoi(3.5,0.0333,0).thresh(0.5,0.1)  )
.mult( voronoi(3.4,0.03319,0).thresh(0.5,0.1)  )

.color(0.7,0,0)

.add( voronoi(3.33,0.0313,0).thresh(0.81,0.025).color(0,0,1) )
.add( voronoi(4.5,0.0322,0).thresh(0.81,0.025).color(0,0,1) )
//.add( voronoi(5.4,0.03339,0).thresh(0.81,0.025).color(0,0,1) )

.modulateScrollX(
  osc(18.845,0).thresh(0.5,0)
)

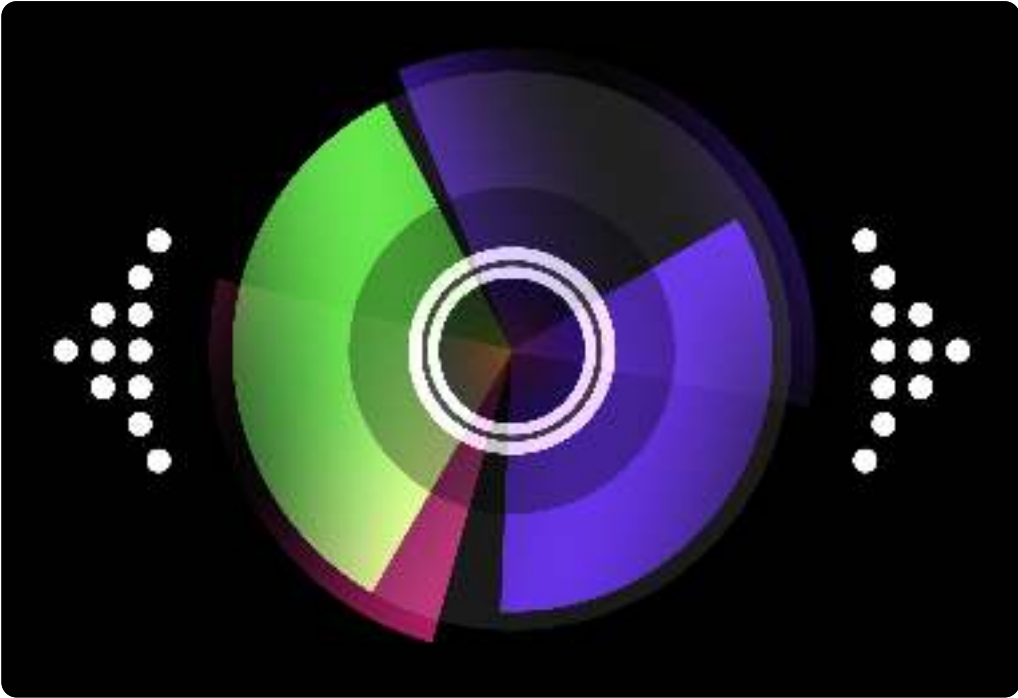
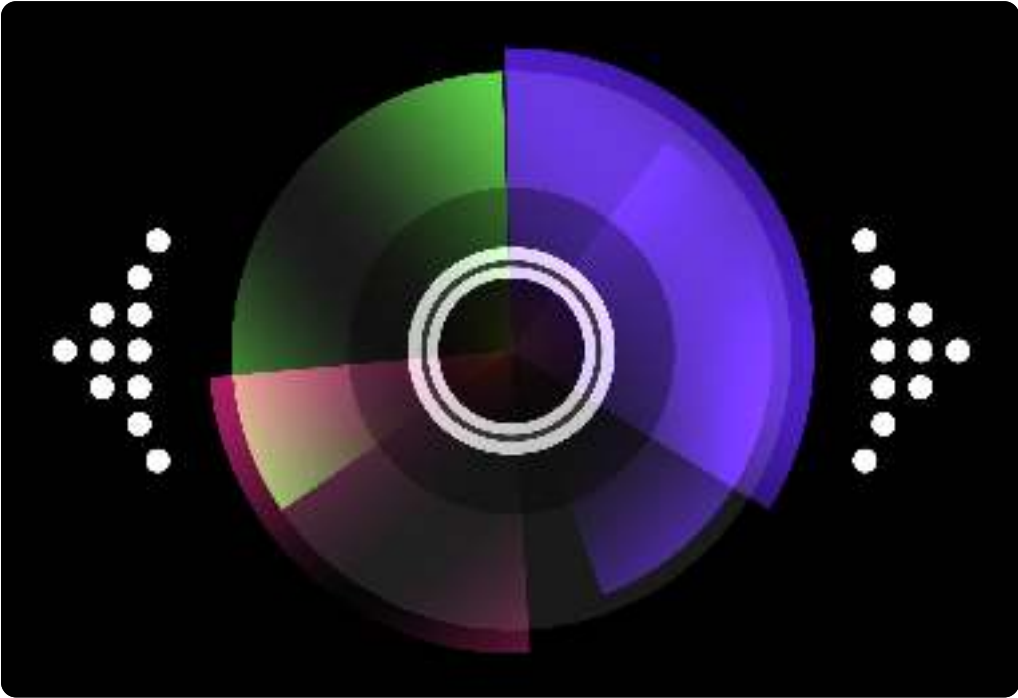
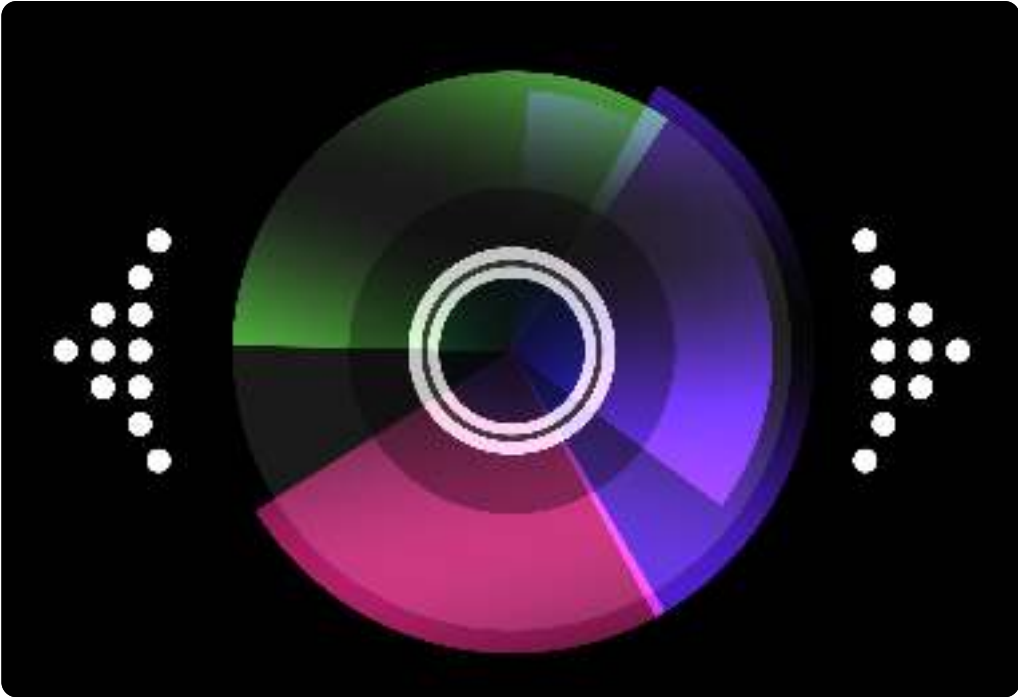
.out(o0)
render(o0)

```

O patch se organiza em torno de limiar (**thresh**), Voronoi (**voronoi**), colorização (**color**), soma de camadas (**add**) e modulação de scroll X (**modulateScrollX**). Osciladores viram padrões binários via **thresh**, e a geometria resultante alimenta o restante da cadeia. A composição usa **mult**, **add** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScrollX**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.9** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

solid(.1,.1,.1).mult( shape(100,0.3,0.001) )

.add(
osc(20,0.01,0).rotate(Math.PI/2).color(0.7,0.1,0.4).contrast(0.95)
).rotate( ()=> time*-0.0225 )
.mult( shape(100,0.325,0.001)
.mult( shape(1,0,0.001).rotate(-0.8) )
.mult( shape(1,0,0.001).rotate(+0.8) )
.rotate( ()=> time*0.0225 )
))

.add(
osc(20,0.01,0).rotate(Math.PI/2).color(0.3,0.1,0.8).contrast(0.95)
).rotate( ()=> time*0.025 )
.mult( shape(100,0.325,0.001)
.mult( shape(1,0,0.001).rotate(-0.8) )
.mult( shape(1,0,0.001).rotate(-1.8) )
.rotate( ()=> time*-0.025 )
))

.add(
osc(20,0.01,0).rotate(Math.PI/2).color(0.3,0.1,0.8).rotate( ()=>
time*-0.0275 )
.mult( shape(100,0.28,0.001)
.mult( shape(1,0,0.001).rotate(-1.8) )
.mult( shape(1,0,0.001).rotate(-2.8) )
.rotate( ()=> time*0.0275 )
))

.add(
osc(20,0.01,0).rotate(Math.PI/2).color(0.3,0.8,0.2).rotate( ()=>
time*0.03 )

```

```

.mult( shape(1,0,0.001).rotate(-3.8) )
.rotate( ()=> time*-0.03 )
))

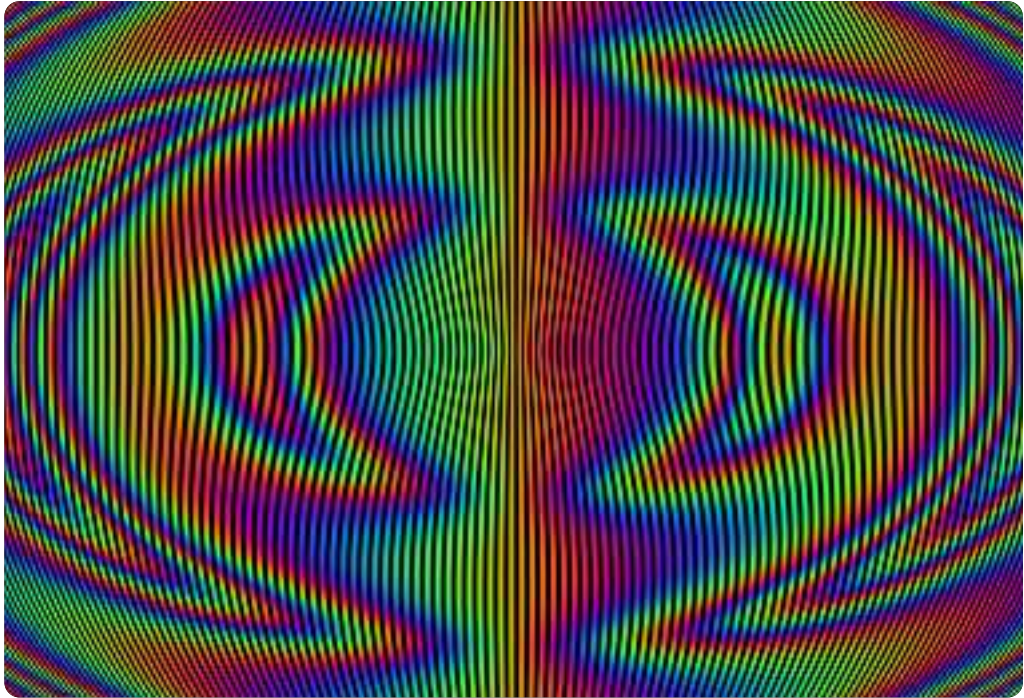
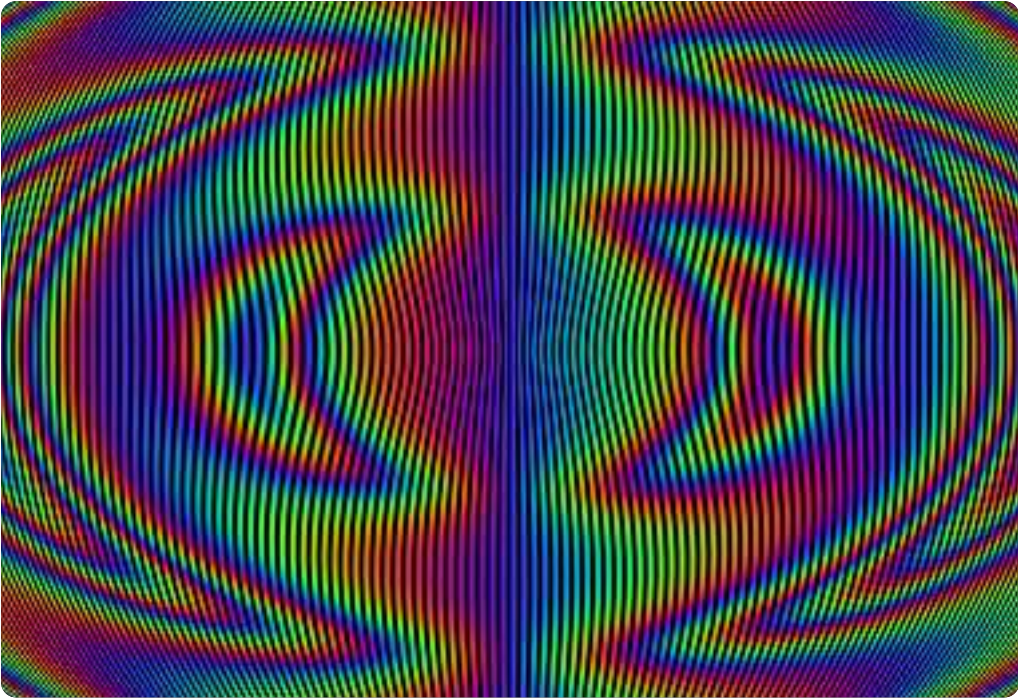
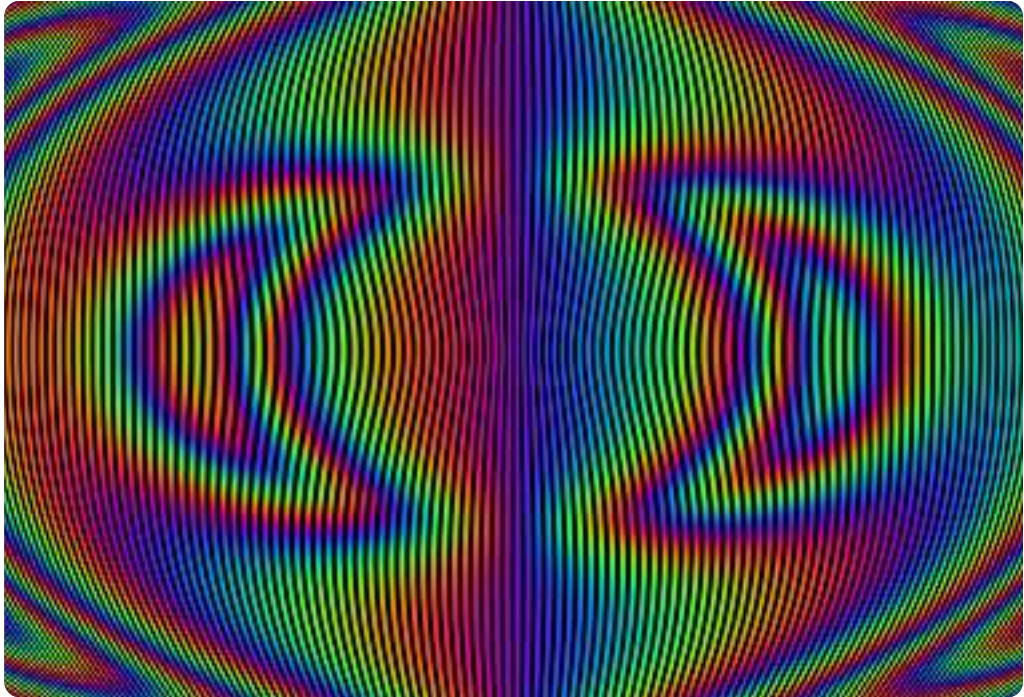
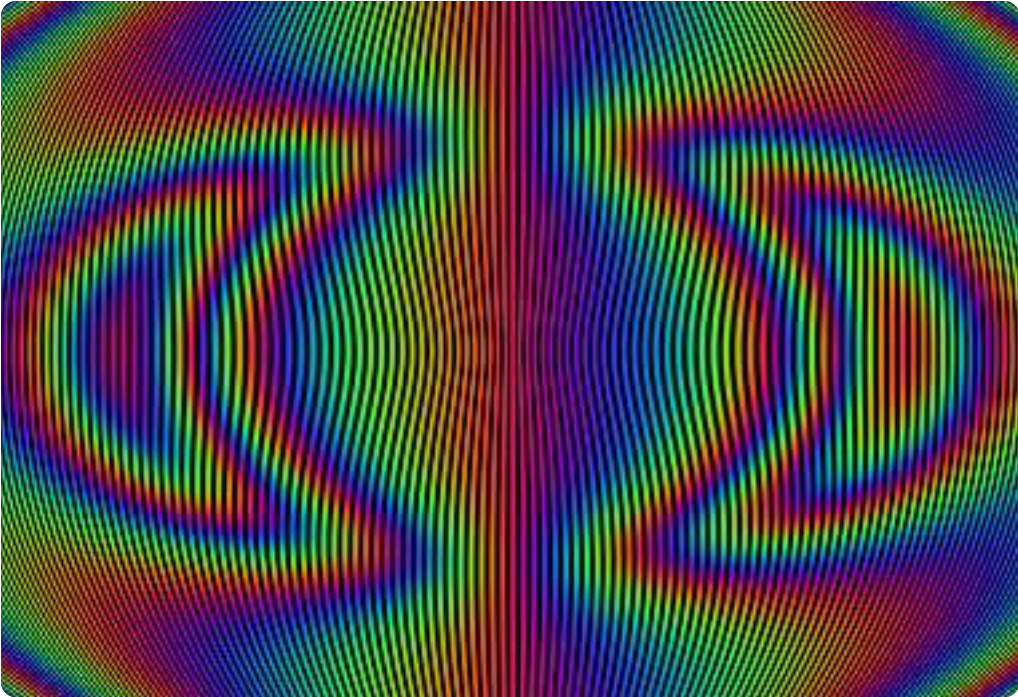
.mult( shape(100,0.175,0.001).invert(), 0.333 )
.mult( osc(40,-0.01,1).kaleid(36).mult(
shape(100,0.078,0.001).color(.6,.6,.6) ).invert() ) // iris
.add( shape(100,0.111,0.001).mult(
shape(100,0.078,0.001).invert() ), 0.7 ) // tarja branca
.mult( shape(100,0.097,0.001).invert().add(
shape(100,0.09,0.001) ), 0.75 ) // linha preta

.scale(1,1,16/9)

.add(
shape(32,0.0125,0.001).scale(1,1,16/9).scrollX(0.20,0) )
//3
.add(
shape(32,0.0125,0.001).scale(1,1,16/9).scrollX(0.22,0) )
//3
.add(
shape(32,0.0125,0.001).scale(1,1,16/9).scrollX(0.24,0) )
//3
.add(
shape(32,0.0125,0.001).scale(1,1,16/9).scrollX(0.20,0).scro
llY(0.035,0) ) //2
.add(
shape(32,0.0125,0.001).scale(1,1,16/9).scrollX(0.22,0).scro
llY(0.035,0) ) //2
.add(
shape(32,0.0125,0.001).scale(1,1,16/9).scrollX(0.20,0).scro
llY(0.070,0) ) //1

```





```

osc(2222,-0.01,0).thresh(0.5,0.5)
.modulateScale( osc(5,-0.0007,0).kaleid(250),2 )

.mult(
  osc(60,-0.01,2)
  //.rotate( )=>time/30 )
.modulateScale( osc( )=>Math.sin(time/30)*14+15
,-0.001,0).kaleid(250),2 )
)

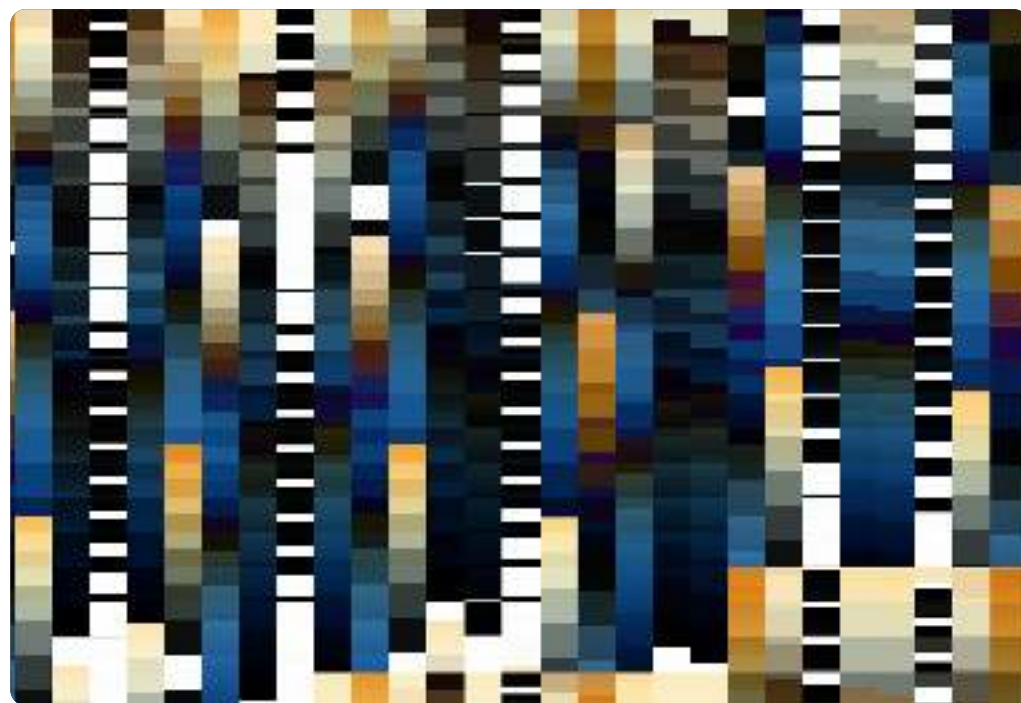
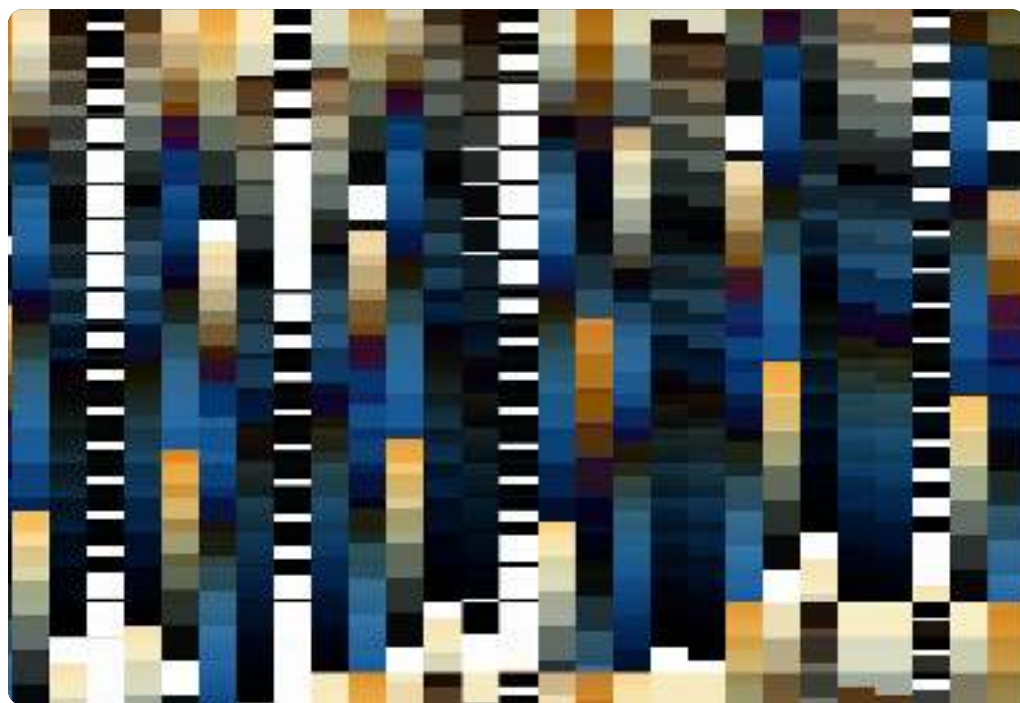
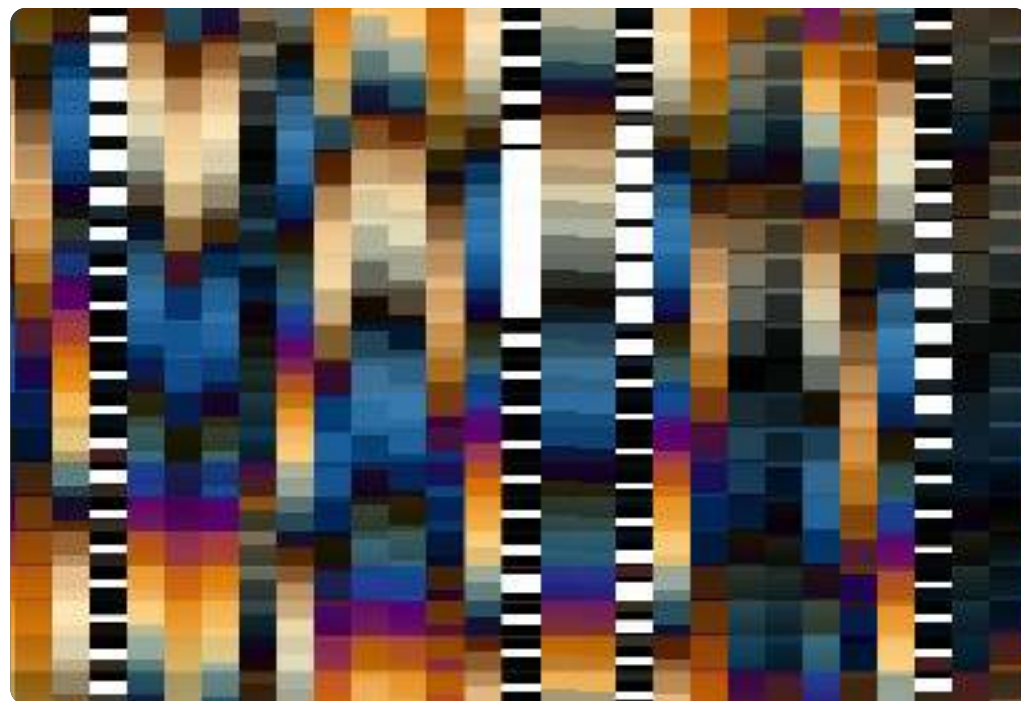
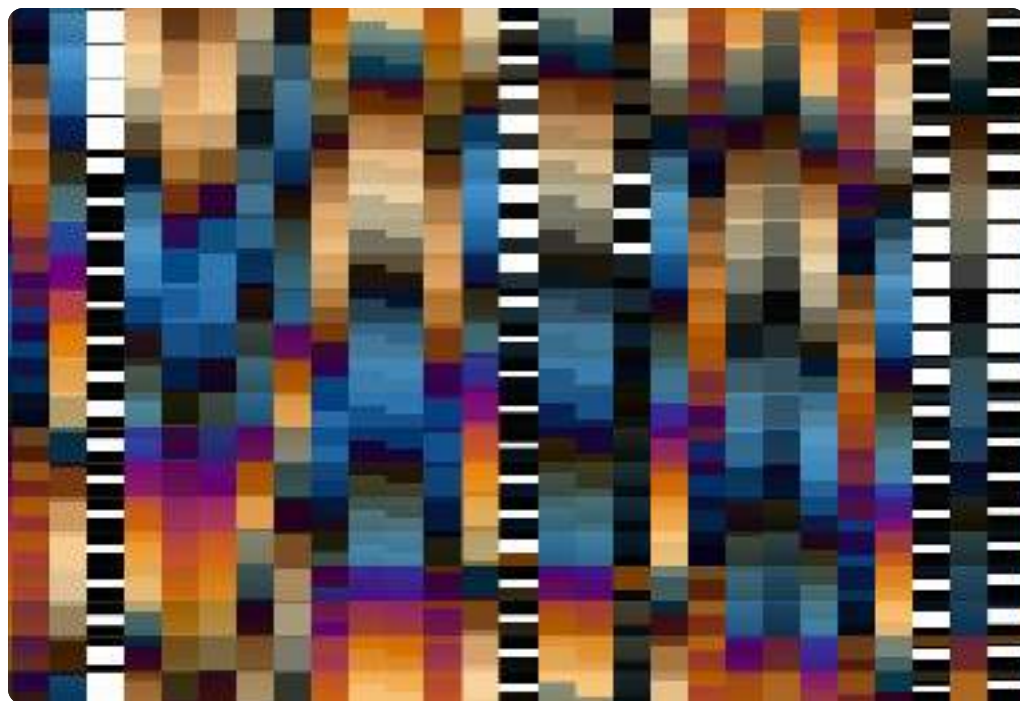
.out()

```

O patch se organiza em torno de osciladores (**osc**), modulação de escala (**modulateScale**), kaleidoscópio (**kaleid**), limiar (**thresh**) e multiplicação de camadas (**mult**). Osciladores viram padrões binários via **thresh**, e a geometria resultante alimenta o restante da cadeia. A composição usa **mult** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra.





```

speed=0.36

osc(10,-0.015,0)
.rotate(Math.PI/2)
.pixelate(10)
.mult( osc(4,0.00333,12).rotate(Math.PI/2) ) //color
.modulateScrollY( osc(20,0.001,0).pixelate(33) ,1)

.out(o0)

src(o0)
.diff(
  src(o0).pixelate(3)
)
.colorama(-0.01)
.out(o1)

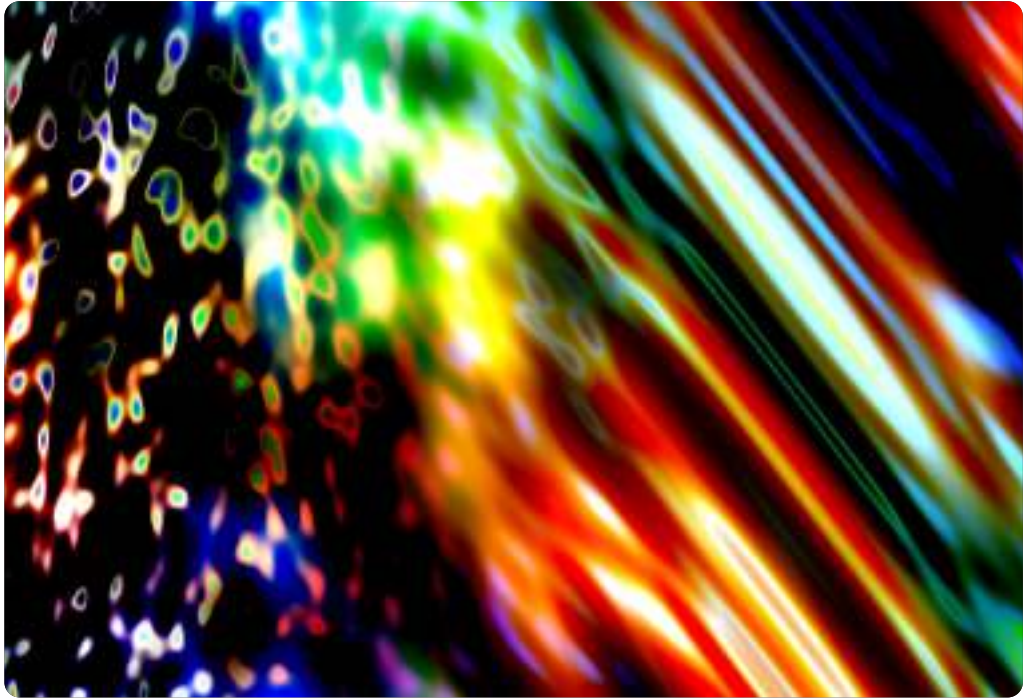
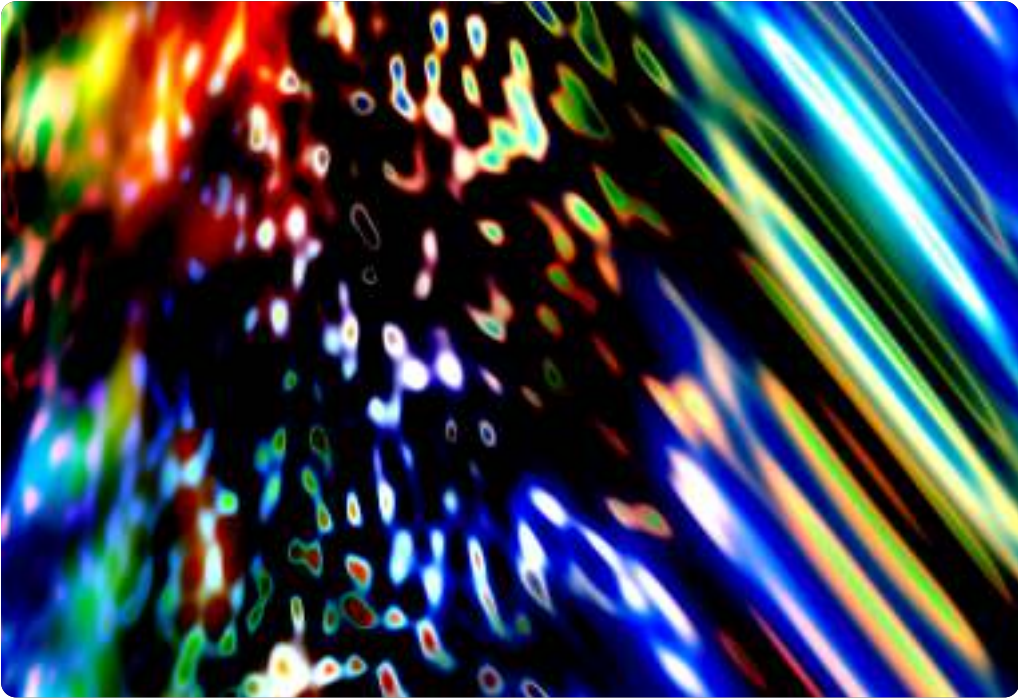
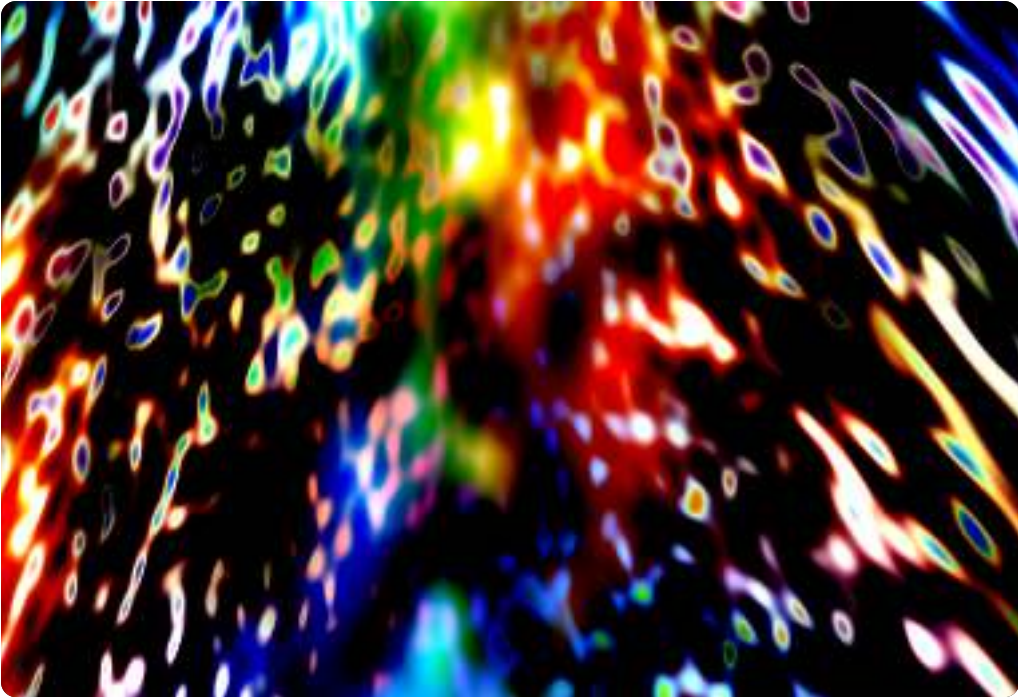
render(o1)

```

O patch se organiza em torno de pixelização (**pixelate**), osciladores (**osc**), rotação (**rotate**), fontes externas (**src**) e modulação de scroll Y (**modulateScrollY**). A composição usa **mult**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScrollY**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.36** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**src**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.



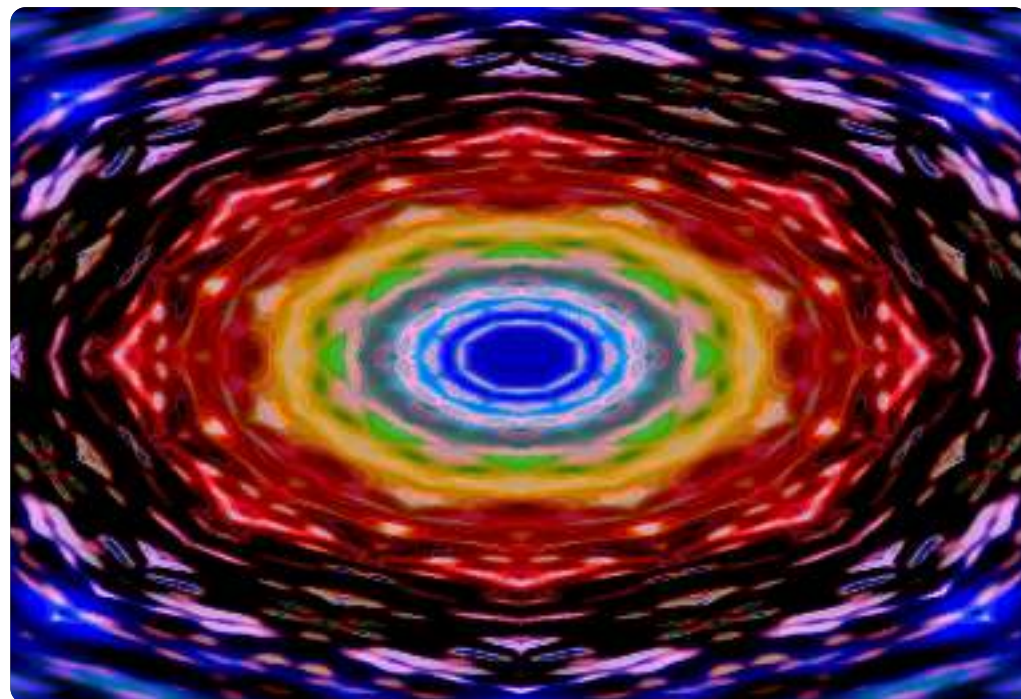
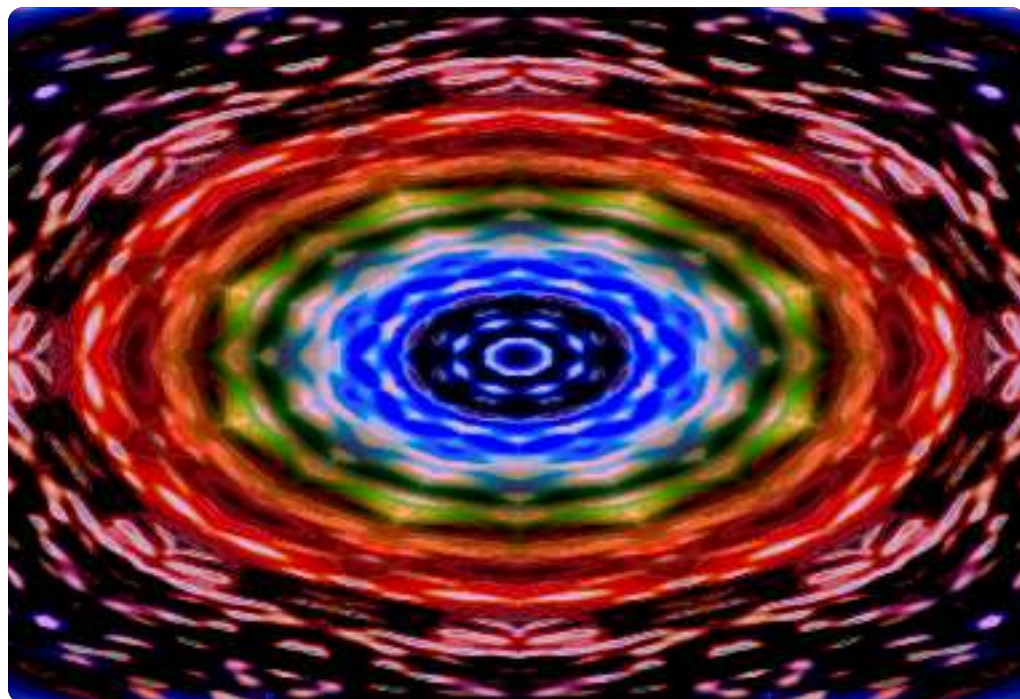
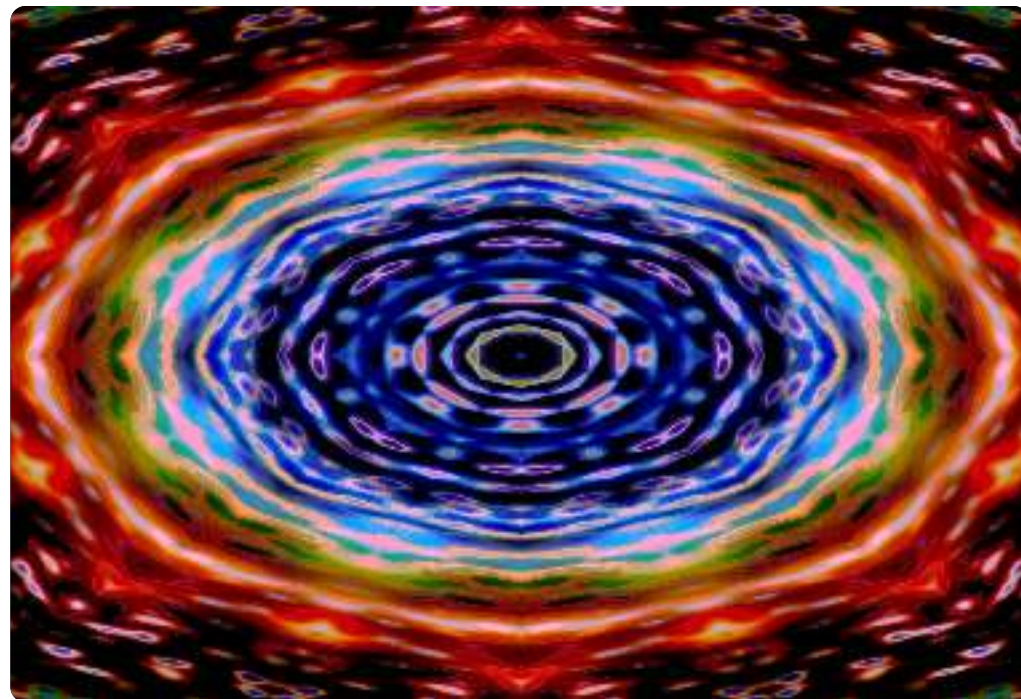
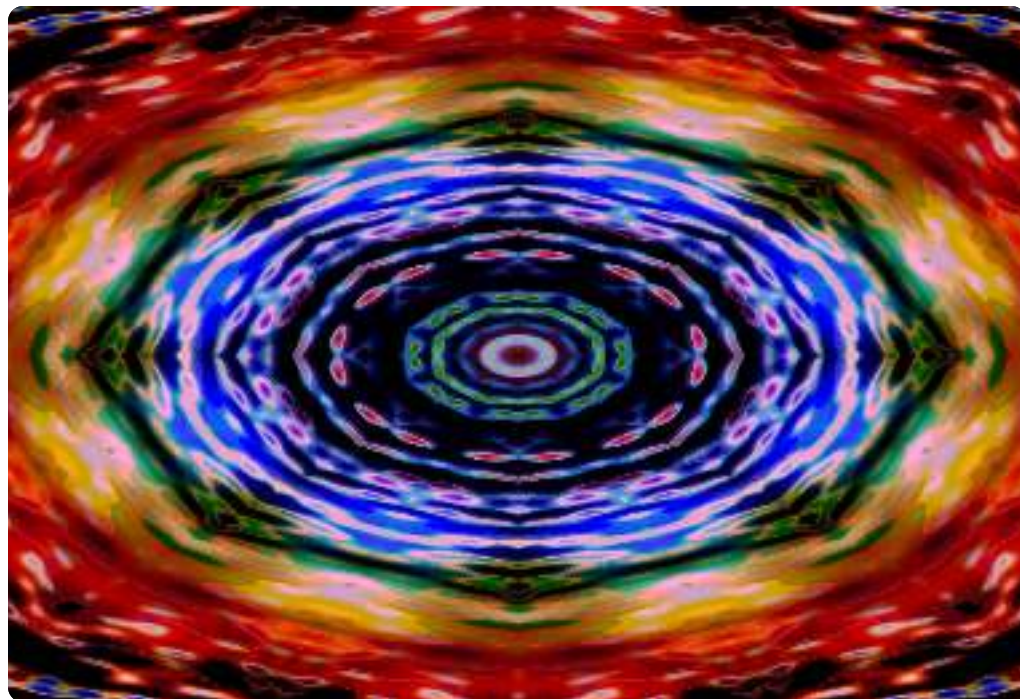


```
speed=0.54
noise(9,0.02).contrast(3.5)
.mult( osc(3,-0.2,1) )
.diff( osc(3,-0.1,1).rotate(Math.PI/2) )
.invert()
.add( noise(3,0.09).rotate( ()=>time/22 ) , 1 )
.scale(0.74)
.modulate( osc( ()=>Math.sin(time/99)*0.5+3 ,0.01,1),1.5 )
.out()
```

O patch se organiza em torno de osciladores (**osc**), rotação (**rotate**), ruído (**noise**), modulação (**modulate**) e contraste (**contrast**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo. A composição usa **mult**, **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.54** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra.





```

speed=-0.16
noise(8.81,0.02).contrast(3.5)
.mult( osc(3,-0.2,1) )
.diff( osc(3,-0.1,1).rotate(Math.PI/2) )
.invert()
.add( noise(3,0.09).rotate( ()=>time/22 ) , 1 )
.scale(0.55)
.modulate( osc( ()=>Math.sin(time/99)*0.5+3 ,0.015,1),1.25 )
.out(o0)

src(o0)
.blend( src(o0).scale(1.005).scrollX(0.001).color(1,0,1),0.222 )
.blend(
src(o0).scale(1.005).scrollX(0.002).color(1.3,0.3,1.3).modulate(s
rc(o0),0.14),0.2 )
.kaleid(8)
.out(o1)

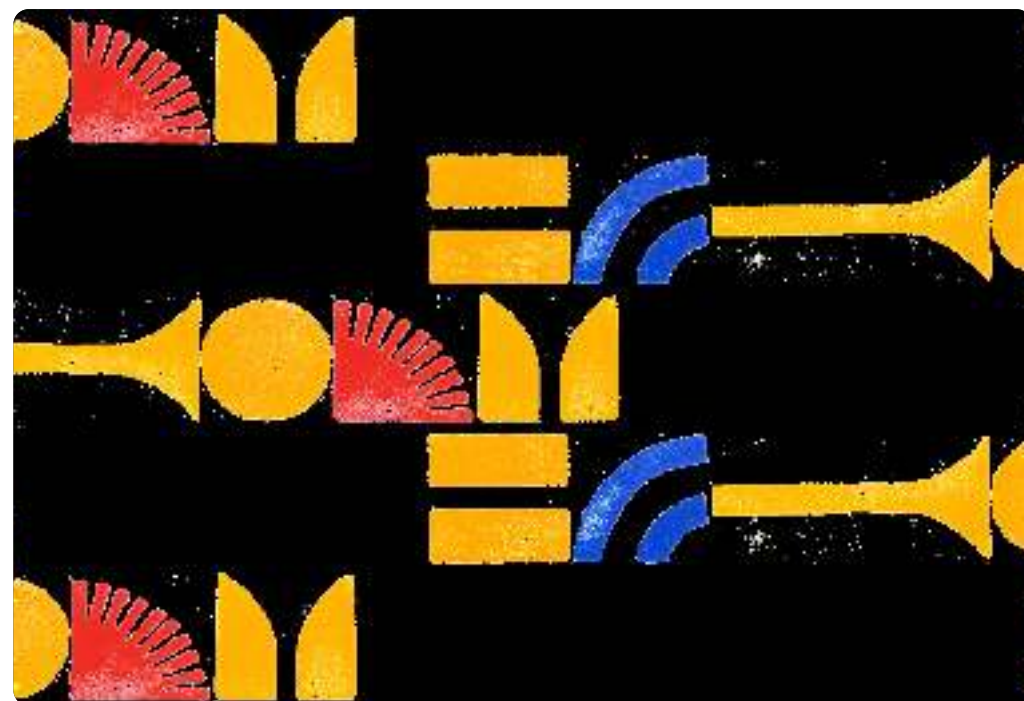
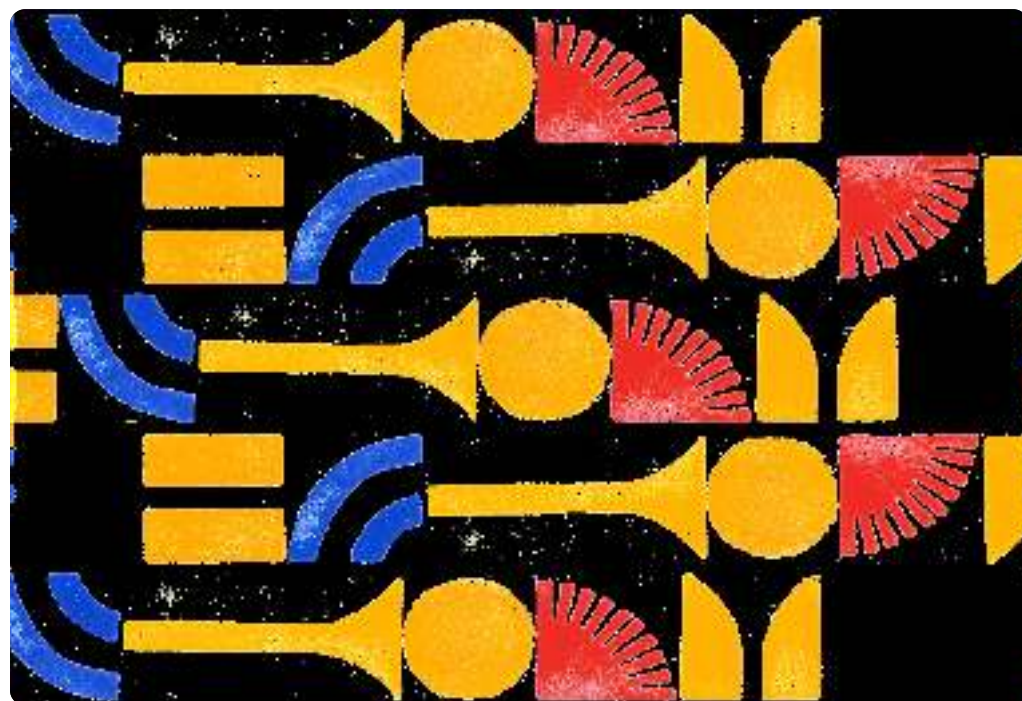
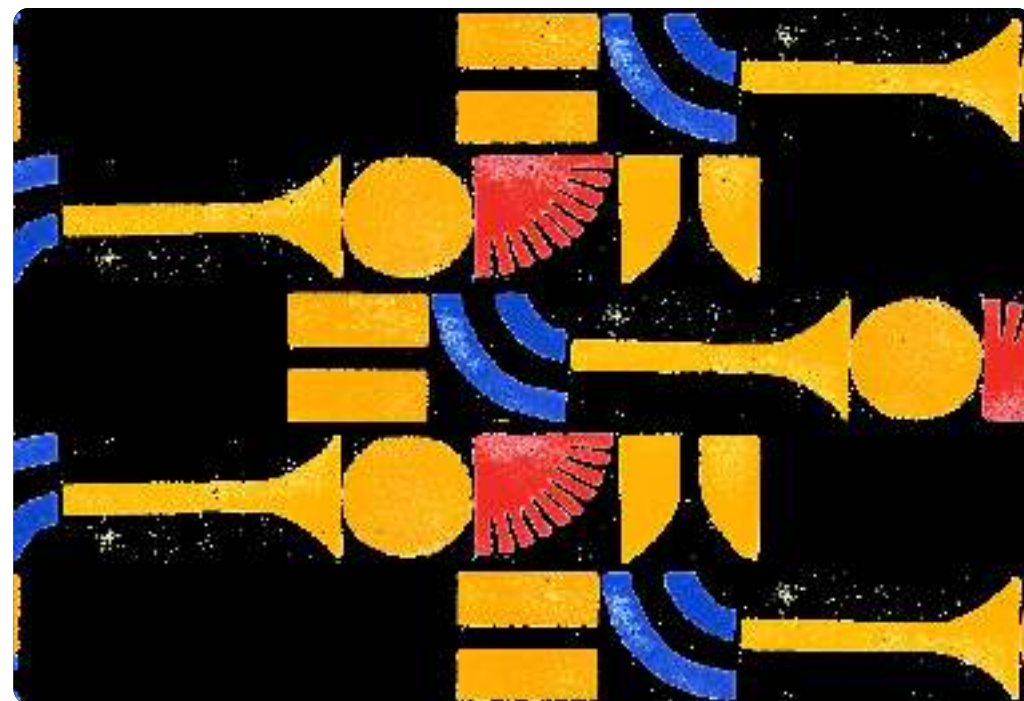
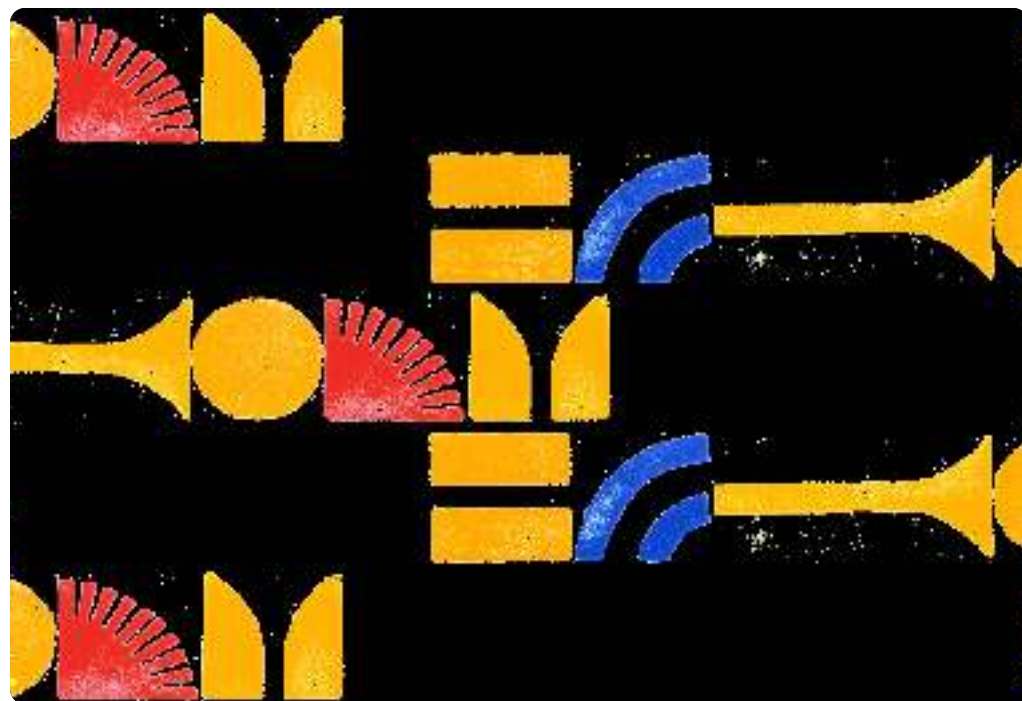
render(o1)

```

O patch se organiza em torno de fontes externas (**src**), escala (**scale**), osciladores (**osc**), modulação (**modulate**) e deslocamento X (**scrollX**). **osc** entra em **kaleid**, multiplicando eixos de simetria até o quadro parecer um ornamento vivo. A composição usa **mult**, **add**, **diff**, **blend** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**src**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```
await loadScript("https://hyper-hydra.glitch.me/hydra-blend.js")
await loadScript("https://hyper-hydra.glitch.me/hydra-wrap.js")
hydraWrap.setNoWrap()
s0.initImage("https://alexandrerangel.art.br/temp/linha-1.png")

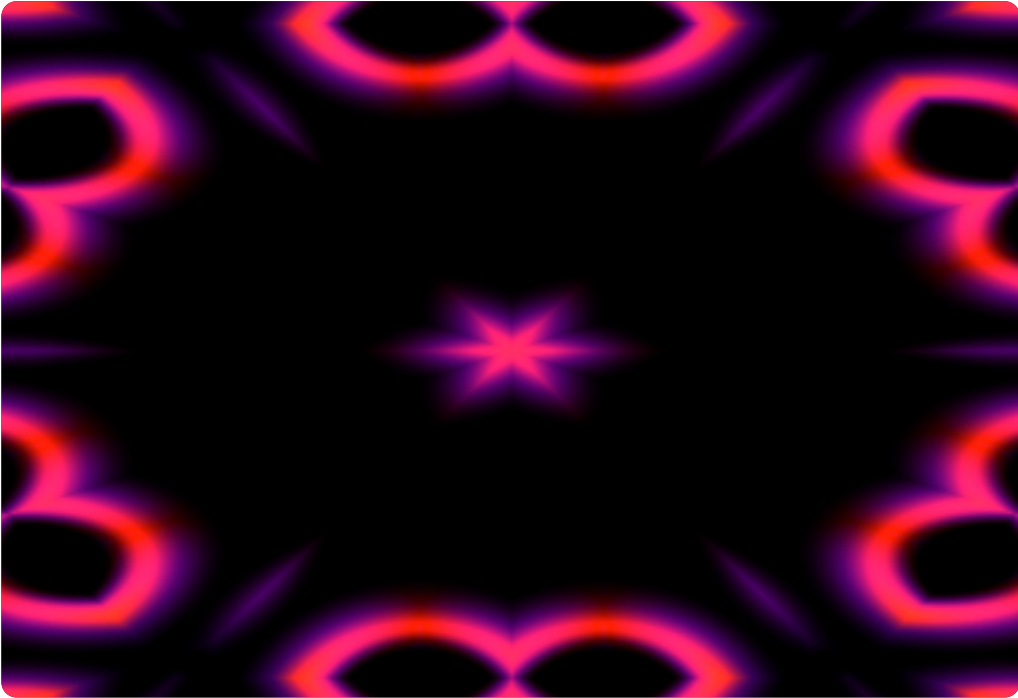
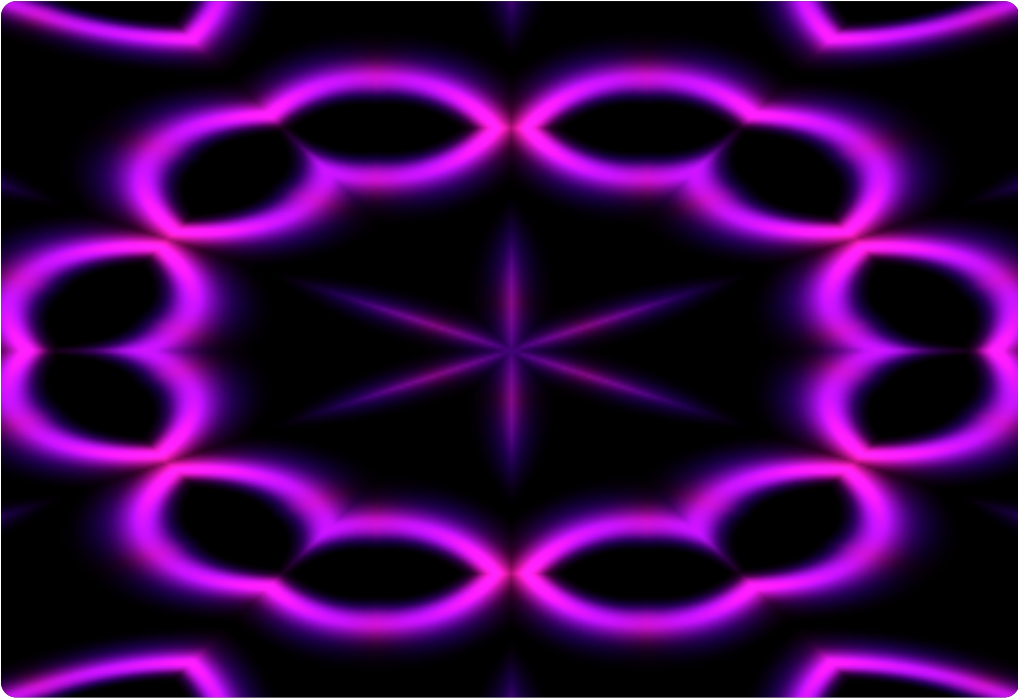
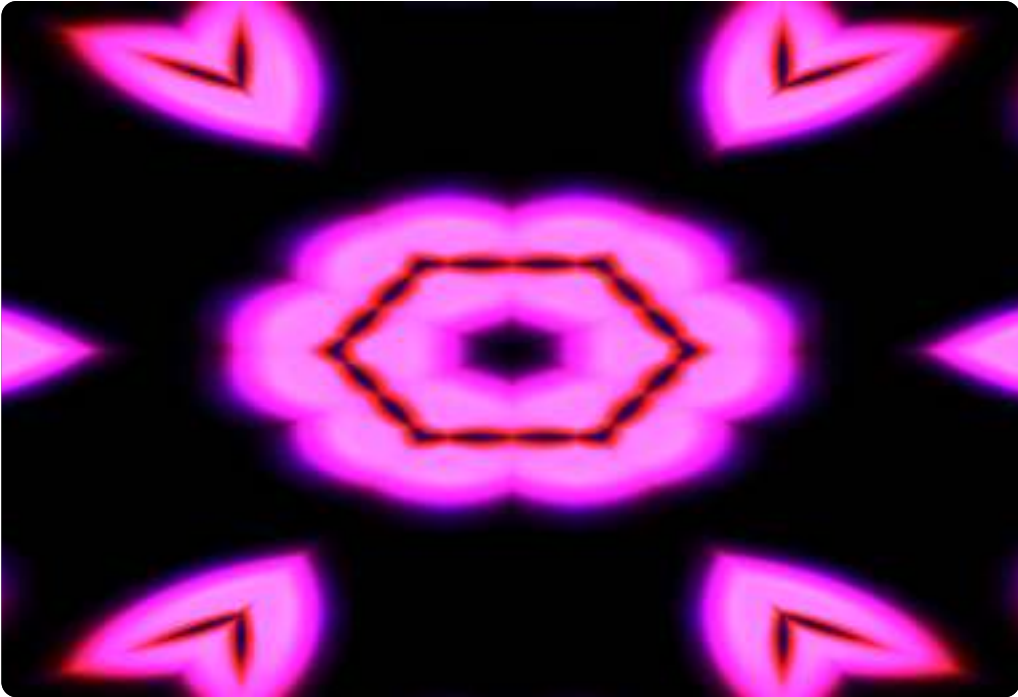
src(s0).scale(0.8,1,0.24).scrollX( ()=>Math.cos(time/7)*0.45
,0).kaleid(2)
.add( src(s0).scale(0.8,1,0.24).scrollX(
()=>Math.cos(time/7)*0.45 ,0) )
.add( src(s0).scale(0.8,1,-0.24).scrollX(
()=>Math.cos(time/7)*-0.45 ,0).scrollY(-0.2) )
.add( src(s0).scale(0.8,1,-0.24).scrollX(
()=>Math.cos(time/7)*-0.45 ,0).scrollY(+0.2) )
.add( src(s0).scale(0.8,1,0.24).scrollX(
()=>Math.cos(time/7)*0.75 ,0).scrollY(-0.4) )
.add( src(s0).scale(0.8,1,0.24).scrollX(
()=>Math.cos(time/7)*0.75 ,0).scrollY(+0.4) )

.out()
```

O patch se organiza em torno de deslocamento X (**scrollX**), escala (**scale**), fontes externas (**src**), soma de camadas (**add**) e deslocamento Y (**scrollY**). A composição usa **add** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**initImage**, **src**, **loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares.





```

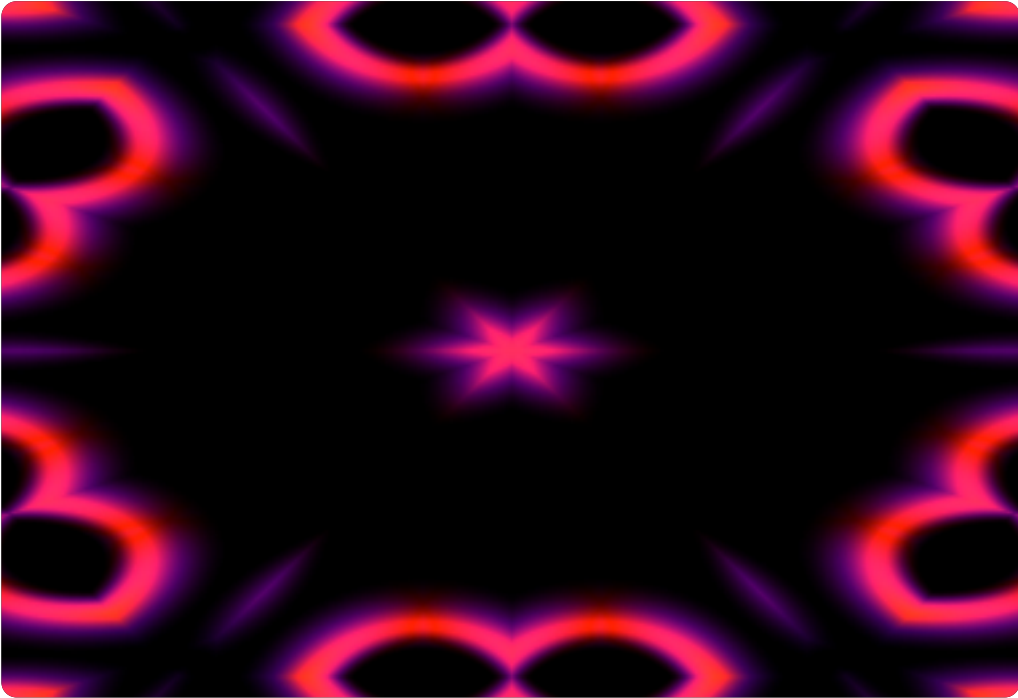
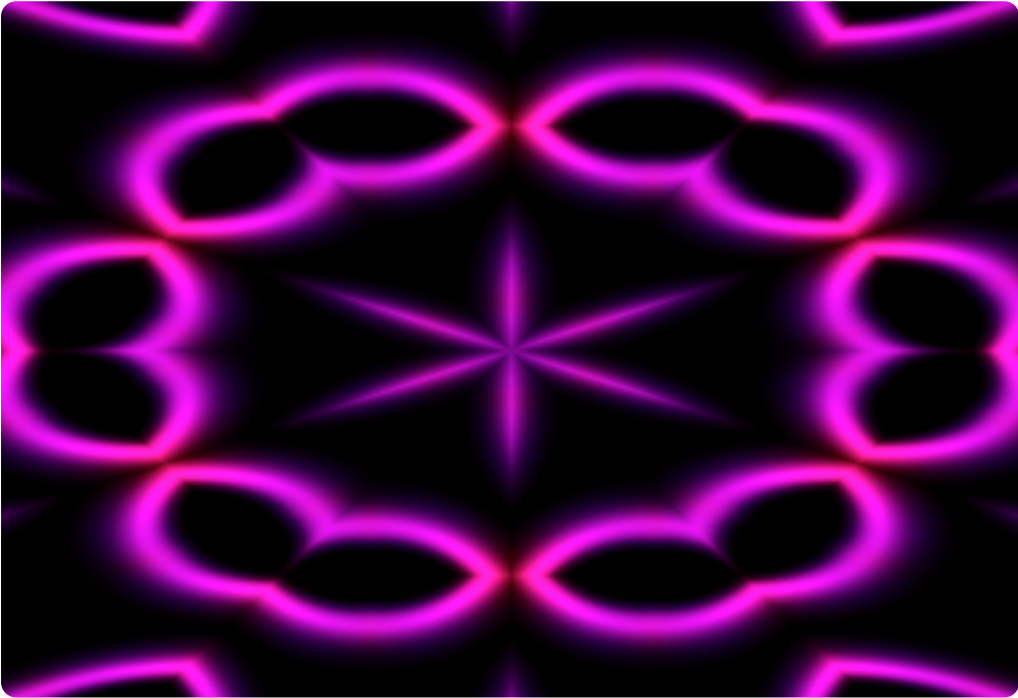
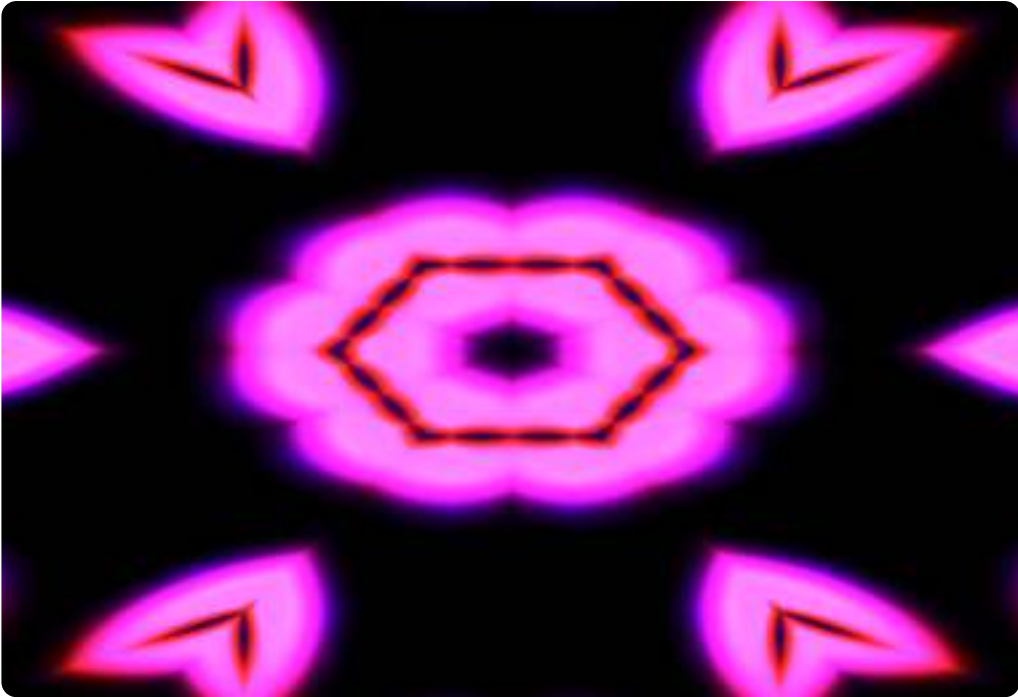
speed=2
noise(0.25, 0.045)
  .color(1, 0, 0)
  .mult(shape(72, 0.45, 0.45, ))
  .add(noise(0.50, 0.050))
  .color(0, 0, 1)
  .mult(shape(72, 0.45, 0.55)))
  .add(noise(0.75, 0.045))
  .color(1, 0, 0)
  .mult(shape(72, 0.55, 0.45)))
  .add(noise(1.00, 0.050))
  .color(0, 0, 1)
  .mult(shape(72, 0.55, 0.45)))
  .add(noise(1.25, 0.055))
  .color(0, 0, 1)
  .mult(shape(72, 0.55, 0.45)))
  .diff(noise(1.50, 0.020))
  .invert().scale(2)
  // .color( ()=>Math.sin(time/22)*0.5+0.333 ,
  ()=>Math.sin(time/31)*2-2, ()=>Math.sin(time/21)*0.5+0.333 )
  .mult(shape(108, 0.55, 0.3)))
  .color(() => Math.sin(time / 37) * 0.5 + 1.4, 0.5, () =>
Math.sin(time / 44) * 0.5 + 1.4)
  .modulate(shape(100, 0.33, 0.41), 0.4)
  .rotate(() => time / -27)
  // .scale(-1, -1, 0.99)
  .kaleid(4)
  .repeat(2)
  .scrollY(0.01, -0.01)
  .scrollX(-0.01, -0.005)
  .kaleid(6)
  .out()

```

O patch se organiza em torno de formas geométricas (**shape**), colorização (**color**), ruído (**noise**), multiplicação de camadas (**mult**) e soma de camadas (**add**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **mult**, **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 2** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

speed=2
noise(0.25, 0.045)
  .color(1, 0, 0)
  .mult(shape(72, 0.45, 0.45, ))
  .add(noise(0.50, 0.050)
    .color(0, 0, 1)
    .mult(shape(72, 0.45, 0.55)))
  .add(noise(0.75, 0.045)
    .color(1, 0, 0)
    .mult(shape(72, 0.55, 0.45)))
  .add(noise(1.00, 0.050)
    .color(0, 0, 1)
    .mult(shape(72, 0.55, 0.45)))
  .add(noise(1.25, 0.055)
    .color(0, 0, 1)
    .mult(shape(72, 0.55, 0.45)))
  .diff(noise(1.50, 0.020)
    .invert().scale(2)
    // .color( )=>Math.sin(time/22)*0.5+0.333 ,
    ()=>Math.sin(time/31)*2-2, ()=>Math.sin(time/21)*0.5+0.333 )
    .mult(shape(108, 0.55, 0.3)))
  .color(() => Math.sin(time / 37) * 0.5 + 1.4, 0.5, () =>
Math.sin(time / 44) * 0.5 + 1.4)
  .modulate(shape(100, 0.33, 0.41), 0.4)
  .rotate(() => time / -27)
  // .scale(-1, -1, 0.99)
  .kaleid(4)
  .repeat(2)
  .scrollY(0.01, -0.01)
  .scrollX(-0.01, -0.005)
  .kaleid(6)
  .out()

```

O patch se organiza em torno de formas geométricas (**shape**), colorização (**color**), ruído (**noise**), multiplicação de camadas (**mult**) e soma de camadas (**add**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **mult**, **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 2** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

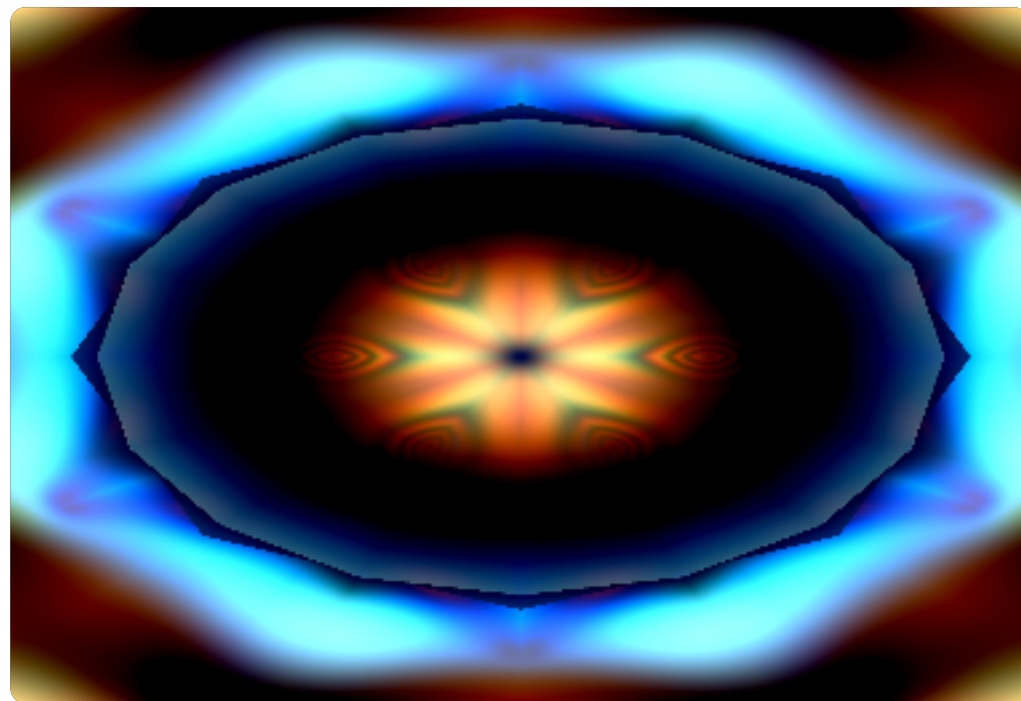
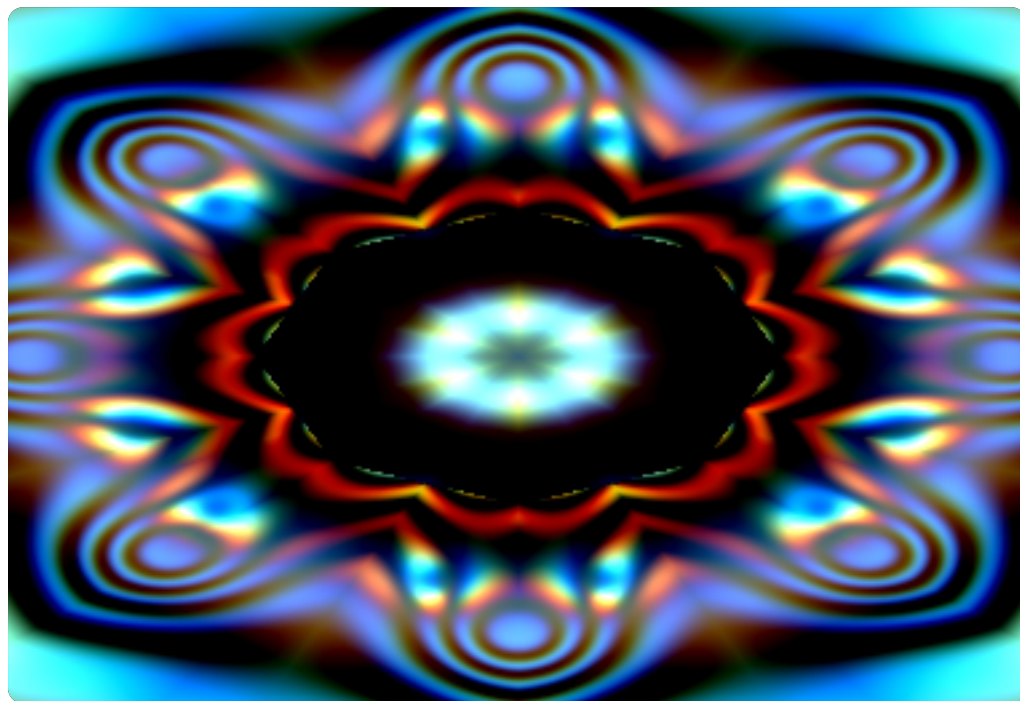
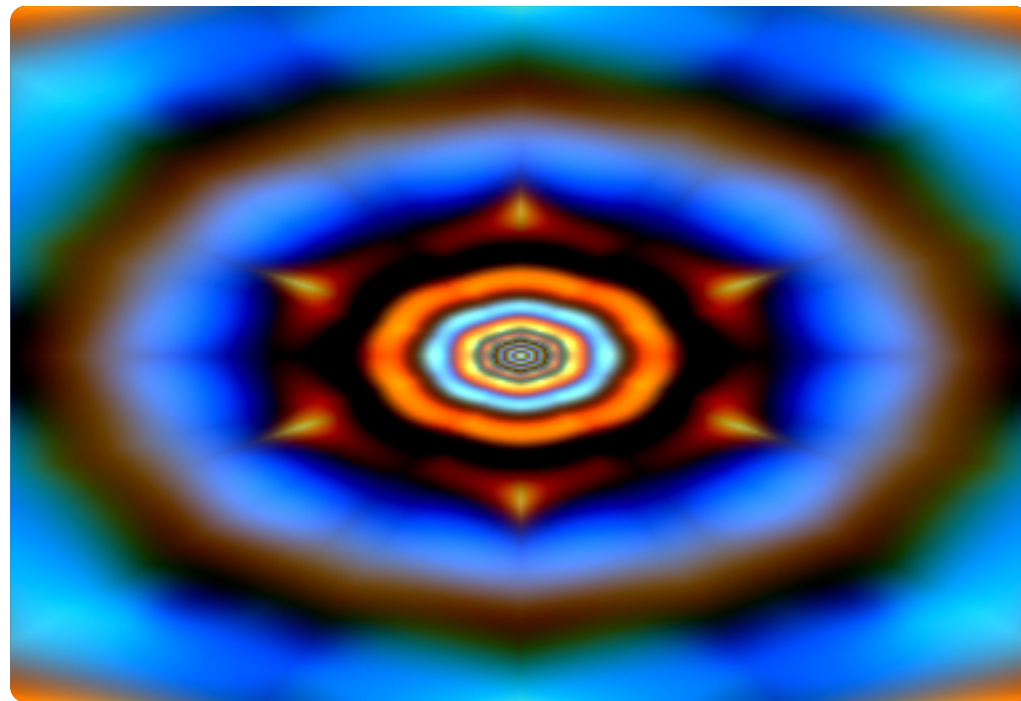
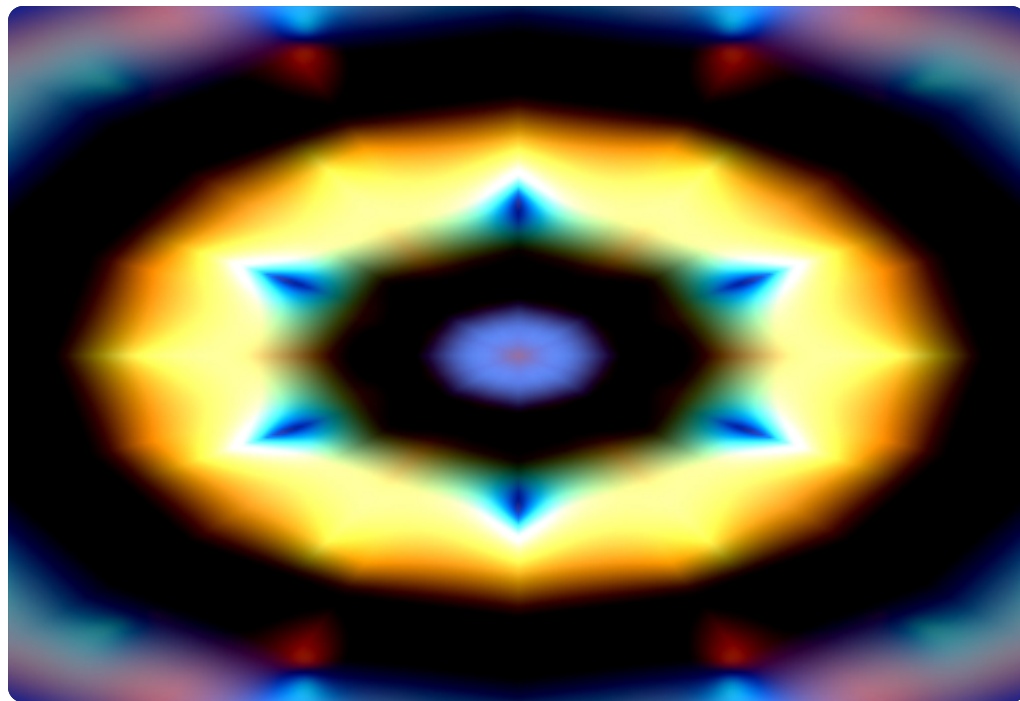
speed=0.3
osc(10,-0.07,1)
  .modulate( noise(4,0.03) )
  // .scrollY(0.1,-0.03)
.kaleid(6)
.diff(
osc(20,0.04,12)
.modulate( noise(2,0.06) )
.scrollX(0.1,-0.02)
.kaleid(8)
)
// .diff(
// osc(20,0.013,0.5)
// .modulate( noise(2,0.06) )
// .scrollX(0.1,-0.02)
// .kaleid(16)
// )
.scale(1,9/16,1)
.out()

```

O patch se organiza em torno de modulação (**modulate**), kaleidoscópio (**kaleid**), ruído (**noise**), osciladores (**osc**) e deslocamento X (**scrollX**). **osc** entra em **kaleid**, multiplicando eixos de simetria até o quadro parecer um ornamento vivo. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.3** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra.





```

speed=0.52
osc(10,-0.07,1)
  .modulateScale( noise(4,0.03), 1.1 )
  // .scrollY(0.1,-0.03)
.kaleid(6)
.mult(
osc(20,0.04,12)
.modulateScale( noise(2,0.06), 0.9 )
.scrollX(0.1,-0.02)
.kaleid(8)
)
.add(
osc(20,0.013,0.5)
.modulateScale( noise(2,0.06), .04 )
.scrollX(0.1,-0.02)
.kaleid(16)
, .75
)
.scale(0.9)
.saturate(1.125)
.brightness(-0.3)
.contrast(1.125)
// .scale(1,9/16,1)

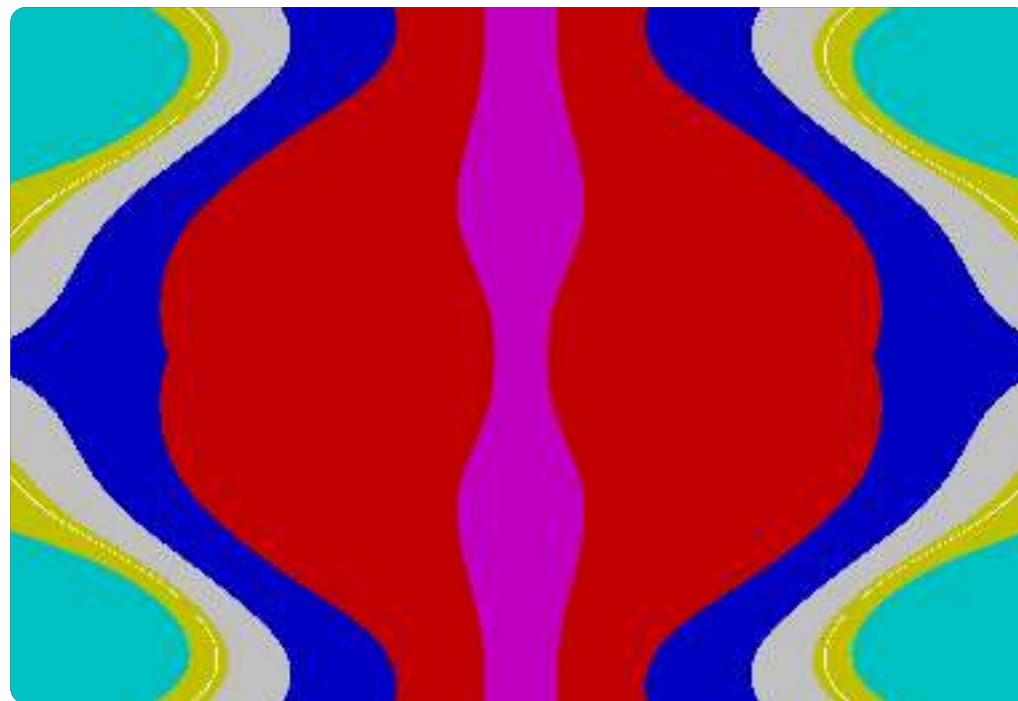
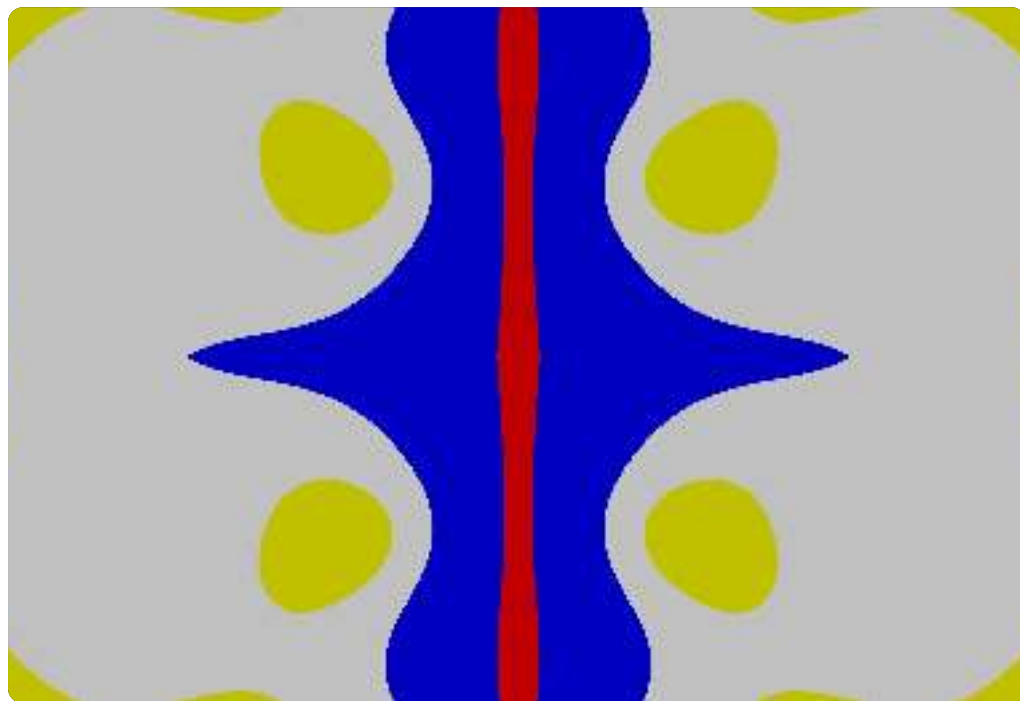
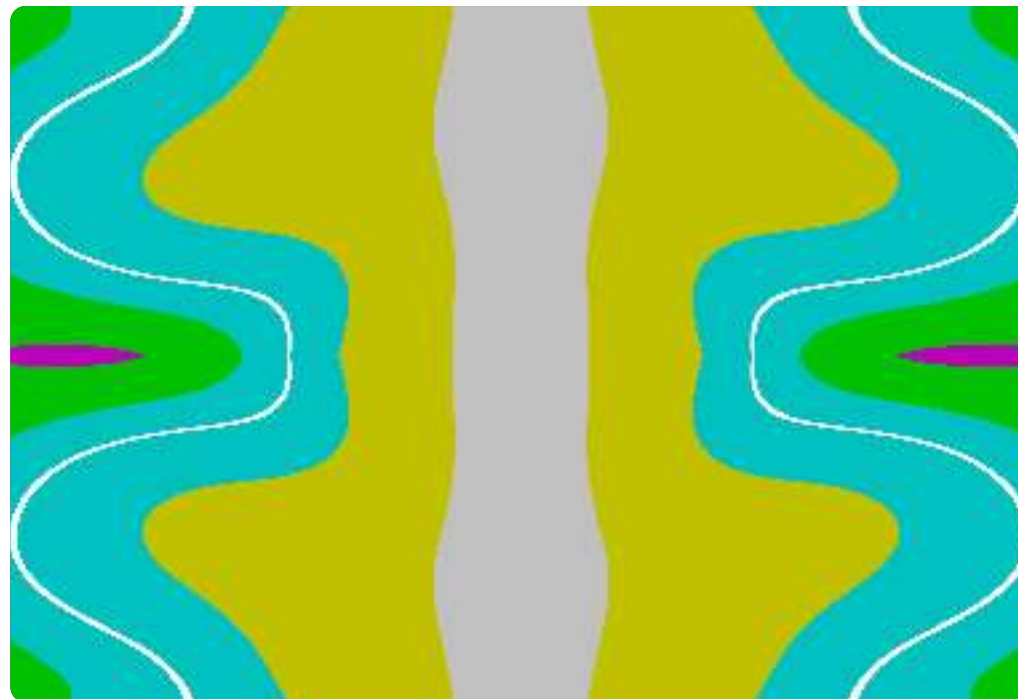
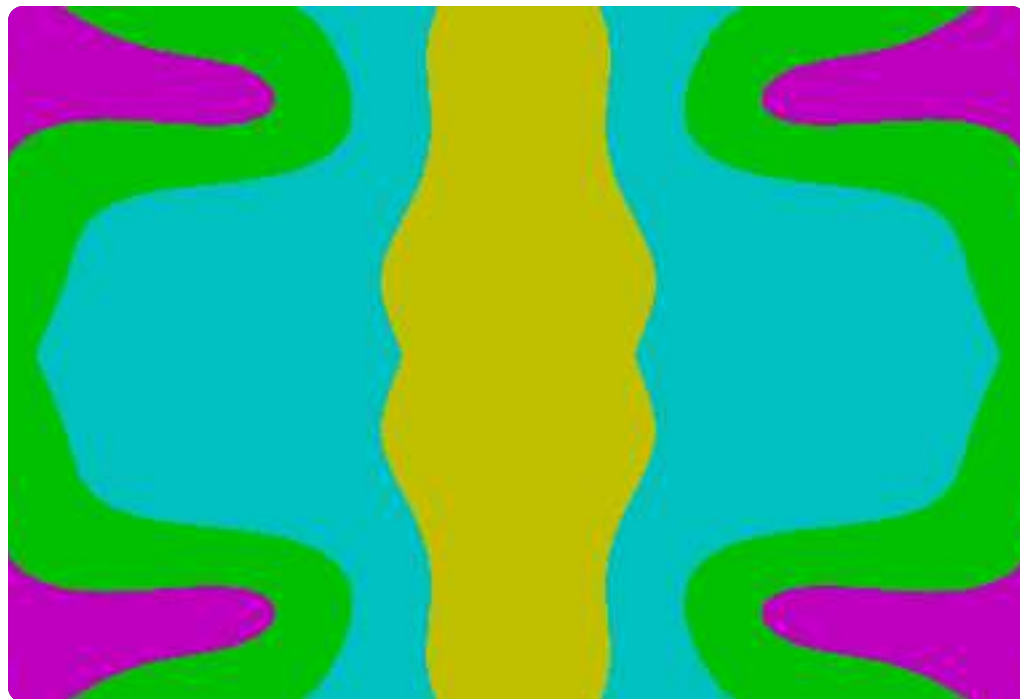
.out()

```

O patch se organiza em torno de modulação de escala (**modulateScale**), kaleidoscópio (**kaleid**), ruído (**noise**), osciladores (**osc**) e deslocamento X (**scrollX**). **osc** entra em **kaleid**, multiplicando eixos de simetria até o quadro parecer um ornamento vivo. A composição usa **mult**, **add** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.52** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```
await loadScript("https://hyper-hydra.glitch.me/hydra-
fractals.js")
await loadScript("https://hyper-hydra.glitch.me/hydra-wrap.js")
//hydraWrap.setMirror()
hydraWrap.setNoWrap()

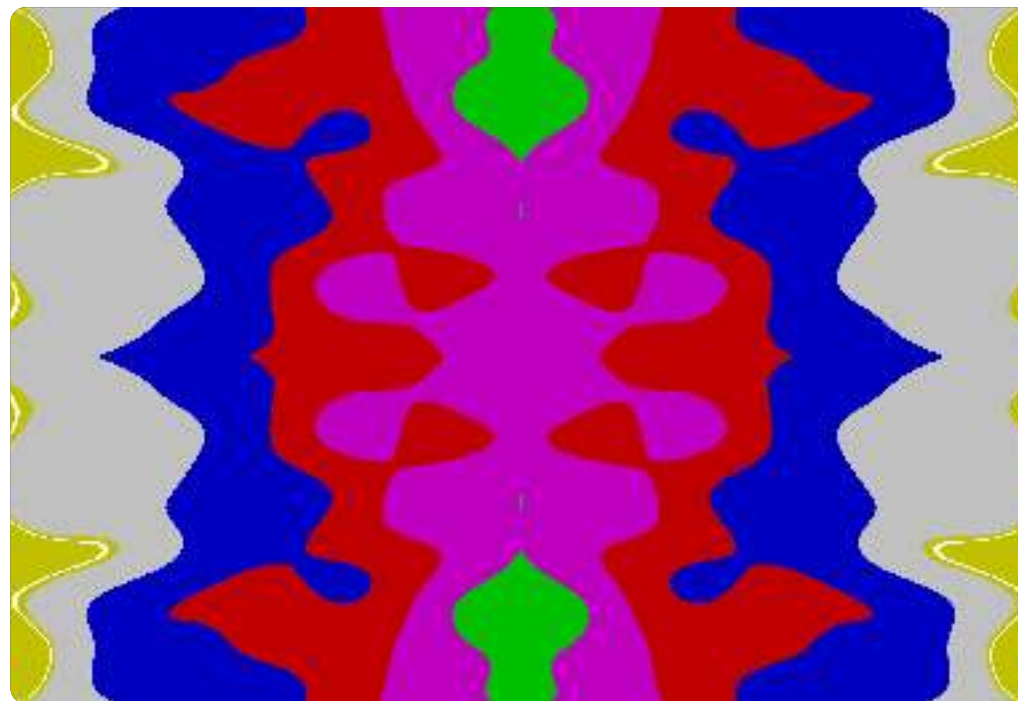
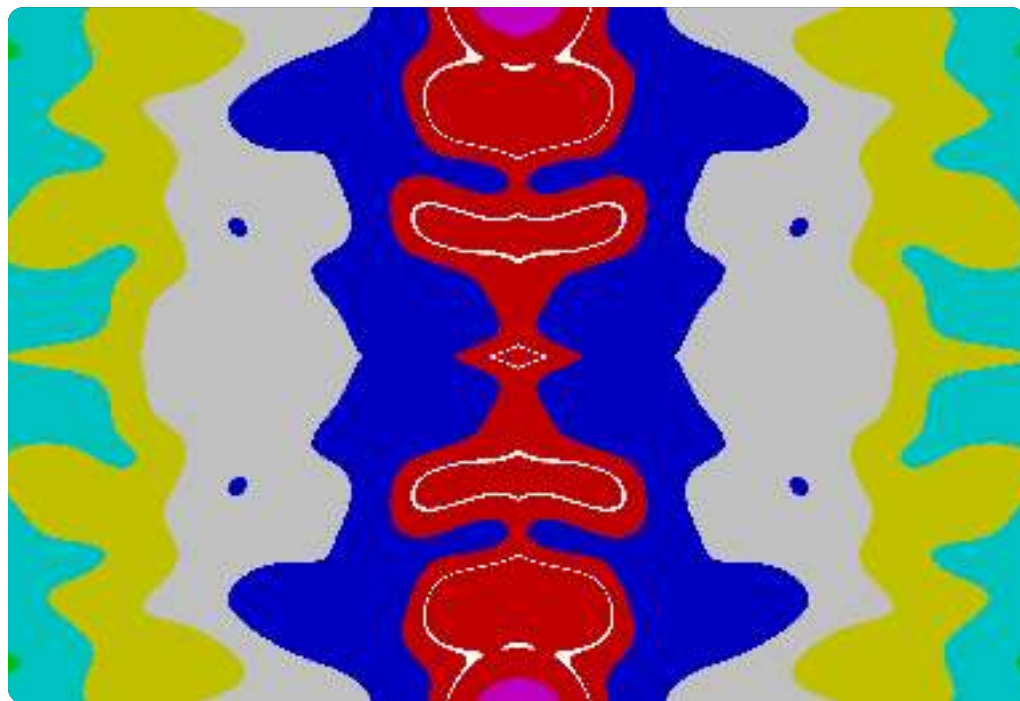
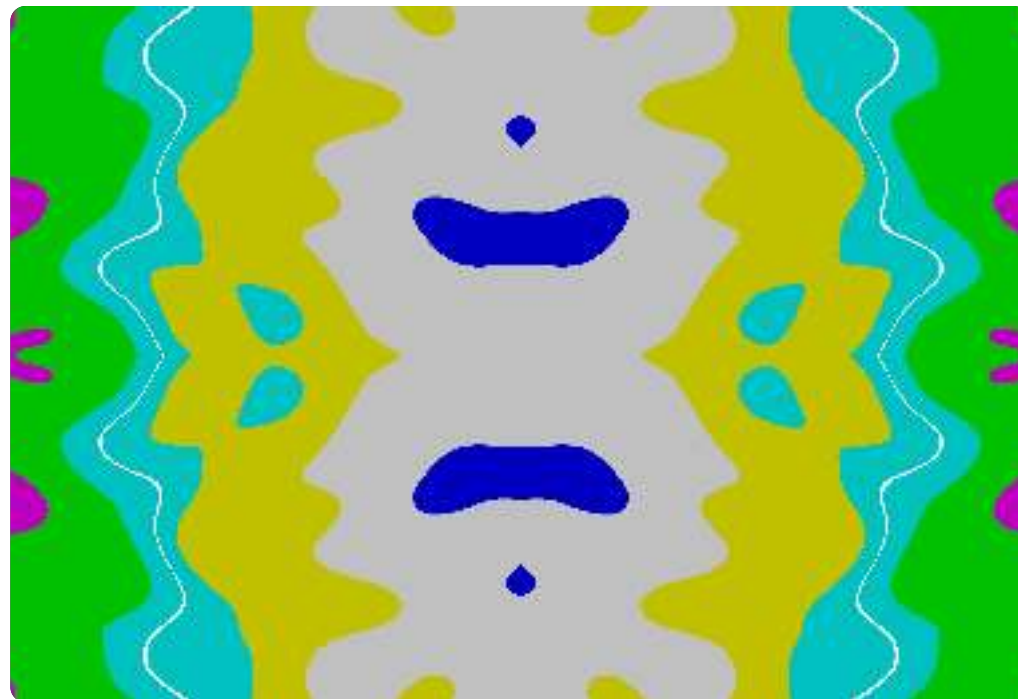
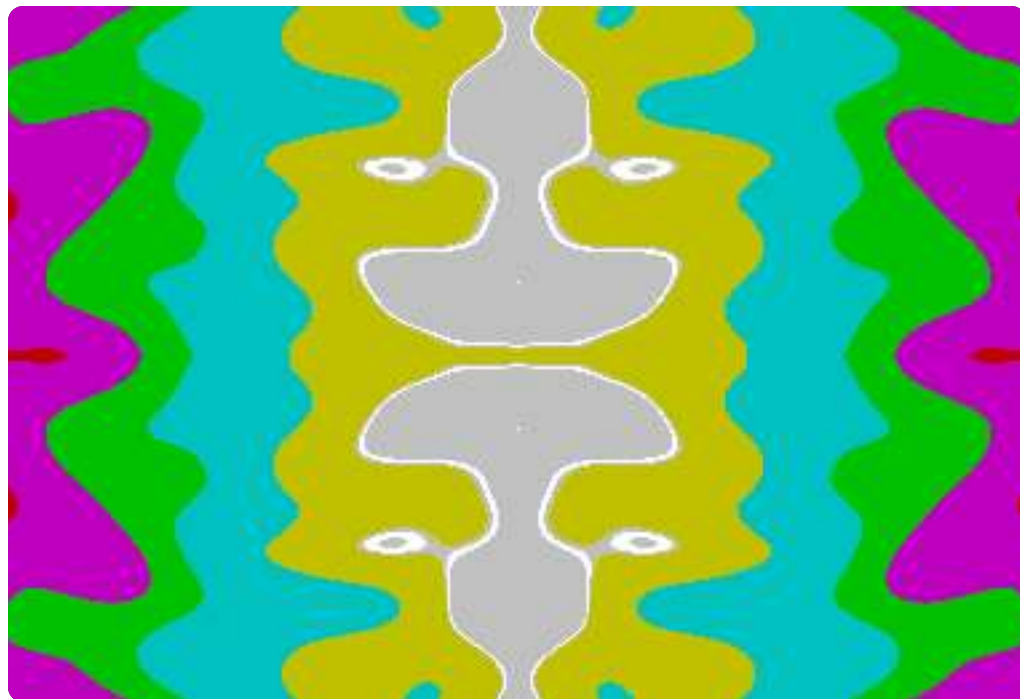
//s0.initScreen()
src(s0).scale(1).scale(-1,1,-1)

    .scrollX(0.25,0)
    .scrollY(0.25,0)
    .modulateScale(noise(2,.1),.9)
    .mirrorX(0,1)
    .mirrorY(0,1)
//.colorama(0.1)
//.inversion()
.out()
```

O patch se organiza em torno de escala (**scale**), modulação de escala (**modulateScale**), deslocamento Y (**scrollY**), deslocamento X (**scrollX**) e ruído (**noise**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo.

A modulação (**modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. Há entrada externa (**src**, **loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares.





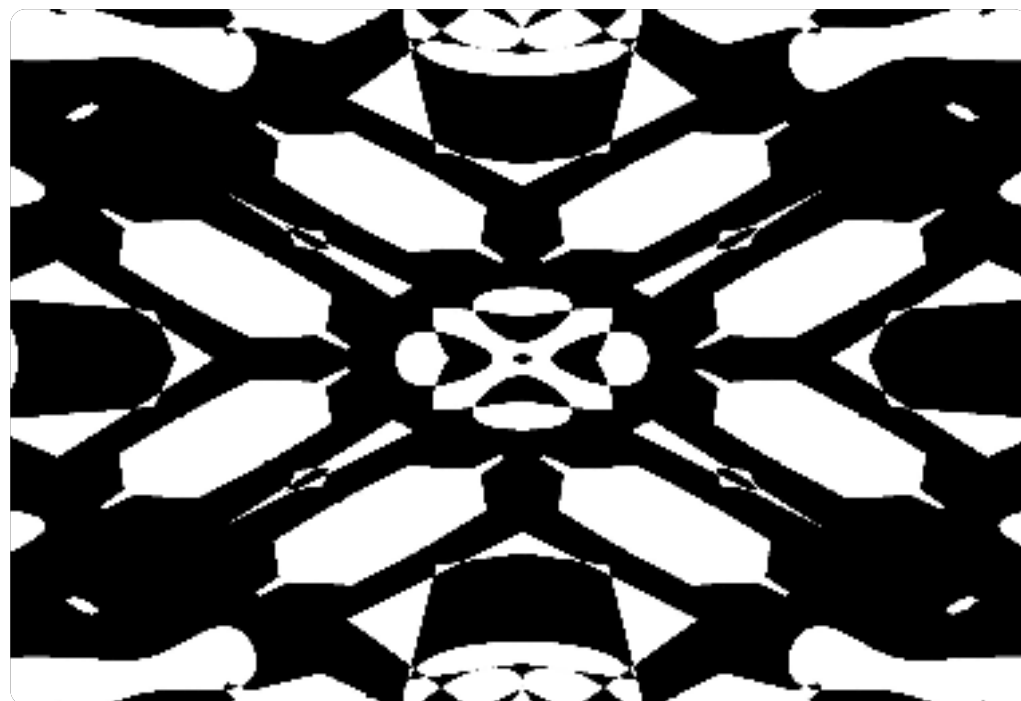
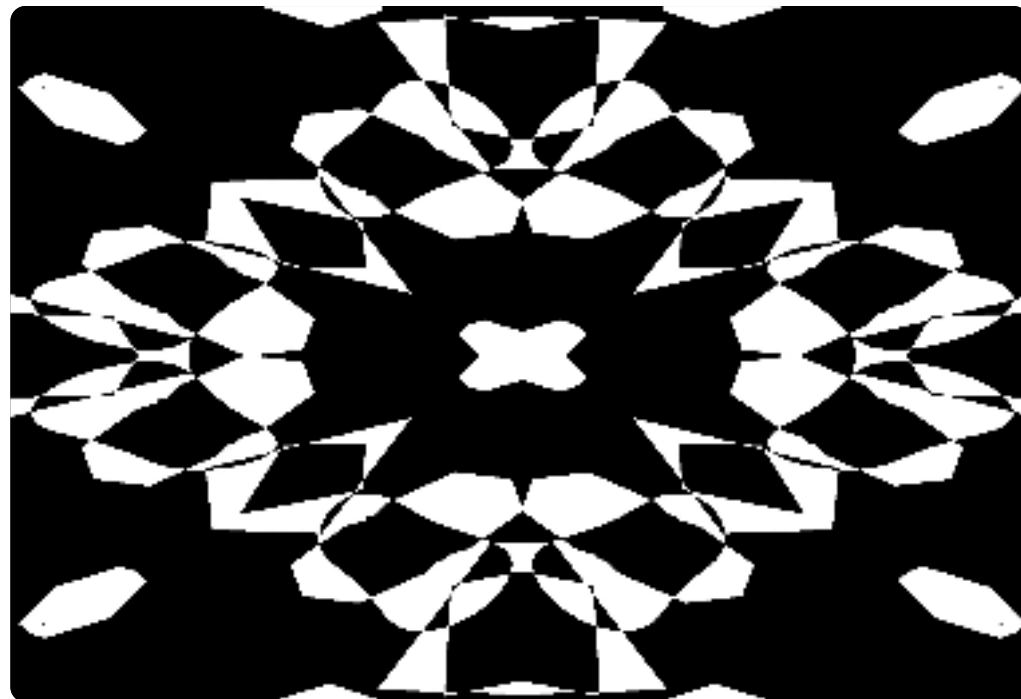
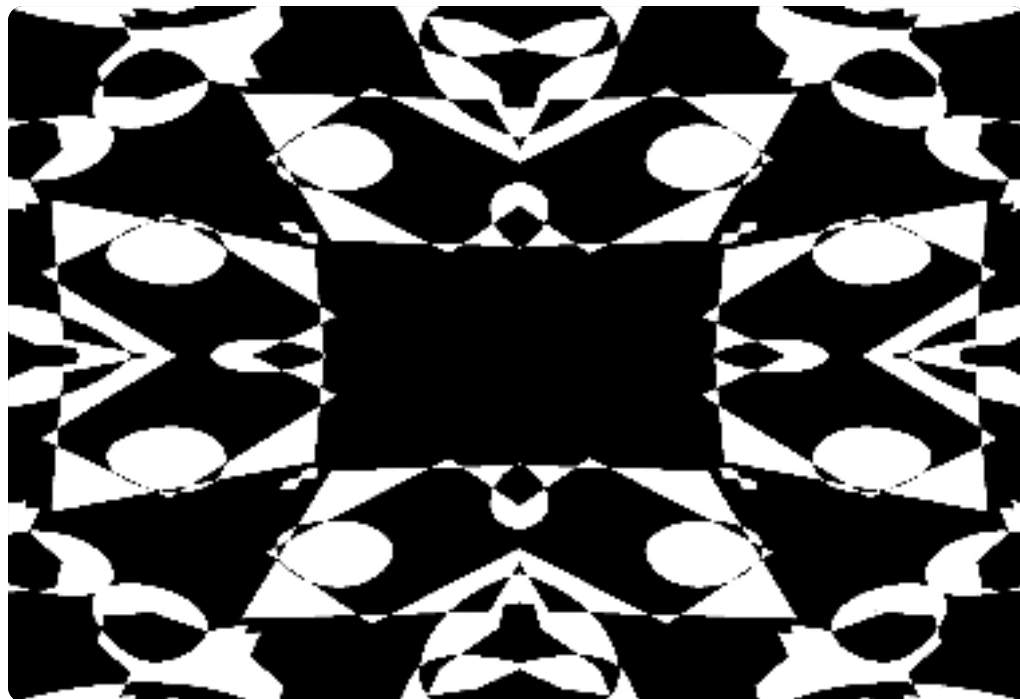
```
await loadScript("https://hyper-hydra.glitch.me/hydra-
fractals.js")
await loadScript("https://hyper-hydra.glitch.me/hydra-wrap.js")
//hydraWrap.setMirror()
hydraWrap.setNoWrap()

//s0.initScreen()
src(s0).scale(1).scale(-1,1,-1)
.scale(.7)
  .scrollX(0.25,0)
  .scrollY(0.25,0)
  .modulate(noise(5.5,.02))
  .mirrorX(0,1)
  .mirrorY(0,1)
//.colorama(0.1)
//.inversion()
.out()
```

O patch se organiza em torno de escala (**scale**), modulação (**modulate**), deslocamento Y (**scrollY**), deslocamento X (**scrollX**) e ruído (**noise**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. Há entrada externa (**src**, **loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares.





```

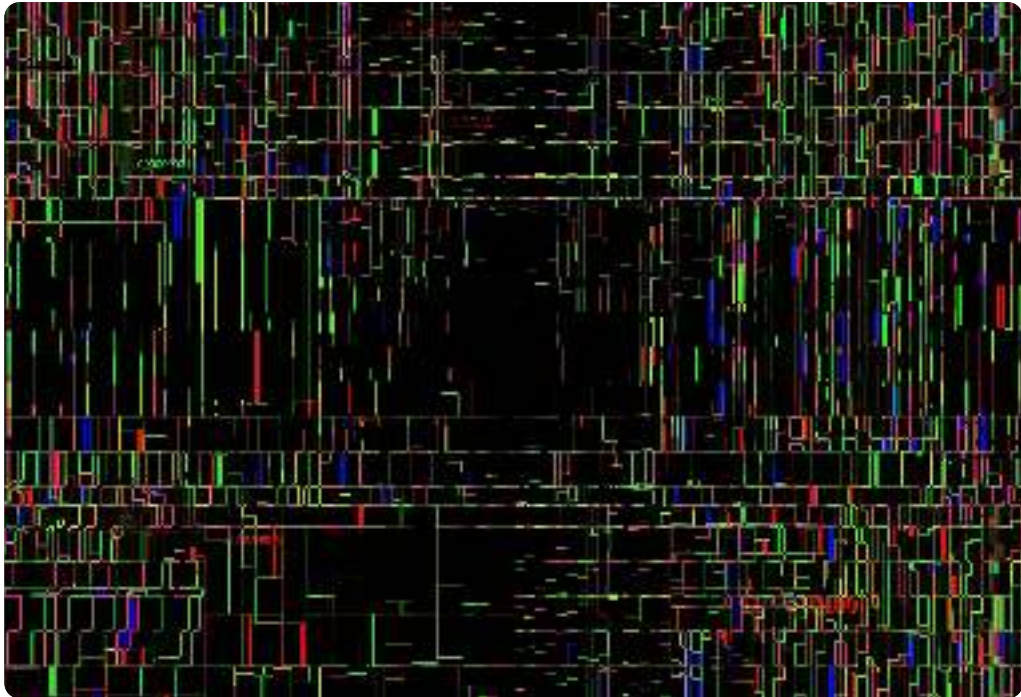
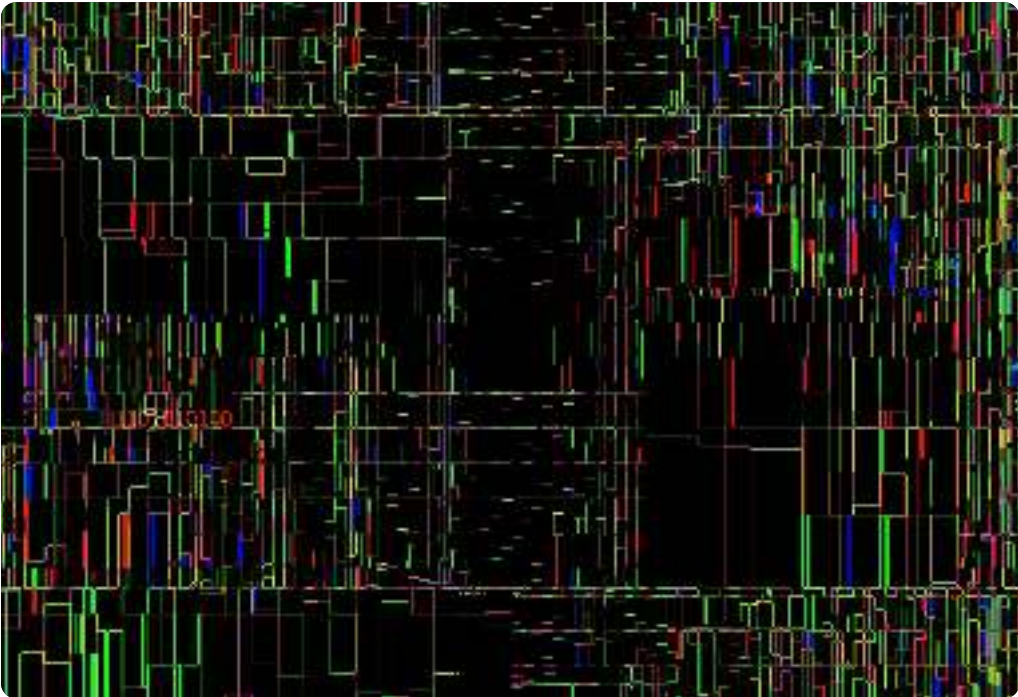
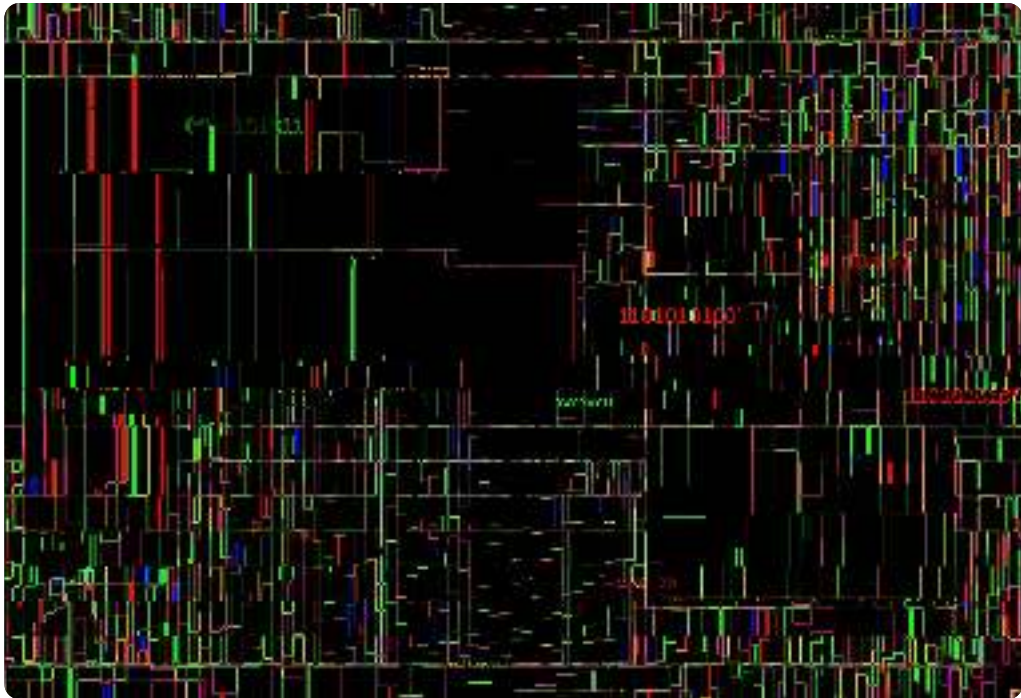
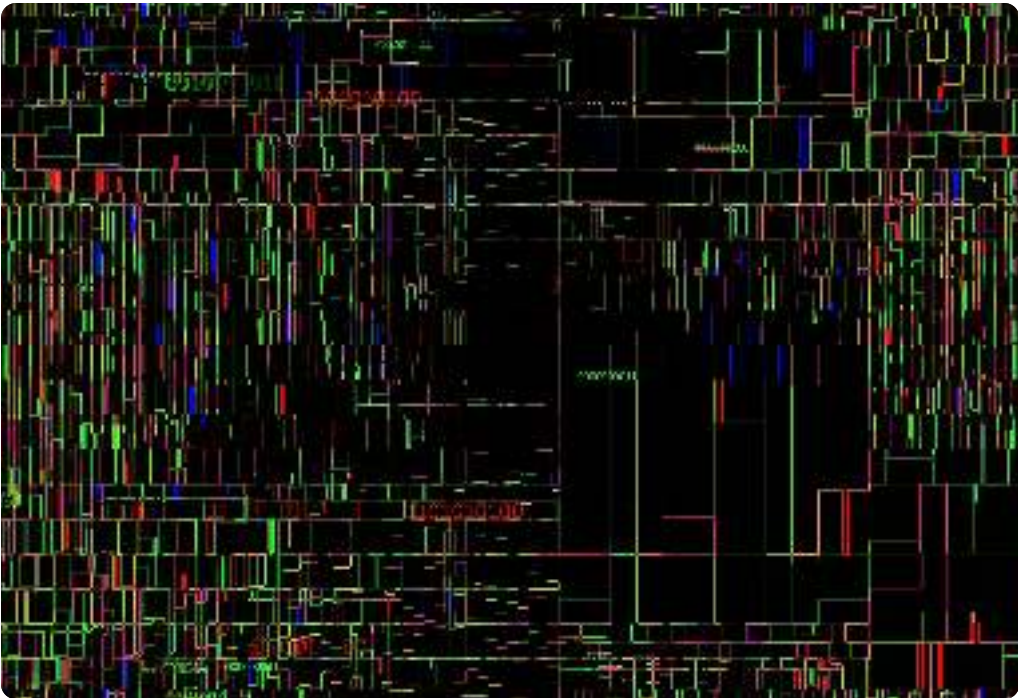
shape(4,0.5,0).rotate(0,10)
.diff( shape(4,0.5,0).rotate(0,-10/5) )
.diff( shape(4,0.4,0).rotate(0,10*2) )
.modulateScale( shape(4,0.6,0).rotate(0,10*2),.5 )
.modulateRotate( shape(4,0.7,0).rotate(0,10*2),1 )
.kaleid(4)
.diff( shape(4,0.5,0).rotate(0,-10/5) )
.diff( shape(4,0.4,0).rotate(0,10*2) )
.repeat(2)
.scrollY(0,0.02)
.kaleid(4)
.diff( noise(4,0.1).thresh(0.5,0).kaleid(4) )
.diff( noise(2,0.05).thresh(0.5,0).kaleid(4) )
.out()

```

O patch se organiza em torno de rotação (**rotate**), formas geométricas (**shape**), diferença (**diff**), kaleidoscópio (**kaleid**) e limiar (**thresh**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateRotate**, **modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa.





```

await loadScript("https://hyper-hydra.glitch.me/hydra-text.js")

speed=0.3
osc(32,-0.004,1.25)//.thresh(0.5,0.6)
.modulateScrollX( noise(4,0.04).pixelate(80), 0.7 )
.modulateScrollY( noise(10,0.01).pixelate(120), 0.7 )
.modulateHue( noise(1,0.009).pixelate(80), 180 )
.color(1,1,0.5)
.modulateScale( noise(2.2,0.04).pixelate(8).thresh(0.5,0.2), 1 )

.modulateScale( noise(1.2,0.03).pixelate(4).thresh(0.5,0.1), 1.5
)
.colorama(0.0001)
.modulateScale( shape(4, ()=>Math.cos(time/10)*0.5+0.5 ,0.000),
()=>Math.sin(time*1)*0.5+1.5 )

.modulate( shape(2, ()=>Math.cos(time/10)*0.5+0.5 ,0.000),
()=>Math.sin(time/2)*0.5+0.5 )

.sub( text( '1100000000'
).scale(0.05).scrollX(()=>Math.random()).scrollY(()=>Math.random(
)).color([1,0,1,0].fast(2),0,0) )
.add( text( '0000000011'
).scale(0.05).scrollX(()=>Math.random()).scrollY(()=>Math.random(
)).color(0,[1,0,1,0].fast(2.1),0,0) )
.sub( text( '1111111111'
).scale(0.06).scrollX(()=>Math.random()).scrollY(()=>Math.random(
)).color([1,0,1,0].fast(2),0,0) )

```

```

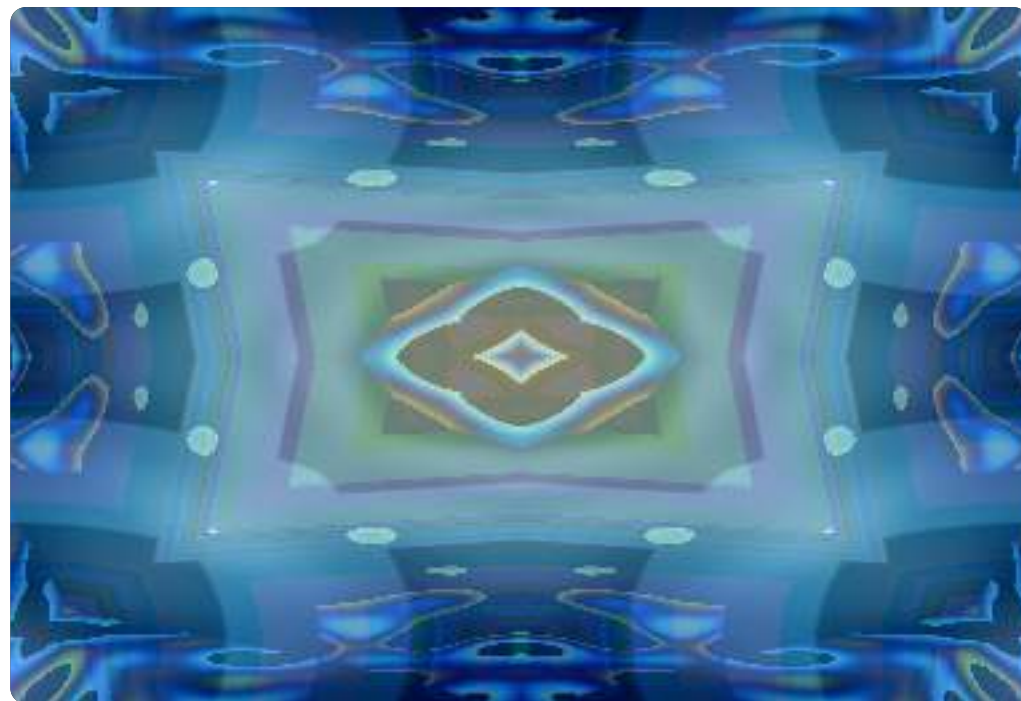
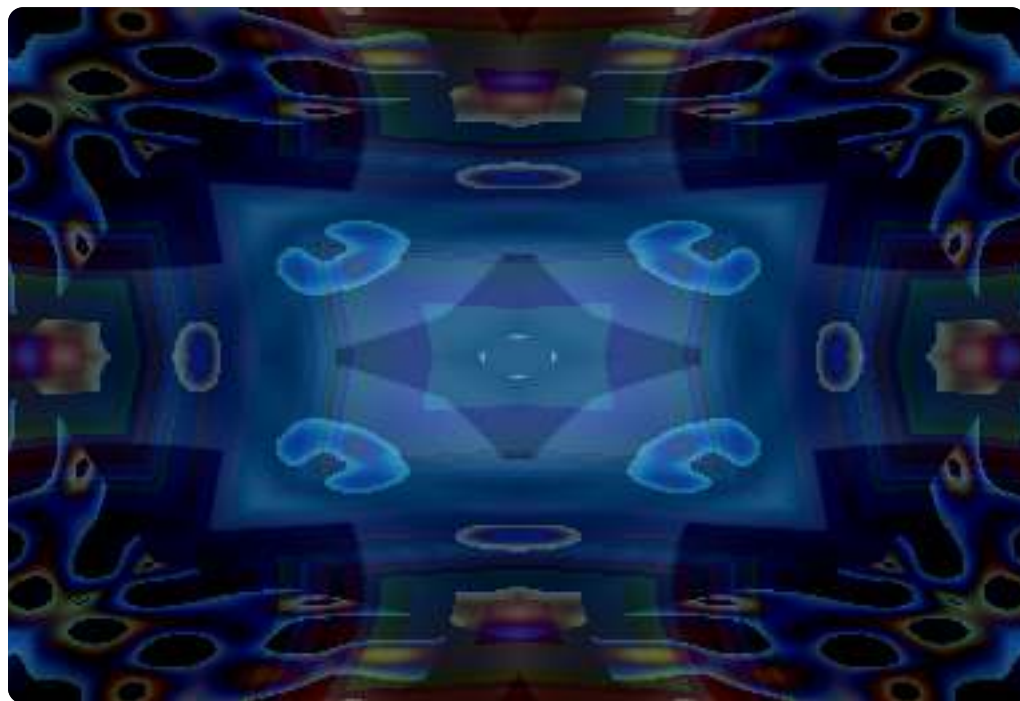
.sub( text( '1010101010'
).scale(0.06).scrollX(()=>Math.random()).scrollY(()=>Math.ra
ndom()).color(0,[1,0,1,0].fast(2.1),0,0) )
.add( text( '1101010100'
).scale(0.1).scrollX(()=>Math.random()).scrollY(()=>Math.ra
ndom()).color([1,0,1,0].fast(2.2),0,0) )
.sub( text( '0000000000'
).scale(0.1).scrollX(()=>Math.random()).scrollY(()=>Math.ra
ndom()).color(0,[1,0,1,0].fast(2.3),0,0) )
.sub( text( '1100000000'
).scale(0.05).scrollX(()=>Math.random()).scrollY(()=>Math.ra
ndom()).color([1,0,1,0].fast(1),0,0) )
.add( text( '1111111111'
).scale(0.05).scrollX(()=>Math.random()).scrollY(()=>Math.ra
ndom()).color(0,[1,0,1,0].fast(1.1),0,0) )
.sub( text( '0000000000'
).scale(0.075).scrollX(()=>Math.random()).scrollY(()=>Math.
random()).color([1,0,1,0].fast(1),0,0) )
.sub( text( '1010101010'
).scale(0.075).scrollX(()=>Math.random()).scrollY(()=>Math.
random()).color(0,[1,0,1,0].fast(1.1),0,0) )
.add( text( '1101010100'
).scale(0.1).scrollX(()=>Math.random()).scrollY(()=>Math.ra
ndom()).color([1,0,1,0].fast(1.3),0,0) )
.sub( text( '0010101011'
).scale(0.1).scrollX(()=>Math.random()).scrollY(()=>Math.ra
ndom()).color(0,[1,0,1,0].fast(1.3),0,0) )

.out(o1)

src(o1).diff(src(o1).scale(1.006),0.5)

```





```

speed=0.1
// Core layer: "Straight-edged circles" simulated with square-
based rotations
shape(4, 0.6, 0.9)
  .scale(() => Math.abs(Math.sin(time * 0.3)) * 0.5 + 0.4) //
Dynamic size change
  .repeat(4, 4) // Create a grid-like structure
  .modulate(osc(10, 0.2, 0.8).rotate(Math.PI / 4)) // Use square-
based distortion
  .rotate(({time}) => Math.sin(time * 0.1) * Math.PI / 4) //
Rotation skewing the idea of curves
  .color(1.0, 0.3, 0.6)
  .luma(0.2)
  .out(o0);

// Background layer: Square "curves" with sharp oscillating
brightness
osc(15, 0.05, 0.8)
  .kaleid(4) // Reinforce symmetry of "4"
  .modulateRotate(shape(4, 0.4).scale(2).luma(0.5).invert(), 0.3)
  .colorama(0.8)
  .scale(() => Math.abs(Math.cos(time * 0.4)) * 0.3 + 1.2) //
Dynamic breathing
  .blend(noise(3, 0.5).modulate(osc(10), 0.3), 0.5)
  .out(o1);

// Secondary modulation layer: Overlays to distort "geometry"
osc(8, 0.03, 0.9)
  .rotate(() => Math.abs(Math.sin(time * 0.5)) * Math.PI / 4) //
Exaggerating square-based rotation
  .mult(shape(4, 0.7).rotate(Math.PI / 4))
  .modulate(noise(6, 0.3), 0.6)

```

```

  .add(shape(4, 0.4).scale(2).invert(), 0.4)
  .color(0.6, 0.9, 1.0)
  .scrolly(0.3, 0.1)
  .out(o2);

// Combine all layers with square-distorted interactions
src(o0)
  .layer(src(o1).blend(src(o2).scale(0.8).rotate(Math.PI /
4), 0.6))
  .modulate(osc(6, 0.05, 0.8).kaleid(4), 0.3)
  .add(shape(4, 0.8).scale(1.5).luma(0.5).invert(), 0.2)
  .blend(osc(5, 0.2,
0.6).kaleid(4).modulateRotate(noise(2), 0.4), 0.5)
  .rotate(0.1, 0.003)
  .kaleid(4)
  .contrast(1.25)
  .out(o3);

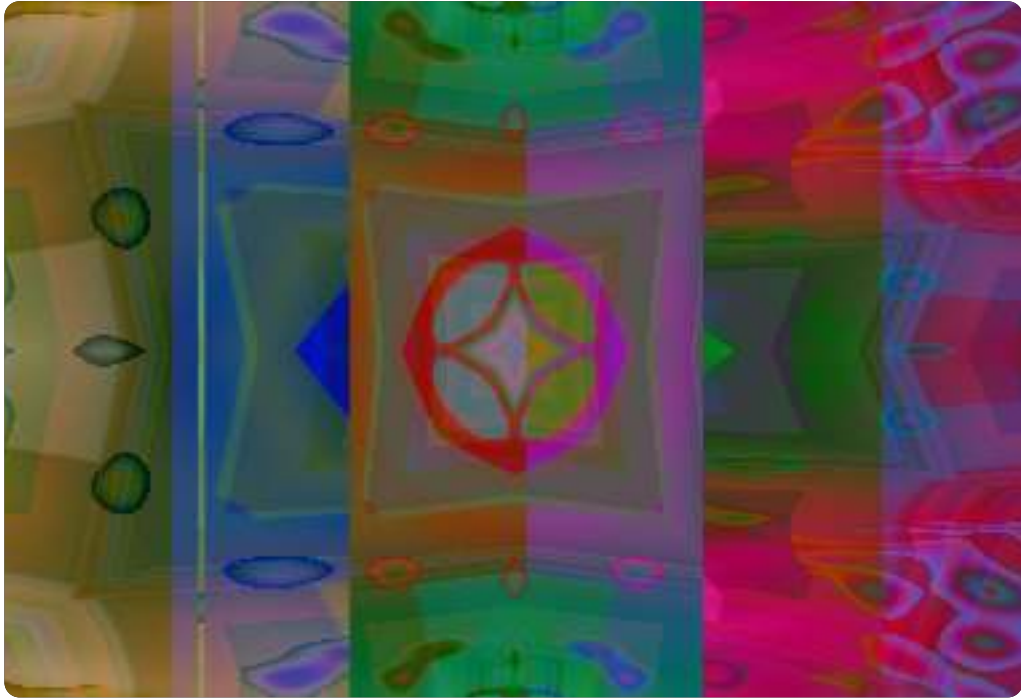
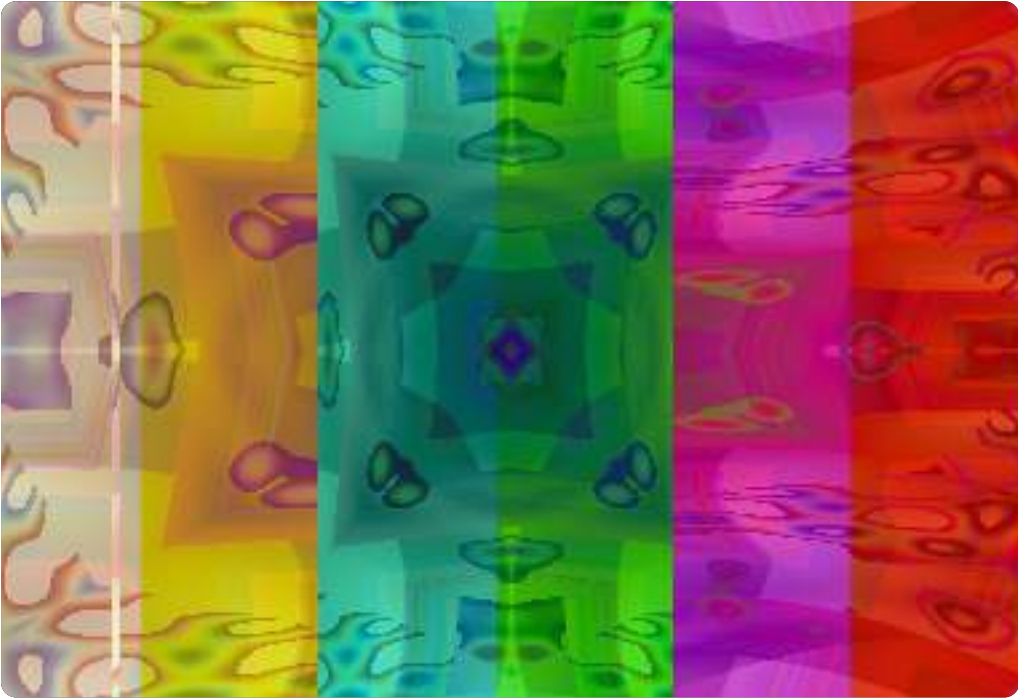
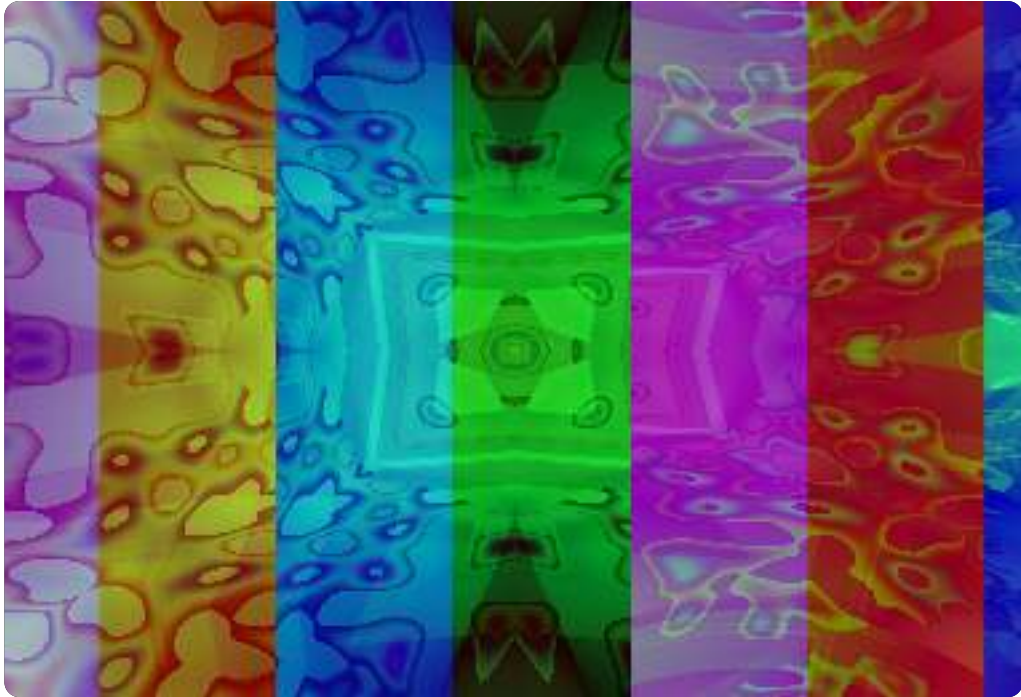
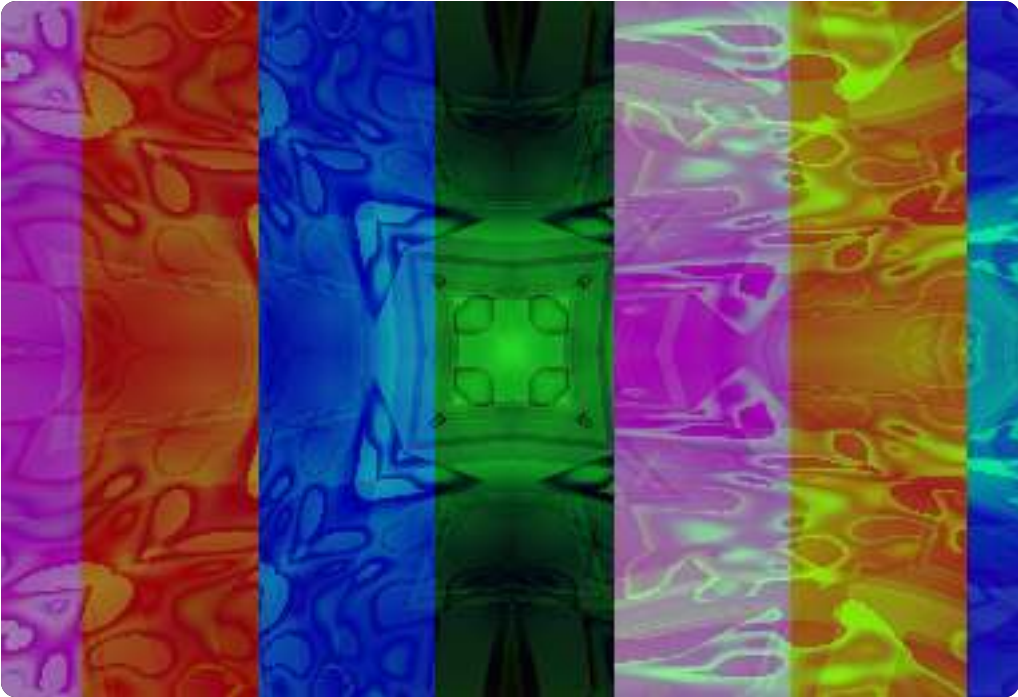
// Render final composite
render(o3);

```

O patch se organiza em torno de rotação (**rotate**), escala (**scale**), osciladores (**osc**), formas geométricas (**shape**) e modulação (**modulate**). **osc** entra em **kaleid**, multiplicando eixos de simetria até o quadro parecer um ornamento vivo. A composição usa **mult**, **add**, **layer**, **blend** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**, **modulateRotate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.1** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada





```
//s0.initCam()
//src(s0).saturate(2).contrast(1.3).layer(src(o0).mask(shape(4,2)
.scale(0.5,0.7).scrollX(0.25)).scrollX(0.001)).modulate(o0,0.001)
.out(o0)

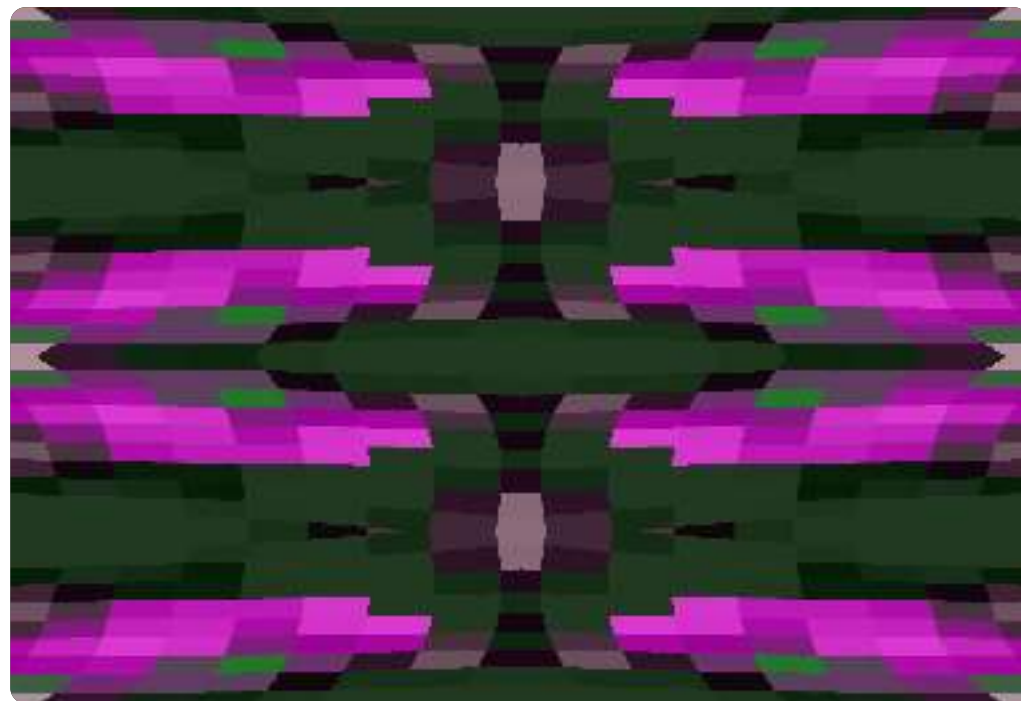
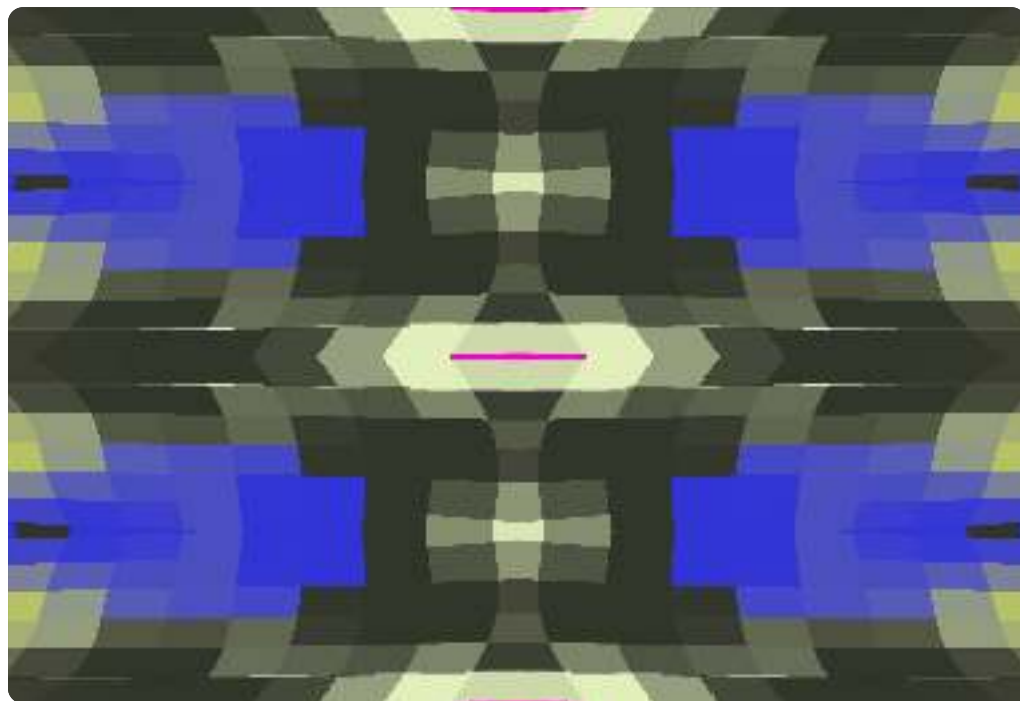
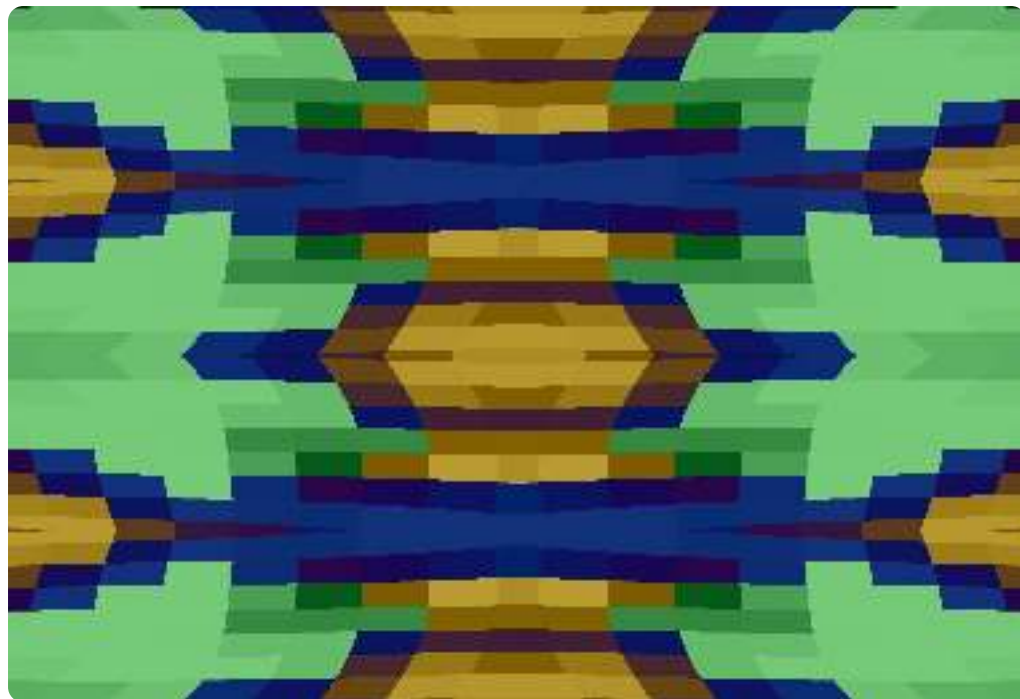
speed=0.1
s0.initCam()
// Core layer: "Straight-edged circles" simulated with square-
based rotations
shape(4, 0.6, 0.9)
.scale(() => Math.abs(Math.sin(time * 0.3)) * 0.5 + 0.4) //
Dynamic size change
.repeat(4, 4) // Create a grid-like structure
.modulate(osc(10, 0.2, 0.8).rotate(Math.PI / 4)) // Use square-
based distortion
.rotate(({time}) => Math.sin(time * 0.1) * Math.PI / 4) //
Rotation skewing the idea of curves
.color(1.0, 0.3, 0.6)
.luma(0.2)
.out(o0);

// Background layer: Square "curves" with sharp oscillating
brightness
osc(15, 0.05, 0.8)
.kaleid(4) // Reinforce symmetry of "4"
.modulateRotate(shape(4, 0.4).scale(2).luma(0.5).invert(), 0.3)
.colorama(0.8)
.scale(() => Math.abs(Math.cos(time * 0.4)) * 0.3 + 1.2) //
Dynamic breathing
.blend(noise(3, 0.5).modulate(osc(10), 0.3), 0.5)
.out(o1);
```

```
// Secondary modulation layer: Overlays to distort
"geometry"
osc(8, 0.03, 0.9)
.rotate(() => Math.abs(Math.sin(time * 0.5)) * Math.PI /
4) // Exaggerating square-based rotation
.mult(shape(4, 0.7).rotate(Math.PI / 4))
.modulate(noise(6, 0.3), 0.6)
.add(shape(4, 0.4).scale(2).invert(), 0.4)
.color(0.6, 0.9, 1.0)
.scrollY(0.3, 0.1)
.out(o2);

// Combine all layers with square-distorted interactions
src(o0)
.layer(src(o1).blend(src(o2).scale(0.8).rotate(Math.PI /
4), 0.6))
.modulate(osc(6, 0.05, 0.8).kaleid(4), 0.3)
.add(shape(4, 0.8).scale(1.5).luma(0.5).invert(), 0.2)
.blend(osc(5, 0.2,
0.6).kaleid(4).modulateRotate(noise(2), 0.4), 0.5)
.rotate(0.1,0.003)
.kaleid(4)
.scale(()=>Math.sin(time/7)*0.4 + 0.6)
.contrast(1.5)
.color( ()=>Math.sin(time/4.3), ()=>Math.cos(time/5.2) ,
()=>Math.sin(time/6.1) )
.modulateScale(
src(s0).contrast(3.5)
, ()=>Math.sin(time*4)* 0.075 + 0.175
)
.diff( src(s0).contrast(0.75)
//.color( ()=>Math.sin(time/3), ()=>Math.cos(time/4) )
```





```

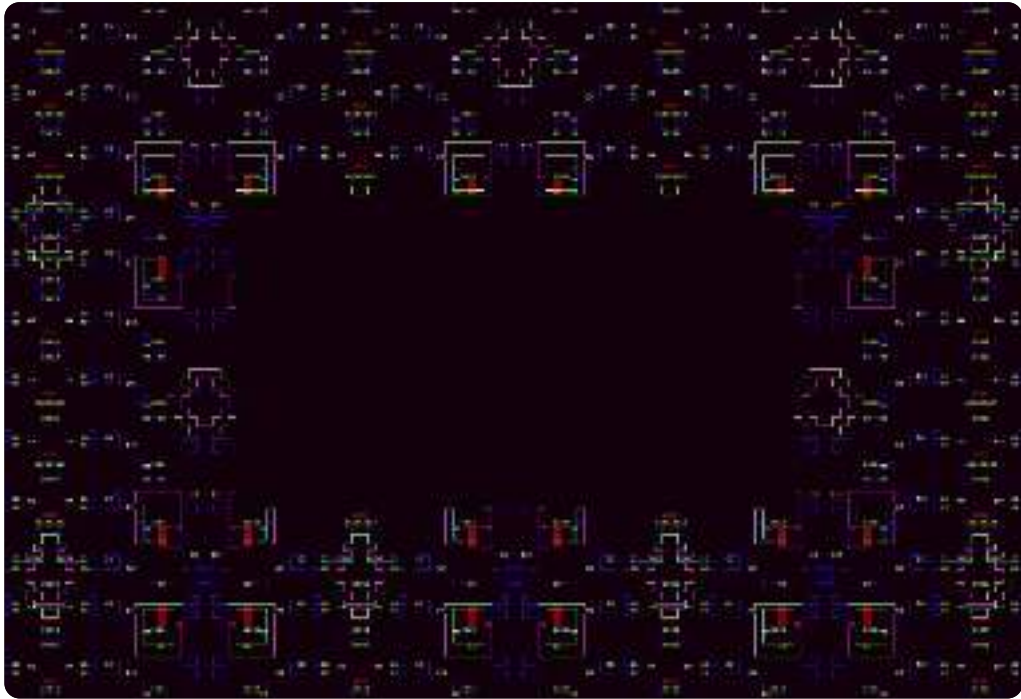
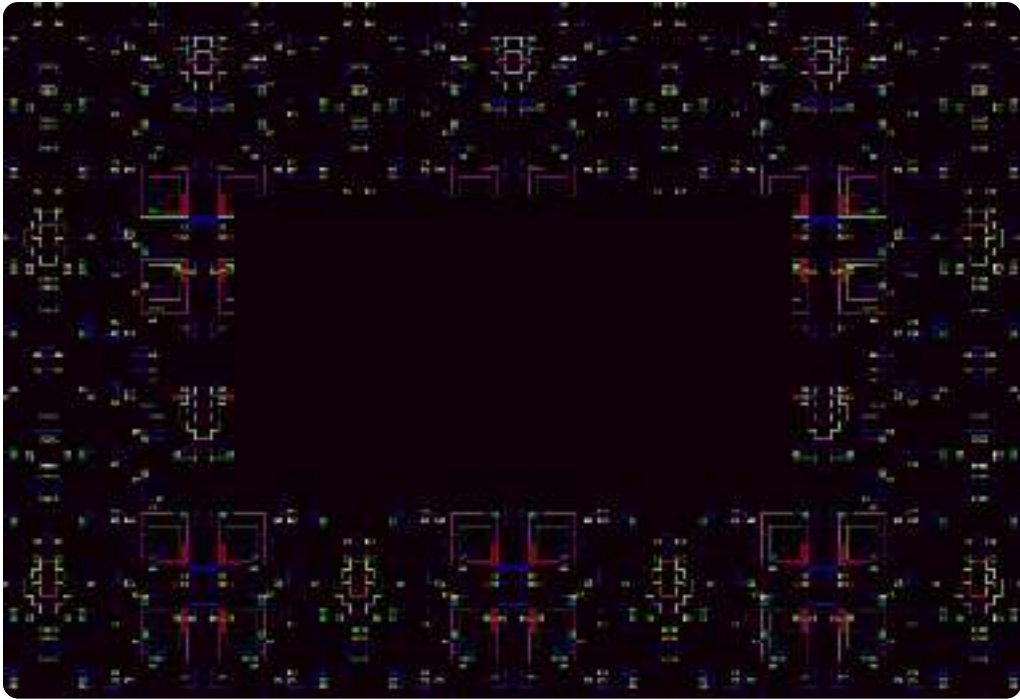
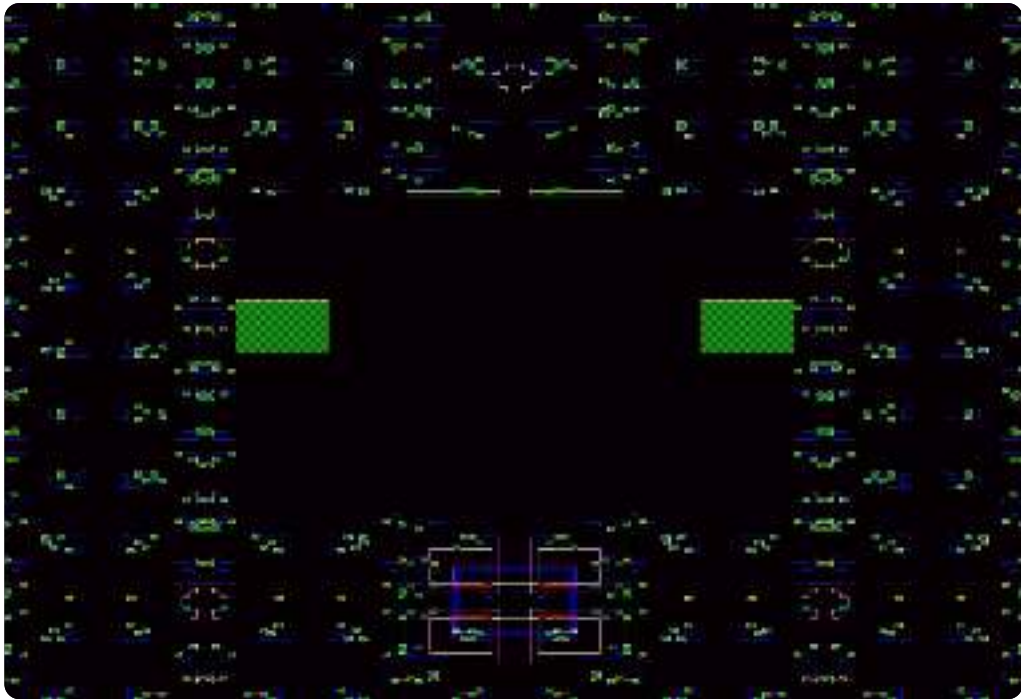
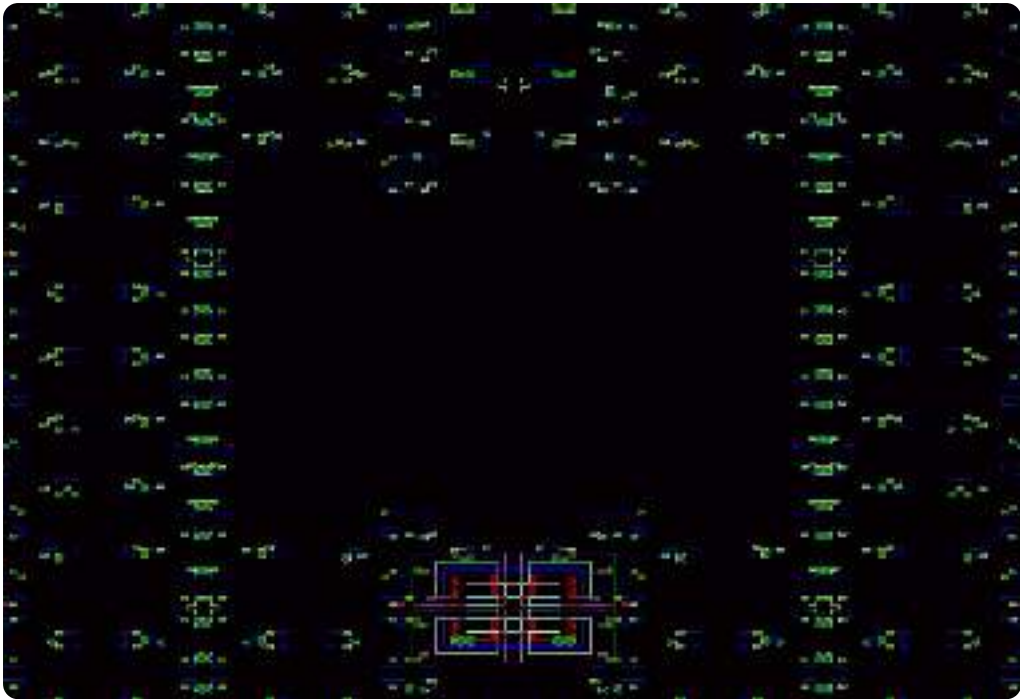
speed=0.75
noise(4,0.00001).thresh(0.5,0.7).scrollY(0.01,0.01).colorama(())=>
Math.sin(time/9.2)*0.1+0.15)
.diff(
noise(2,0.000001).thresh(0.5,0).scrollY(0.01,-0.01).colorama(())=>
Math.cos(time/7.25)*0.31+0.31) )
.pixelate(20,20)
.scale(1,1,())=>Math.sin(time/10.1)*0.6+1.2)
.scrollY(0.01,-0.02)
.modulate( noise(0.9,0.0003) )
.modulateScale( noise(0.8,0.0002) )
.kaleid(2).repeat(2,1).kaleid(2)
.out()

```

O patch se organiza em torno de ruído (**noise**), deslocamento Y (**scrollY**), deslocamento de matiz (**colorama**), limiar (**thresh**) e kaleidoscópio (**kaleid**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**, **modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.75** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

await loadScript("https://hyper-hydra.glitch.me/hydra-blend.js")

await loadScript("https://cdn.statically.io/gl/metagrowing/extra-
shaders-for-hydra/main/lib/lib-screen.js")

speed=0.05
shape(4,0.3,0.02).scrollX(0.025,-0.04).scrollY(0.0250,-0.0401).color(0,0,1)
.diff(
shape(4,0.25,0).scrollX(0.025,-0.045).scrollY(0.025,-0.0451) )
.diff(
shape(4,0.2,0).scrollX(0.025,-0.055).scrollY(0.025,-0.0551) )

.diff( shape(4,0.15,0).scrollX(0.025,0.035).scrollY(0.025,0.0352)
)
.diff( shape(4,0.10,0).scrollX(0.025,0.031).scrollY(0.025,0.032)
)
.diff(
shape(4,0.05,0.02).scrollX(0.025,0.021).scrollY(0.025,0.022).color(1,0,0) )
.out(o1)

src(o1).kaleid(2)
.diff( src(o1).kaleid(4) )
.diff( src(o1).repeat(2).invert().kaleid(4)
.repeat( ()=>Math.sin(time/19)*3.5+5 )
.color( ()=>Math.cos(time/10)*1,
()=>Math.sin(time/11.1)*0.8-0.4,
()=>Math.cos(time/12.2)*1
)
.modulateScale(src(o1).kaleid(4)) )

```

```

.rotate(Math.PI/2).repeat([2,4,4,8,8,10,4,2].fast(0.7),
[2,2,4,8].fast(0.35))
.scrollY(0.1,0.02)
.pixelate(160,160)

.modulateScale( shape(4,0.45,0), 10.99 )
.modulateRotate( shape(4,0.45,0), Math.PI/-1 )

.out(o2)

src(o2).edge().out(o3)

src(o2)

.negate(src(o3).dilate().dither2(), 1)

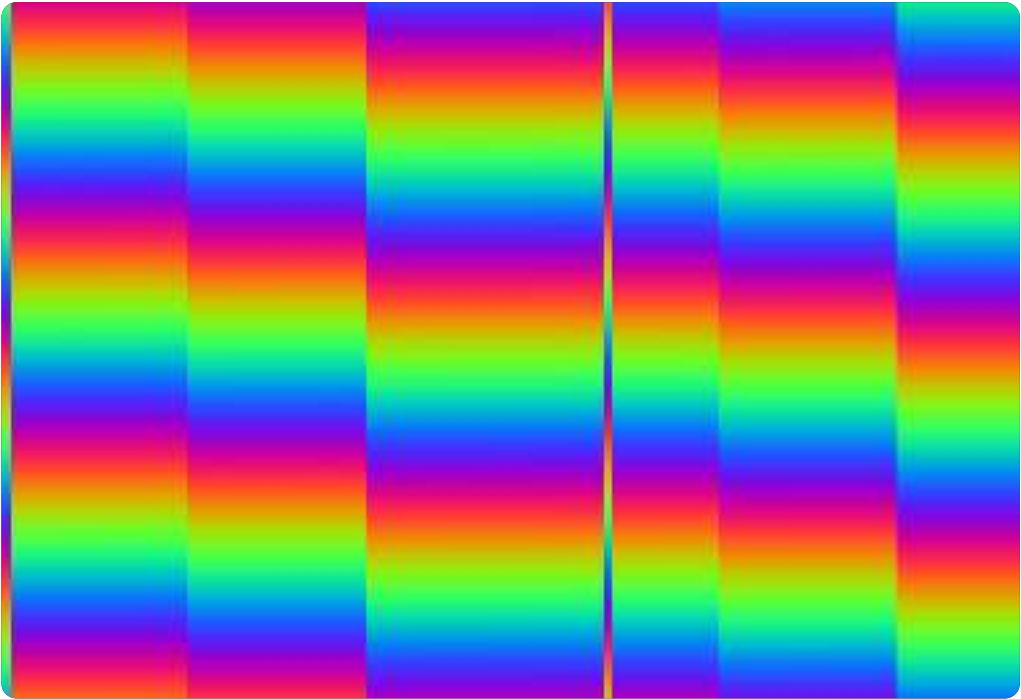
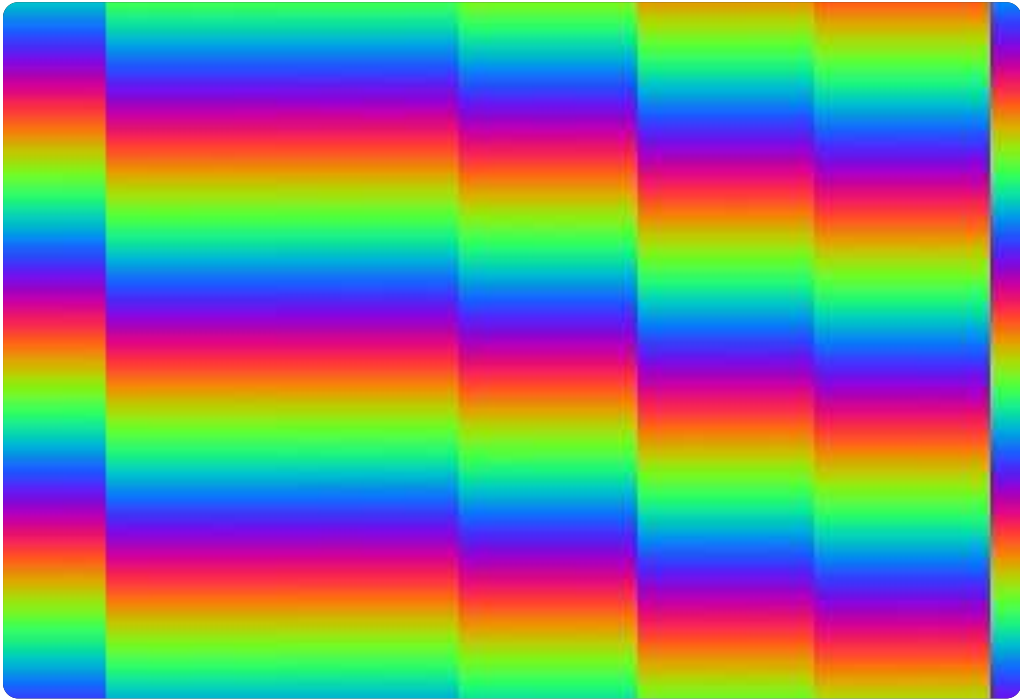
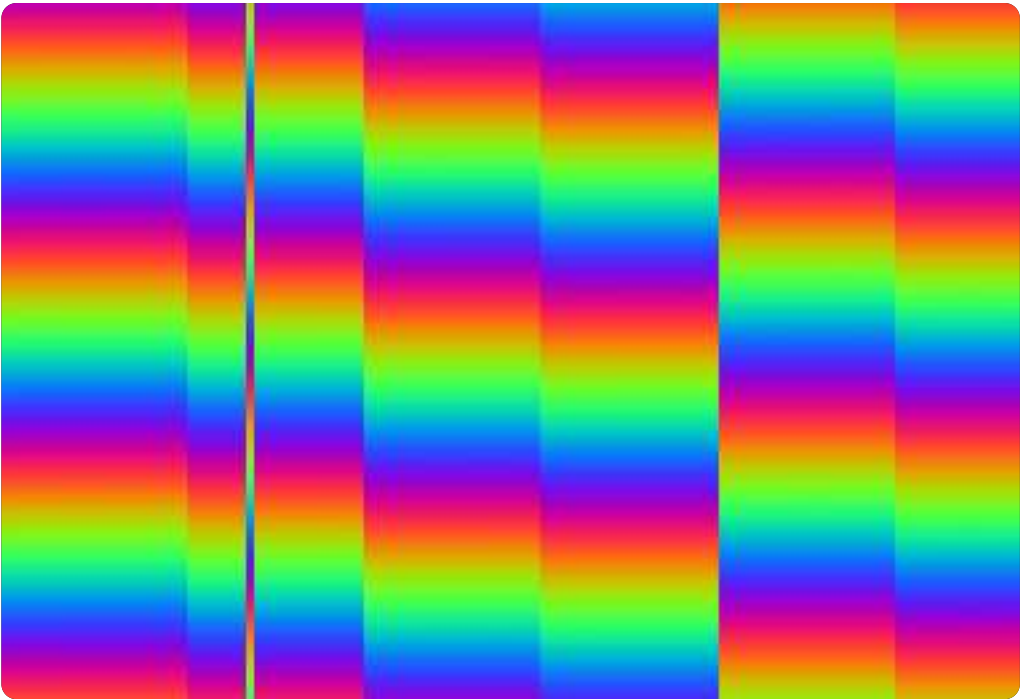
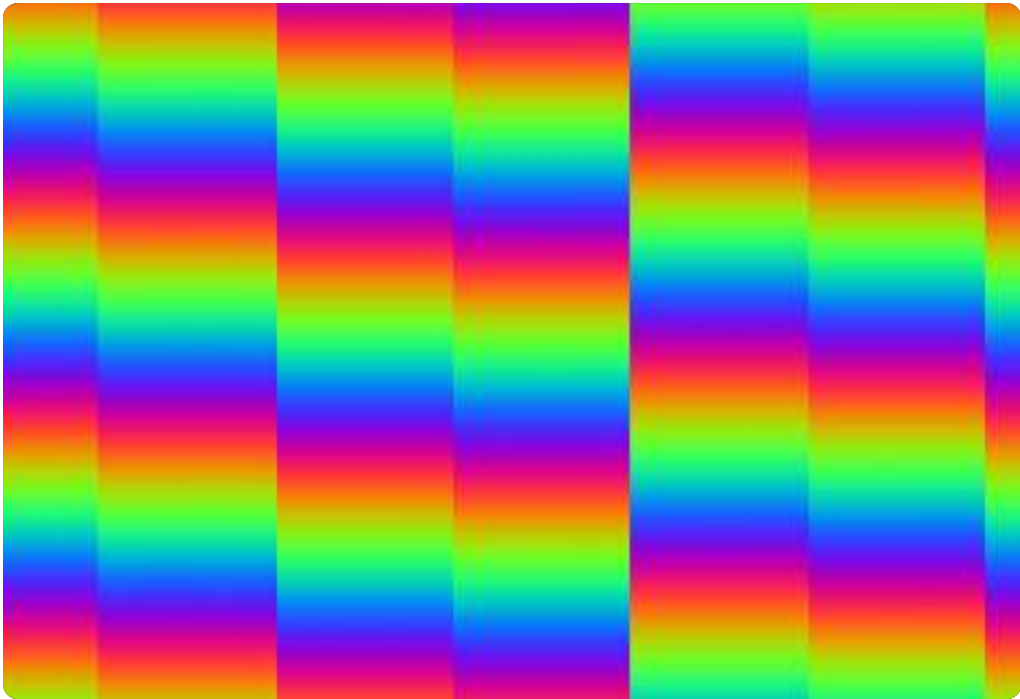
.out()

```

O patch se organiza em torno de formas geométricas (**shape**), deslocamento Y (**scrollY**), diferença (**diff**), fontes externas (**src**) e deslocamento X (**scrollX**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**, **modulateRotate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed** = **0.05** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**src**, **loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima





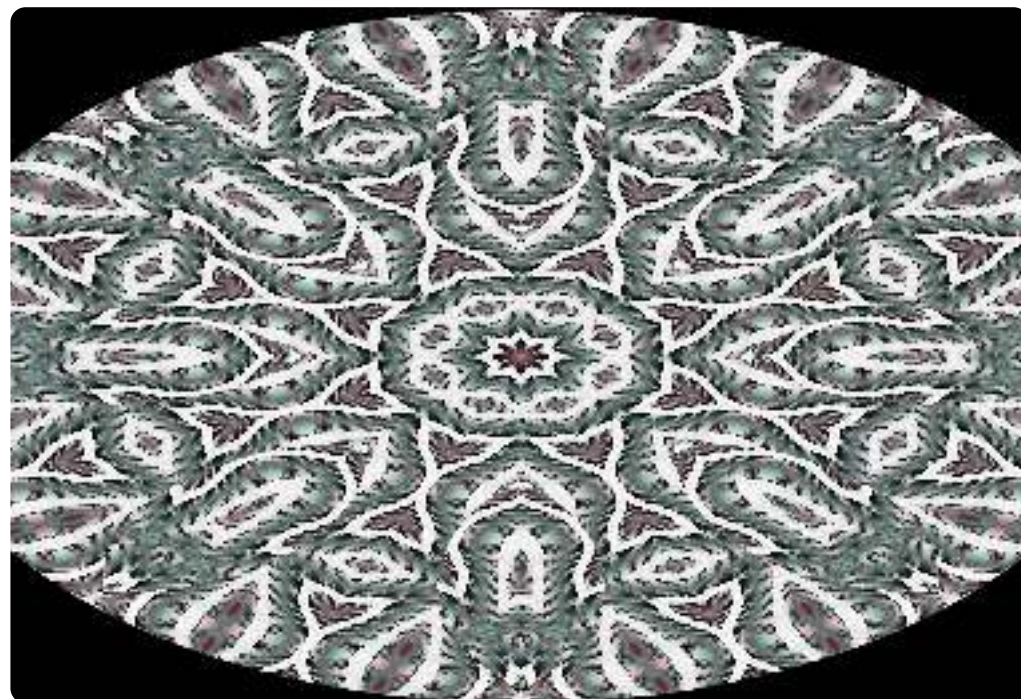
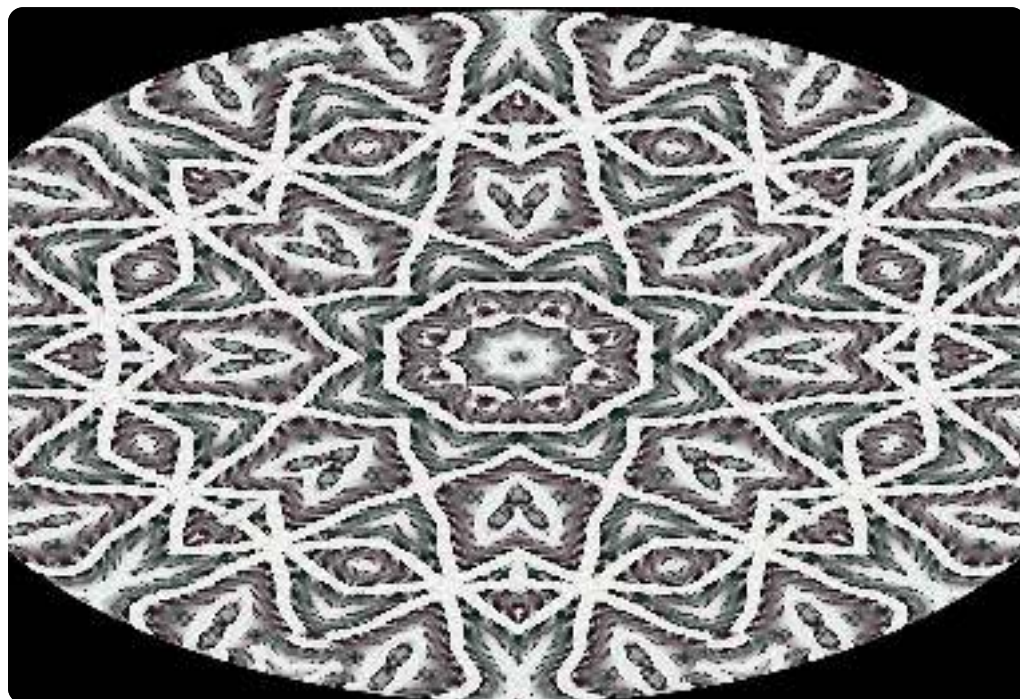
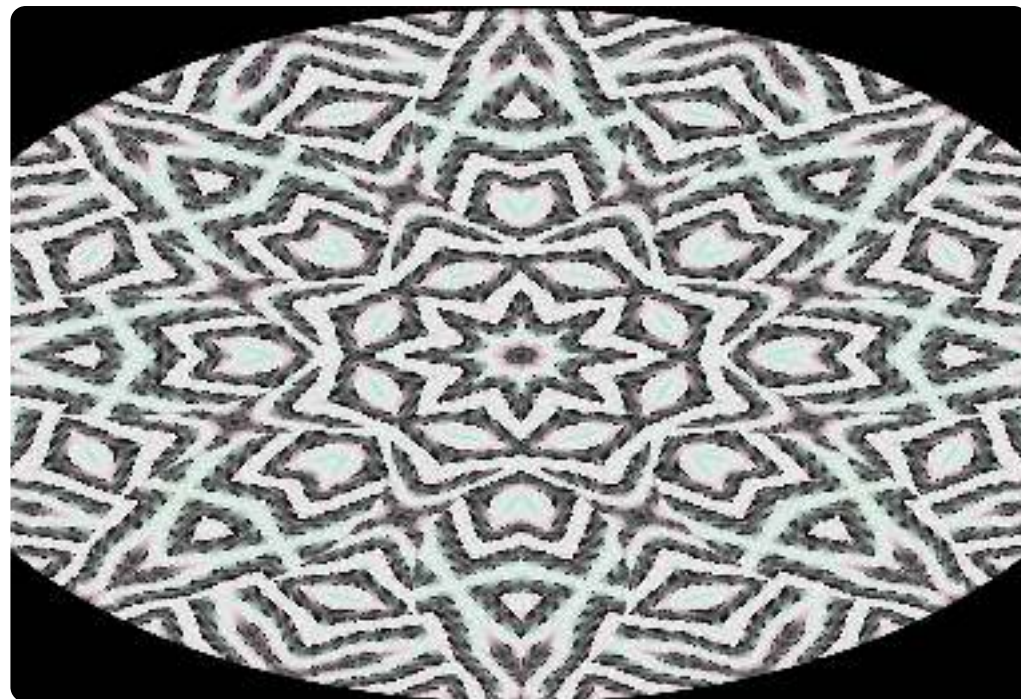
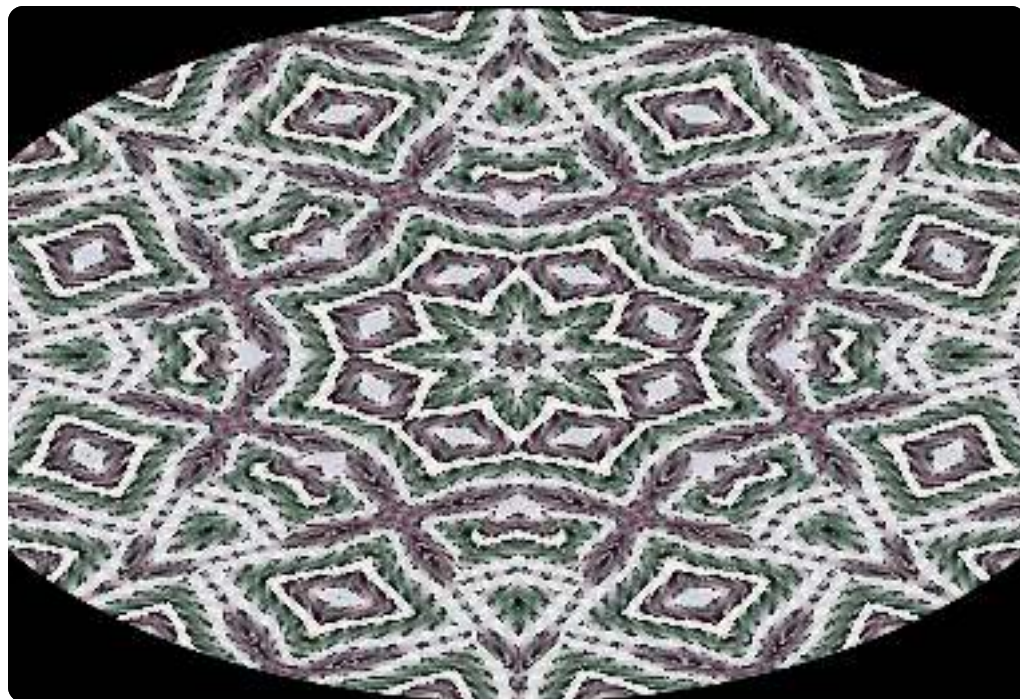
```
//s0.initScreen()

osc(20,-0.09,2).rotate(Math.PI/-2)
  //.colorama( ()=>Math.sin(time/2)*0.2 )
.modulate(src(s0)
  .saturate(0)
  .contrast(0.9)
  .scale(1),0.5 )
.out(o0)
```

O patch se organiza em torno de modulação (**modulate**), saturação (**saturate**), contraste (**contrast**), rotação (**rotate**) e escala (**scale**).

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. Há entrada externa (**src**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-
shaders-for-hydra/main/lib/lib-screen.js")
```

```
speed=0.19
```

```
osc(100,-0.01)
```

```
//.thresh(0.5,0.5)
```

```
.kaleid(4)
```

```
.rotate(0.05,0.04)
```

```
.brightness(-0.25)
```

```
.dither2()
```

```
.diff(
```

```
  osc(20,-0.01, ()=>Math.sin(time)*0.5+12.5 )
```

```
  .kaleid(2)
```

```
  .rotate(0.05,0.04)
```

```
)
```

```
.modulate(
```

```
  noise( ()=>Math.sin(time/13)*1+2 ,0.005)
```

```
)
```

```
.kaleid(4)
```

```
.colorama( ()=>Math.sin(time/91)*0.75-0.2).repeat(2,2)
```

```
.modulate(osc(10,-0.041).thresh(0.5,0).kaleid(64) )
```

```
.repeat(2,2)
```

```
.kaleid(8)
```

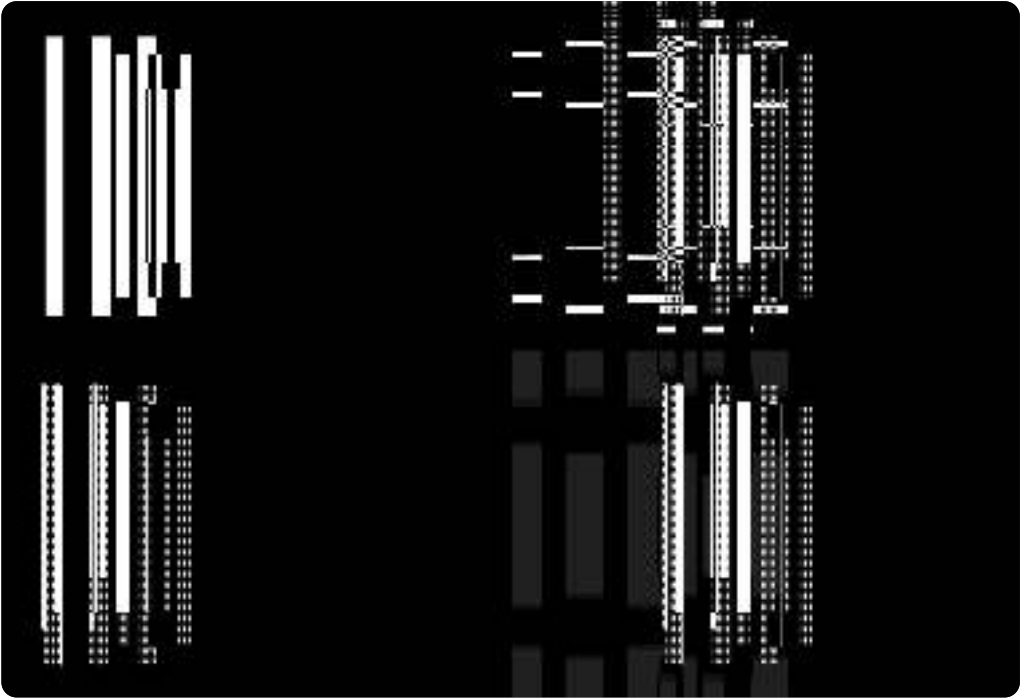
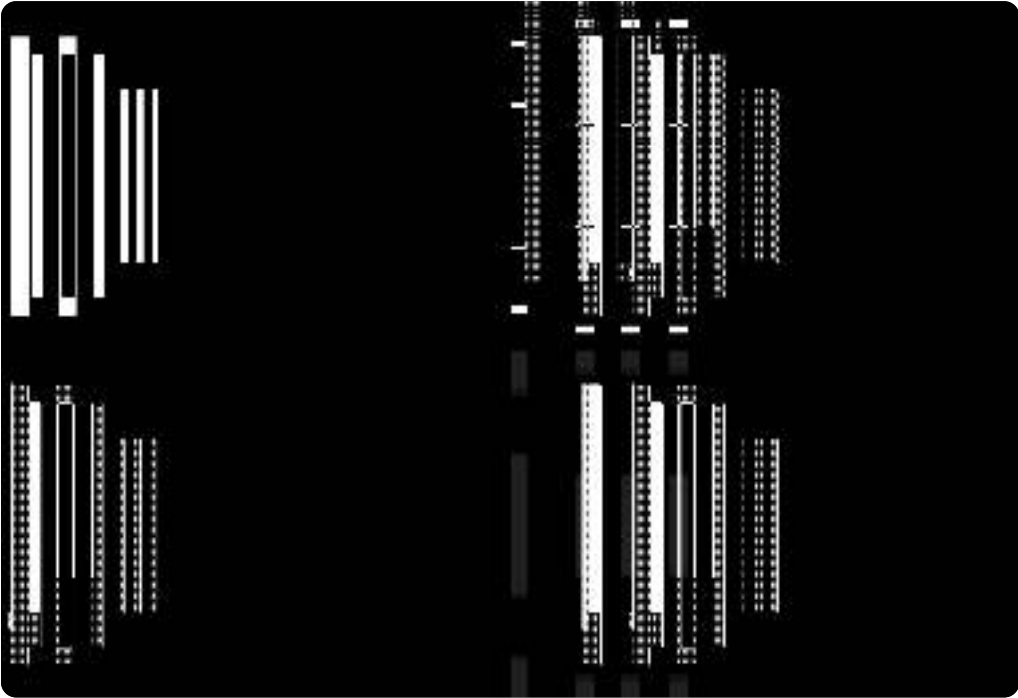
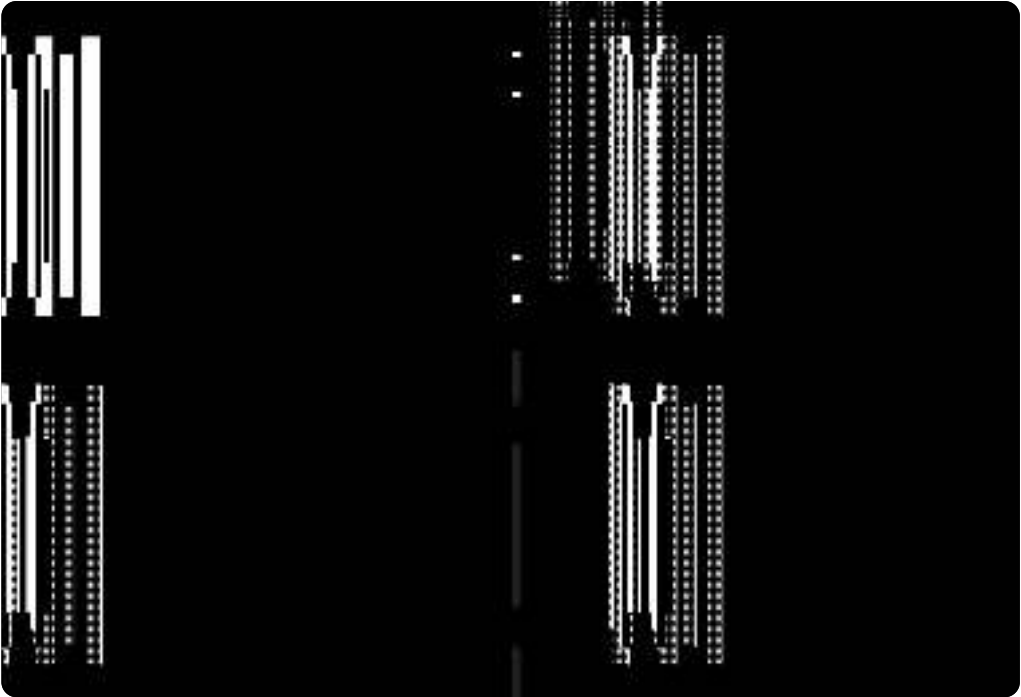
```
.mult( shape(128,0.992,0.0025) )
```

```
.out()
```

O patch se organiza em torno de kaleidoscópio (**kaleid**), osciladores (**osc**), modulação (**modulate**), rotação (**rotate**) e repetição (**repeat**). Osciladores viram padrões binários via **thresh**, e a geometria resultante alimenta o restante da cadeia. A composição usa **mult**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.19** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```
await loadScript("https://cdn.statically.io/gl/metagrowing/extra-
shaders-for-hydra/main/lib/lib-screen.js")
```

```
speed=0.5
```

```
shape(4,1,0.001).scale(1,0.01,0.5).scrollX(
()=>Math.sin(time/5)*0.2,0.01).scrollX(-0.025,0)
.add( shape(4,1,0.001).scale(1,0.01,0.5).scrollX(
()=>Math.sin(time/5)*0.2,0.01) )
.add( shape(4,1,0.001).scale(1,0.01,0.5).scrollX(
()=>Math.sin(time/5)*0.2,0.01).scrollX(+0.025,0) )
```

```
.add(shape(4,1,0.001).scale(1,0.02,0.7).scrollX(
()=>Math.sin(time/6)*0.2,0.01).scrollX(-0.05,0) )
.add( shape(4,1,0.001).scale(1,0.02,0.7).scrollX(
()=>Math.sin(time/6)*0.2,0.01) )
.add( shape(4,1,0.001).scale(1,0.02,0.7).scrollX(
()=>Math.sin(time/6)*0.2,0.01).scrollX(+0.05,0) )
```

```
.diff( shape(4,1,0.001).scale(1,0.03,0.8).scrollX(
()=>Math.sin(time/7)*0.2,0.01).scrollX(-0.075,0) )
.diff( shape(4,1,0.001).scale(1,0.03,0.8).scrollX(
()=>Math.sin(time/7)*0.2,0.01) )
.diff( shape(4,1,0.001).scale(1,0.03,0.8).scrollX(
()=>Math.sin(time/7)*0.2,0.01).scrollX(+0.075,0) )
.out(o0)
```

```
src(o0)
.modulate( src(o0).scale(1.01).dither4() )
.add( src(o0).scale(3,1,0.2) , 0.05 )
.add( src(o0).scale(3,1,0.19) , 0.075 )
```

```
src(o0)
.modulate( src(o0).scale(1.01).dither4() )
.diff( src(o0).scale(3,1,0.2) , 0.05 )
.diff( src(o0).scale(3,1,0.19) , 0.075 )
//.scale(1,-1,1)
.out(o2)
```

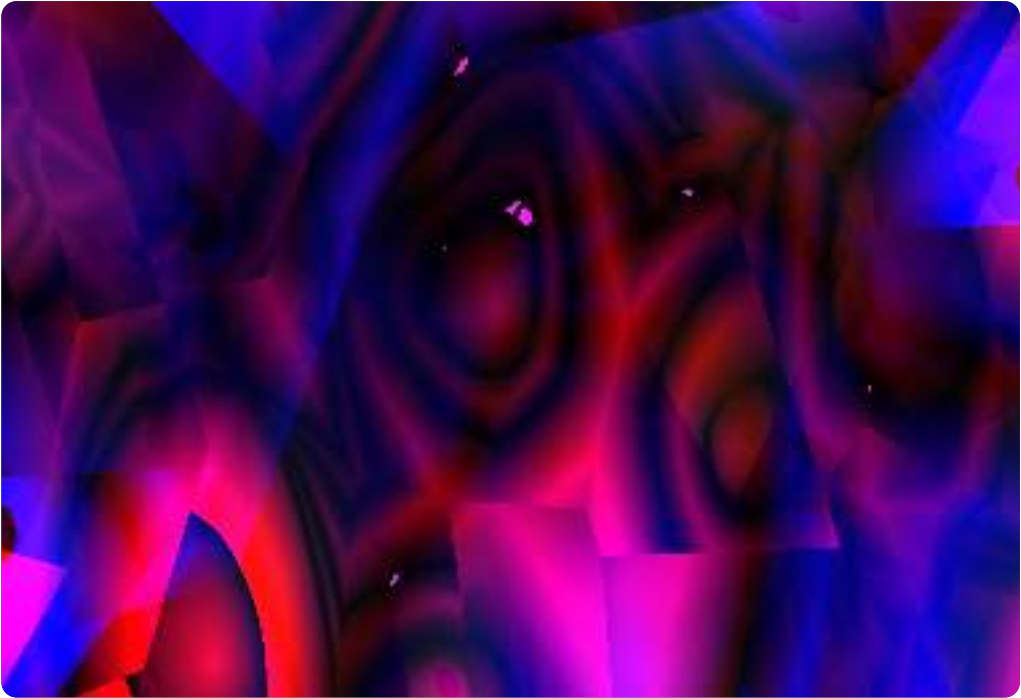
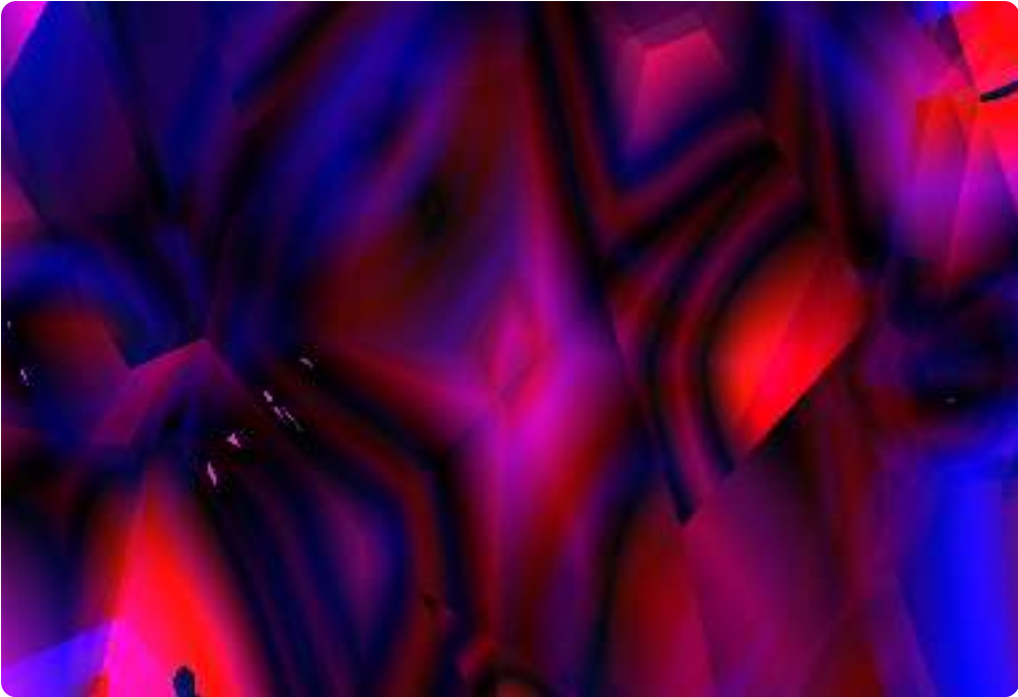
```
src(o0)
.modulate( src(o0).scale(1.01).dither4() )
.out(o1)
```

```
render()
```

O patch se organiza em torno de escala (**scale**), deslocamento X (**scrollX**), fontes externas (**src**), formas geométricas (**shape**) e soma de camadas (**add**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.5** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**src**, **loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares.





```
speed=-0.5
```

```
voronoi(1.5,0.03/2,1.25).color(2,0,0)
.diff( voronoi(1.6,-0.04/2,1.25).color(0,2,0) )
.diff( voronoi(1.7,0.05/2,1.25).color(0,0,2) )
.kaleid(2)

.diff( voronoi(1.8,-0.06/2,1.25).color(2,0,0) )
.diff( voronoi(1.9,0.07/2,1.25).color(0,2,0) )
.diff( voronoi(1.0,-0.08/2,1.25).color(0,0,2) )
.rotate(0.005,-0.005)

.diff( voronoi(1.8/2,-0.06/3,1.25).color(2,0,0) )
.diff( voronoi(1.9/2,0.07/3,1.25).color(0,2,0) )
.diff( voronoi(1.0/2,-0.08/3,1.25).color(0,0,2) )
.rotate(0.003,-0.003)

.diff( voronoi(1.8/3,-0.06/4,1.25).color(1,0,0) )
.diff( voronoi(1.9/3,0.07/4,1.25).color(0,1,0) )
.diff( voronoi(1.0/3,-0.08/4,1.25).color(0,0,2) )
.rotate(0.002,-0.002)

//.kaleid(4)

.colorama(-0.025)

//.kaleid(16)

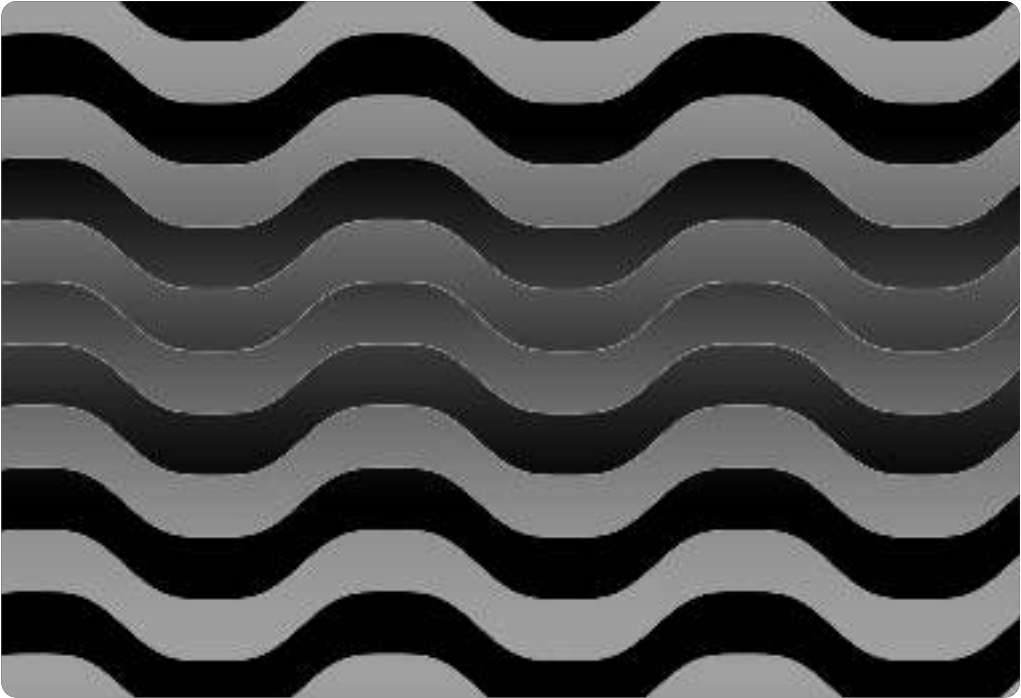
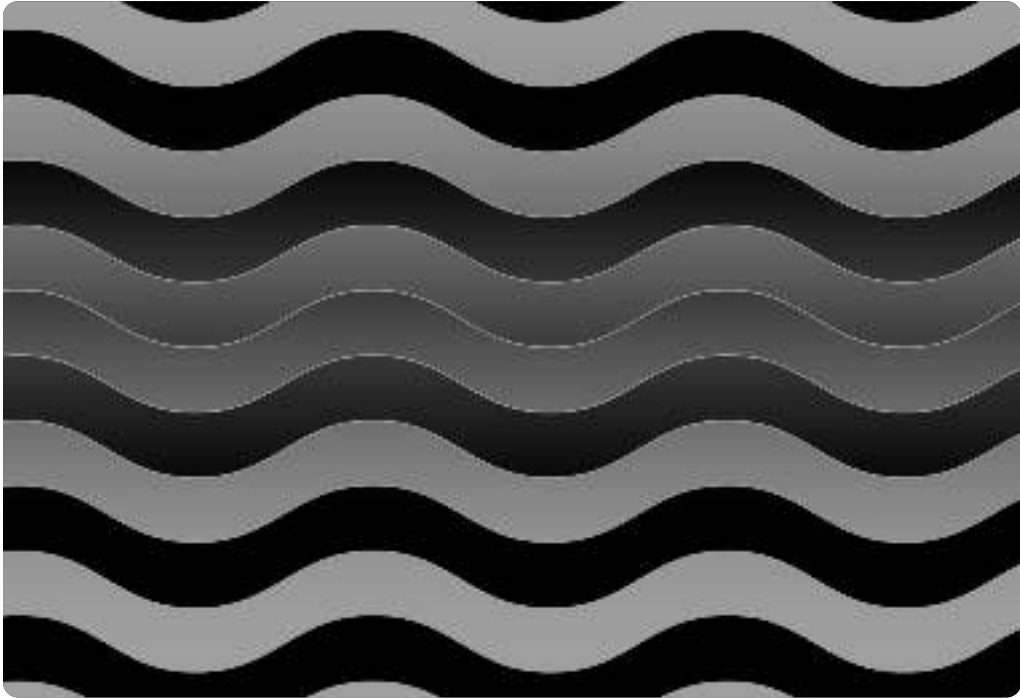
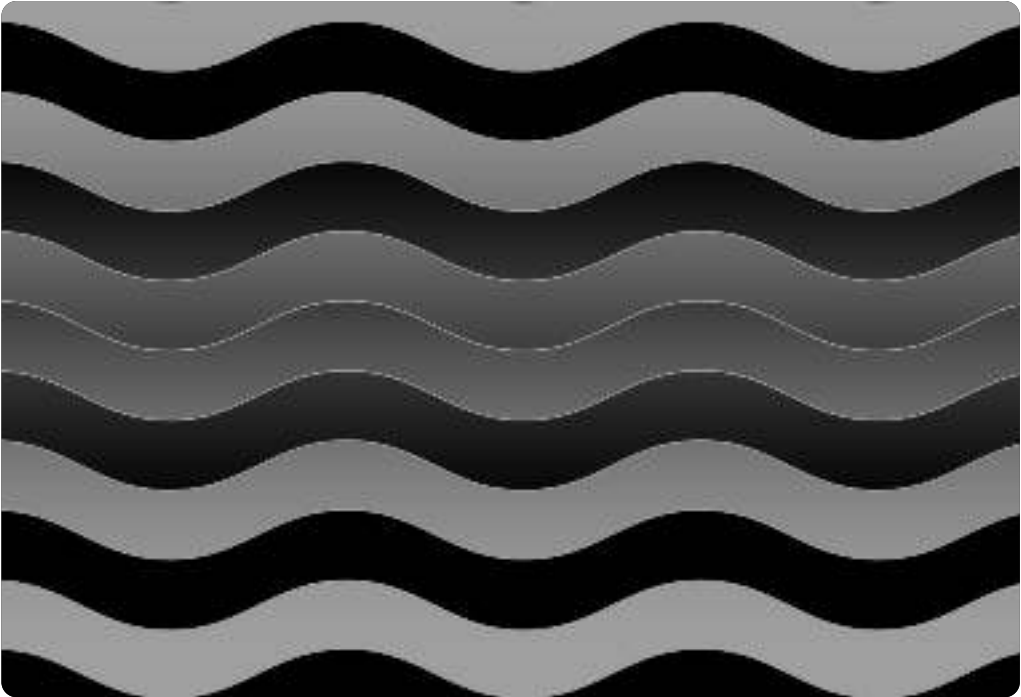
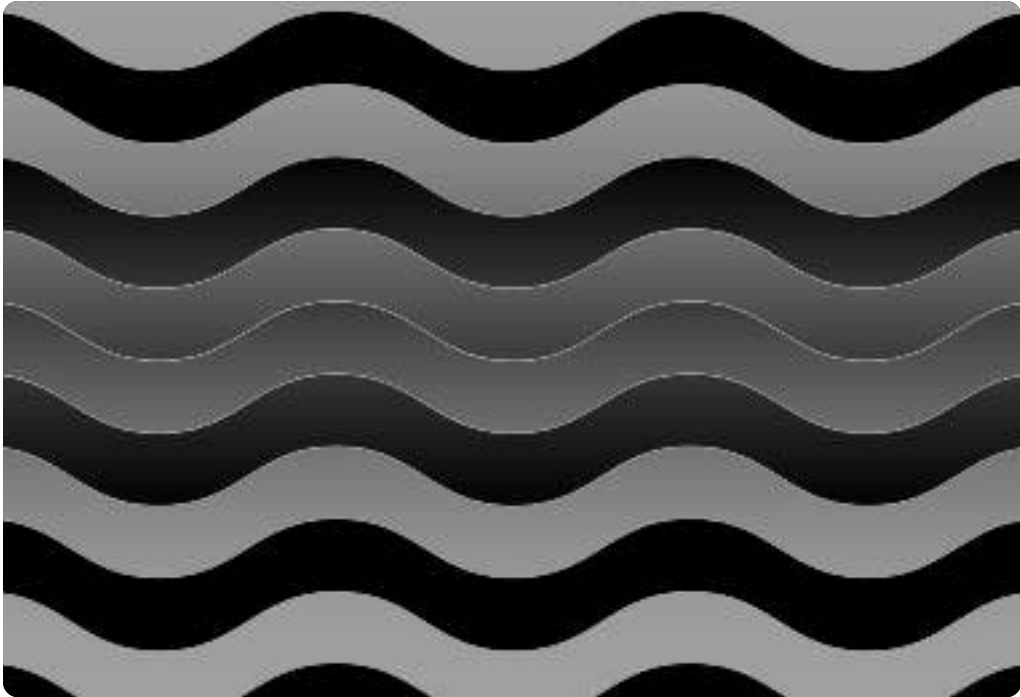
.scale(1,1/3,1)
.color(1.5,0.1,1.75)

.out()
```

O patch se organiza em torno de colorização (**color**), Voronoi (**voronoi**), diferença (**diff**), rotação (**rotate**) e deslocamento de matiz (**colorama**). A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```
speed=0.25

osc(22,-0.004).thresh(0.5, ()=>Math.sin(time*0.4)*0.25+0.75)
.out(o1)

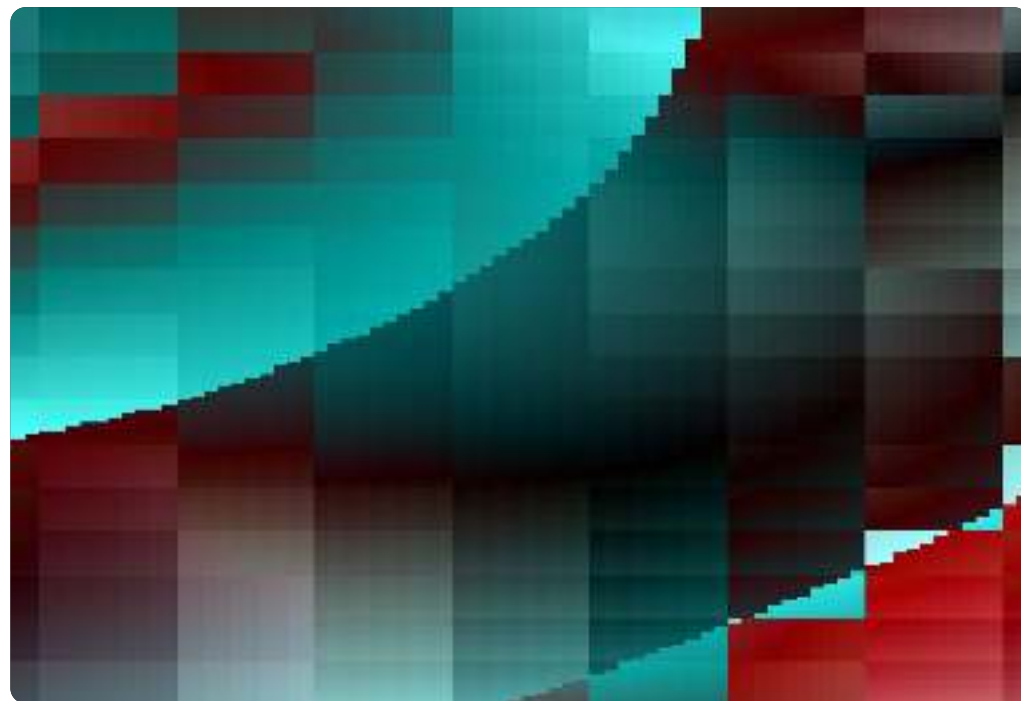
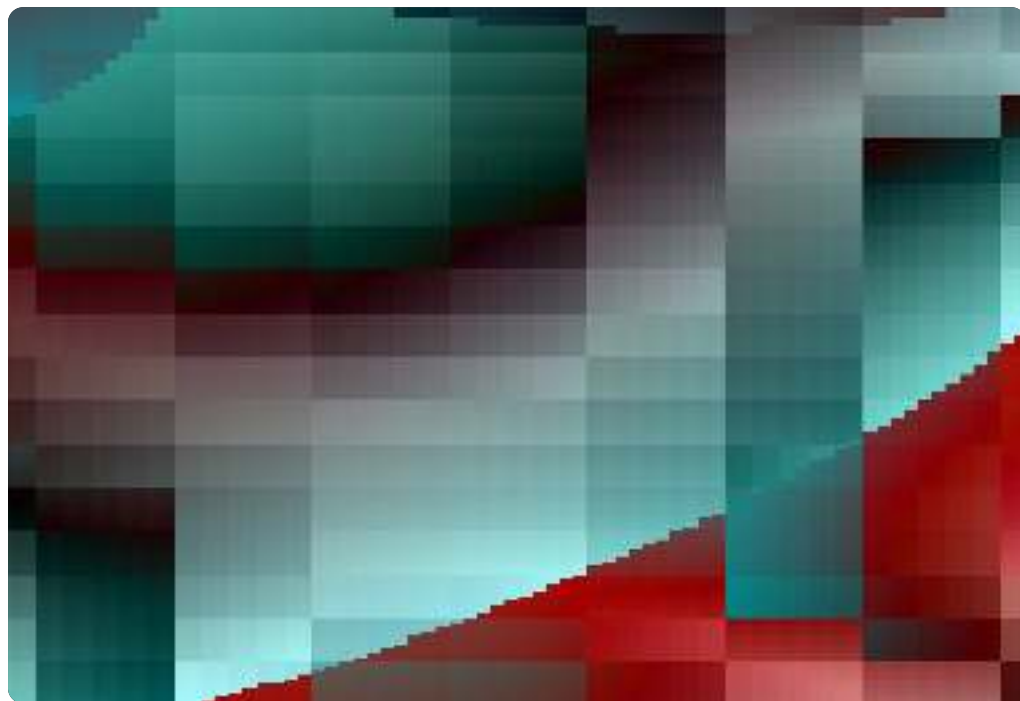
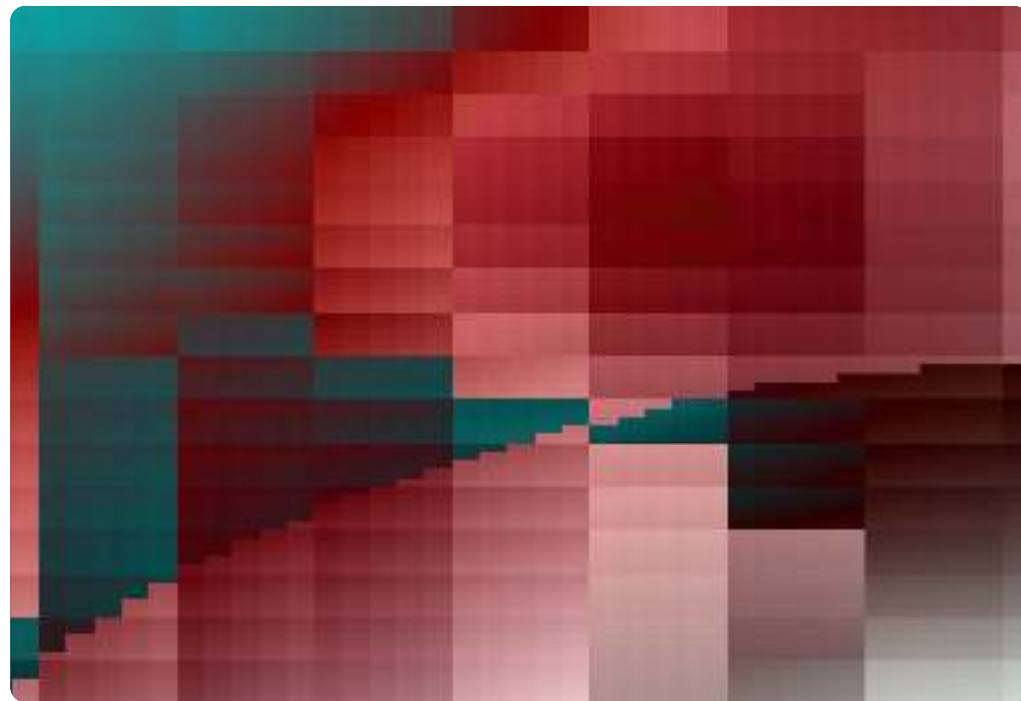
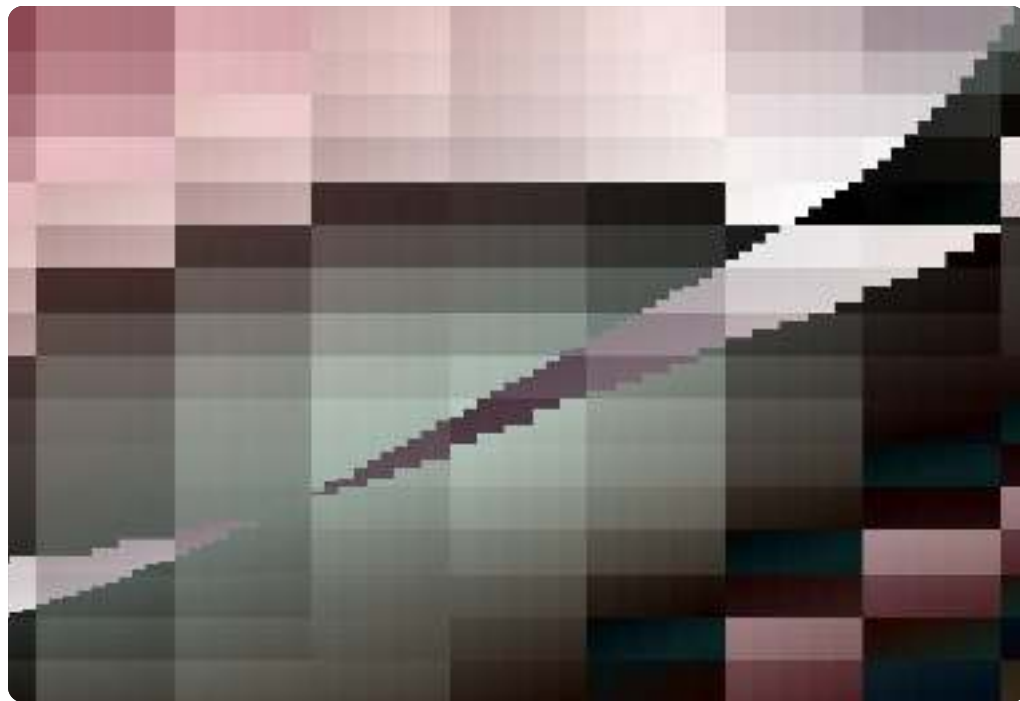
osc( ()=>Math.sin(time*0.05)*10+30
,-0.005).rotate(Math.PI/2).thresh().modulate( src(o1) ).out(o2)

osc(3,0.002).rotate(Math.PI/2).contrast(0.75).brightness(-0.25)
.modulate( src(o2).contrast(10) )
.out()
```

O patch se organiza em torno de osciladores (**osc**), modulação (**modulate**), contraste (**contrast**), rotação (**rotate**) e limiar (**thresh**). Osciladores viram padrões binários via **thresh**, e a geometria resultante alimenta o restante da cadeia.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.25** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**src**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares.





```
noise(1,0.04)
.pixelate(9,16).colorama(-0.1)

.diff(
  noise(0.5,0.04/2)
  .pixelate(9*5,16*5).colorama(-0.05)
)

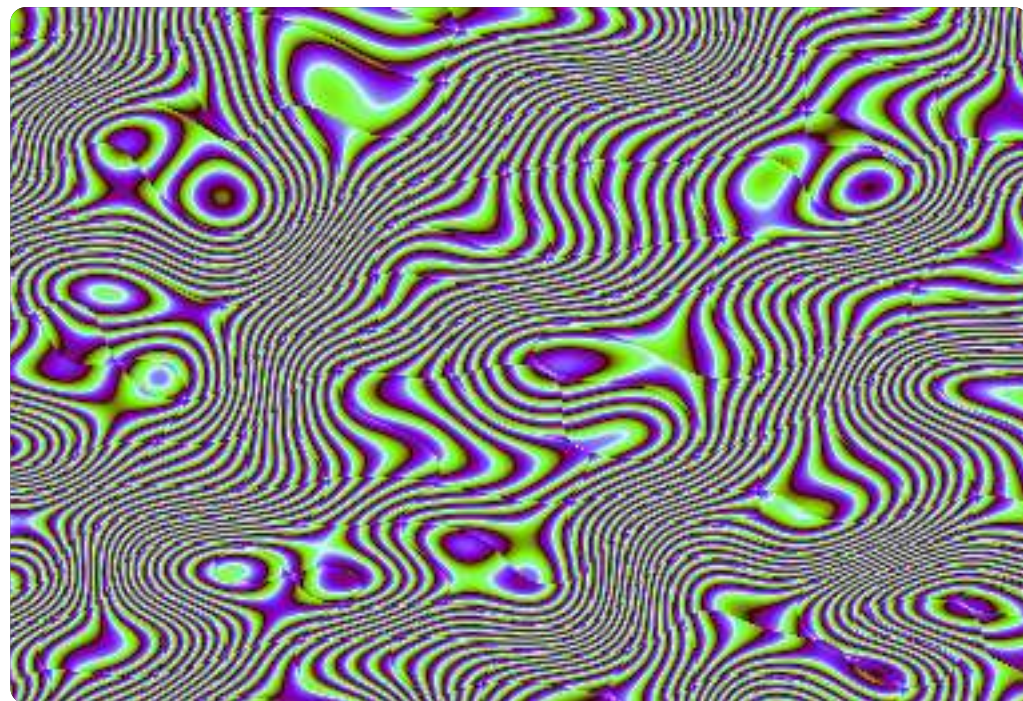
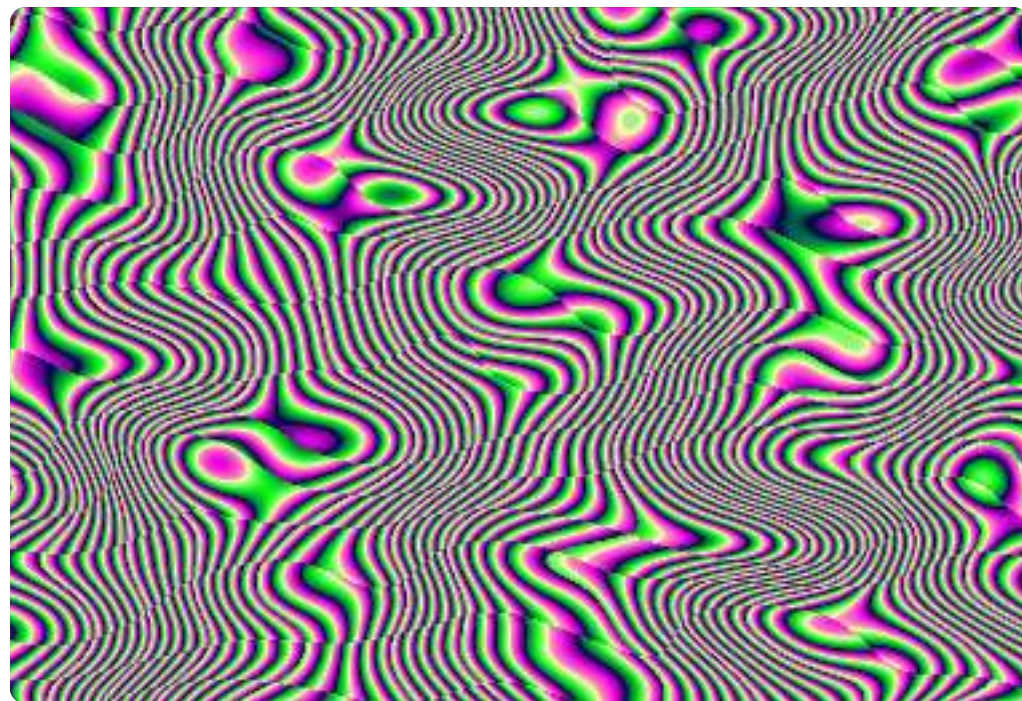
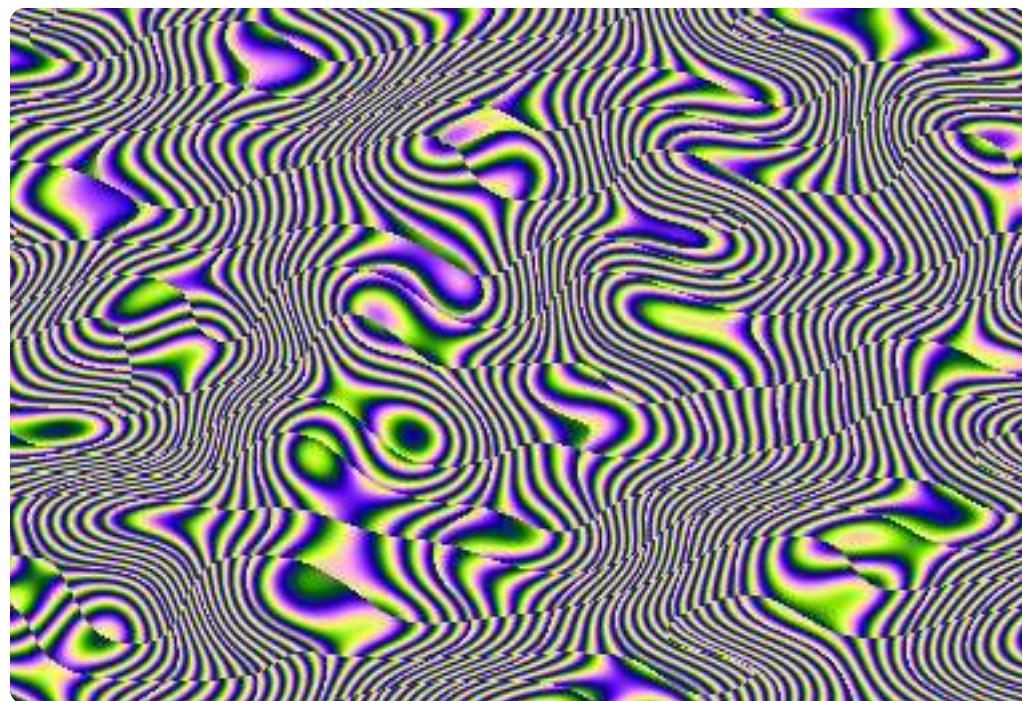
.diff(
  noise(0.25,0.04/8)
  .pixelate(9*10,16*10).colorama(-2)
)

.out()
```

O patch se organiza em torno de pixelização (**pixelate**), deslocamento de matiz (**colorama**), ruído (**noise**) e diferença (**diff**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.



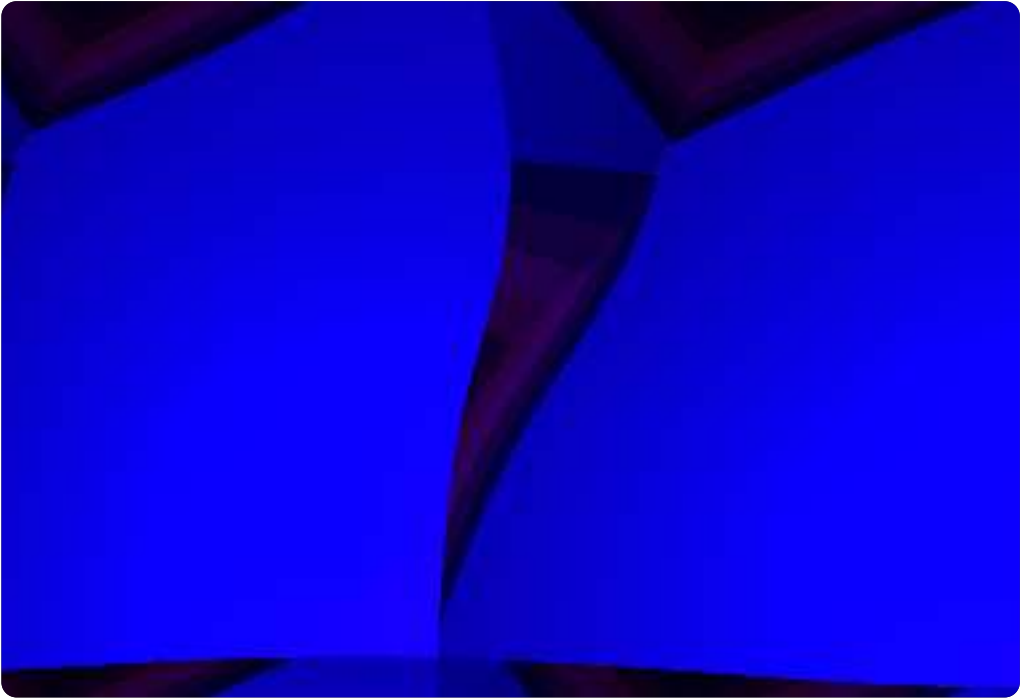
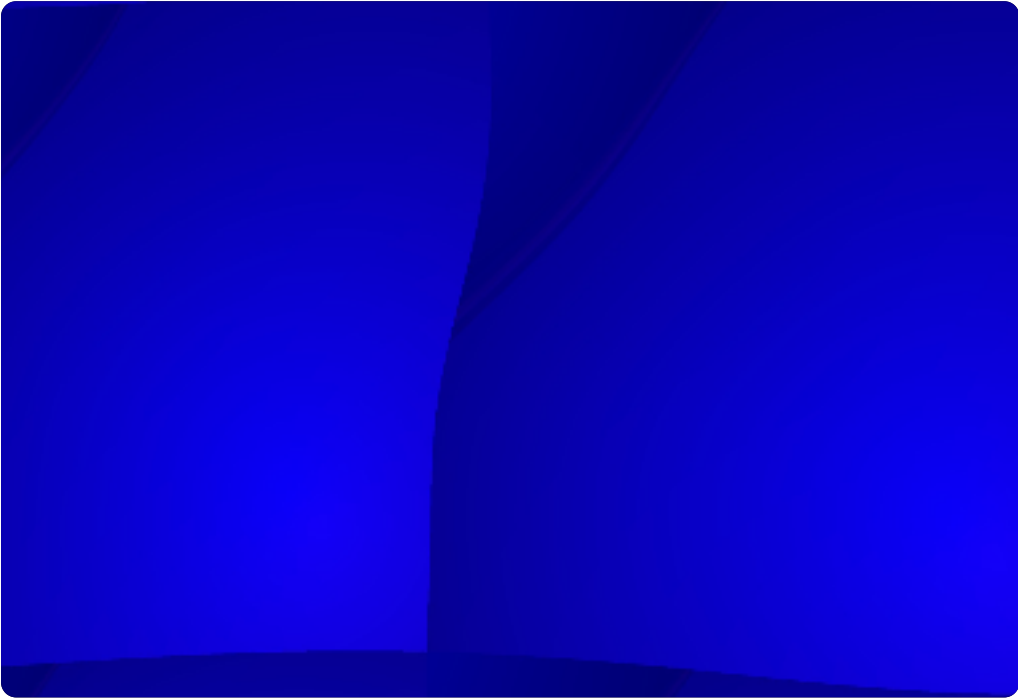


```
osc(30, 0.1, 1)
.hue(() => time * 0.1)
.rotate(() => Math.sin(time) * 0.5)
.repeat(12, 12)
.modulate(noise(4,0.1))
//.mask(shape(100, 0.5, 0.1))
.out(o0)
```

O patch se organiza em torno de modulação (**modulate**), rotação (**rotate**), repetição (**repeat**), ruído (**noise**) e osciladores (**osc**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra.





```
voronoi(1,0.015).color(0,0,1)
.add( voronoi(1,0.016).color(1,0,0) )
.add( voronoi(1,0.017).color(1,0,1) )
.add( voronoi(1,0.018).color(0,0,1) )
.diff( voronoi(1,0.019).color(1,0,1) )
.diff( voronoi(1,0.020).color(1,0,0) )
.mult( voronoi(1,0.021).color(1,0,1) )
.repeat(2,1)

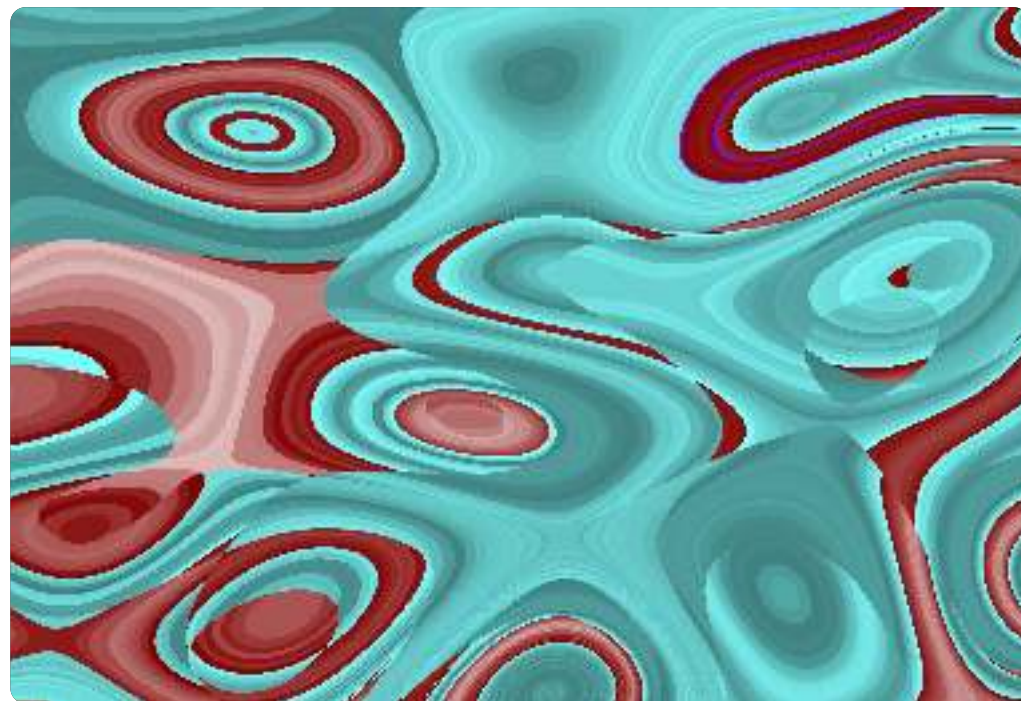
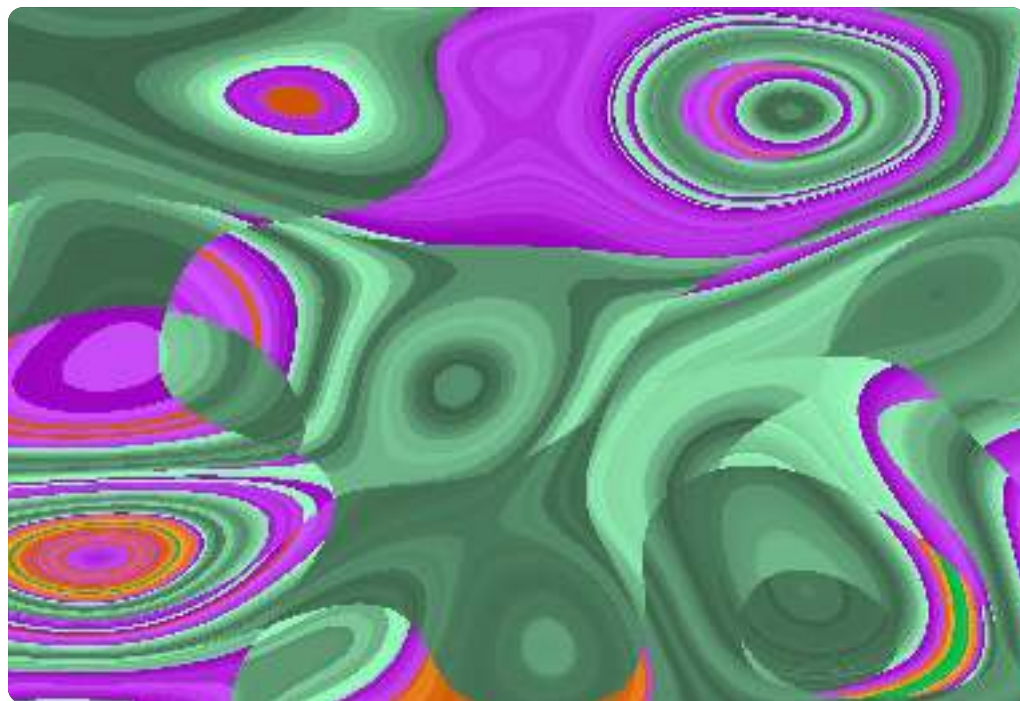
.modulate( noise(1,0.01).contrast(0.5) )

.out()
```

O patch se organiza em torno de Voronoi (**voronoi**), colorização (**color**), soma de camadas (**add**), diferença (**diff**) e modulação (**modulate**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo. A composição usa **mult**, **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.



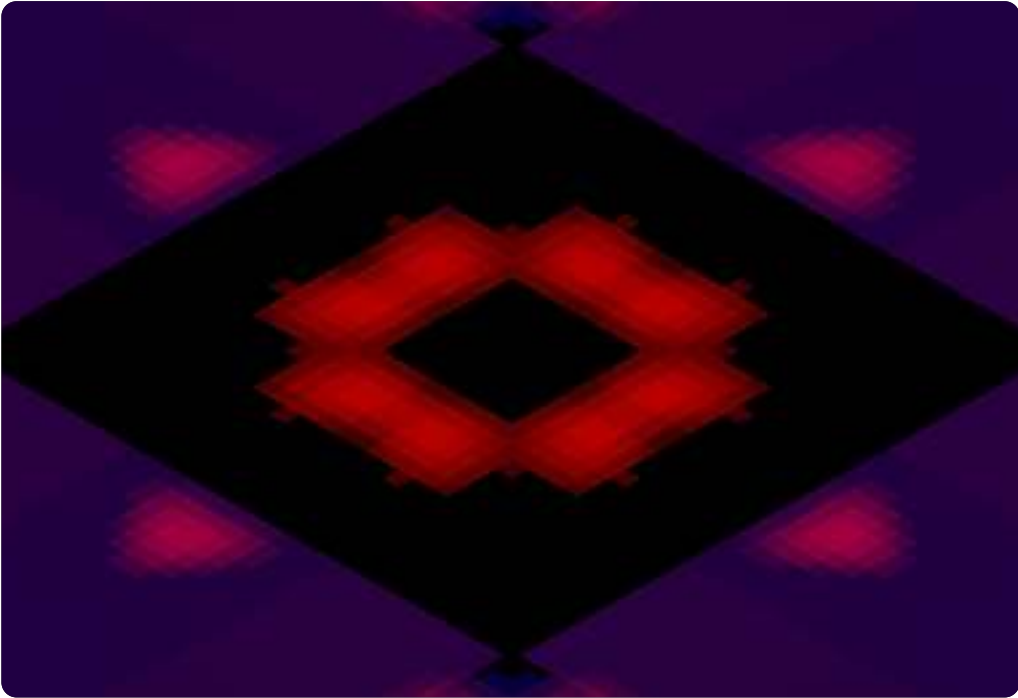
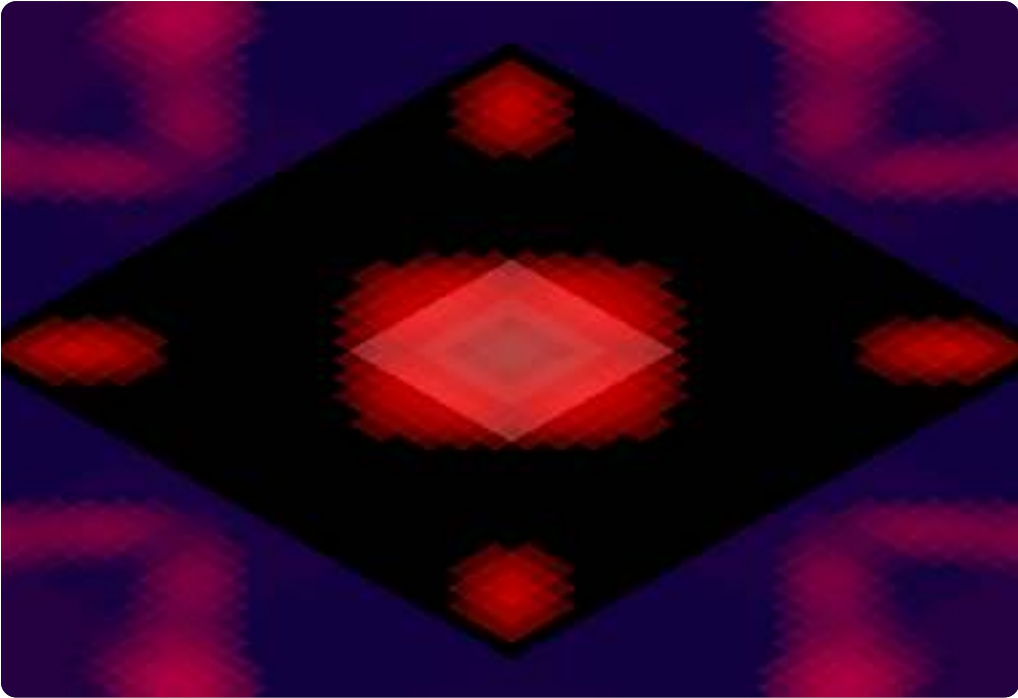
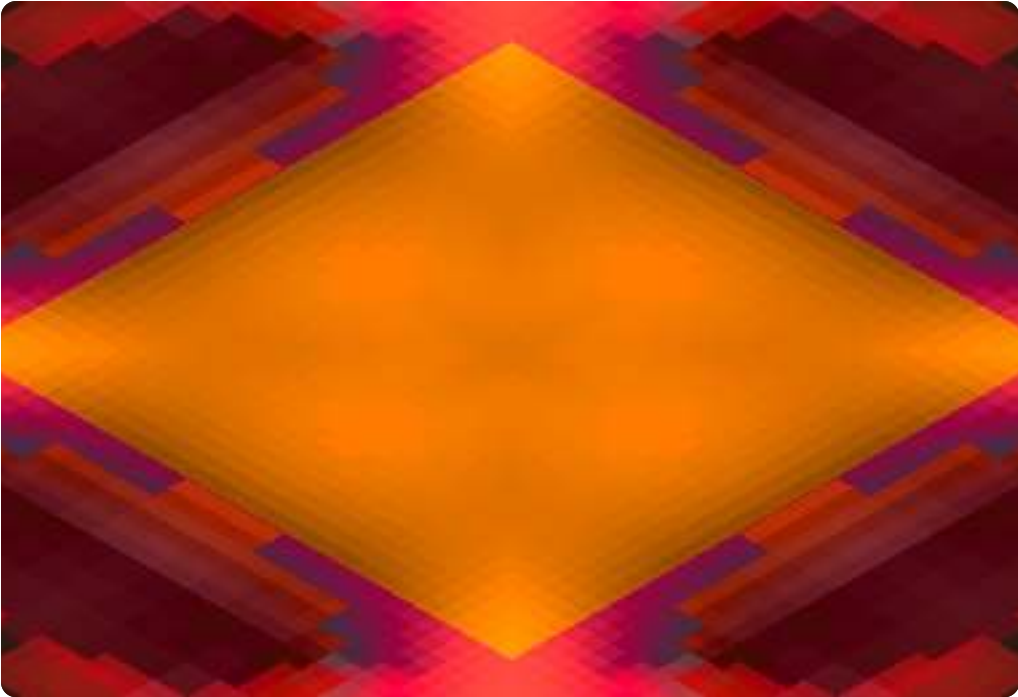


```
speed=0.3
noise(1,0.1/2).pixelate(16,9)
.mult( noise(2,0.1/4).pixelate(16*2,9*2) )
.diff( noise(2.5,0.1/6).pixelate(16*4,9*4) )
.diff( noise(3,0.1/8).pixelate(16*8,9*8) )
.modulatePixelate( noise(2,0.1/2), ()=>Math.sin(time/9)+1.5 )
.colorama( ()=>Math.sin(time/9)-0.5 )
.out()
```

O patch se organiza em torno de ruído (**noise**), pixelização (**pixelate**), diferença (**diff**), modulação por pixelate (**modulatePixelate**) e deslocamento de matiz (**colorama**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo. A composição usa **mult**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulatePixelate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.3** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

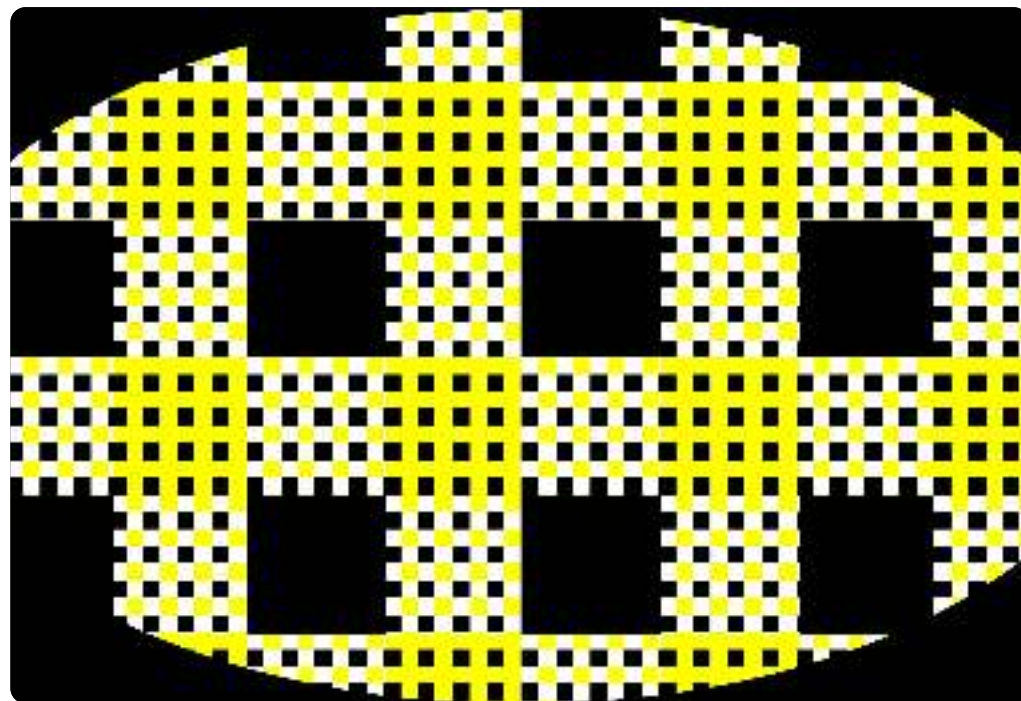
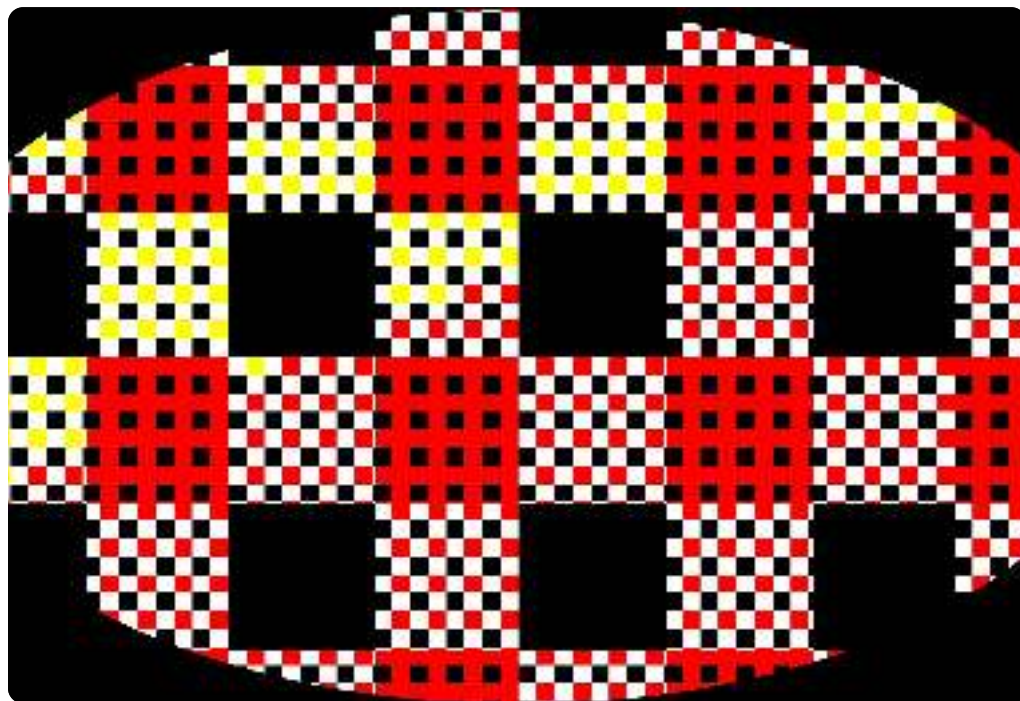
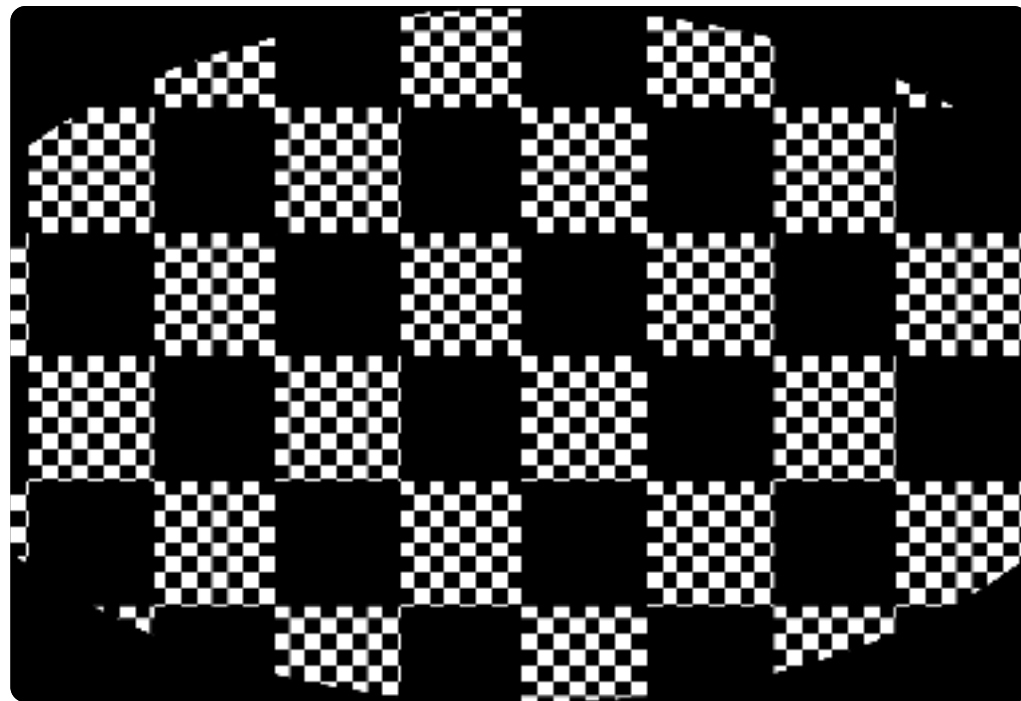
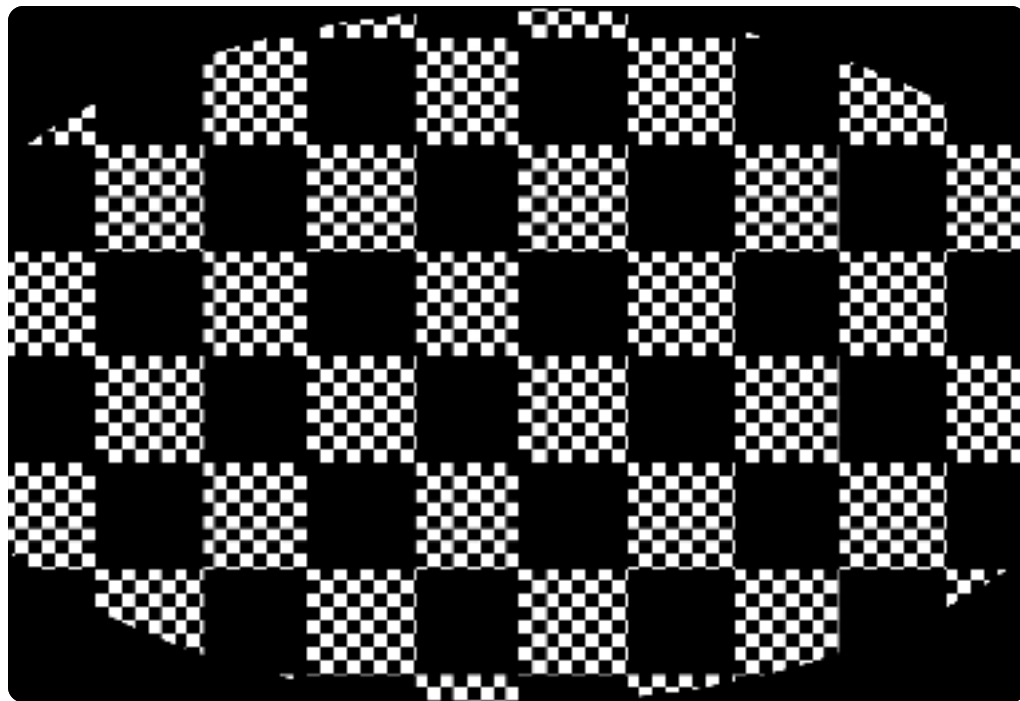
speed=0.9
noise(1,0.1/3).pixelate(9,9).color(2,2,2)
.mult( noise(2,0.1/3).pixelate(9*4,9*4).luma(0.25,0) )
.add( noise(5,0.2/3).pixelate(9*8,9*8).luma(0.25,0).color(1,0,0)
)
.add( noise(1,0.1/3).pixelate(9*8,9*8).luma(0.5,0.1).color(2,1,0)
)
.out(o1)
src(o1).modulateScale( shape(2,0.28,0), 4.2).out(o2)
src(o1).diff(
src(o2).invert().color(0.25,-0.5,0.25).scale(2).mask(
shape(2,0.28,0) ) )
.scale(1.333)
.rotate(Math.PI/2,0)
//.repeat(2,1)
.kaleid(4)
.out()

```

O patch se organiza em torno de pixelização (**pixelate**), colorização (**color**), ruído (**noise**), máscara de luma (**luma**) e fontes externas (**src**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **mult**, **add**, **diff**, **mask** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.9** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**src**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

await loadScript("https://cdn.statically.io/gl/metagrowing/extra-
shaders-for-hydra/main/lib/lib-screen.js")

speed=0.1111

voronoi(1,0.1,0.2222)

.out(o1)

src(o1)
.modulate( voronoi( ()=>Math.sin(time/33)+2 ,0.01),
()=>Math.sin(time/30)*2+5 )

.scale(0.2222)

.colorama( ()=>Math.sin(time/44) )
.dither2()
.scale( ()=>Math.sin(time/55)+1 )
.out(o2)

src(o2)
.mult( src(o2).scale(8) )
.scale(2.5)
.modulateScale( shape(72, ()=>Math.cos(time/3)*1+1.5 ,0.4444),
()=>Math.sin(time/3)*7+7 )
.mult( shape(160,0.995,0.002) )
.out()

```

O patch se organiza em torno de escala (**scale**), fontes externas (**src**), Voronoi (**voronoi**), formas geométricas (**shape**) e multiplicação de camadas (**mult**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **mult** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**, **modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.1111** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**src**, **loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

speed=0.2222

voronoi(1.2222,0.05)
.out(o1)

src(o1)
.modulate( voronoi( ()=>Math.sin(time/33)+2 ,0.01) ,
 ()=>Math.sin(time/30)*2+5 )
.scale(0.8)
.colorama( ()=>Math.cos(time/21) )
.out(o2)

src(o1).invert()
.layer( src(o2).scale(2).color(2,0,0).mask(src(o1).luma(0.1,0.4)
) )

.colorama( ()=>Math.sin(time/55) )

.modulateRotate( shape(2,0.2,0).rotate(Math.PI/2) , ()=>time/11 )
.modulateScale( shape(2,0.16,0).rotate(Math.PI/2) , 1.4444)

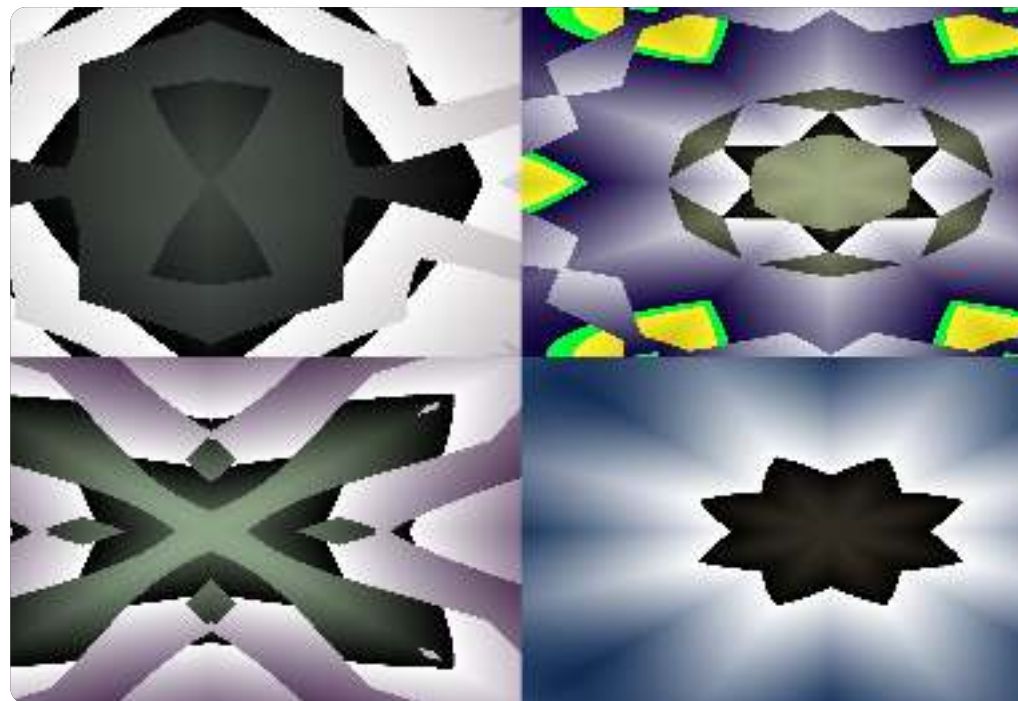
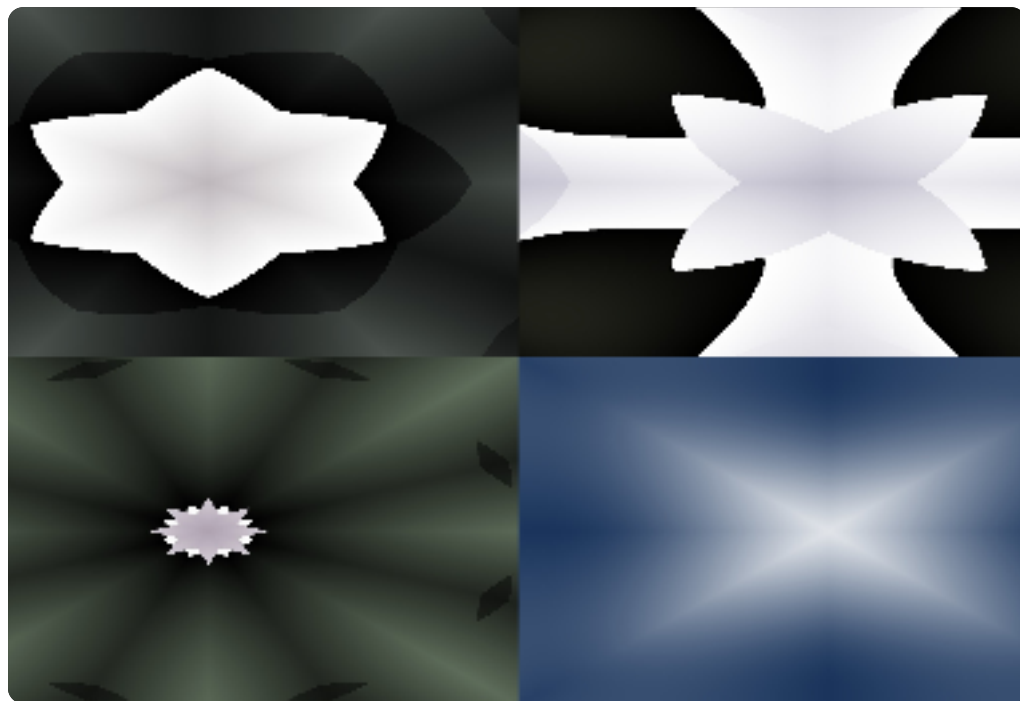
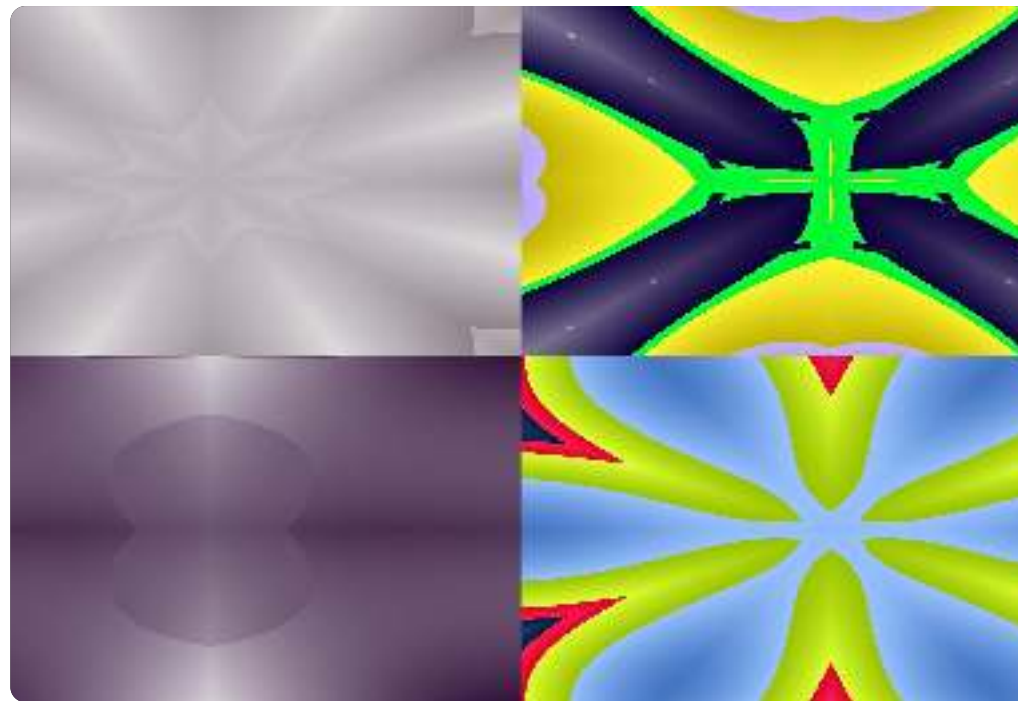
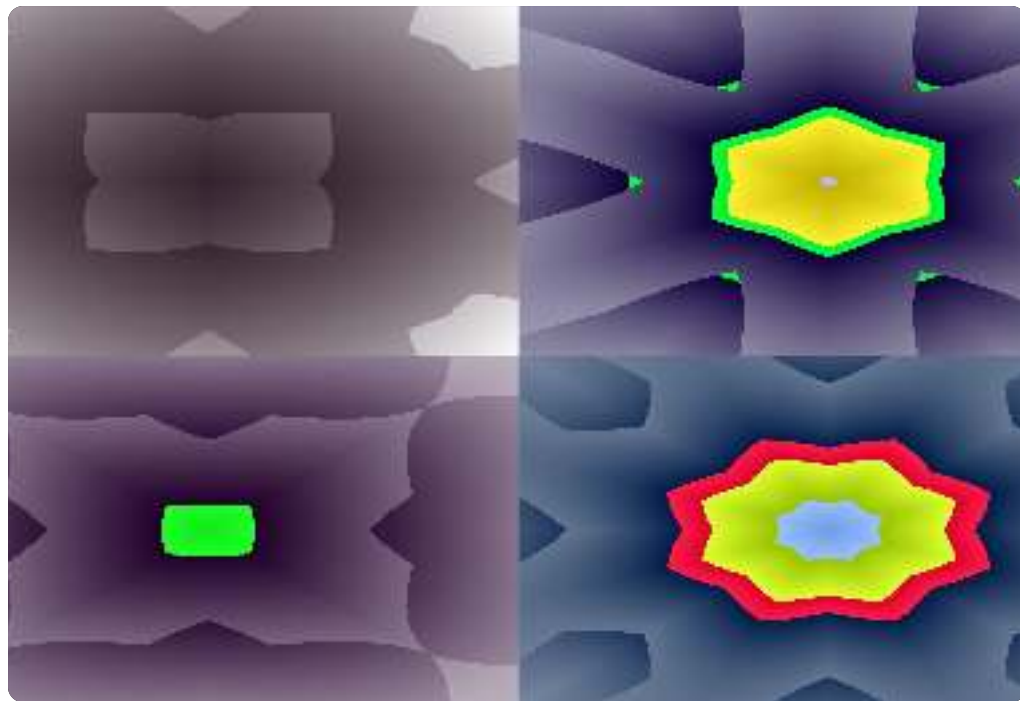
.out()

```

O patch se organiza em torno de fontes externas (**src**), deslocamento de matiz (**colorama**), Voronoi (**voronoi**), rotação (**rotate**) e escala (**scale**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **layer**, **mask** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateRotate**, **modulateScale**, **modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.2222** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**src**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.



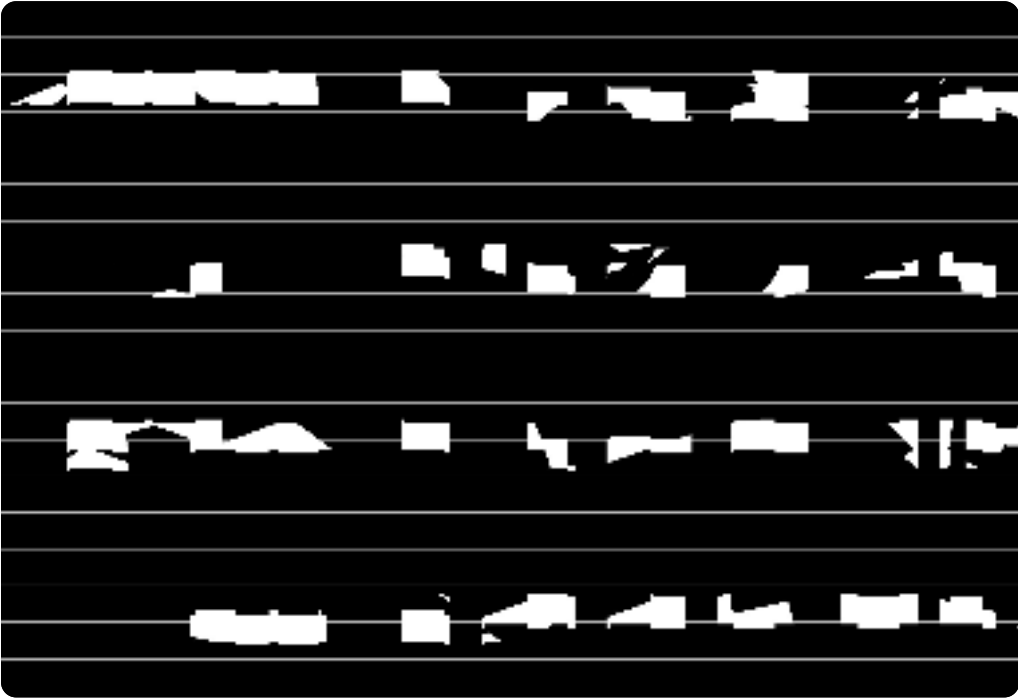
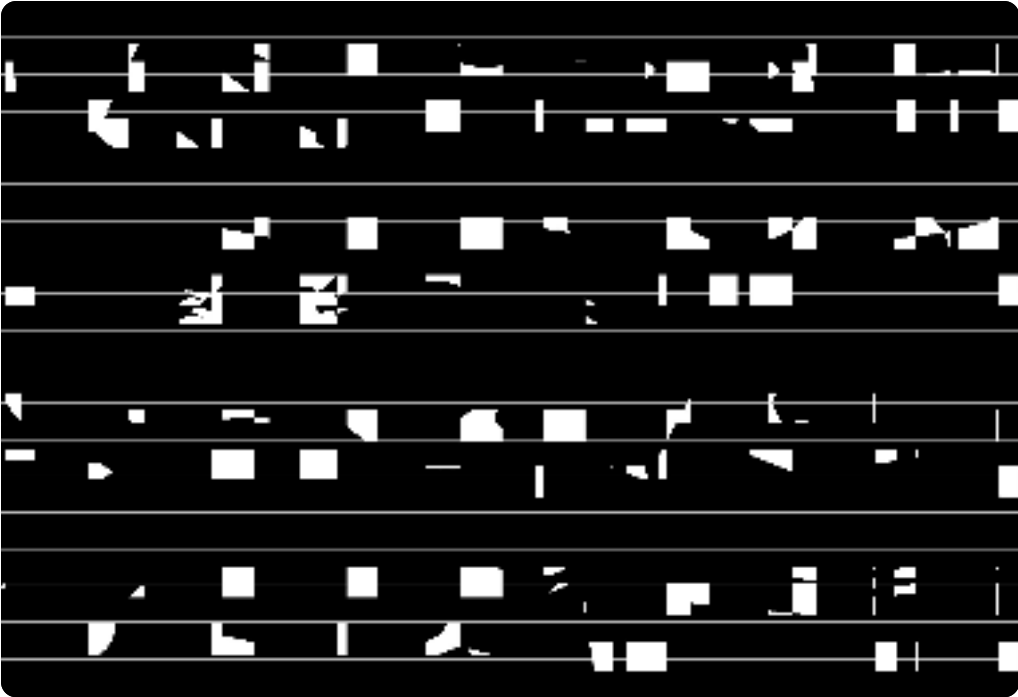


```
noise(0.5,0.11/3).modulate( src(o3).thresh(0.5,0) )  
  .colorama(-0.1).kaleid([2,4,6,8].fast(0.333))  
  .out(o0)  
noise(0.6,0.12/3).modulate( src(o2).thresh(0.5,0) )  
  .colorama(-0.2).kaleid([4,6,8,2].fast(0.333))  
  .out(o1)  
noise(0.7,0.13/3).modulate( src(o1).thresh(0.5,0) )  
  .colorama(-0.3).kaleid([6,8,2,4].fast(0.333))  
  .out(o2)  
noise(0.8,0.14/3).modulate( src(o0).thresh(0.5,0) )  
  .colorama(-0.4).kaleid([8,2,4,6].fast(0.333))  
  .out(o3)  
  
render()
```

O patch se organiza em torno de modulação (**modulate**), deslocamento de matiz (**colorama**), limiar (**thresh**), kaleidoscópio (**kaleid**) e ruído (**noise**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. Há entrada externa (**src**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

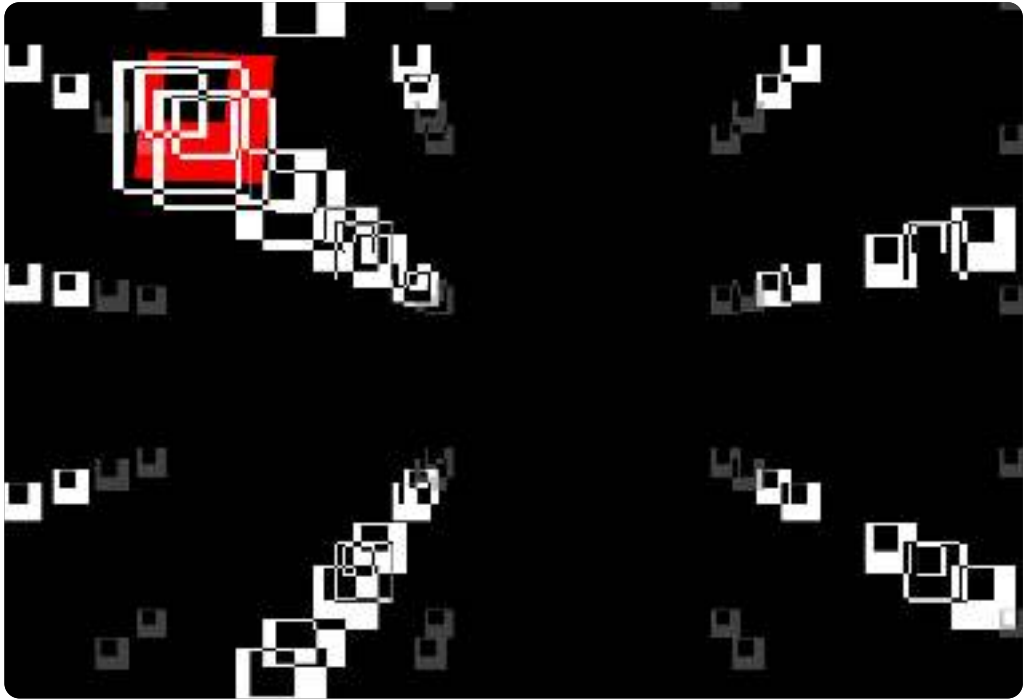
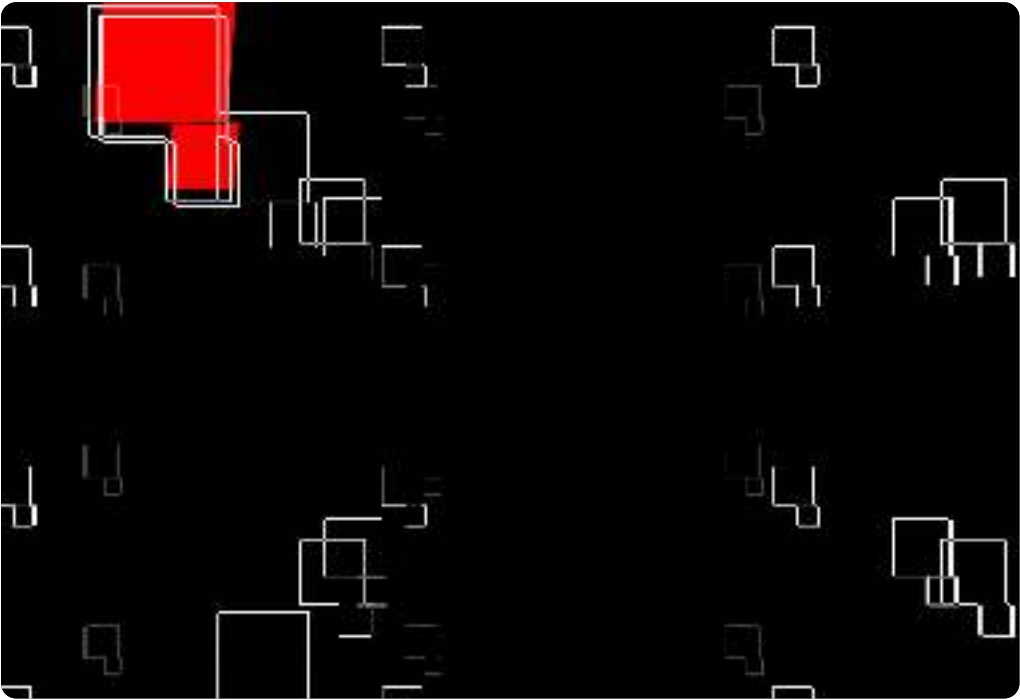
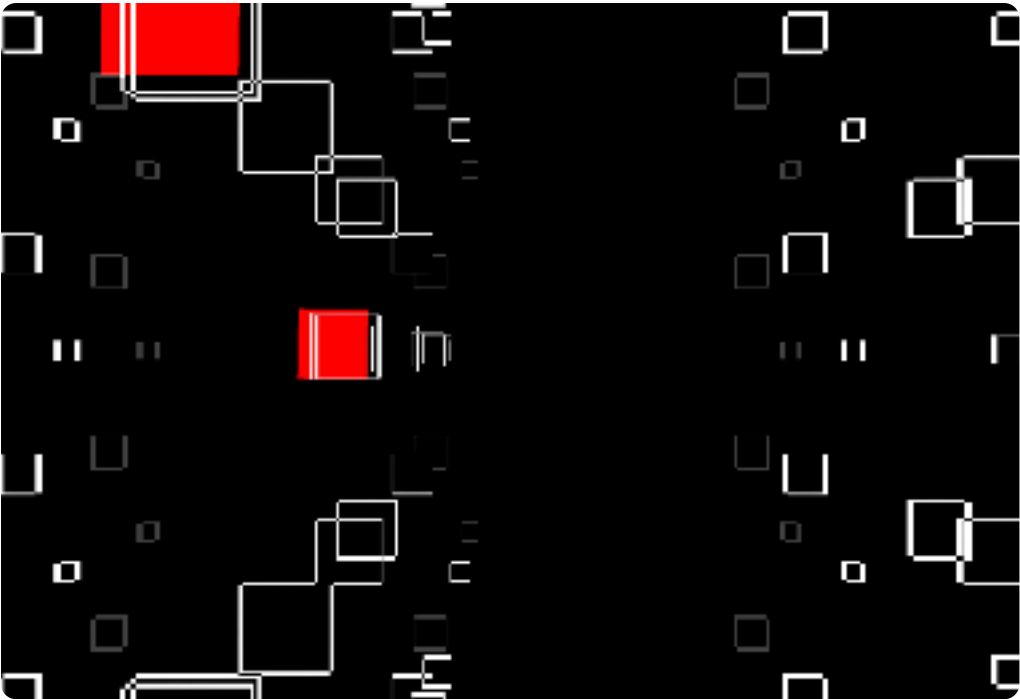
shape(4,0.1,0).scale(1,1,16/9).scrollY(0.01,0.01)
.add(
shape(4,0.1,0).scale(1,1,16/9).scrollX(-0.1).scrollY(0.01,-0.01)
)
.add(
shape(4,0.1,0).scale(1,1,16/9).scrollX(+0.1).scrollY(0.01,-0.01)
)
.add(
shape(4,0.1,0).scale(1,1,16/9).scrollX(-0.2).scrollY(0.01,0.01) )
//.add(
shape(4,0.1,0).scale(1,1,16/9).scrollX(+0.2).scrollY(0.01,0.01) )
.repeat(3,4)
.modulateScrollX( osc(50,-0.005).thresh(0.5,0), 0.1 )
.modulateScrollX( voronoi(5,0.05).thresh(0.5,0), 0.1 )
.modulateScrollX( voronoi(10,0.075).thresh(0.5,0.00), 0.1 )
.modulateScrollY( voronoi(5,0.02).thresh(0.5,00), -0.025 )
.scrollX(0.01,0.005)
.add(
  osc(120,0).rotate(Math.PI/2).thresh(0.995,0).scrollY(0.014,0),
0.7
)
.out()

```

O patch se organiza em torno de deslocamento Y (**scrollY**), limiar (**thresh**), deslocamento X (**scrollX**), escala (**scale**) e formas geométricas (**shape**). Osciladores viram padrões binários via **thresh**, e a geometria resultante alimenta o restante da cadeia. A composição usa **add** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScrollX**, **modulateScrollY**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa.





```

speed=0.3
shape(4,0.05,0.002).scale(1,1,16/9)
.scrollX( ()=>Math.sin(time/15)*0.25 ).scrollY(
()=>Math.sin(time/15)*0.45 ).scrollX(
()=>Math.cos(time/15)*0.25/2 )

.diff(
  shape(4,0.1,0.002).scale(1,1,16/9)
  .scrollX( ()=>Math.cos(time/17)*0.25 ).scrollY(
()=>Math.cos(time/17)*0.45 ).scrollX(
()=>Math.sin(time/17)*0.25/2 )
).out(o1)

src(o1)
.diff( src(o1).scale(1.1/3.5) )
.diff( src(o1).scale(1.6/3.5) )
.diff( src(o1).scale(1.8/3.5) )
.diff( src(o1).scale(2.5/3.5) )
.diff( src(o1).scale(3.6/3.5) )
.add( src(o1).scale(0.9/3.5), 0.25 )
.out(o2)

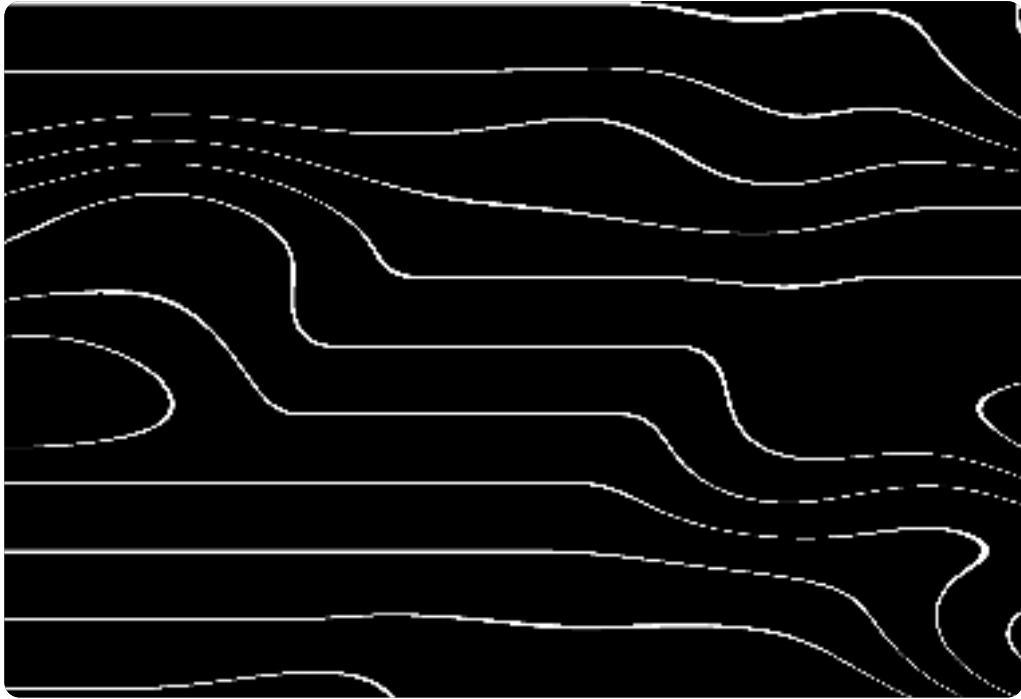
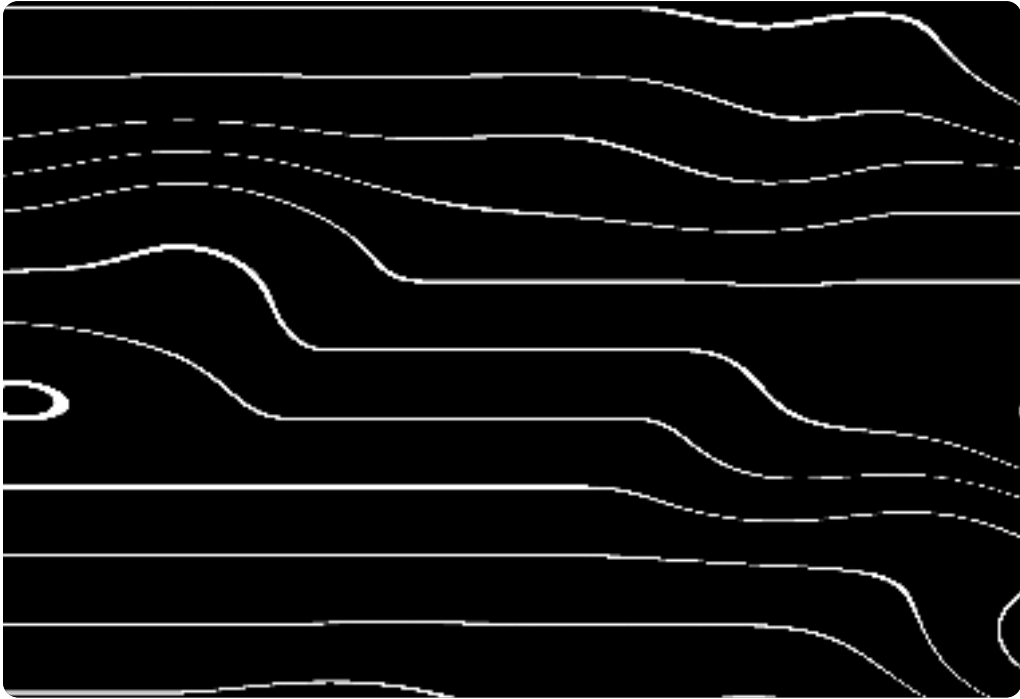
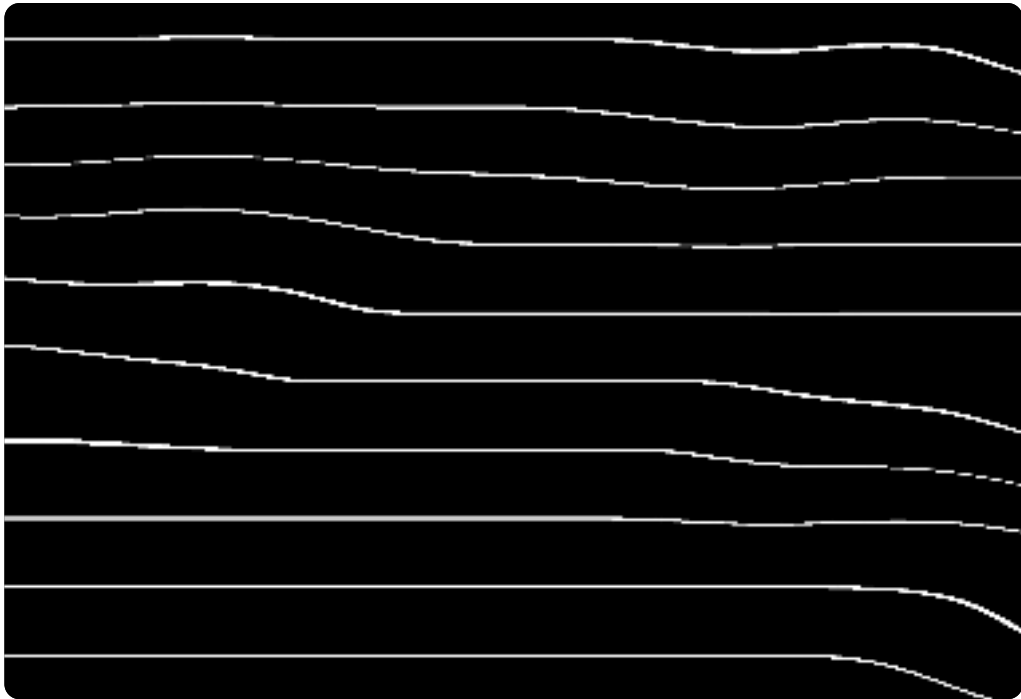
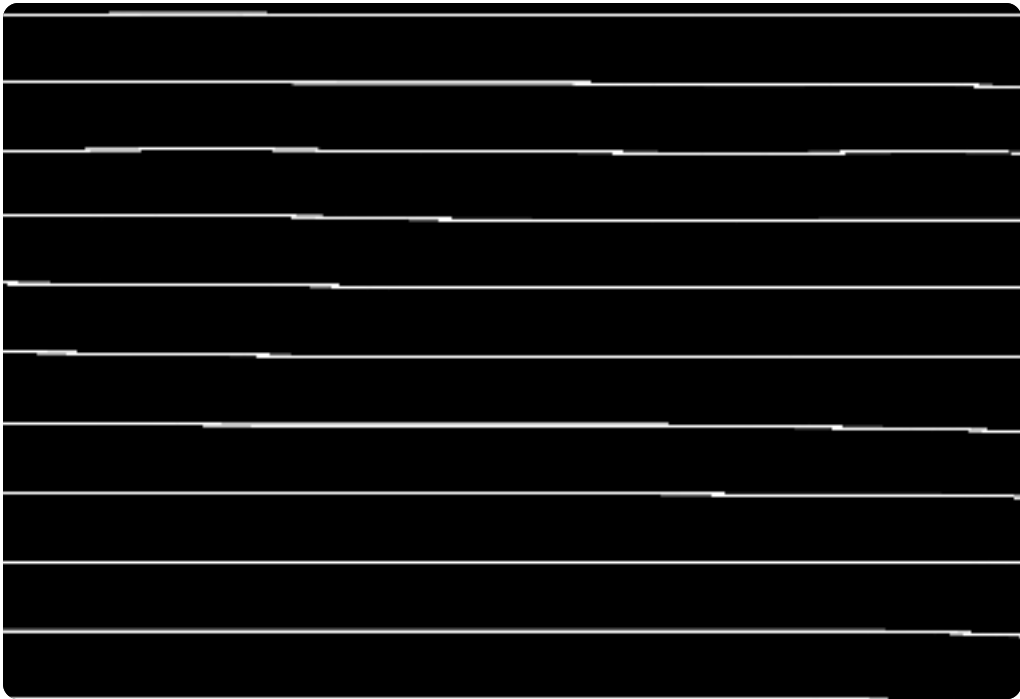
src(o2)
.diff( src(o2).scale( ()=>Math.sin(time/3)*0.1+1 ) )
.add( src(o1).scale( ()=>Math.cos(time/10)*0.1+1
).color(1,0,0).rotate(0.004,0.004) )
.out()

```

O patch se organiza em torno de escala (**scale**), fontes externas (**src**), diferença (**diff**), deslocamento X (**scrollX**) e deslocamento Y (**scrollY**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

speed = 0.3 define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**src**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.



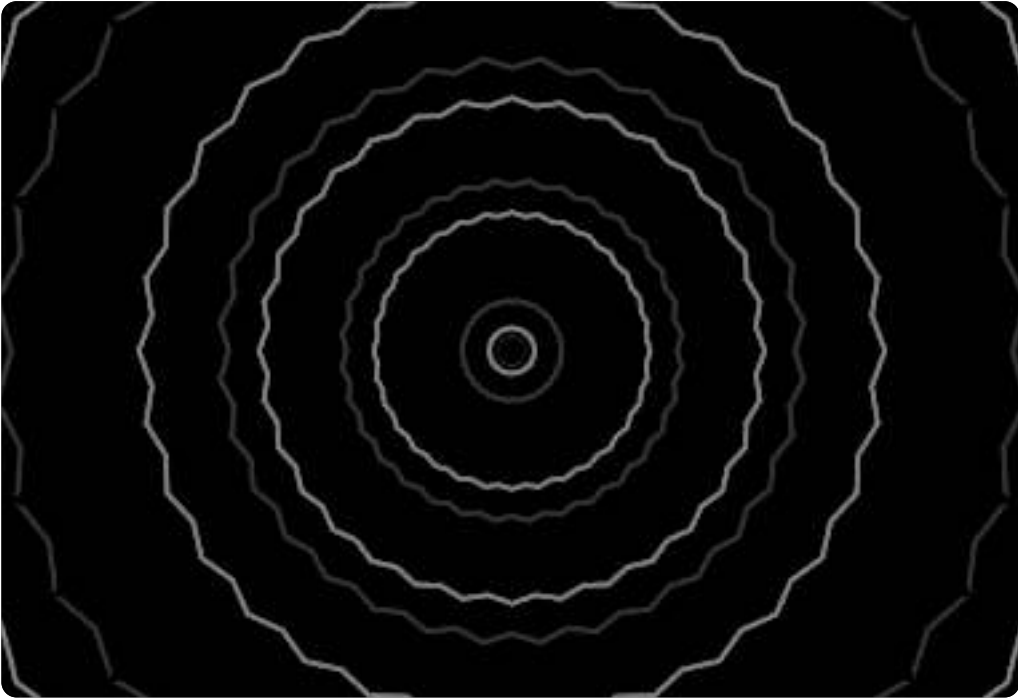
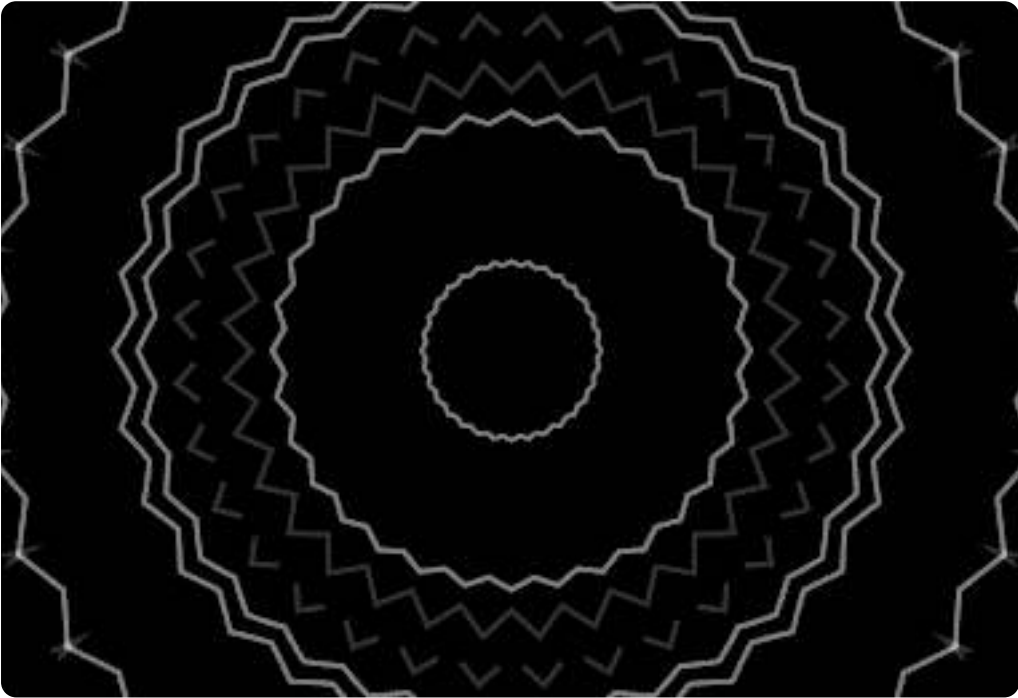
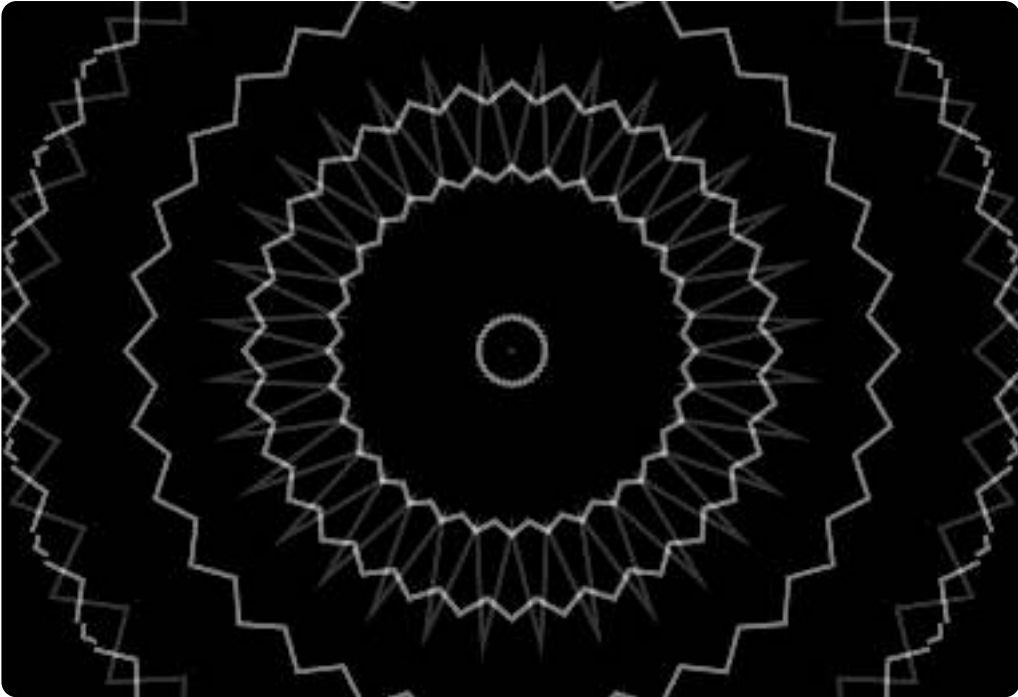


```
speed=0.7
osc(64,0.01,0).luma(0.995,0).rotate(Math.PI/2)
//.modulate( noise( 2 ,0.01,0) )
.modulateRotate( noise(2,0.005,0).thresh(0.5,0.5),
()=>Math.sin(time/33) )
//.colorama( ()=>Math.sin(time/33)*0.5 )
.out()
```

O patch se organiza em torno de modulação de rotação (**modulateRotate**), rotação (**rotate**), limiar (**thresh**), ruído (**noise**) e máscara de luma (**luma**). Osciladores viram padrões binários via **thresh**, e a geometria resultante alimenta o restante da cadeia.

A modulação (**modulateRotate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.7** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra.





```
osc(70,0.01/2.5).rotate(Math.PI/2).luma(0.99,0.01)
.modulate( voronoi(3,0.1) )
.kaleid(4).rotate(0.1,0.01)
.brightness(-0.5)
.out(o1)

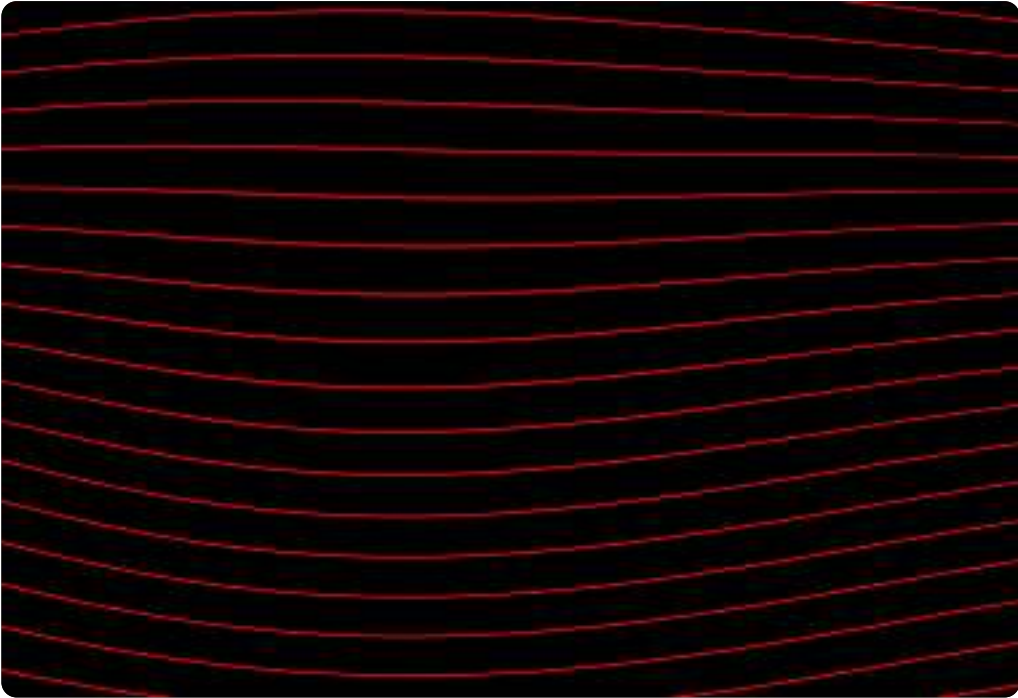
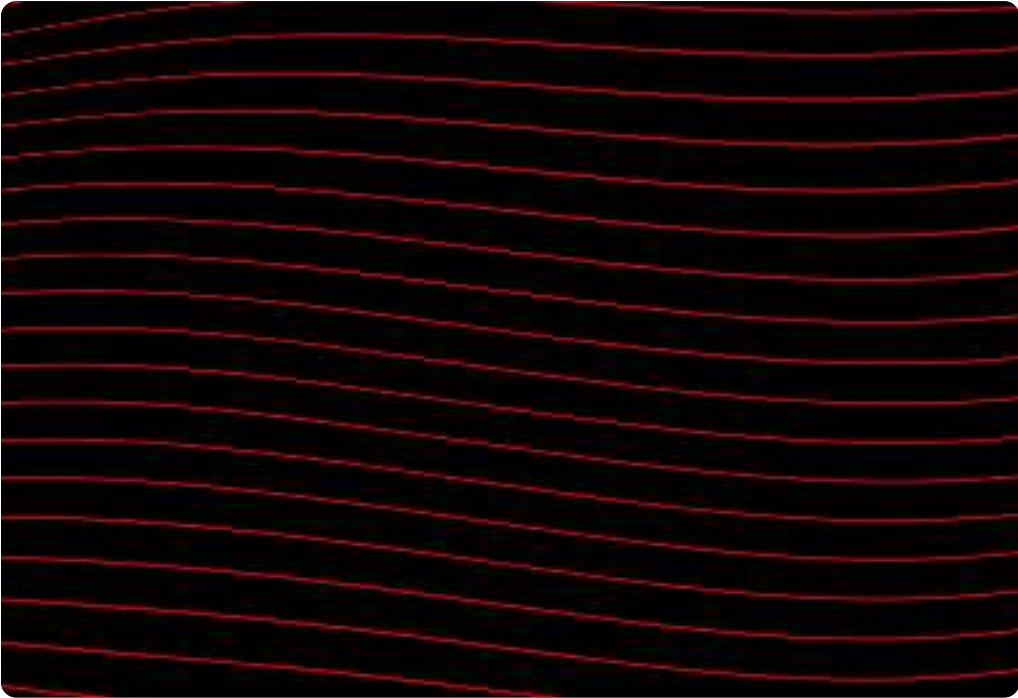
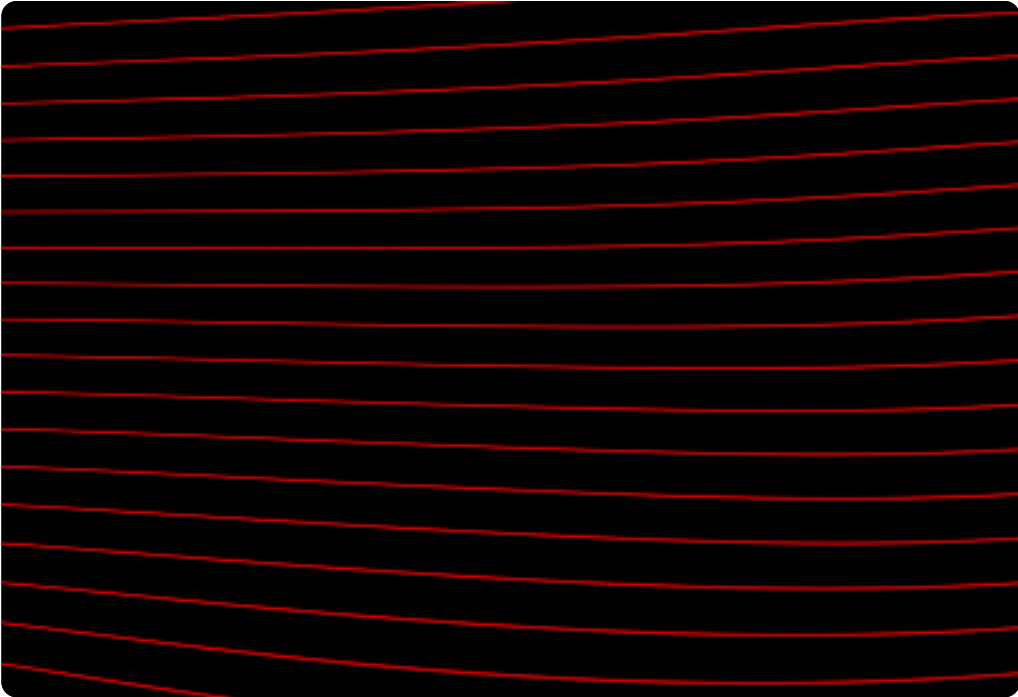
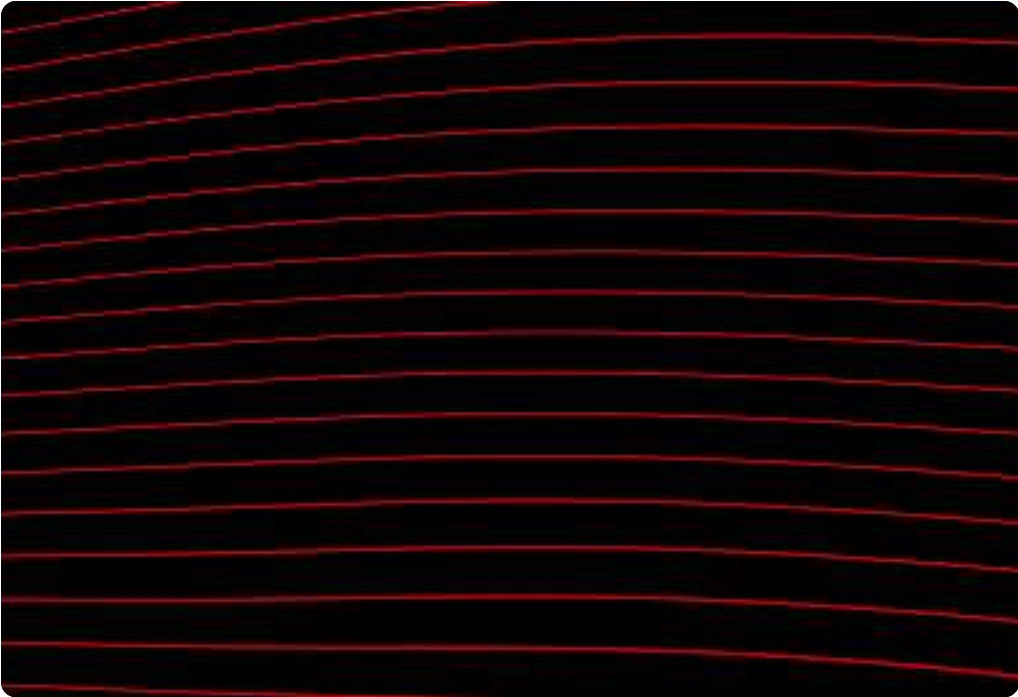
src(o1)
.add(
  src(o1)
  .rotate(0.1,0.02)
  .kaleid(8)
  , 0.4 )

.kaleid(32)
.scale(1,1,16/9)
.out()
```

O patch se organiza em torno de rotação (**rotate**), kaleidoscópio (**kaleid**), fontes externas (**src**), brilho (**brightness**) e modulação (**modulate**). **osc** entra em **kaleid**, multiplicando eixos de simetria até o quadro parecer um ornamento vivo. A composição usa **add** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. Há entrada externa (**src**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares.





// many red lines

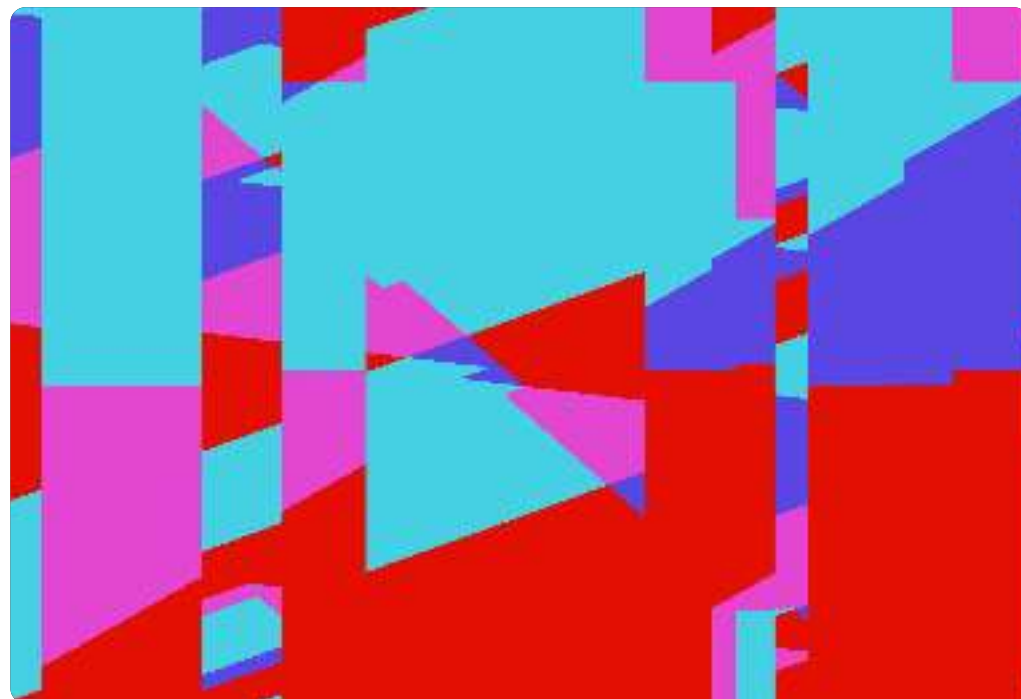
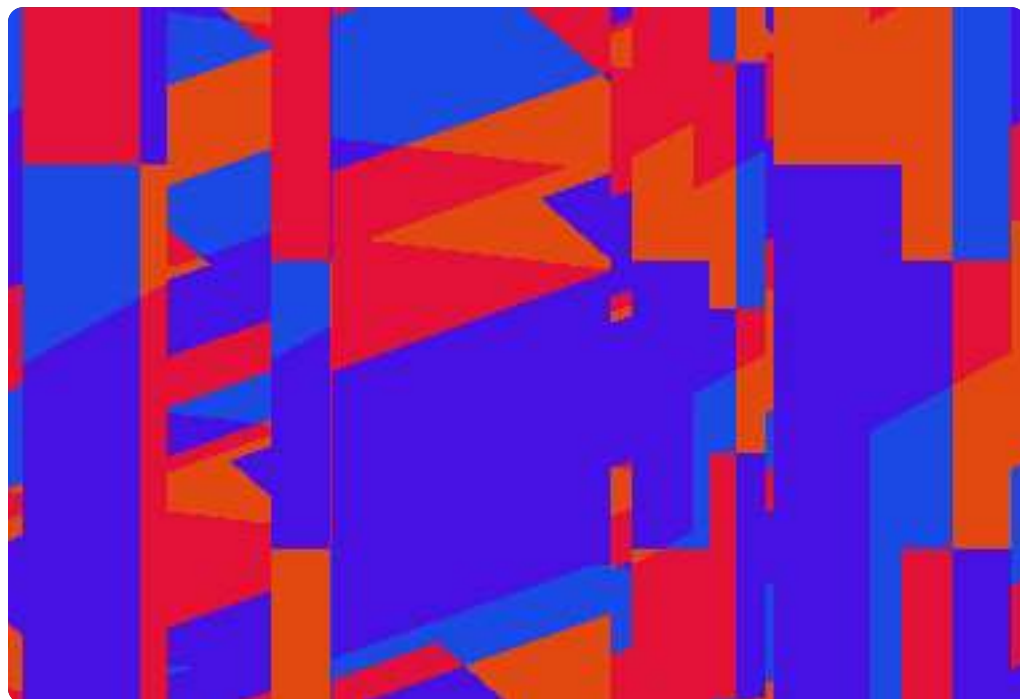
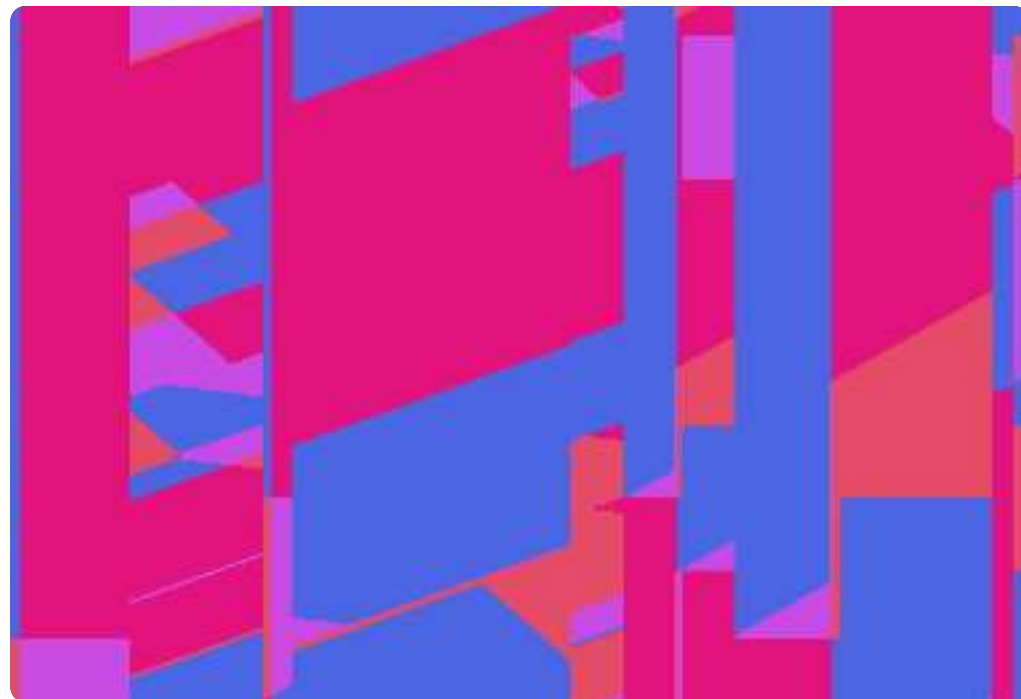
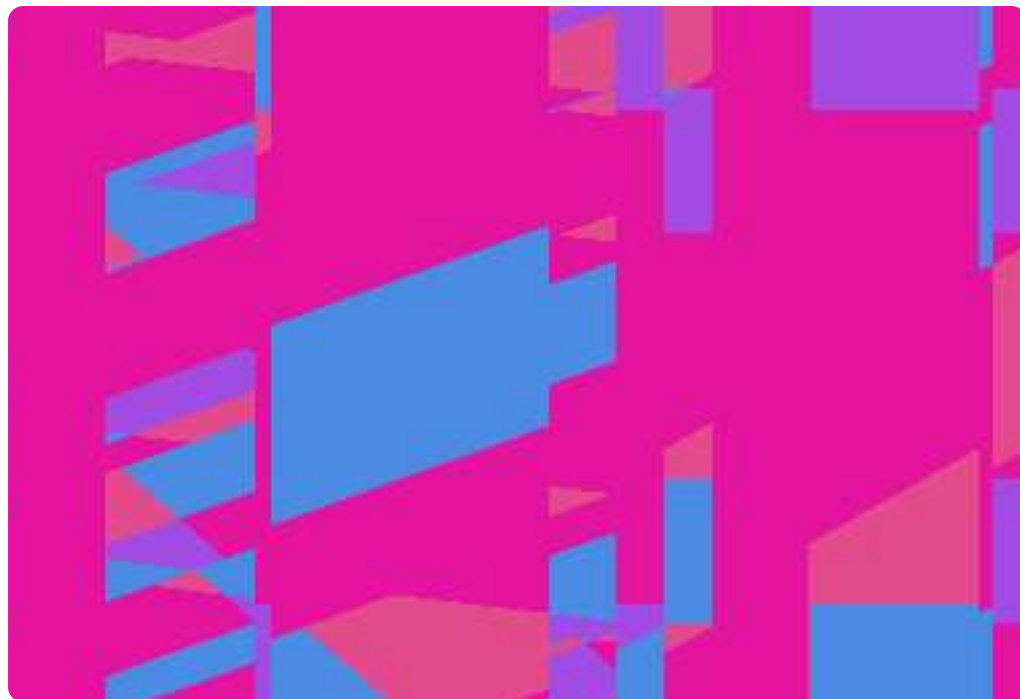
```
osc( ()=>Math.sin(time/2)*30+130,0.1,0)
.rotate(Math.PI/2)
.thresh(0.99,0.02)
.modulate( noise(()=>Math.sin(time/3)*0.1+0.9,1.25),0.2 )
.modulateScale( osc(2,0.1),0.5 )
.color(1,0,0)
.out()

speed=0.1
osc( ()=>Math.sin(time/2)*30+130,0.02,0)
.rotate(Math.PI/2)
.thresh(0.99,0.02)
.modulate( noise(()=>Math.sin(time/3)*0.1+0.9,1.25),0.1 )
.modulateScale( osc(2,0.05),0.5 )
.color(1,0,0)
.out()
```

O patch se organiza em torno de osciladores (**osc**), modulação de escala (**modulateScale**), modulação (**modulate**), rotação (**rotate**) e limiar (**thresh**). Osciladores viram padrões binários via **thresh**, e a geometria resultante alimenta o restante da cadeia.

A modulação (**modulateScale**, **modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.1** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```
// licensed with CC BY-NC-SA 4.0
https://creativecommons.org/licenses/by-nc-sa/4.0/
// "the-wall"
// Alexandre Rangel
// www.alexandrangel.art.br/hydra.html

speed=.034
osc(48,-.1,0).thresh([.3,.7].fast(.75),0).color(0,0,1)

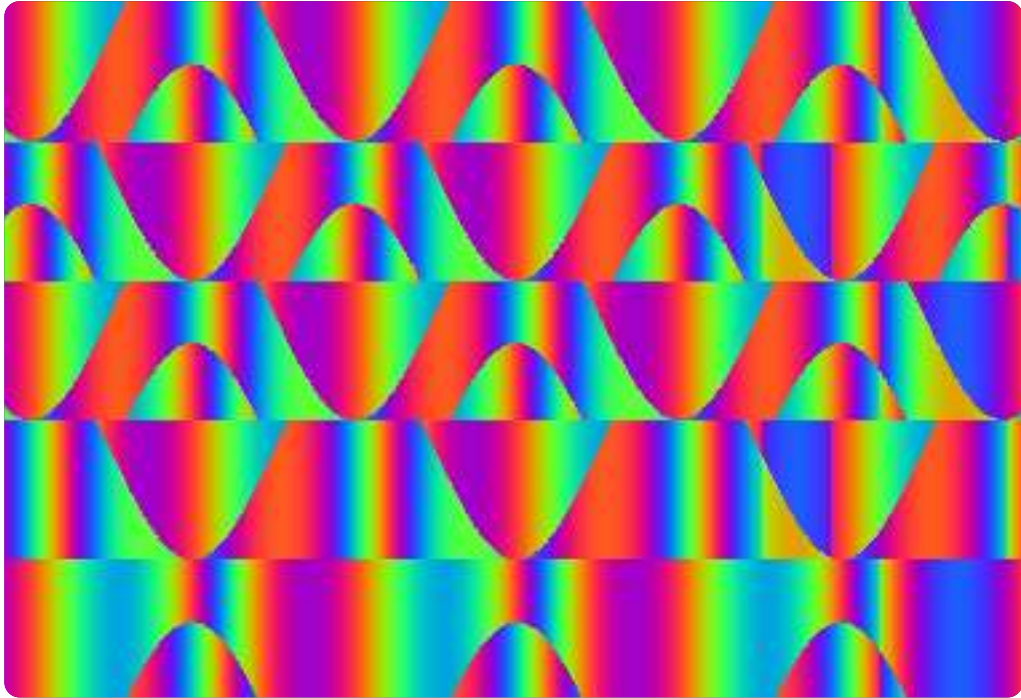
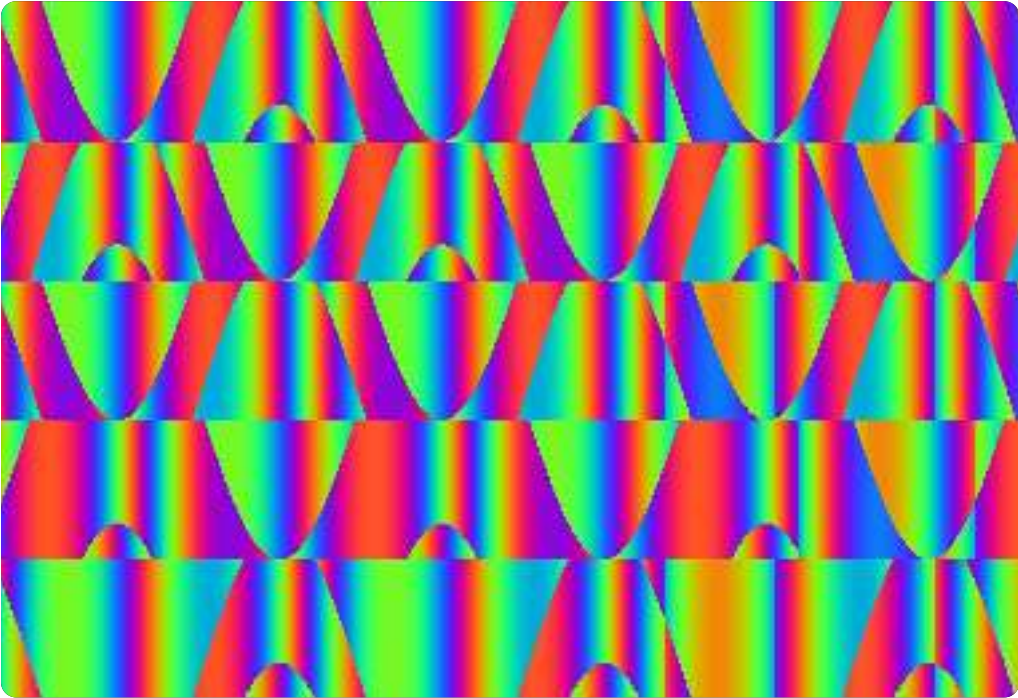
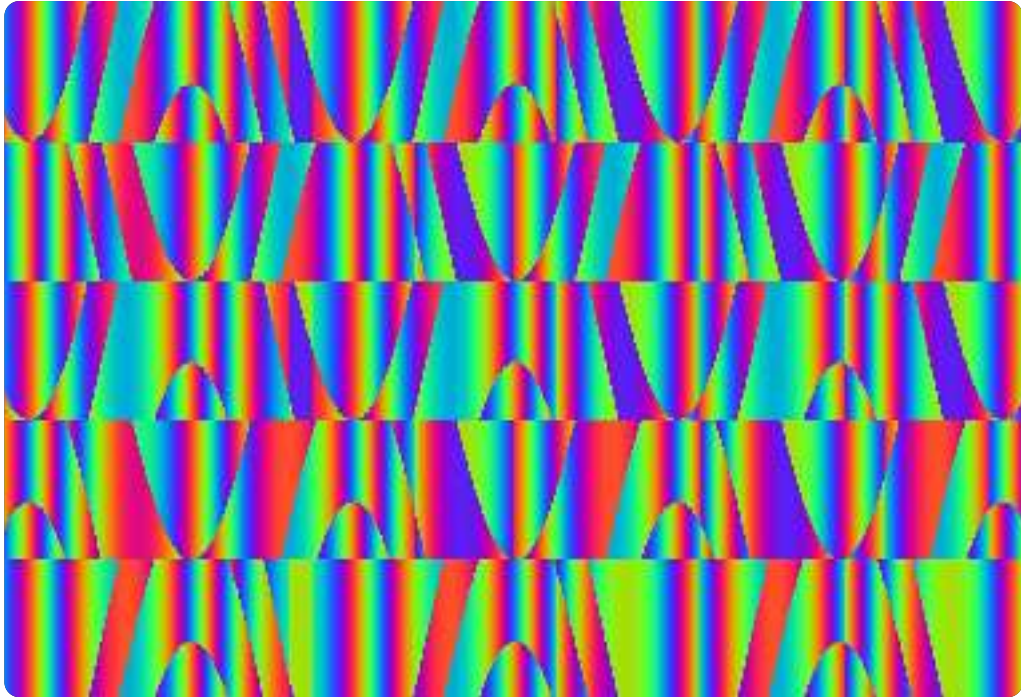
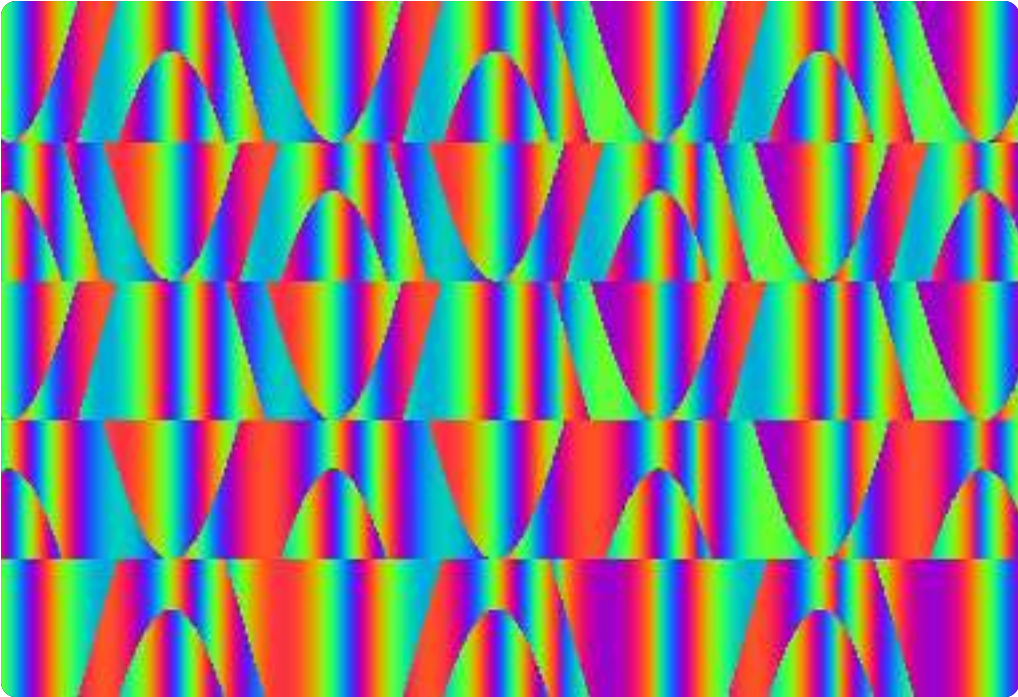
.add(
  osc(28,.1,0).thresh([.3,.7].fast(.75),0).rotate(3.14/4)
  .color(1,0,0)
  .modulateScale( osc(64,-.01,0).thresh([.3,.7].fast(.75),0) )
)
.diff(
  osc(28,.1,0).thresh([.3,.7].fast(.5),0).rotate(3.14/2)
  .color(1,0,1)
  .modulateScale( osc(64,-.015,0).thresh([.3,.7].fast(.5),0) )
)
.modulateRotate( osc(54,-.005,0).thresh([.3,.7].fast(.25),0) )
.modulateScale( osc(44,-.020,0).thresh([.3,.7].fast(.25),0) )
.colorama( ()=>Math.sin(time/18)*.01222+9.89)
.scale(2.5)

.out()
```

O patch se organiza em torno de limiar (**thresh**), arrays acelerados (**fast**), osciladores (**osc**), modulação de escala (**modulateScale**) e colorização (**color**). Osciladores viram padrões binários via **thresh**, e a geometria resultante alimenta o restante da cadeia. A composição usa **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**, **modulateRotate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed** = **.034** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

osc(24,-0.03,2)
.diff(shape(2,0.2,0).scrollY(0.4,0))
.diff(shape(2,0.2,0).scrollY(+0.0,0))
.diff(shape(2,0.2,0).scrollY(-0.4,0))
.out(o1)

src(o1)
.modulate( src(o1), ()=>Math.sin(time/9)*.2+0.5 )

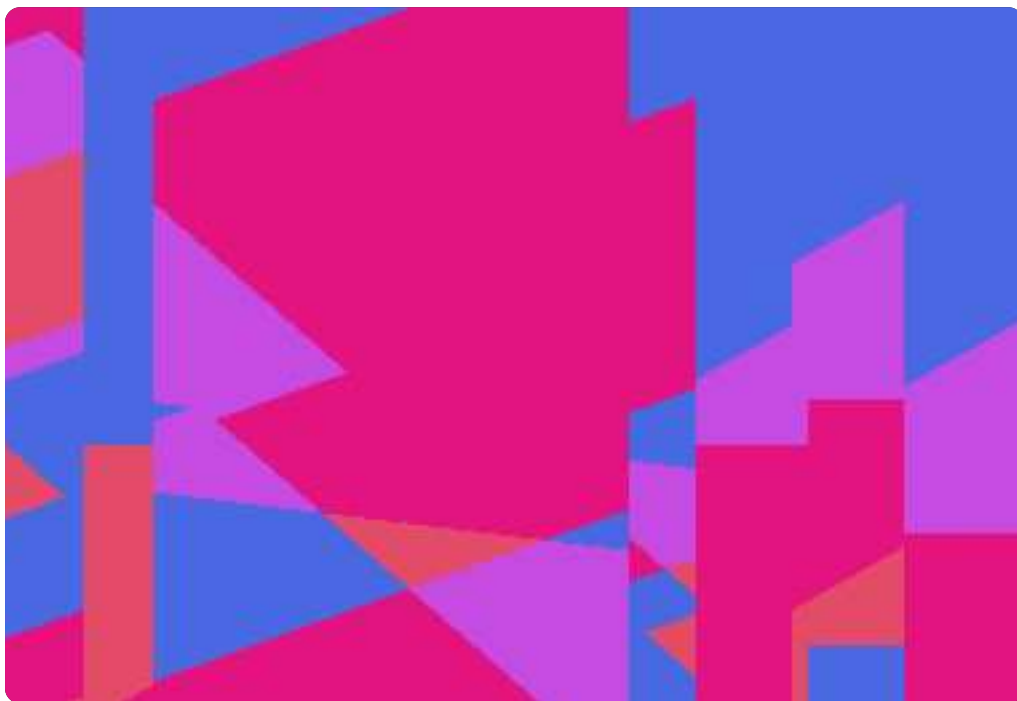
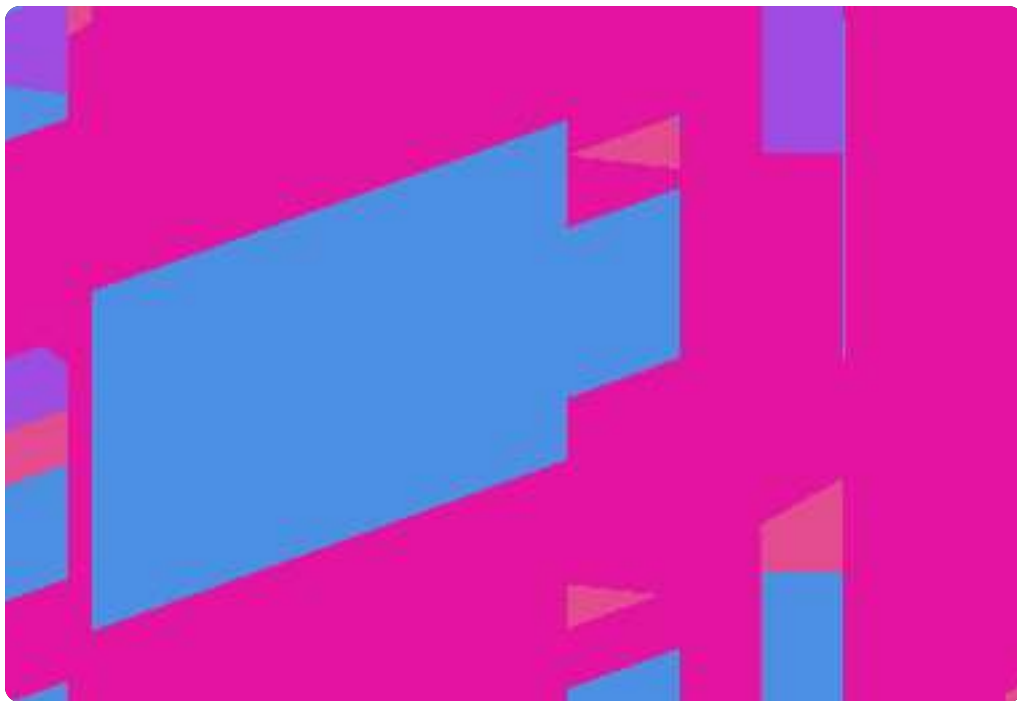
.out()

```

O patch se organiza em torno de deslocamento Y (**scrollY**), formas geométricas (**shape**), diferença (**diff**), fontes externas (**src**) e modulação (**modulate**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**src**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares.





```
// licensed with CC BY-NC-SA 4.0
https://creativecommons.org/licenses/by-nc-sa/4.0/
// "the-wall"
// Alexandre Rangel
// www.alexandrangel.art.br/hydra.html

speed=0.02
osc(48,-.1,0).thresh([.3,.7].fast(.75),0).color(0,0,1)

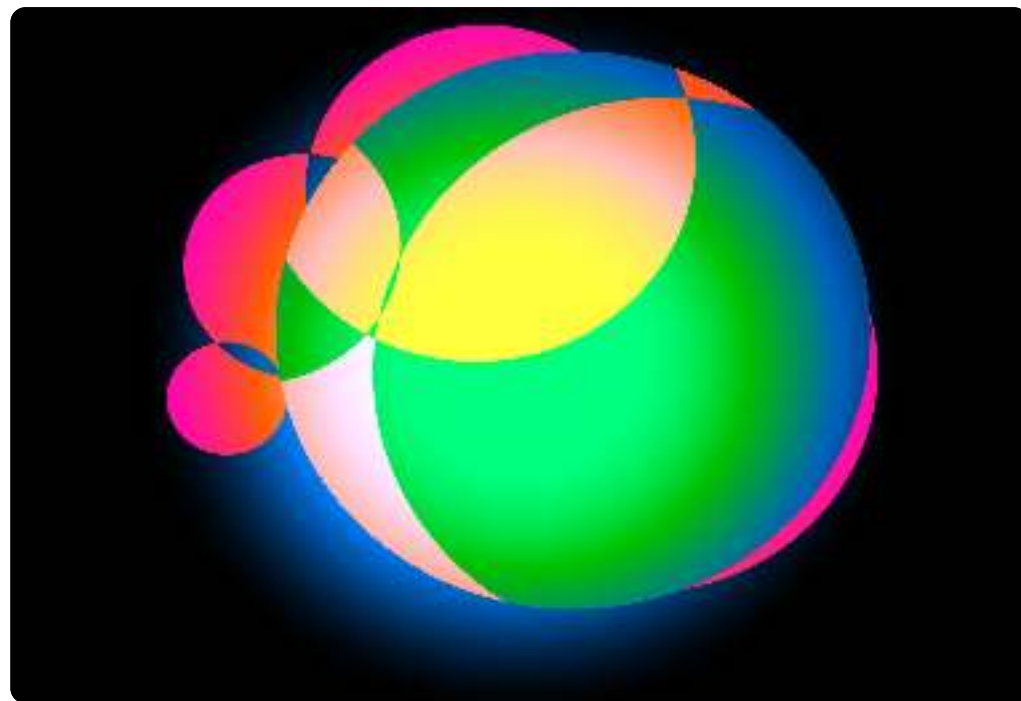
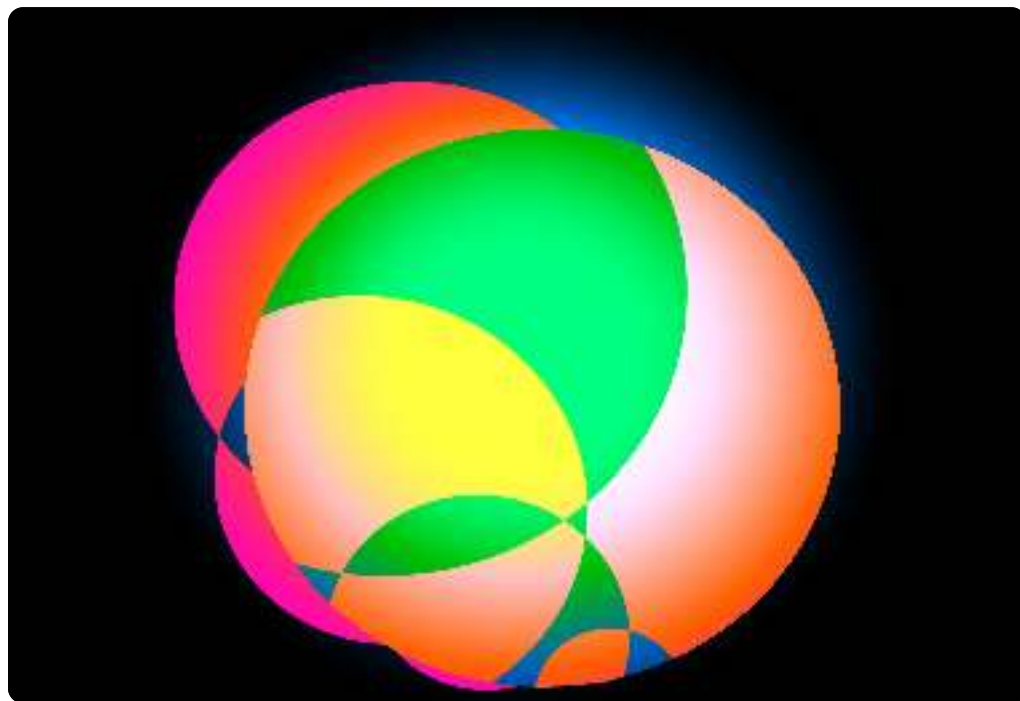
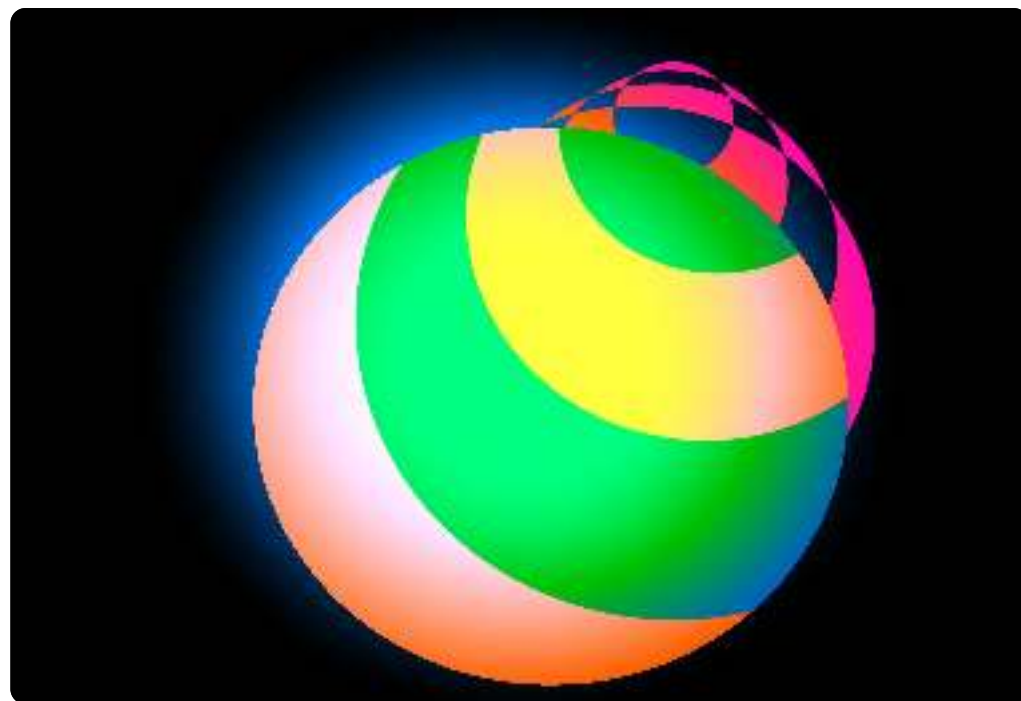
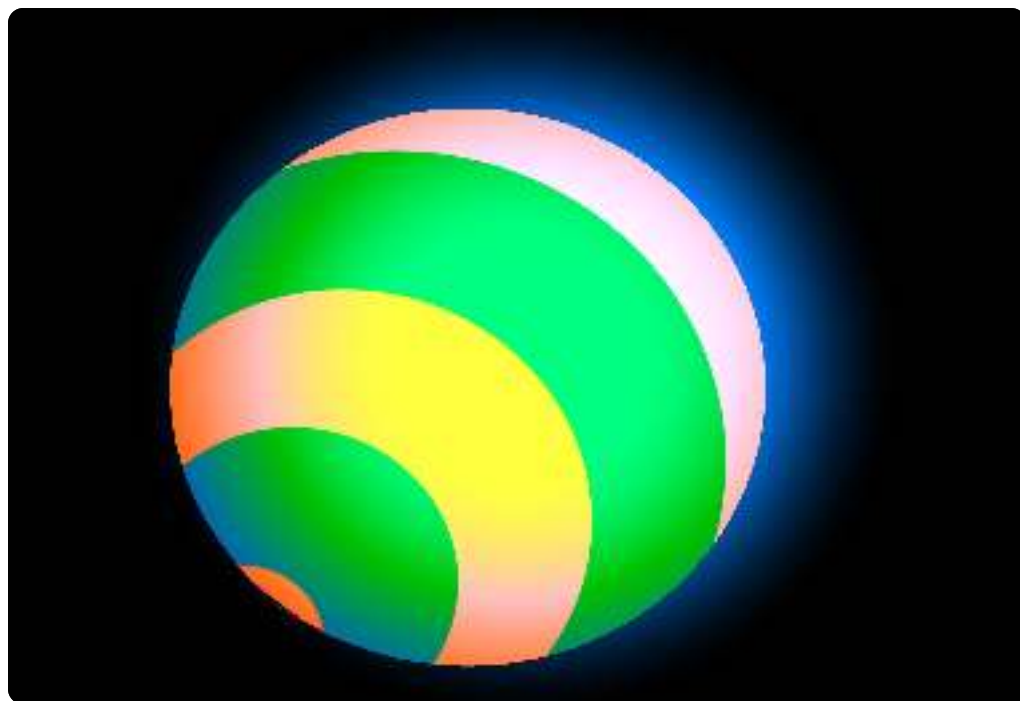
.add(
  osc(28,.1,0).thresh([.3,.7].fast(.75),0).rotate(3.14/4)
  .color(1,0,0)
  .modulateScale( osc(64,-.01,0).thresh([.3,.7].fast(.75),0) )
)
.diff(
  osc(28,.1,0).thresh([.3,.7].fast(.5),0).rotate(3.14/2)
  .color(1,0,1)
  .modulateScale( osc(64,-.015,0).thresh([.3,.7].fast(.5),0) )
)
.modulateRotate( osc(54,-.005,0).thresh([.3,.7].fast(.25),0) )
.modulateScale( osc(44,-.020,0).thresh([.3,.7].fast(.25),0) )
.colorama( ()=>Math.sin(time/33)*.01222+9.89)
.scale(2.122*2)

.out()
```

O patch se organiza em torno de limiar (**thresh**), arrays acelerados (**fast**), osciladores (**osc**), modulação de escala (**modulateScale**) e colorização (**color**). Osciladores viram padrões binários via **thresh**, e a geometria resultante alimenta o restante da cadeia. A composição usa **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**, **modulateRotate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed** = 0.02 define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```
// licensed with CC BY-NC-SA 4.0
https://creativecommons.org/licenses/by-nc-sa/4.0/
// "egg of the phoenix"
// Alexandre Rangel
// www.alexandrangel.art.br/hydra.html

speed=2.5

shape(99,.15,.5).color(0,1,2)

.diff( shape(240,.5,0).scrollX(.05).rotate( ()=>time/10
).color(1,0,.75) )
.diff( shape(99,.4,.002).scrollX(.10).rotate( ()=>time/20
).color(1,0,.75) )
.diff( shape(99,.3,.002).scrollX(.15).rotate( ()=>time/30
).color(1,0,.75) )
.diff( shape(99,.2,.002).scrollX(.20).rotate( ()=>time/40
).color(1,0,.75) )
.diff( shape(99,.1,.002).scrollX(.25).rotate( ()=>time/50
).color(1,0,.75) )

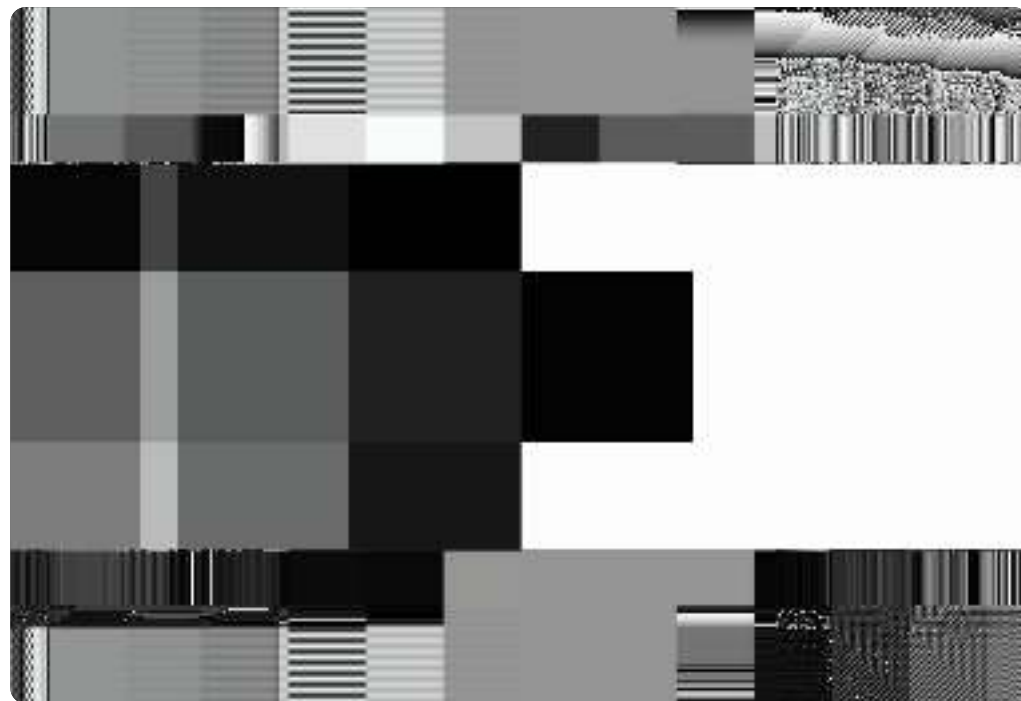
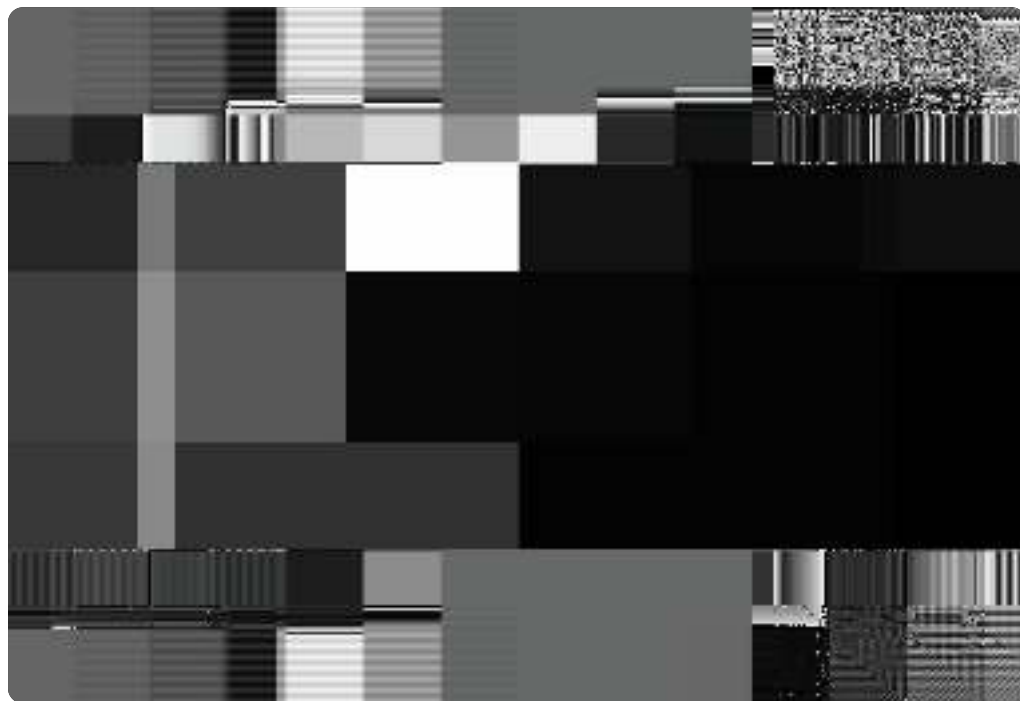
.modulateScale(
  shape(240,.5,0).scrollX(.05).rotate( ()=>time/10 )
  , ()=>(Math.sin(time/3)*.2)+.2 )

.rotate( ()=>Math.sin(time/222)*5555 )
.scale(1.6,.6,1)
.out()
```

O patch se organiza em torno de rotação (**rotate**), formas geométricas (**shape**), deslocamento X (**scrollX**), colorização (**color**) e diferença (**diff**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 2.5** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.



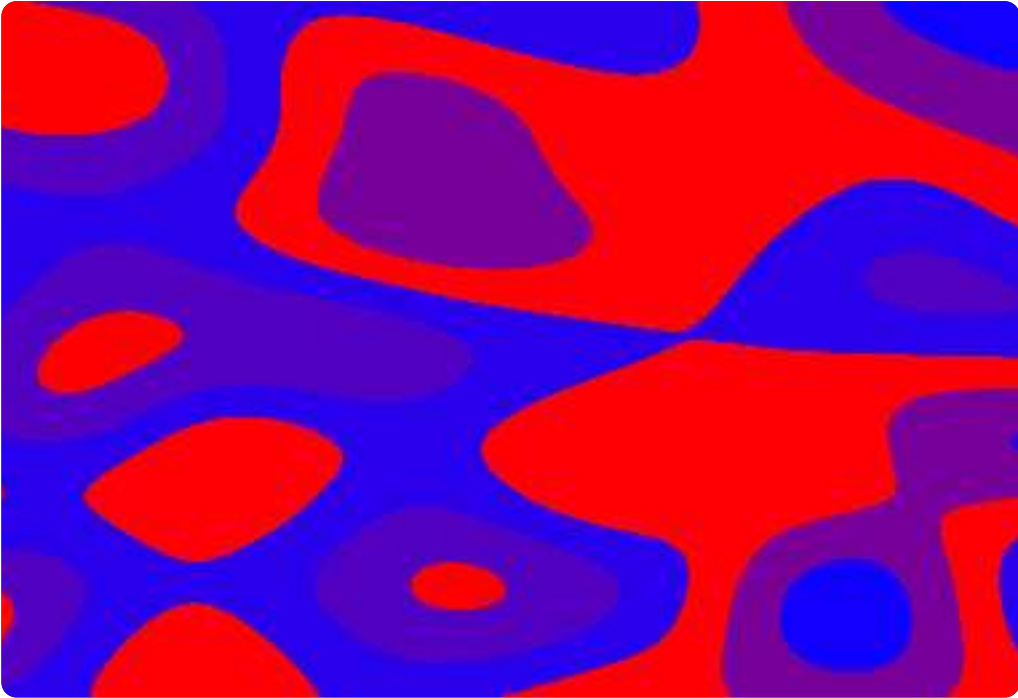
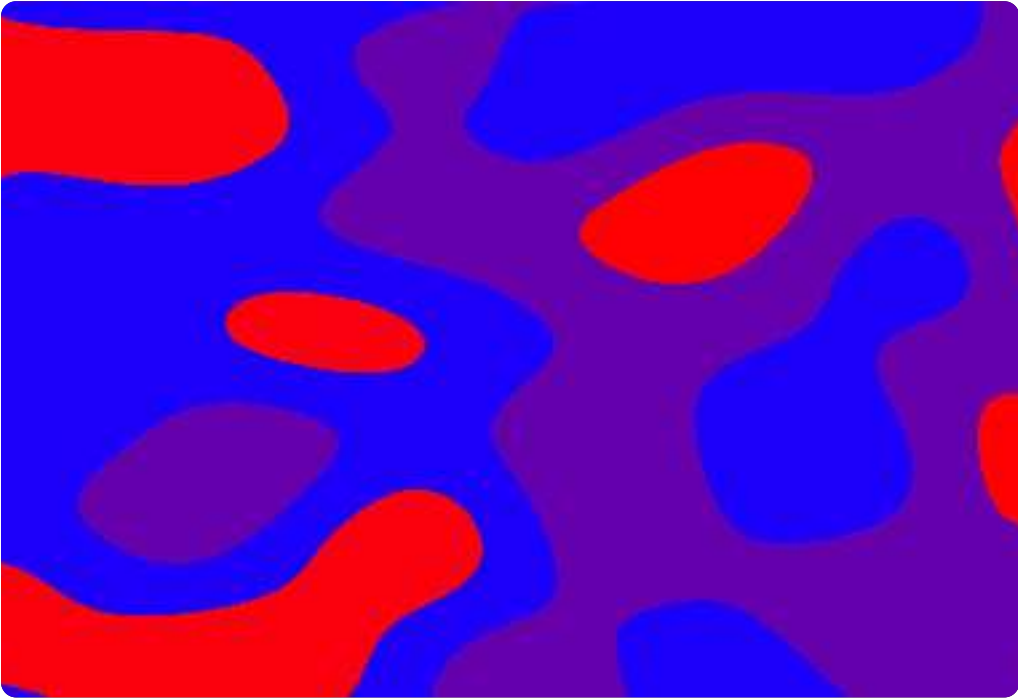
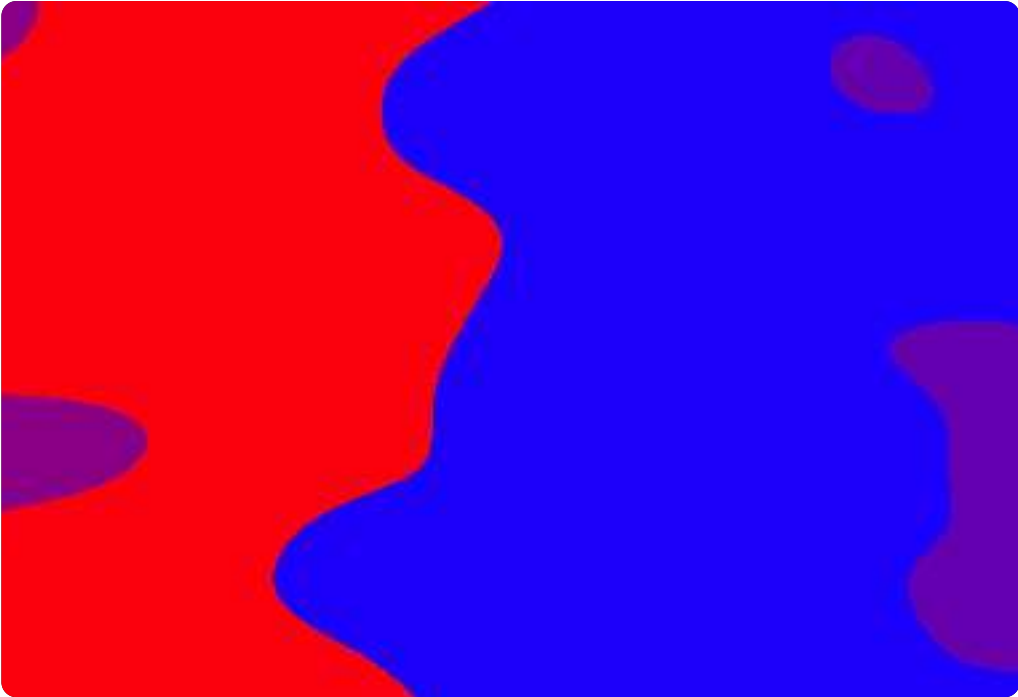


```
noise(1.5,0.01).thresh(0.5,0.5).pixelate(16,9)
.diff(src(o0).scale(0.999),1)
.colorama(-0.01)
.modulateScale(shape(2,0.55,0),1.22)
.out()
```

O patch se organiza em torno de modulação de escala (**modulateScale**), pixelização (**pixelate**), deslocamento de matiz (**colorama**), limiar (**thresh**) e escala (**scale**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulateScale**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. Há entrada externa (**src**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.



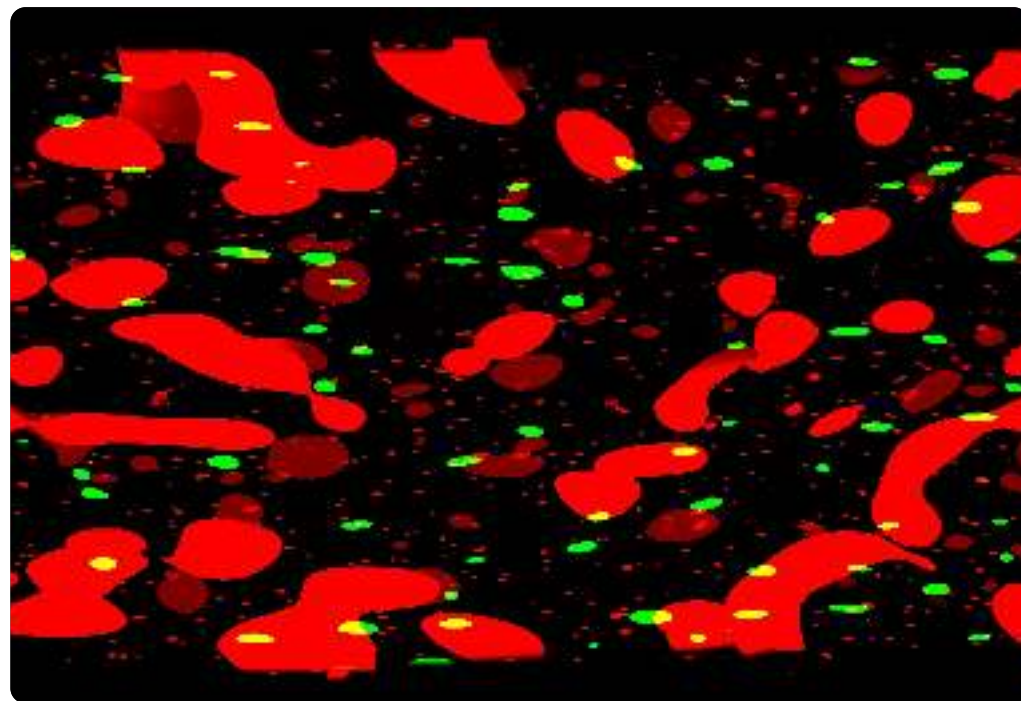
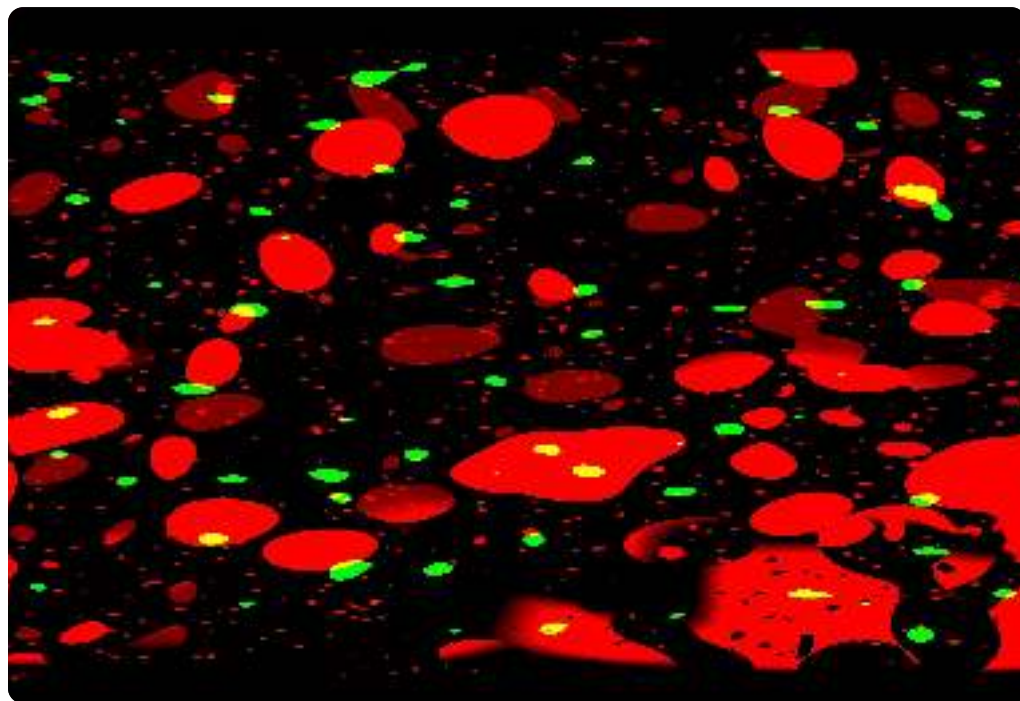
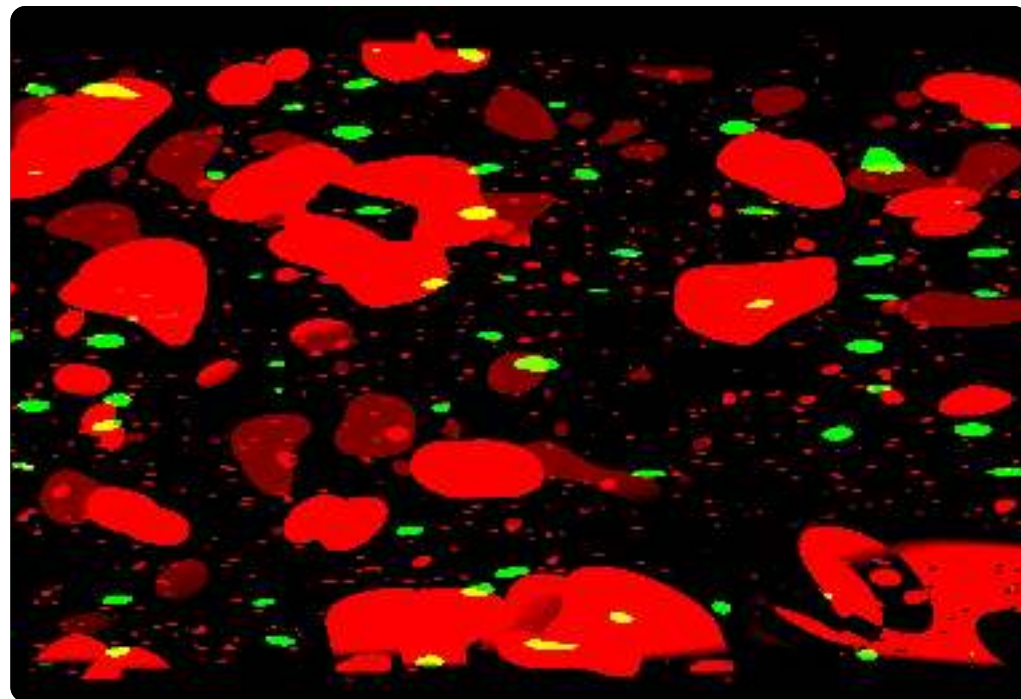
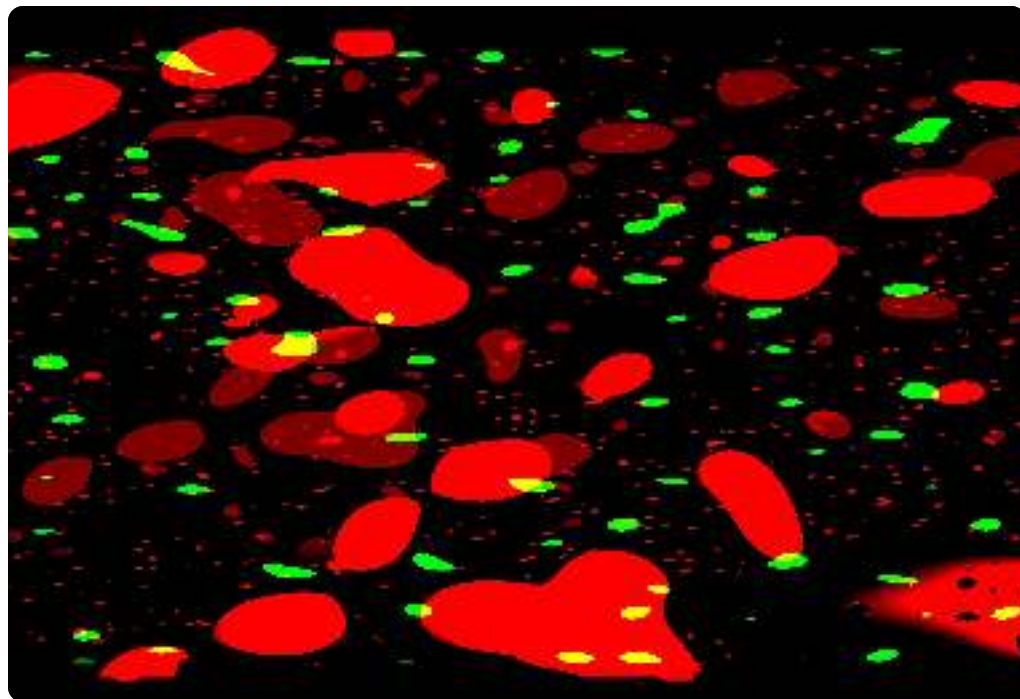


```
speed=0.25
a.setSmooth(0.89)
osc(4, 0.1, ()=>a.fft[0]*2.4 ).color(1.04,0, -1.1)
.rotate(()=>a.fft[2], ()=>a.fft[2]/5).pixelate(2, 20)
.modulate(noise(2.5), () => 1.5 * Math.sin(0.08 * time)).out(o0)
```

O patch se organiza em torno de modulação (**modulate**), pixelização (**pixelate**), rotação (**rotate**), colorização (**color**) e ruído (**noise**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. **speed = 0.25** define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```

noise(5,0.02).thresh(0.5,0).color(1,0.1,0.1)
.add(
noise(5,0.022).thresh(0.5,0).color(1,0.1,0).rotate(-0.01,0.01) ,
0.75)
.add( noise(7,0.024).thresh(0.5,0).color(1,0,0) , 0.5)
.modulate( noise (1,0.02) )

.add(
noise(15,0.10).thresh(0.7,0).color(1,0,0).rotate(0.01,0.012) ,
0.5)
.add(
noise(25,0.09).thresh(0.8,0).scale(1,1,0.61).color(1,0.333,0) ,
0.75)
.add(
noise(30,0.08).thresh(0.8,0).scale(1,1,0.61).color(1,0.333,0) ,
0.75)

.add( noise(10,0.08).thresh(0.7,0).scale(1,1,0.61).color(0,1,0) ,
0.75)

.add(
noise(35,0.07).thresh(0.75,0).scale(1,1,0.61).color(1,0.333,0).ro
tate(0.01,0.008) , 0.75)

.add(
noise(2,0.01).thresh(0.8,0.5).scale(1,1,0.5).color(1,0.25,0) ,
0.75)
.diff(
noise(0.5,0.011).thresh(0.8,0.5).scale(1,1,0.5).color(2,0,0) ,
0.75)

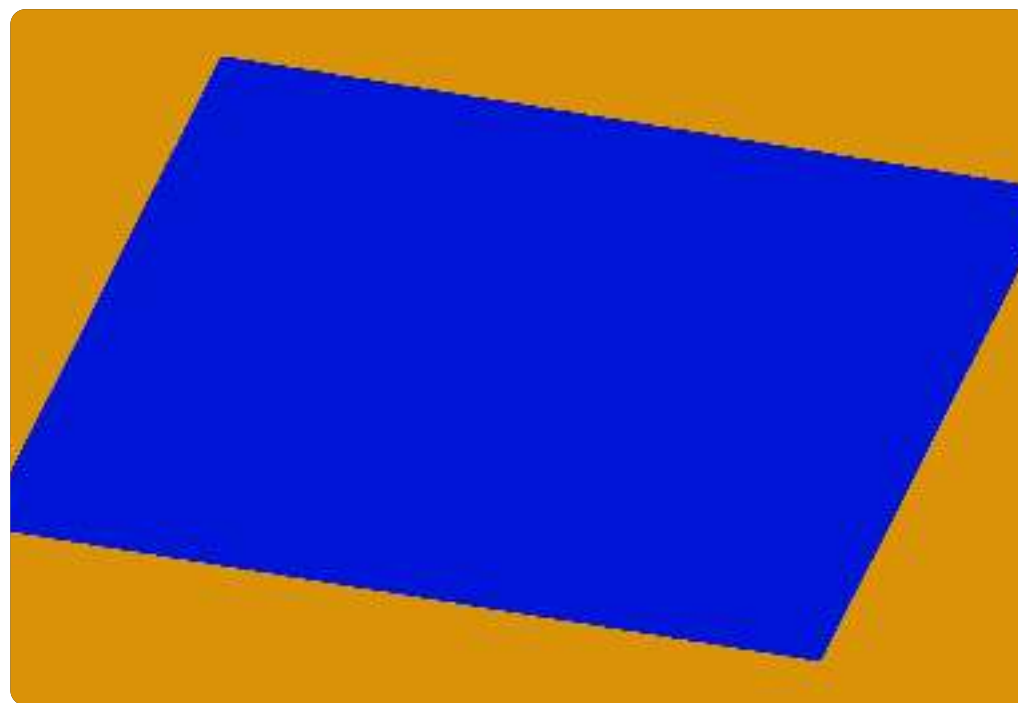
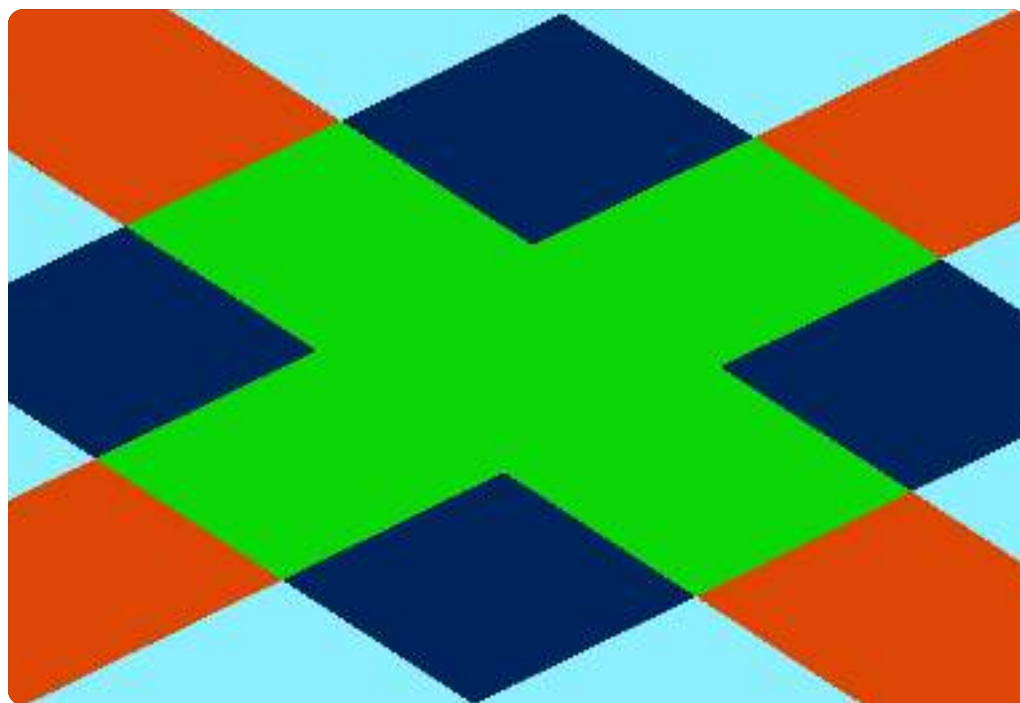
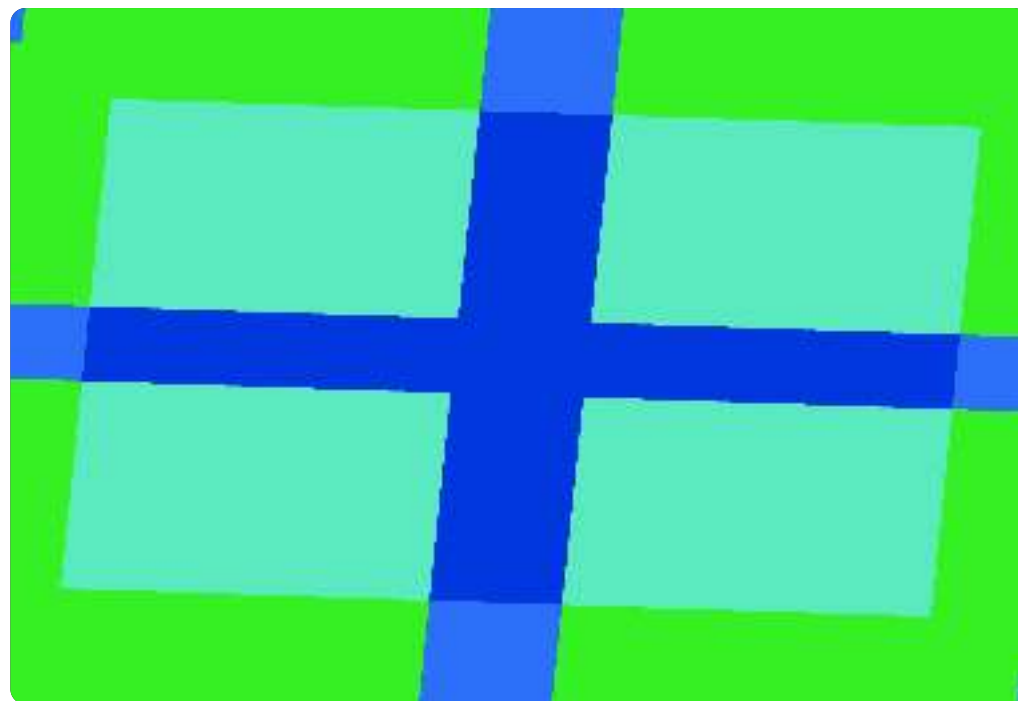
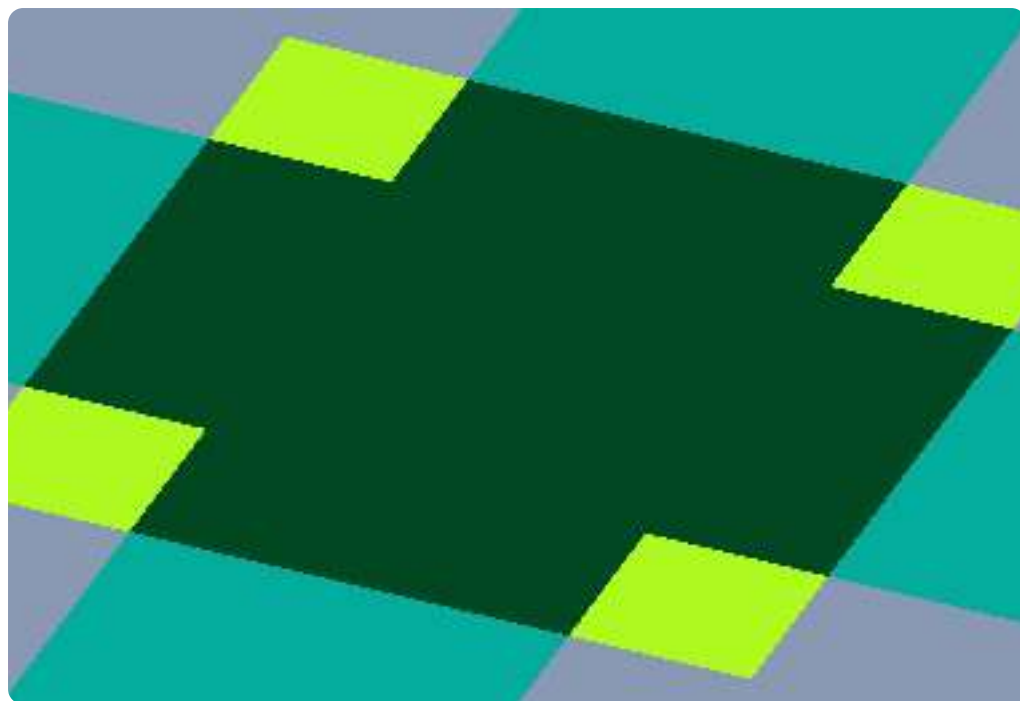
.mult( shape(4,0.8,0.2) )
.contrast(10)
.out()

```

O patch se organiza em torno de ruído (**noise**), limiar (**thresh**), colorização (**color**), soma de camadas (**add**) e escala (**scale**). **shape** define máscaras e contornos que recortam cor e movimento. A composição usa **mult**, **add**, **diff** para misturar ou recortar sinais em vez de apenas empilhar efeitos.

A modulação (**modulate**) faz um sinal alterar outro — o quadro nunca é estático, mesmo quando a paleta parece fixa. Cor e saturação fecham a leitura cromática — **color**, **colorama** ou **saturate** deslocam o clima sem redesenhar a geometria.





```
// quad H
await loadScript("https://metagrowing.org/extra-shaders-for-hydra/lib-noise.js")
speed=0.0012
colornoise(900,0.0004)
.scrollY(0.025,13)
.kaleid(4)
.rotate(0.1,-32)
.out()
```

O patch se organiza em torno de ruído colorido (**colornoise**), deslocamento Y (**scrollY**), rotação (**rotate**) e kaleidoscópio (**kaleid**). O ruído não é textura passiva: ele empurra posição, escala ou cor ao longo do tempo.

speed = 0.0012 define o pulso global; expressões com **time** (e às vezes **Math.sin/Math.cos**) amarram rotação, saturação ou deslocamento ao relógio do Hydra. Há entrada externa (**loadScript**): o patch deixa de ser só síntese e passa a negociar com imagem, vídeo ou scripts auxiliares.

